

Manual do Usuário do Painel 3X-UI

العربية · English · Español · فارسی · Bahasa Indonesia · 日本語 · Português · Русский ·
Türkçe · Українська · Tiếng Việt · 简体中文 · 繁體中文

Versão do 3X-UI: 3.4.0. O manual foi elaborado para esta versão e é válido para ela. Um resumo das alterações da versão 3.4.0 em relação à 3.3.1 está na seção «[O que há de novo na 3.4.0](#)».

Manual detalhado em português do painel web **3X-UI** (gerenciamento do Xray-core): funcionalidades, configuração e operação, com explicação de cada campo e opção da interface.

Os nomes e rótulos correspondem à interface do painel; os termos estão apresentados conforme exibidos na interface. As palavras *inbound* / *outbound* não são traduzidas.

Sumário

- O que há de novo na 3.4.0
- 1. Introdução, requisitos e instalação
 - 1.1. O que é o 3X-UI
 - 1.2. Sistemas operacionais e arquiteturas suportados
 - 1.3. Métodos de instalação
 - 1.4. Primeiro acesso e credenciais padrão
 - 1.5. Localização dos arquivos
 - 1.6. Comando de gerenciamento `x-ui` (menu do script)
 - 1.7. Subcomandos `x-ui` (sem menu interativo)
 - 1.8. Migração SQLite → PostgreSQL
- 2. Acesso ao painel e segurança
 - 2.1. Formulário de login
 - 2.2. Autenticação de dois fatores (2FA / TOTP)
 - 2.3. Limite de tentativas de login (login limiter / proteção contra força bruta)
 - 2.4. Alteração do login e senha do administrador
 - 2.5. Caminho secreto (URI path / webBasePath) e porta do painel
 - 2.6. Tempo de vida da sessão (timeout)
 - 2.7. LDAP (sincronização e autenticação)
- 3. Visão geral / Dashboard
 - 3.1. Princípios gerais de coleta de dados
 - 3.2. CPU
 - 3.3. Memória (RAM)
 - 3.4. Swap
 - 3.5. Disco (Storage)
 - 3.6. Tempo de atividade do sistema (Uptime)
 - 3.7. Carga do sistema (Load average)
 - 3.8. Rede: velocidade e volume total de tráfego
 - 3.9. Endereços IP do servidor
 - 3.10. Conexões TCP/UDP
 - 3.11. Status do Xray e controle do processo
 - 3.12. Atualização do painel (3X-UI)
 - 3.13. Atualização dos arquivos geográficos (GeoIP / GeoSite)
 - 3.14. Backup e restauração do banco de dados
 - 3.15. Elementos adicionais da interface
- 4. Inbounds: criação e parâmetros gerais
 - 4.1. Campos gerais do formulário
 - 4.2. Sniffing
 - 4.3. Allocate (estratégia de distribuição de portas)
 - 4.4. External Proxy (Proxy externo)
 - 4.5. Fallbacks
 - 4.6. Reset periódico de tráfego
 - 4.7. JSON do inbound (advanced)
 - 4.8. Ações sobre o inbound: QR / Edit / Reset / Delete e estatísticas

- 5. Protocolos
 - 5.1. Lista de protocolos suportados
 - 5.2. Quais protocolos suportam TLS / REALITY / transporte
 - 5.3. VLESS
 - 5.4. VMess
 - 5.5. Trojan
 - 5.6. Shadowsocks
 - 5.7. Dokodemo-door / Tunnel (encaminhador transparente)
 - 5.8. SOCKS / HTTP (protocolo `mixed`)
 - 5.9. WireGuard (inbound)
 - 5.10. Hysteria (padrão v2)
 - 5.11. MTPROTO (proxy para Telegram)
 - 5.12. Guia rápido de escolha de protocolo
- 6. Transporte (Stream Settings)
 - 6.1. Escolha da rede de transmissão
 - 6.2. RAW / TCP (`tcpSettings`)
 - 6.3. mKCP (`kcpSettings`)
 - 6.4. WebSocket (`wsSettings`)
 - 6.5. gRPC (`grpcSettings`)
 - 6.6. HTTPUpgrade (`httpupgradeSettings`)
 - 6.7. XHTTP / SplitHTTP (`xhttpSettings`)
 - 6.8. Transporte Hysteria (`hysteriaSettings`)
 - 6.9. Parâmetros adicionais
- 7. Segurança da conexão: TLS, XTLS e REALITY
 - 7.1. Diferença entre TLS vs XTLS vs REALITY
 - 7.2. Modo «Nenhum» (`none`)
 - 7.3. Modo TLS
 - 7.4. Modo REALITY
 - 7.5. Recomendações práticas de configuração
- 8. Clientes
 - 8.1. Campos do cliente
 - 8.2. Associação ao inbound
 - 8.3. Operações sobre o cliente
 - 8.4. Operações em massa
 - 8.5. Pesquisa, filtros e ordenação
 - 8.6. Ícones e status
- 9. Grupos de clientes
 - 9.1. O que é um grupo de clientes e para que serve
 - 9.2. Relação do grupo com clientes, inbounds, nós e protocolos
 - 9.3. Diretório de grupos e grupos «vazios»
 - 9.4. Campos e colunas do grupo
 - 9.5. Criação de grupo
 - 9.6. Renomeação de grupo
 - 9.7. Adição de clientes ao grupo
 - 9.8. Remoção de clientes do grupo (sem excluir os clientes)
 - 9.9. Reset de tráfego do grupo

- 9.10. Exclusão do grupo e dos clientes do grupo
- 9.11. Relação com a página «Clientes»
- 9.12. Resumo dos endpoints de API
- 9.13. Tráfego por grupo
- 10. Assinaturas (Subscription)
 - 10.1. O que é subld e como o link é formado
 - 10.2. Configurações do servidor de assinaturas
 - 10.3. Formatos de saída
 - 10.4. Página de informações da assinatura e QR codes
 - 10.5. Templates personalizados da página de assinatura
- 11. Xray: roteamento, outbounds, DNS e extensões
 - 11.1. Estrutura do editor: abas/modos
 - 11.2. Configurações gerais (General)
 - 11.3. Regras de roteamento (routing)
 - 11.4. Outbounds (conexões de saída)
 - 11.5. Balanceadores (Balancers)
 - 11.6. DNS
 - 11.7. Fake DNS
 - 11.8. WireGuard / WARP / NordVPN
 - 11.9. Proxy reverso e TUN
 - 11.10. Logs e estatísticas (Stats, metrics)
 - 11.11. Salvar, reiniciar e transformações automáticas
 - 11.12. Outbound de assinatura (com atualização automática)
 - 11.13. Rotação de IP no WARP
- 12. Nós (multipainel, master/slave)
 - 12.1. Resumo no topo da lista
 - 12.2. Adição e edição de nó
 - 12.3. Verificação TLS (para nós https)
 - 12.4. O que é exibido por nó
 - 12.5. Ações sobre o nó
 - 12.6. Histórico de métricas
 - 12.7. Como inbounds e clientes são sincronizados
 - 12.8. Cadeias de nós (sub-nós / nós de trânsito)
 - 12.9. Nós: novidades na 3.3.0
- 13. Configurações do painel
 - 13.1. Salvar e reiniciar o painel
 - 13.2. Configurações gerais (aba «Painel» / *General*)
 - 13.3. Acesso ao painel: IP, porta, caminho, domínio, certificado
 - 13.4. Sessão, proxy do painel e proxies confiáveis (aba «Proxy e Servidor» / *Proxy and Server*)
 - 13.5. Bot do Telegram (aba «Bot do Telegram» / *Telegram Bot*)
 - 13.6. Data e hora (aba «Data e Hora» / *Date and Time*)
 - 13.7. Tráfego externo e comportamento do Xray (aba «Tráfego Externo» / *External Traffic*)
 - 13.8. Outros: template de configuração do Xray e URL de verificação
 - 13.9. Conta de administrador e tokens de API
 - 13.10. Alterações de API na 3.3.0 (importante para integrações)

- 14. Bot do Telegram
 - 14.1. Ativação e configuração do bot
 - 14.2. Menu principal e botões
 - 14.3. Comandos do bot
 - 14.4. Gerenciamento de cliente (somente administrador)
 - 14.5. Notificações e relatórios
 - 14.6. Backup e logs
 - 14.7. Particularidades de funcionamento
 - 15. Bases geográficas (geoip / geosite e personalizadas)
 - 15.1. O que são geoip.dat e geosite.dat
 - 15.2. Arquivos geográficos padrão e sua atualização
 - 15.3. Recursos geográficos personalizados (Custom GeoSite / GeoIP)
 - 15.4. Validação e limitações
 - 15.5. Verificação automática na inicialização do painel
 - 15.6. Uso das bases geográficas nas regras de roteamento
 - 16. Operação: backups, logs, atualização, CLI
 - 16.1. Backup e restauração do banco de dados
 - 16.2. Visualização de logs
 - 16.3. Nível e configuração de logging do Xray
 - 16.4. Gerenciamento do Xray: parada e reinício
 - 16.5. Reinício e atualização do painel
 - 16.6. Tarefas periódicas (cron)
 - 16.7. Menu de console e CLI (`x-ui`)
 - 16.8. Remoção do painel
 - 16.9. Comando `x-ui migrateDB`
-

O que há de novo na 3.4.0

Esta seção lista brevemente as alterações da versão **3.4.0** em relação à 3.3.1, visíveis ao usuário do painel, agrupadas por seção do manual. Os detalhes de cada funcionalidade estão na seção correspondente abaixo.

3. Visão geral / Dashboard

- **Histórico de métricas do sistema: novos intervalos de agregação 12h/24h/48h** — Na janela do histórico de métricas do sistema foram adicionados valores 12h, 24h e 48h aos intervalos de média — agora os gráficos (CPU, RAM, tráfego, pacotes, conexões, disco, online, carga) podem ser visualizados por um período de até dois dias.

4. Inbounds: criação e parâmetros gerais

- **Inbound: aviso de conflito com porta reservada da API interna do Xray** — Ao criar ou editar um inbound, o painel agora impede o uso da porta reservada da API interna do Xray (por padrão 62789 em 127.0.0.1): um inbound TCP local nessa porta no loopback é recusado com erro de conflito de porta. Em nós (nodes) a restrição não se aplica — eles têm seu próprio Xray.
- **Tunnel/TProxy: aceitar streamSettings sem a chave security** — Inbounds do tipo tunnel/TProxy cujas streamSettings não contêm o bloco security agora abrem e salvam sem erro de validação.
- **Inbounds: coluna Speed (velocidade em tempo real) na lista** — Na lista de inbounds apareceu a coluna Speed com a velocidade atual de tráfego (upload/download) por inbound.

5. Protocolos

- **Shadowsocks-2022: regeneração das PSK dos clientes ao mudar o método com tamanho de chave diferente** — Para Shadowsocks-2022: ao trocar o método de criptografia por um método com tamanho de chave diferente (por exemplo entre aes-256 e aes-128), as PSK dos clientes agora são automaticamente regeneradas para o novo comprimento ao salvar o inbound. Consequência: os clientes afetados precisam obter a assinatura novamente (os links antigos deixarão de funcionar).
- **WireGuard: campo workers removido** — O campo workers foi removido dos formulários WireGuard (inbound e outbound): o xray-core v26.6.22 não o utiliza mais. Configurações salvas anteriormente continuam funcionando sem alterações — o campo é simplesmente ignorado.
- **VLESS+XHTTP: flow xtls-rprx-vision em links e assinaturas** — Para VLESS sobre XHTTP, o flow xtls-rprx-vision agora aparece corretamente nos links e assinaturas (incluindo para XHTTP+REALITY e no formato Clash/Mihomo). Antes, o painel salvava o flow, mas os clientes recebiam a configuração sem vision.

6. Transporte (Stream Settings)

- **XHTTP: renomeação dos campos sessionId + Session ID Table / Length** — No transporte XHTTP, os campos de sessão foram renomeados: Session ID Placement e Session ID Key (antes — Session Placement / Session Key). Foram adicionados dois parâmetros. Session ID Table — conjunto de caracteres para geração de identificadores de sessão: é possível escolher um predefinido (ALPHABET, Base62, hex, number etc.) ou inserir uma string ASCII arbitrária; vazio = valor padrão do xray-core. Session ID Length — comprimento ou intervalo (por exemplo 8-16) dos identificadores gerados; considerado apenas quando Session ID Table

está definido, o mínimo deve ser maior que 0. Inbounds salvos anteriormente são migrados automaticamente.

- **Inbound: preset Real client IP para determinar o IP real atrás de CDN/relay** – Nas configurações de socket (sockopt) do inbound foi adicionada a seleção Real client IP – um preset para determinar o IP real do visitante quando o tráfego chega via CDN ou relay (caso contrário o endereço do intermediário é registrado). Há três opções: Off / direct (sem processamento), Cloudflare CDN (confiar no X-Forwarded-For) e L4 relay / Spectrum (PROXY) (aceitar o cabeçalho do protocolo PROXY). Os presets são mutuamente exclusivos e avisam se o transporte selecionado não os suporta. Esses campos nunca são enviados aos clientes nas assinaturas.
- **gRPC: cabeçalho Trusted X-Forwarded-For agora é considerado** – O cabeçalho Trusted X-Forwarded-For agora é considerado também no transporte gRPC (antes – apenas WebSocket, HTTPUpgrade e XHTTP). Para inbounds gRPC, o painel não mostra mais o aviso sobre cabeçalho não suportado.

7. Segurança da conexão: TLS, XTLS e REALITY

- **TLS: novos campos Verify Peer Cert By Name, Curve Preferences, Master Key Log, ECH Sockopt** – Verify Peer Cert By Name – nomes (separados por vírgula) pelos quais o cliente verifica o certificado do servidor em vez do SNI; substituto moderno do allowInsecure removido, transmitido em links e assinaturas, não gravado no servidor. Curve Preferences – restrição e ordem das curvas de troca de chaves TLS (por exemplo X25519MLKEM768, X25519); vazio = valores padrão. Master Key Log – caminho para gravação de chaves TLS (formato SSLKEYLOGFILE) para depuração no Wireshark; deixar vazio em produção. ECH Sockopt – parâmetros de socket para obter configuração ECH (dialerProxy, Domain Strategy, TCP Fast Open, Multipath TCP).
- **REALITY: limitação de velocidade do fallback (Limit Fallback) e Master Key Log** – Para cada direção (Upload e Download) são definidos: After Bytes – quantos bytes passar em velocidade máxima antes de iniciar a limitação (0 = limitar desde o primeiro byte); Bytes Per Sec – teto de velocidade do tráfego de fallback, para que sondas não usem o servidor como canal gratuito (0 = sem limite, desativa a direção); Burst Bytes Per Sec – reserva para picos momentâneos. Também foi adicionado o campo Master Key Log (caminho para o arquivo SSLKEYLOGFILE para depuração).
- **TLS: botões de preenchimento de Pinned Peer Cert SHA-256 a partir do certificado do inbound e por SNI** – Ao lado do campo Pinned Peer Cert SHA-256 há agora dois botões de preenchimento automático em vez do antigo botão de hash aleatório. O primeiro insere o SHA-256 do certificado do próprio inbound. O segundo obtém o hash do certificado ativo do servidor, realizando uma conexão TLS pelo SNI indicado (serverName deve estar preenchido). Os hashes obtidos são adicionados ao campo (separados por vírgula) e incluídos nos links para fixação do certificado no cliente.
- **TLS: OCSP stapling desativado por padrão para novos certificados de inbound** – Para novos inbounds o OCSP stapling está desativado por padrão (intervalo 0). Isso elimina erros nos logs do xray para certificados sem responder OCSP (por exemplo Let's Encrypt). O campo permanece – pode ser ativado para certificados que suportem stapling.
- **REALITY: compatibilidade com o campo dest (alias de target)** – Se um inbound REALITY foi criado com o campo dest (por versões antigas do painel, via API ou ferramentas externas), agora carrega corretamente: o valor de dest é inserido no campo Target. Antes, o Target ficava vazio e ao salvar novamente o REALITY era corrompido.

8. Clientes

- **Aba «Links» no editor de cliente (links externos e assinaturas)** — Nela, pelo botão **Add External Link** são adicionados links de compartilhamento de terceiros (`vless://` , `vmess://` , `trojan://` , `ss://` , `hysteria2://` , `wireguard://`), e pelo botão **Add External Subscription** — URLs de assinaturas remotas. Tudo isso é mesclado na saída da assinatura deste cliente (formatos raw, JSON e Clash): os links são adicionados como estão, e as assinaturas remotas são periodicamente baixadas pelo painel e suas configurações são combinadas com as próprias.
- **Campo «IP Limit» agora é desativado sem o Fail2ban** — O campo **IP Limit** agora funciona apenas com o Fail2ban instalado e ativo. Se o Fail2ban não estiver instalado (ou o sistema for Windows, ou a função estiver desativada no servidor), o campo no editor de cliente fica bloqueado, e ao passar o mouse é exibida uma dica sugerindo instalar o Fail2ban pelo menu `bash x-ui`. Ao atualizar o painel, em clientes em servidores sem Fail2ban o limite de IP salvo é zerado, pois não era aplicado de qualquer forma.
- **Remoção de clientes não vinculados, exportação e importação de clientes** — No menu **Mais** da página **Clientes** foram adicionadas três operações. **Exportar clientes** exibe uma lista JSON de todos os clientes (no formato `{client, inboundIds}`) com botões de cópia e download (`clients-export.json`). **Importar clientes** aceita o mesmo JSON: clientes com vínculos são recriados e associados aos inbounds, clientes sem vínculos são restaurados como registros separados, e e-mails já existentes não são sobrescritos (são marcados como ignorados). **Remover clientes não vinculados** exclui todos os clientes não associados a nenhum inbound, junto com seu tráfego, histórico de IP e links externos; a ação é irreversível.
- **Histórico de IP do cliente: hora de conexão e nome do nó** — No histórico de IP do cliente (botão de visualização ao lado do campo «IP Limit» e no cartão «Informações do cliente»), cada registro agora contém, além do próprio IP, a hora do último acesso e a etiqueta do nó (`@ nome_do_nó`) pelo qual a conexão foi registrada — em configuração multipainel é possível ver por qual nó o cliente se conectou.
- **Reset da etiqueta de grupo no editor individual de cliente** — Agora, se no editor de um único cliente o campo **Grupo** for limpo e salvo, a etiqueta de grupo é corretamente removida — anteriormente o cliente podia continuar sendo exibido no grupo anterior até um novo salvamento.
- **Lista de clientes atualizada automaticamente (polling em segundo plano)** — A lista de clientes agora é atualizada automaticamente: o painel consulta a página atual a cada poucos segundos, portanto clientes recém-conectados e alterações na ordem de classificação aparecem sem atualização manual.

10. Assinaturas (Subscription)

- **Hosts gerenciados: substituição dos links de assinatura por hosts** — Na versão 3.4.0 foi adicionada a seção Hosts (item do menu lateral). Para cada inbound é possível definir um ou mais endpoints de Host que substituem o endereço, porta e parâmetros TLS do próprio inbound nos links de assinatura — conveniente para distribuir tráfego via CDN ou relay. Para o host são definidos Remark e descrição, associação ao inbound, Address (vazio = herda o endereço do inbound) e Port (0 = herda a porta do inbound), parâmetros de Security (same/tls/none/reality), além de Host header, Path, Mux, Sockopt, Final Mask, exclusão de formatos de assinatura (raw/json/clash) e parâmetros para Clash/mihomo. Os hosts são ordenados dentro do inbound e suportam operações em massa.
- **Remark Template substituiu o construtor do modelo de remark; variáveis {{VAR}}** — O antigo construtor de nome de perfil (escolha de Inbound/Email/External Proxy e separador) foi substituído pelo campo «Remark Template». Nele você escreve um formato de nome arbitrário, inserindo variáveis pelo botão: identificação do cliente (`{{EMAIL}}` , `{{INBOUND}}` , `{{HOST}}` , `{{ID}}` , `{{SUB_ID}}` , `{{COMMENT}}` , `{{TELEGRAM_ID}}`), tráfego (`{{TRAFFIC_USED}}` , `{{TRAFFIC_LEFT}}` , `{{TRAFFIC_TOTAL}}` , `{{UP}}` , `{{DOWN}}` , `{{USAGE_PERCENTAGE}}`) e

prazo/status ({{DAYS_LEFT}}, {{TIME_LEFT}}, {{EXPIRE_DATE}}, {{JALALI_EXPIRE_DATE}}, {{STATUS}}, {{STATUS_EMOJI}}). As variáveis são substituídas individualmente para cada cliente na geração da assinatura, com pré-visualização disponível. Segmentos separados pelo caractere «|» com valores ilimitados são ocultados automaticamente, e informações de consumo e prazo são exibidas apenas no primeiro link do cliente. Se o campo for deixado vazio, o modelo de remark anterior é utilizado.

- **Links externos e assinaturas remotas por cliente (aba Links)** – Aqui, para um cliente individual, é possível anexar links de compartilhamento de terceiros (Add External Link) e endereços de assinaturas externas (Add External Subscription) – eles serão incluídos na assinatura própria do cliente (formatos RAW, JSON e Clash). As assinaturas externas são baixadas pelo painel e mescladas com as configurações do cliente. Útil para fornecer ao cliente servidores adicionais além dos seus inbounds.
- **Happ: ocultar configurações do servidor no cliente (Hide server settings)** – Na aba Happ das configurações de assinatura foi adicionado o switch «Hide server settings». Quando ativado, o cliente Happ oculta a possibilidade de visualizar e alterar os parâmetros do servidor. A opção se aplica apenas ao cliente Happ.
- **O nome do nó não é mais adicionado ao nome do perfil na assinatura** – O nome do nó (Node) não é mais acrescentado ao nome do perfil na assinatura. No aplicativo cliente é exibido apenas o remark do inbound definido pelo administrador, sem o sufixo interno do tipo «@nome-do-nó».
- **Rótulo do modelo de remark renomeado Other → External Proxy (depois substituído pelo template)** – Não precisa de documentação separada: a renomeação do item do modelo de remark «Other» para «External Proxy» foi absorvida pela transição para o novo campo Remark Template, onde o construtor do modelo foi removido da UI.
- **Correção dos links de assinatura: SS2022, Shadowrocket, SIP002 obfs, XHTTP no Clash** – Melhorada a compatibilidade dos links de assinatura gerados: corrigida a codificação SS2022, deep-link para Shadowrocket, saída de Shadowsocks+obfs no formato SIP002 (plugin obfs-local) e conjunto completo de campos XHTTP nas assinaturas Clash/Mihomo. Não são necessárias configurações separadas – os links simplesmente são reconhecidos de forma mais correta pelos clientes.
- **Modelo de remarks de assinatura: item «Other» renomeado para «External Proxy»** – Nas configurações de assinatura no modelo de remarks, o item «Other» foi renomeado para «External Proxy» (fonte – remark do proxy externo). O comportamento não foi alterado, as configurações existentes permanecem compatíveis.
- **Assinaturas: seleção de variáveis de remark clicando nos chips (Remark variable picker)** – Ao lado do campo Remark Template há um conjunto agrupado de chips de variáveis: clicar na variável {{VAR}} a insere no template, ao passar o mouse é exibida a descrição. Nos campos de remark e nome do host também é aceita a notação simplificada com chaves simples – {DATA_LEFT}, {EXPIRE_DATE}, {PROTOCOL}, {TRANSPORT} etc.; o painel converte automaticamente para o formato interno {{...}}.

11. Xray: roteamento, outbounds, DNS e extensões

- **Roteamento e Outbounds movidos para itens separados do menu lateral** – A partir desta versão «Outbounds» e «Roteamento» (Routing) foram movidos para itens separados do menu lateral (logo abaixo de «Hosts»), cada um com seu próprio endereço – `/outbound` e `/routing`. Antes, o roteamento abria dentro do submenu «Configurações do Xray» e os outbounds como uma aba da página do Xray. No submenu «Configurações do Xray» agora ficam apenas: Principal, Balanceador, DNS e Template avançado. Links diretos para `/outbound` e `/routing` e o recarregamento da página funcionam como esperado.
- **Regras de roteamento podem ser ativadas e desativadas por switch** – Uma regra de roteamento individual agora pode ser **desativada** temporariamente por switch, sem excluí-la. Na tabela de regras há a coluna «Ativar» com switch, e no formulário da regra o campo «Ativar» também é um switch. Uma regra

desativada não entra na configuração final do Xray. A regra de serviço de estatísticas (api) não pode ser desativada – seu switch está bloqueado.

- **Exportação de regras de roteamento e outbounds abre em janela modal de prévia** – O botão «Exportar» das regras de roteamento e outbounds agora não baixa o arquivo imediatamente, mas abre uma janela modal com prévia do JSON e botões «Copiar» e «Baixar». Na aba «Roteamento», «Importar» e «Exportar» estão agrupados no menu suspenso «mais» (como na aba Outbounds).
- **Teste de todos os outbounds agora verifica também os outbounds de assinaturas; direct/dns não são mais testados** – O botão «Testar todos» na página «Outbounds» agora verifica também os outbounds obtidos de assinaturas (tabela «de assinaturas») – suas linhas também são destacadas com o resultado. Os outbounds `freedom` («direct») e `dns` não são mais testados em nenhum modo (não são proxies): o botão de teste fica indisponível para eles, e «Testar todos» os ignora.
- **FinalMask: arrays por fragmento Lengths/Delays em vez dos campos únicos Length/Delay** – Na máscara fragment (FinalMask), os campos únicos Length e Delay foram substituídos por listas Lengths e Delays: para cada segmento é possível definir um intervalo de comprimento separado (por exemplo 100-200) e atraso em ms (por exemplo 10-20 ou 0). Linhas podem ser adicionadas e removidas; valores salvos anteriormente são transferidos automaticamente.
- **Outbound loopback: bloco Sniffing adicionado** – O outbound do tipo loopback ganhou um bloco Sniffing com os mesmos parâmetros que o inbound: ativação, destOverride, Metadata Only, Route Only e lista de domínios excluídos.
- **Hysteria2 / Salamander: modo Gecko (packetSize) e TLS para máscara Realm** – Na máscara UDP (FinalMask) para Hysteria2 as possibilidades foram expandidas. Na máscara Salamander foi adicionado o seletor Mode: o modo Gecko adiciona preenchimento aleatório de pacotes com campos Min/Max de tamanho (de 1 a 2048, padrão 512-1200) para proteção contra análise de comprimento de pacotes. Na máscara Realm apareceu um bloco TLS Config opcional: Server Name (SNI), ALPN (h3/h2/http/1.1), Fingerprint e Allow Insecure.
- **Importação de share-link em outbound preserva configurações de xmux** – Ao importar um outbound de share-link, as configurações do multiplexador **xmux** (XHTTP) agora são preservadas: antes eram silenciosamente perdidas. Após a importação os valores são preenchidos no subformulário XMUX.
- **Tags de outbounds de assinaturas preservam caracteres não-ASCII (cirílico)** – As tags de outbounds obtidos de assinaturas agora preservam caracteres não-ASCII (por exemplo cirílico) e permanecem legíveis, em vez de serem reduzidas a apenas dígitos.

12. Nós (multipainel, master/slave)

- **Nós: novo modo de verificação TLS – Mutual TLS (certificado de cliente)** – No formulário do nó, o modo de verificação TLS agora tem quatro opções: Verify (CA do sistema), Pin (fixação do certificado por SHA-256), Skip (sem verificação) e o novo Mutual TLS (certificado de cliente). No modo Mutual TLS o painel se autentica adicionalmente ao nó com um certificado de cliente emitido pela sua própria CA; o token de API para tal nó passa a ser opcional. Para ativar mTLS: no nó defina o modo Mutual TLS, copie a CA do painel de controle na seção Node mTLS, configure-a no nó como CA pai confiável e reinicie o nó.
- **Nós: seção «Node mTLS» – copiar CA do painel e CA pai confiável** – Na página Nós foi adicionada a seção Node mTLS para configurar autenticação TLS mútua entre painéis. O botão «Copiar CA deste painel» copia o certificado raiz do painel para a área de transferência – deve ser passado aos nós gerenciados que verificarão o certificado de cliente do painel. O campo «CA pai confiável» é usado quando o próprio painel é um nó: cole aqui a CA do painel de controle para exigir seu certificado de cliente e reinicie o painel. O

TLS mútuo é opcional; se os campos estiverem vazios, os nós funcionam pelo esquema anterior — apenas com token de API.

- **Roteamento da conexão de saída do painel ao nó (Connection outbound)** — No formulário do nó apareceu o campo «**Connection outbound**» (conexão de saída). Se um tag de Xray-outbound for selecionado, o tráfego do painel para a API deste nó passará pelo outbound indicado (o painel adicionará automaticamente à configuração ativa uma ponte-inbound no loopback e a aplicará sem reinicialização). Valor vazio = conexão direta. Na lista, os tags estão agrupados em «Outbounds» e «Balanceadores», outbounds do tipo blackhole estão ocultos.
- **Nós: roteamento do tráfego painel→nó pelo outbound selecionado («Connection outbound»)** — No formulário do nó apareceu o campo «Connection outbound»: permite direcionar o tráfego das solicitações do painel ao nó pelo Xray outbound selecionado (disponíveis outbounds comuns e balanceadores). O painel adiciona automaticamente à configuração ativa um inbound de loopback-bridge e aplica as alterações sem reinicialização. Deixe o campo vazio para conexão direta.
- **Nós: exclusão de nó bloqueada enquanto há inbounds associados** — Um nó só pode ser excluído depois que todos os inbounds forem desassociados dele. Se pelo menos um inbound ainda estiver associado ao nó, o painel recusará a exclusão com erro — primeiro desassocie ou exclua esses inbounds, depois exclua o nó.
- **Nós: velocidade em tempo real dos inbounds hospedados no nó é exibida na página de nós** — Na página Nós, para os inbounds hospedados no nó, agora são exibidos clientes online, contadores e velocidade de transmissão atual. O chip «encerrados» (ended) conta apenas clientes expirados e com tráfego esgotado (os desativados não entram mais nele).

14. Bot do Telegram

- **Notificações: novo barramento de eventos com canais Telegram e Email (SMTP)** — Foi adicionado um sistema de notificações por eventos com dois canais de entrega — Telegram e Email. Na aba de notificações os eventos estão agrupados em cartões: Outbound (queda/recuperação), Xray Core (encerramento inesperado), Nodes (nó indisponível/recuperado), System (alta carga de CPU e memória com limite configurável em %), Security (tentativa de login). Cada grupo tem um switch mestre e contador de eventos selecionados. O conjunto de eventos ativados é configurado separadamente para Telegram e Email; por padrão estão ativados «tentativa de login» e «alta carga de CPU».
- **Notificações: novo canal Email/SMTP e configurações do servidor SMTP** — Foi adicionado um novo canal de notificações por e-mail via SMTP. Na aba de configurações SMTP são definidos: ativação de notificações por e-mail, host e porta SMTP (padrão 587), nome de usuário, senha (armazenada de forma oculta), lista de destinatários (separados por vírgula) e tipo de criptografia — none, STARTTLS (padrão) ou TLS. O botão «Enviar e-mail de teste» verifica a conexão e mostra em qual etapa (conexão, autenticação, envio) ocorreu o erro. Na segunda aba são selecionados os eventos sobre os quais serão enviados e-mails.
- **Notificações: alerta de alta utilização de memória (limite de RAM)** — Aos alertas de alta carga de CPU foi adicionado um alerta de alta utilização de memória RAM. No grupo de eventos «System» apareceu «Memory high (%)» com seu próprio campo de limite (padrão 80%); o painel verifica a carga de RAM a cada minuto e, ao ultrapassar o limite, envia notificação nos canais selecionados.

15. Bases geográficas (geoip / geosite e personalizadas)

- **Atualização de bases geográficas: status por arquivo e pulo de reinício sem alterações** — A atualização das bases geográficas (geoip/geosite, incluindo os conjuntos IR e RU) agora mostra o status de cada arquivo:

atualizado, já atual ou erro de download. O reinício do Xray (e consequente interrupção das conexões ativas) ocorre apenas se pelo menos um arquivo foi realmente atualizado; na ausência de alterações o painel não reinicia. O mesmo comportamento se aplica ao comando `x-ui update-all-geofiles`.

16. Operação: backups, logs, atualização, CLI

- **IP limit do cliente funciona apenas com fail2ban instalado; campo fica bloqueado caso contrário** — A restrição por quantidade de IP do cliente agora funciona apenas se o fail2ban estiver instalado no servidor. Se não estiver, o campo «IP Limit» no formulário do cliente e no cadastro em massa fica indisponível com uma dica explicativa (no Windows — com uma mensagem separada), e os limites anteriormente definidos nesses servidores são zerados automaticamente, pois não eram aplicados de qualquer forma. O bloqueio pelo fail2ban agora se aplica tanto a TCP quanto a UDP.
- **Instalação automática do fail2ban durante instalação e atualização do painel** — Durante a instalação e atualização do painel em um servidor comum, o fail2ban agora é instalado e configurado automaticamente (antes isso ocorria apenas no Docker), para que a função de IP limit funcione imediatamente. O comportamento é controlado pela variável de ambiente `XUI_ENABLE_FAIL2BAN`: a configuração é executada se a variável não estiver definida ou for igual a `true`. A execução manual está disponível pelo comando `x-ui setup-fail2ban`; um erro no fail2ban não interrompe a instalação ou atualização.
- **Substituição da porta do painel pela variável**
XUI_PORT — Foi adicionada a variável de ambiente `XUI_PORT`, que define a porta do painel web apenas durante a execução do processo atual, sem alterar o valor salvo de `webPort` no banco de dados. Valores de 1 a 65535 são aceitos; valor vazio, inválido ou fora do intervalo é ignorado (usa `webPort`) com aviso no log. Ao usar Docker com rede bridge, a porta publicada do contêiner deve coincidir com `XUI_PORT`, por exemplo `XUI_PORT=8080` e `ports: «8080:8080»`.
- **CLI: flags -webCert/-webCertKey agora funcionam no subcomando setting** — Na CLI, os flags `-webCert` e `-webCertKey` agora funcionam também no subcomando `x-ui setting` (antes eram silenciosamente ignorados e o painel ficava sem HTTPS). Especificando-os, é possível definir imediatamente o caminho para o certificado e a chave do painel web, sem chamar um subcomando separado `cert`.
- **Nome do arquivo de backup do banco de dados é formado pelo endereço do servidor** — Os arquivos de backup do banco de dados agora recebem o nome baseado no endereço do servidor, e não o fixo `x-ui.db` / `x-ui.dump`. Ao baixar pelo navegador, o nome é obtido do endereço do painel na barra de endereços, caso contrário do domínio web configurado, e na ausência deste — do IP público (primeiro IPv4, depois IPv6). Assim os backups de servidores diferentes são fáceis de distinguir. A extensão permanece `.db` para SQLite e `.dump` para PostgreSQL.
- **Suporte a instalação e atualização em hosts somente com IPv6** — Os scripts de instalação e atualização agora funcionam corretamente em servidores somente com IPv6: o download do release e dos arquivos auxiliares não força mais o uso de IPv4, portanto o painel pode ser instalado e atualizado em um host sem endereço IPv4.

1. Introdução, requisitos e instalação

1.1. O que é o 3X-UI

3X-UI é um painel de administração web de código aberto para servidores [Xray-core](#). O painel oferece uma interface web multilíngue unificada para implantação, configuração e monitoramento de uma ampla gama de protocolos proxy e VPN: de um único VPS a configurações distribuídas com múltiplos nós.

O 3X-UI é um fork avançado do projeto original X-UI. Em relação a ele, foram adicionados suporte a mais protocolos, maior estabilidade, contabilização de tráfego por cliente e diversas funcionalidades convenientes.

Principais recursos:

- **Inbound de diferentes protocolos** – VLESS, VMess, Trojan, Shadowsocks, WireGuard, Hysteria2, HTTP, SOCKS (Mixed), Dokodemo-door / Tunnel, TUN e **MTPProto** (proxy do Telegram, adicionado na versão 3.3.0).
- **Transportes modernos e criptografia** – TCP (Raw), mKCP, WebSocket, gRPC, HTTPUpgrade e XHTTP, protegidos por TLS, XTLS e REALITY.
- **Fallback** – atendimento de múltiplos protocolos na mesma porta (por exemplo, VLESS e Trojan na 443) por meio do mecanismo de fallback do Xray.
- **Gerenciamento por cliente** – cotas de tráfego, datas de expiração, limites de IP, exibição do status "online", links de convite em um clique, QR codes e assinaturas.
- **Estatísticas de tráfego** – por inbound, cliente e outbound, com possibilidade de redefinição.
- **Suporte a múltiplos nós** – gerenciamento e escalabilidade para vários servidores a partir de um único painel.
- **Outbound e roteamento** – WARP, NordVPN, regras de roteamento personalizadas, balanceadores de carga, encadeamento de proxies.
- **Servidor de assinaturas integrado** com vários formatos de saída.
- **Bot do Telegram** para monitoramento e gerenciamento remotos.
- **REST API** com documentação Swagger integrada.
- **Armazenamento flexível** – SQLite (padrão) ou PostgreSQL.
- **13 idiomas de interface**, temas claro e escuro.
- **Integração com Fail2ban** para aplicar limites de IP por cliente.

Importante: o projeto é destinado apenas ao uso pessoal. Não é recomendado utilizá-lo para fins ilegais ou em ambientes de produção.

1.2. Sistemas operacionais e arquiteturas suportados

Sistemas operacionais

O script de instalação identifica a distribuição pelo campo `ID` do arquivo `/etc/os-release` (ou `/usr/lib/os-release`). São oficialmente suportados:

Ubuntu, Debian, Armbian, Fedora, CentOS, RHEL, AlmaLinux, Rocky Linux, Oracle Linux, Amazon Linux, Virtuozzo, Arch, Manjaro, Parch, openSUSE (Tumbleweed / Leap), Alpine e Windows.

Em sistemas baseados em Alpine é utilizado o serviço OpenRC (`rc-service` / `rc-update`); nos demais, o `systemd`. Para CentOS 7, os pacotes são instalados via `yum` ; para versões mais recentes, via `dnf` . Caso a distribuição não seja reconhecida, o script tentará usar o gerenciador de pacotes `apt-get` como padrão.

Arquiteturas de processador

A arquitetura é determinada pela saída de `uname -m` e mapeada para um dos valores suportados:

Valor de <code>uname -m</code>	Arquitetura do 3X-UI
<code>x86_64</code> , <code>x64</code> , <code>amd64</code>	<code>amd64</code>
<code>i*86</code> , <code>x86</code>	<code>386</code>
<code>armv8*</code> , <code>arm64</code> , <code>aarch64</code>	<code>arm64</code>
<code>armv7*</code> , <code>arm</code>	<code>armv7</code>
<code>armv6*</code>	<code>armv6</code>
<code>armv5*</code>	<code>armv5</code>
<code>s390x</code>	<code>s390x</code>

Se a arquitetura não estiver nessa lista, o script exibirá a mensagem «Unsupported CPU architecture!» e encerrará a instalação.

Dependências básicas

Antes de instalar o painel, o script instala automaticamente um conjunto básico de pacotes (os nomes variam conforme a distribuição): `cron` / `cronie` / `dcron` , `curl` , `tar` , `tzdata` / `timezone` , `socat` , `ca-certificates` , `openssl` .

1.3. Formas de instalação

Forma 1. Script de instalação (recomendado)

A instalação é feita com um único comando como root:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh)
```

O script exige obrigatoriamente privilégios de root: se executado sem permissão de root, exibe «Please run this script with root privilege» e encerra com erro.

O que o instalador faz passo a passo:

1. Identifica o sistema operacional e a arquitetura.
2. Instala as dependências básicas.
3. Baixa o arquivo de release `x-ui-linux-<arch>.tar.gz` e o descompacta no diretório `/usr/local/x-ui` .
4. Baixa o script de gerenciamento `x-ui.sh` e o instala como comando `/usr/bin/x-ui` .

5. Cria o diretório de logs `/var/log/x-ui`.
6. Executa a configuração inicial: escolha do banco de dados, geração de credenciais, seleção de porta, configuração opcional de SSL.
7. Instala e inicia o serviço de inicialização automática (unit do systemd `x-ui.service` ou script init OpenRC para Alpine).

Escolha do banco de dados durante a instalação. O instalador oferece:

- 1) SQLite (padrão, recomendado com menos de 500 clientes) – um único arquivo `/etc/x-ui/x-ui.db`, sem necessidade de configuração.
- 2) PostgreSQL (recomendado para grande número de clientes ou múltiplos nós). O PostgreSQL pode ser instalado localmente (é criado um usuário e banco de dados dedicados com o nome `xui`) ou pode-se informar um DSN para um servidor já existente. Os parâmetros de conexão são gravados no arquivo de variáveis de ambiente do serviço (`/etc/default/x-ui`, `/etc/conf.d/x-ui` ou `/etc/sysconfig/x-ui`, dependendo da distribuição) na forma das variáveis `XUI_DB_TYPE` e `XUI_DB_DSN`.

Exemplo: gravação dos parâmetros do PostgreSQL no arquivo de variáveis de ambiente do serviço. Após escolher PostgreSQL e informar o DSN, o instalador adicionará ao arquivo de ambiente linhas semelhantes a estas:

```
XUI_DB_TYPE=postgres
XUI_DB_DSN=postgres://xui:S3cretPass@127.0.0.1:5432/xui?sslmode=disable
```

Aqui `xui` é o nome do usuário e do banco, `127.0.0.1:5432` é o endereço e a porta do servidor, e `sslmode=disable` é adequado para conexão local (para servidor remoto, normalmente se usa `require`).

Instalação de uma versão específica (antiga). É possível indicar explicitamente uma tag de versão – o instalador baixará o release correspondente:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/v2.4.0/install.sh)
v2.4.0
```

A versão mínima suportada para esse tipo de instalação é `v2.3.5`; se uma versão mais antiga for especificada, será exibida a mensagem «Please use a newer version (at least v2.3.5)».

Forma 2. Docker

Execução com banco de dados SQLite padrão:

```
docker compose up -d
```

Para executar com o serviço PostgreSQL integrado, é necessário descomentar as linhas `XUI_DB_*` no `docker-compose.yml` e iniciar com o perfil:

```
docker compose --profile postgres up -d
```

A imagem inclui o Fail2ban (ativo por padrão) para aplicar limites de IP por cliente. O Fail2ban bloqueia infratores via `iptables`, o que requer a capability `NET_ADMIN`. No `docker-compose.yml` ela já é fornecida via `cap_add`. Ao executar manualmente com `docker run`, as capabilities precisam ser adicionadas manualmente; caso contrário, os bloqueios serão apenas registrados nos logs, sem serem aplicados:

Exemplo: comando completo `docker run`. Versão mínima com mapeamento da porta do painel, capabilities de rede e volume persistente para o banco de dados:

```
docker run -d \
  --name 3x-ui \
  --restart unless-stopped \
  --cap-add=NET_ADMIN --cap-add=NET_RAW \
  -v $PWD/db:/etc/x-ui \
  -v $PWD/cert:/root/cert \
  -p 2053:2053 \
  ghcr.io/mhsanaei/3x-ui:latest
```

O volume `/etc/x-ui` preserva o arquivo `x-ui.db` entre reinicializações do contêiner; sem ele, as configurações e as contas serão perdidas.

```
docker run -d --cap-add=NET_ADMIN --cap-add=NET_RAW ... ghcr.io/mhsanaei/3x-ui
```

No Docker, o painel é o processo principal do contêiner: a inicialização automática é controlada pela política de reinício do contêiner (por exemplo, `restart: unless-stopped`), e não por um serviço interno ao contêiner.

1.4. Primeiro acesso e credenciais padrão

Na primeira instalação (quando ainda são utilizadas as credenciais padrão), o instalador **gera valores aleatórios** para nome de usuário, senha, caminho web e porta:

Parâmetro	Como é definido na instalação	Observação
Nome de usuário (Username)	string aleatória de 10 caracteres	gerado automaticamente
Senha (Password)	string aleatória de 10 caracteres	gerada automaticamente
Caminho web do painel (WebBasePath)	string aleatória de 18 caracteres	protege o painel de ser descoberto pela URL raiz
Porta do painel (Port)	por padrão, uma porta aleatória no intervalo 1024–62000; é possível definir manualmente	o valor «de fábrica» de <code>webPort</code> é <code>2053</code> , mas o instalador o substitui

Ao final da instalação, o script exibe um resumo com: nome de usuário, senha, porta, caminho web, token de API e o link de acesso pronto (Access URL) no formato:

```
https://<domínio-ou-IP>:<porta>/<caminho-web>
```

Se o certificado SSL não estiver configurado, o link será por `http://` e o script exibirá um aviso sobre a necessidade de configurar o SSL (item de menu 19).

Alteração obrigatória das credenciais. Como o login e a senha são gerados aleatoriamente, é necessário **salvá-los imediatamente após a instalação**. É possível alterá-los a qualquer momento pelo item de menu «Reset Username & Password» (veja abaixo) ou pela interface web nas configurações do painel. Após a redefinição, o script lembra: «Please use the new login username and password to access the X-UI panel. Also remember them!».

Após a instalação, o comando `x-ui` abre o menu de gerenciamento (veja a seção 1.6).

1.5. Localização dos arquivos

Caminho	Finalidade
<code>/usr/local/x-ui/</code>	diretório de instalação do painel (binário <code>x-ui</code> , script <code>x-ui.sh</code>)
<code>/usr/local/x-ui/bin/xray-linux-<arch></code>	binário do Xray-core (em armv5/armv6/armv7 é renomeado para <code>xray-linux-arm</code>)
<code>/usr/bin/x-ui</code>	script de gerenciamento (comando <code>x-ui</code>)
<code>/etc/x-ui/x-ui.db</code>	arquivo do banco de dados SQLite (padrão)
<code>/var/log/x-ui/</code>	diretório de logs do painel
<code>/etc/systemd/system/x-ui.service</code>	unit systemd do serviço (exceto Alpine)
<code>/etc/init.d/x-ui</code>	script init OpenRC (somente Alpine)
<code>/etc/default/x-ui</code> · <code>/etc/conf.d/x-ui</code> · <code>/etc/sysconfig/x-ui</code>	arquivo de variáveis de ambiente do serviço (o caminho depende da distribuição); aqui são gravadas <code>XUI_DB_TYPE</code> / <code>XUI_DB_DSN</code>

O diretório do banco de dados pode ser redefinido pela variável de ambiente `XUI_DB_FOLDER` (padrão `/etc/x-ui`), e o diretório dos binários do Xray pela variável `XUI_BIN_FOLDER` (padrão `bin` relativo ao diretório do painel). O nome do arquivo de banco de dados é `x-ui.db`.

Exemplo: mover o banco de dados para um disco separado. Para armazenar `x-ui.db` não em `/etc/x-ui`, mas em um disco montado como `/data`, defina a variável no arquivo de variáveis de ambiente do serviço e reinicie o painel:

```
echo 'XUI_DB_FOLDER=/data/x-ui' >> /etc/default/x-ui
mkdir -p /data/x-ui
systemctl restart x-ui
```

O caminho completo para o banco de dados será `/data/x-ui/x-ui.db`.

Principais variáveis de ambiente

Variável	Finalidade	Padrão
<code>XUI_DB_TYPE</code>	backend do banco de dados: <code>sqlite</code> ou <code>postgres</code>	<code>sqlite</code>

Variável	Finalidade	Padrão
XUI_DB_DSN	string de conexão do PostgreSQL (quando XUI_DB_TYPE=postgres)	–
XUI_DB_FOLDER	diretório do arquivo de banco de dados SQLite	/etc/x-ui
XUI_INIT_WEB_BASE_PATH	caminho URI inicial do painel web (somente na primeira inicialização)	/
XUI_DB_MAX_OPEN_CONNS	número máximo de conexões abertas (pool do PostgreSQL)	–
XUI_DB_MAX_IDLE_CONNS	número máximo de conexões ociosas (pool do PostgreSQL)	–
XUI_ENABLE_FAIL2BAN	habilitar a aplicação de limites de IP via Fail2ban	true
XUI_LOG_LEVEL	nível de log (debug , info , warning , error)	info
XUI_DEBUG	modo de depuração	false

Exemplo: habilitar temporariamente o log detalhado. Para diagnosticar um problema, eleve o nível de log para `debug` e reinicie o serviço:

```
echo 'XUI_LOG_LEVEL=debug' >> /etc/default/x-ui
systemctl restart x-ui
x-ui log      # visualizar o log de depuração
```

Após o diagnóstico, retorne o valor para `info` para evitar que o log cresça excessivamente.

Caminho inicial do painel web via variável de ambiente. A variável `XUI_INIT_WEB_BASE_PATH` define o caminho URI do painel web (`webBasePath`) durante a inicialização inicial das configurações. Isso é útil ao implantar via Docker ou systemd, para fixar desde o início o caminho de acesso ao painel. O valor é normalizado automaticamente – barras iniciais e finais são adicionadas quando necessário, e um valor vazio ou composto apenas de espaços é ignorado (nesse caso, o caminho padrão `/` é aplicado). A variável afeta **apenas a inicialização inicial**: se as configurações já foram criadas, o caminho é alterado pela interface web ou pelo item de menu «Reset Web Base Path».

1.6. Comando de gerenciamento `x-ui` (menu do script)

Após a instalação, o comando `x-ui` (executado como root) abre o menu interativo «3X-UI Panel Management Script». A seleção de um item é feita digitando seu número (intervalo 0–27). Muitos itens também estão disponíveis como subcomandos para uso em scripts (veja a seção 1.7).

O menu é dividido em blocos temáticos.

Instalação e atualização

- **1. Install** – instala o painel (executa `install.sh`). Antes da instalação, verifica se o painel ainda não está instalado.
- **2. Update** – atualiza todos os componentes do `x-ui` para a versão mais recente. Os dados não são perdidos; após a atualização, o painel é reiniciado automaticamente. Requer confirmação.
- **3. Update Menu** – atualiza apenas o script de gerenciamento (`x-ui.sh` / comando `x-ui`) para a versão atual, sem reinstalar o painel.

- **4. Legacy Version** – instala uma versão específica (antiga) do painel. O script solicita o número da versão (por exemplo, 2.4.0) e baixa o release correspondente.
- **5. Uninstall** – remove completamente o painel **junto com o Xray**. O serviço é parado e desativado, os diretórios `/etc/x-ui/` e `/usr/local/x-ui/`, o arquivo de variáveis de ambiente do serviço e o próprio script de gerenciamento são removidos. Requer confirmação (padrão «não»).

Credenciais e configurações

- **6. Reset Username & Password** – redefine o nome de usuário e a senha do painel. É possível digitar novos valores ou deixar em branco para geração aleatória (nome aleatório – 10 caracteres, senha aleatória – 18 caracteres). Opcionalmente, oferece a desativação da autenticação de dois fatores (2FA), se estiver configurada. Após a redefinição, o painel é reiniciado.
- **7. Reset Web Base Path** – redefine o caminho web do painel: um novo caminho aleatório é gerado (18 caracteres) e o painel é reiniciado. Útil quando o caminho anterior foi comprometido ou esquecido.
- **8. Reset Settings** – redefine todas as configurações do painel para os valores padrão. **As credenciais (nome de usuário e senha) e os dados das contas não são perdidos.** Requer confirmação; após a redefinição, o painel é reiniciado.
- **9. Change Port** – altera a porta do painel web. É solicitado o número da porta (1–65535); após a definição, é necessário reiniciar para que a porta entre em vigor.
- **10. View Current Settings** – exibe as configurações atuais (`x-ui setting -show`). Mostra, entre outros, o backend de banco de dados em uso (SQLite ou PostgreSQL com a senha mascarada no DSN) e o link de acesso pronto (Access URL). Se o SSL não estiver configurado, oferece a emissão de um certificado Let's Encrypt para o endereço IP.

Gerenciamento do serviço

- **11. Start** – inicia o serviço do painel. Se o painel já estiver em execução, exibe uma mensagem informando que a reinicialização não é necessária.
- **12. Stop** – para o serviço do painel.
- **13. Restart** – reinicia o serviço do painel.
- **14. Restart Xray** – reinicia apenas o núcleo Xray-core sem reiniciar o próprio painel (via `systemctl reload x-ui`; no Docker, por sinal `USR1` ao processo do painel).
- **15. Check Status** – verifica o estado do serviço (`systemctl status x-ui` ou `rc-service x-ui status`).
- **16. Logs Management** – gerenciamento de logs: visualização do log de depuração (Debug Log, via `journalctl`) e, exceto no Alpine, limpeza de todos os logs (Clear All logs).

Inicialização automática

- **17. Enable Autostart** – habilita a inicialização automática do painel ao iniciar o sistema operacional (`systemctl enable x-ui` ou `rc-update add`).
- **18. Disable Autostart** – desabilita a inicialização automática ao iniciar o sistema operacional.

No Docker, a inicialização automática é controlada pela política de reinício do contêiner, portanto esses itens exibem apenas a dica correspondente.

Segurança e rede

- **19. SSL Certificate Management** – gerenciamento de certificados SSL via acme.sh: emissão de certificado para domínio, revogação, renovação forçada, visualização de domínios existentes, indicação dos caminhos do certificado para o painel e emissão de certificado de curta duração (~6 dias, com renovação automática) para endereço IP.
- **20. Cloudflare SSL Certificate** – emissão de certificado SSL via validação DNS da Cloudflare.
- **21. IP Limit Management** – gerenciamento de limites de número de IPs por cliente (baseado no Fail2ban): visualização e remoção de bloqueios, entre outros.
- **22. Firewall Management** – gerenciamento do firewall (abertura/fechamento de portas e visualização de regras).
- **23. SSH Port Forwarding Management** – configuração de redirecionamento de portas SSH para acessar o painel da máquina local via túnel SSH.

Desempenho e manutenção

- **24. Enable BBR** – habilita/desabilita o algoritmo de controle de congestionamento TCP BBR (submenu com os itens Enable BBR / Disable BBR).
- **25. Update Geo Files** – atualiza as bases geo (arquivos `.dat`) com escolha da fonte: Loyalsoldier (`geoip.dat`, `geosite.dat`), chocolate4u (`geoip_IR.dat`, `geosite_IR.dat`), runetfreedom (`geoip_RU.dat`, `geosite_RU.dat`) ou All (todos de uma vez). Após a atualização, o painel é reiniciado.
- **26. Speedtest by Ookla** – executa o teste de velocidade de rede via Speedtest by Ookla.
- **27. PostgreSQL Management** – gerenciamento da instância PostgreSQL integrada/associada (habilitação e operações relacionadas).
- **0. Exit Script** – sair do menu.

1.7. Subcomandos do `x-ui` (sem menu interativo)

Para uso em scripts, o comando `x-ui` suporta subcomandos diretos (executar `x-ui` sem argumentos abre o menu):

Comando	Ação
<code>x-ui</code>	abrir o menu de gerenciamento
<code>x-ui start</code>	iniciar o painel
<code>x-ui stop</code>	parar o painel
<code>x-ui restart</code>	reiniciar o painel
<code>x-ui restart-xray</code>	reiniciar o Xray
<code>x-ui status</code>	estado atual do serviço
<code>x-ui settings</code>	configurações atuais
<code>x-ui enable</code>	habilitar inicialização automática ao iniciar o sistema
<code>x-ui disable</code>	desabilitar inicialização automática
<code>x-ui log</code>	visualizar logs

Comando	Ação
<code>x-ui banlog</code>	visualizar logs de bloqueios do Fail2ban
<code>x-ui update</code>	atualizar o painel
<code>x-ui update-all-geofiles</code>	atualizar todos os arquivos geo
<code>x-ui migrateDB [file]</code>	conversão <code>.db</code>  <code>.dump</code> (SQLite)
<code>x-ui legacy</code>	instalar versão antiga
<code>x-ui install</code>	instalar o painel
<code>x-ui uninstall</code>	remover o painel

1.8. Migração SQLite → PostgreSQL

Uma instalação existente em SQLite pode ser migrada para PostgreSQL:

```
x-ui migrate-db --dsn "postgres://xui:password@127.0.0.1:5432/xui?sslmode=disable"
# em seguida, definir XUI_DB_TYPE e XUI_DB_DSN em /etc/default/x-ui e reiniciar:
systemctl restart x-ui
```

O arquivo SQLite original permanece intacto – exclua-o manualmente somente após verificar que o novo backend está funcionando corretamente.

Exemplo: verificação da mudança para PostgreSQL. Após a migração, confirme que o painel está realmente funcionando no novo backend com o comando de visualização de configurações – a saída deve indicar PostgreSQL (a senha no DSN é mascarada):

```
x-ui settings | grep -i -E 'db|dsn'
```

Se o painel abrir e as contas estiverem presentes, o arquivo `x-ui.db` original pode ser excluído.

2. Acesso ao painel e segurança

Esta seção descreve tudo o que diz respeito à autenticação do administrador do painel 3X-UI: o formulário de login, a autenticação de dois fatores (TOTP), a proteção contra tentativas de força bruta, a alteração de credenciais, a mudança do caminho secreto e da porta do painel, o tempo de vida da sessão, bem como a sincronização/authenticação via LDAP.

2.1. Formulário de login

A página de login é servida na raiz do caminho secreto do painel (`webBasePath`). Se o usuário já estiver autenticado, ele é redirecionado automaticamente para `.../panel/` . A página conta com um seletor de tema, seleção de idioma da interface e o próprio formulário.

Campos do formulário:

Campo	Dica/título	Obrigatório	Descrição
Имя пользователя	«Имя пользователя»	Sim	Login do administrador. Um valor vazio é rejeitado no lado do cliente e, no servidor, com a mensagem «Введите имя пользователя».
Пароль	«Пароль»	Sim	Senha do administrador. Um valor vazio é rejeitado com a mensagem «Введите пароль».
Код 2FA	«Код 2FA»	Somente quando a 2FA está habilitada	O campo aparece somente se a autenticação de dois fatores estiver habilitada no painel. Código de 6 dígitos gerado pelo aplicativo autenticador.

O botão «**Войти**» envia o formulário para `POST /login` .

Comportamento e mensagens:

- Em caso de login bem-sucedido, é exibida a mensagem «Вход выполнен успешно» e ocorre o redirecionamento para `.../panel/` .
- Em qualquer erro de credenciais ou código 2FA inválido, o servidor retorna uma **única** mensagem: «Неверные данные учетной записи.» (em inglês: *Invalid username or password or two-factor code.*). Isso é intencional – o painel não indica o que está errado (login, senha ou código), para não facilitar ataques de força bruta.
- O campo «Код 2FA» é exibido ou ocultado pelo painel com base na requisição `POST /getTwoFactorEnable` , que retorna o status atual da 2FA antes mesmo da autenticação.
- Se a sessão do servidor expirou, na próxima requisição é exibida a mensagem «Сессия истекла. Войдите в систему снова», e o usuário é redirecionado para a página de login.

Observação sobre CSRF: antes do envio do formulário, o cliente obtém um token CSRF (`GET /csrf-token`); as requisições `/login` e `/logout` são protegidas por verificação CSRF.

Exemplo: login via API. Quando a 2FA está desativada, basta o login e a senha; com a 2FA habilitada, adiciona-se o campo `twoFactorCode` :

```
# Sem 2FA
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль'

# Com 2FA habilitada – adiciona-se o código de 6 dígitos
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль&twoFactorCode=123456'
```

Em caso de sucesso, o servidor retornará `Set-Cookie` com o cookie de sessão – que deve ser enviado nas requisições subsequentes a `/panel/api/...`.

2.2. Autenticação de dois fatores (2FA / TOTP)

A 2FA no 3X-UI é implementada segundo o padrão **TOTP** e é compatível com qualquer aplicativo autenticador (Google Authenticator, Aegis, FreeOTP, entre outros). Os parâmetros são definidos de forma fixa: algoritmo **SHA1**, 6 dígitos, período de **30** segundos, emissor (issuer) `3x-ui`, rótulo `Administrator`.

Exemplo: otpauth-URI codificado no QR-code. Se o aplicativo autenticador não conseguir escanear pela câmera, o token pode ser adicionado manualmente por este link (substitua seu segredo Base32 no lugar de `JBSWY3DPEHPK3PXP`):

```
otpauth://totp/3x-ui:Administrator?secret=JBSWY3DPEHPK3PXP&issuer=3x-
ui&algorithm=SHA1&digits=6&period=30
```

Os parâmetros `algorithm=SHA1`, `digits=6`, `period=30` correspondem aos valores fixos do painel – não é necessário alterá-los.

As configurações ficam em **Настройки** → **Учетная запись**, aba «**Двухфакторная аутентификация**».

Elemento	Texto	Descrição
Alternância	«Включить 2FA»	Habilita/desabilita a autenticação de dois fatores.
Descrição	«Добавляет дополнительный уровень аутентификации для повышения безопасности.»	Dica exibida abaixo da alternância.

Como habilitar a 2FA

Ao ativar a alternância, o painel **gera localmente um novo segredo** – uma string aleatória em codificação Base32 (alfabeto `A–Z` e `2–7`). Uma janela é aberta com o título «Включить двухфакторную аутентификацию» e instruções passo a passo:

1. «Отсканируйте этот QR-код в приложении для аутентификации или скопируйте токен рядом с QR-кодом и вставьте его в приложение». Abaixo do QR-code, o próprio segredo é exibido em formato de texto – ao clicar no QR-code, o segredo é copiado para a área de transferência (exibe «Скопировано»).
2. «Введите код из приложения» – é necessário digitar o código de 6 dígitos gerado pelo aplicativo. O código é verificado **no lado do navegador**: o painel calcula o TOTP atual com base no segredo recém-

gerado e o compara com o digitado. Se o código estiver incorreto – «Неверный код»; o campo aceita somente exatamente 6 dígitos.

Somente após a confirmação bem-sucedida o segredo e o sinalizador de habilitação são salvos. Ao salvar, é exibida a mensagem «Двухфакторная аутентификация была успешно установлена».

Importante: as alterações na seção de configurações são aplicadas pelo botão geral «**Сохранить**», após o qual normalmente é necessário reiniciar o painel («Сохраните изменения и перезапустите панель для их применения»). Na primeira habilitação da 2FA, o servidor adicionalmente **invalida todas as sessões ativas** (incrementa o «login epoch»), portanto, após aplicar a configuração, será necessário fazer login novamente – desta vez com o código 2FA.

Como desabilitar a 2FA

Pressionar a alternância novamente abre a janela «Отключить двухфакторную аутентификацию» com a dica «Введите код из приложения, чтобы отключить двухфакторную аутентификацию.». Após inserir o código correto, o sinalizador e o segredo são apagados, e é exibida a mensagem «Двухфакторная аутентификация была успешно удалена».

Verificação do código no login

Durante o login, o servidor recupera o segredo salvo e compara o TOTP atual com o código 2FA enviado. Uma divergência é tratada como login malsucedido, mas ao usuário é exibida a mesma mensagem unificada «Неверные данные учетной записи.».

Recuperação de acesso

Não existe um mecanismo separado de «códigos de recuperação» no 3X-UI. Se o acesso ao aplicativo autenticador for perdido, não é possível recuperar o login pela interface do painel. O único caminho é desabilitar a 2FA diretamente no banco de dados no servidor: redefinir a chave `twoFactorEnable` para `false` (e, se necessário, limpar `twoFactorToken`) na tabela de configurações, após o que reiniciar o painel. Por isso, recomenda-se guardar o segredo (token Base32) em um local seguro ao habilitar a 2FA.

Exemplo: desabilitação de emergência da 2FA no servidor. Obtendo acesso ao servidor via SSH, pare o painel, redefina as chaves na tabela de configurações e reinicie o painel:

```
x-ui stop
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='false' WHERE
key='twoFactorEnable';"
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='' WHERE key='twoFactorToken';"
x-ui start
```

Após isso, o login é feito apenas com login e senha, e a 2FA pode ser configurada novamente, se desejado.

Relação com a alteração de credenciais: ao alterar o login/senha (ver 2.4), a 2FA é **desabilitada automaticamente** no servidor, para que o segredo antigo não bloqueie o acesso com a nova conta.

2.3. Limitador de tentativas de login (proteção contra força bruta)

O painel possui um limitador integrado de logins malsucedidos (semelhante ao fail2ban no nível da aplicação). Os parâmetros são definidos no código e **não são configuráveis** pela interface:

Parâmetro	Valor	Finalidade
Máximo de falhas	5	Quantas tentativas malsucedidas são permitidas dentro da janela.
Janela de contagem	5 minutos	Janela deslizante em que as falhas se acumulam (as mais antigas são descartadas).
Bloqueio (cooldown)	15 minutos	Por quanto tempo a chave fica bloqueada após ultrapassar o limite.

Como funciona:

- A chave de bloqueio é construída a partir da **combinação «IP + login»** (o login é convertido para minúsculas e os espaços são removidos). Ou seja, o bloqueio se aplica ao par específico «endereço + nome de usuário», e não ao painel inteiro.
- A cada tentativa malsucedida (login/senha incorretos ou código 2FA inválido), o contador aumenta. Ao atingir **5 falhas em 5 minutos**, a chave é bloqueada por **15 minutos**. Durante o bloqueio, qualquer tentativa desse par é imediatamente rejeitada com a mesma mensagem «Неверные данные учетной записи.», mesmo que os dados estejam corretos.
- **Um login bem-sucedido redefine imediatamente** o contador e remove o bloqueio para esse par.
- O endereço IP do cliente é determinado levando em conta proxies confiáveis (ver `trustedProxyCIDRs`): os cabeçalhos `X-Real-IP` e `X-Forwarded-For` são aceitos somente se a requisição veio de um endereço confiável. Caso contrário, é usado o endereço real da conexão; se não for possível extraí-lo, é usada a string `unknown`.

Todas as tentativas são registradas em log. Para as malsucedidas, é gravado um aviso no log do servidor com o nome de usuário, IP, motivo e, em caso de bloqueio, o horário de `blocked_until`. Se as notificações de login via bot do Telegram estiverem habilitadas (`tgNotifyLogin` — «Уведомление о входе»), o administrador também recebe o nome de usuário, IP e horário tanto das tentativas bem-sucedidas quanto das malsucedidas e bloqueadas.

Exemplo: notificação de login no Telegram. Com `tgNotifyLogin` habilitado, após cada tentativa o administrador recebe uma mensagem semelhante a esta:

```
Уведомление о входе
Пользователь: admin
IP: 203.0.113.45
Время: 2026-06-10 14:32:07
Статус: успешно
```

Para o par «IP + login» bloqueado, o status indicará que a tentativa foi rejeitada pelo limitador.

2.4. Alteração de login e senha do administrador

Seção **Настройки** → **Учетная запись**, aba «Учетные данные администратора». Campos:

Campo	Texto	Descrição
Текущий логин	«Текущий логин»	Nome de usuário atual. Deve corresponder ao login atual; caso contrário, a alteração é rejeitada.
Текущий пароль	«Текущий пароль»	Senha atual para confirmação de identidade.
Новый логин	«Новый логин»	Novo nome de usuário. Não pode estar vazio.
Новый пароль	«Новый пароль»	Nova senha. Não pode estar vazia.

A alteração é aplicada pelo botão «Подтвердить» e enviada para `POST /panel/setting/updateUser`.

Lógica e mensagens do servidor:

- Se «Текущий логин» não corresponder ao real ou «Текущий пароль» estiver incorreto – «Произошла ошибка при изменении учетных данных администратора.» com a explicação «Неверное имя пользователя или пароль».
- Se o novo login ou a nova senha estiver vazio – explicação «Новое имя пользователя и новый пароль должны быть заполнены».
- Em caso de sucesso – «Вы успешно изменили учетные данные администратора.». A senha é armazenada como hash bcrypt.

Exemplo: alteração de credenciais via API. A requisição requer um cookie de sessão ativo (obtido no login) e a confirmação do login/senha atuais:

```
curl -X POST https://panel.example.com:2053/мой-секрет/panel/setting/updateUser \
  -b 'session=БАША_СЕССИОННАЯ_COOKIE' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'oldUsername=admin&oldPassword=СтарыйПароль&newUsername=root&newPassword=НовыйСложныйПароль'
```

Após o sucesso, a sessão atual é invalidada – será necessário fazer login novamente com as novas credenciais.

Efeitos importantes da alteração de credenciais:

- **Todas as sessões existentes são invalidadas** (o contador `login_epoch` do usuário é incrementado), portanto, após a alteração, o painel realiza logout automaticamente e redireciona para a página de login – é necessário fazer login novamente.
- Se a **2FA estava habilitada** no momento da alteração, ela é **automaticamente desabilitada** (o sinalizador e o segredo são redefinidos). A autenticação de dois fatores precisará ser configurada novamente após a alteração do login/senha.

Se a 2FA estiver habilitada, antes do envio do formulário é aberta a janela «Изменить учетные данные» com a dica «Введите код из приложения, чтобы изменить учетные данные администратора.» – as credenciais só podem ser alteradas mediante confirmação do código 2FA atual.

2.5. Caminho secreto (URI-путь / webBasePath) e porta do painel

Esses parâmetros ficam em **Настройки** → **Панель** e afetam diretamente a «invisibilidade» e a acessibilidade do painel. São aplicados após salvar e **reiniciar o painel**.

Campo	Texto	Valor padrão	Descrição
Порт панели	«Порт панели» (<code>panelPort</code>), dica «Порт, на котором работает панель»	2053	Porta TCP da interface web.
URI-путь	«URI-путь» (<code>panelUrlPath</code>), dica «Должен начинаться с '/' и заканчиваться '/'»	/	Caminho base secreto (<code>webBasePath</code>). O painel só é acessível por ele (por exemplo, <code>/мой-секрет/</code>).
IP-адрес для управления панелью	«IP-адрес для управления панелью» (<code>panelListeningIP</code>), dica «Оставьте пустым для подключения с любого IP»	vazio	Endereço em que o painel escuta. Vazio = todas as interfaces.
Домен панели	«Домен панели» (<code>panelListeningDomain</code>), dica «Оставьте пустым для подключения с любых доменов и IP.»	vazio	Restrição de acesso por domínio (Host).
Путь к публичному ключу сертификата панели	<code>publicKeyPath</code> , dica «Введите полный путь, начинающийся с '/'»	vazio	Certificado TLS para acesso HTTPS ao painel.
Путь к приватному ключу сертификата панели	<code>privateKeyPath</code> , mesma dica	vazio	Chave privada TLS.

Comportamento do caminho base (`webBasePath`):

- O valor é normalizado automaticamente: se não começar com `/` , o caractere é adicionado no início; se não terminar com `/` , é adicionado no final. Ou seja, o caminho sempre tem a forma `/.../` .
- O caminho base se aplica ao próprio painel, aos assets e ao cookie de sessão (o cookie é emitido apenas para esse caminho).

Recomendações de segurança (seção «Предупреждения безопасности»): o painel exibe avisos quando a configuração está «excessivamente pública»:

- «Панель работает по обычному HTTP — настройте TLS для продакшна.»
- «Стандартный порт 2053 широко известен — измените его на случайный.»
- «Базовый путь по умолчанию "/" широко известен — измените его на случайный.»

Em outras palavras, para um servidor em produção, deve-se definir uma **porta não padrão**, um **URI-путь não trivial** e um **certificado TLS**.

Exemplo: configuração «discreta» do painel para produção. Em **Настройки** → **Панель**, defina valores semelhantes a estes:

Campo	Valor
Porto панели	34571 (aleatória, em vez de 2053)
URI-путь	/aXf9Qm2/ (não trivial, começa e termina com /)
Путь к публичному ключу сертификата панели	/etc/letsencrypt/live/panel.example.com/fullchain.pem
Путь к приватному ключу сертификата панели	/etc/letsencrypt/live/panel.example.com/privkey.pem

Após salvar e reiniciar, o painel estará acessível apenas em `https://panel.example.com:34571/aXf9Qm2/`, e os avisos de segurança desaparecerão.

2.6. Tempo de vida da sessão (timeout)

O campo «**Продолжительность сессии**» (`sessionMaxAge`) encontra-se entre as configurações do painel/ intervalos.

Campo	Texto	Valor padrão	Unidade	Descrição
Продолжительность сессии	«Продолжительность сессии», dica «Продолжительность сессии в системе (значение: минута)»	360	minutos	Tempo de vida do cookie de sessão do administrador.

Comportamento:

- O valor é definido em **minutos** (padrão: 360 minutos = 6 horas) e é convertido para segundos na configuração do cookie.
- Se o valor for **maior que 0**, o cookie de sessão recebe o `MaxAge` correspondente. Após esse prazo, o cookie deixa de ser válido e, na próxima requisição, o usuário recebe «Сессия истекла. Войдите в систему снова».
- A sessão também se torna inválida antecipadamente ao alterar as credenciais ou na primeira habilitação da 2FA (por meio do mecanismo `login_epoch`, ver 2.4 e 2.2) e no logout explícito (`POST /logout`).
- O cookie de sessão é marcado como `HttpOnly`, com política `SameSite=Lax`; o sinalizador `Secure` é definido no acesso HTTPS direto ao painel.

Além do próprio timeout, há uma notificação relacionada: «**Задержка уведомления об истечении сессии**» (`expireTimeDiff`, dica «Получение уведомления об истечении срока действия сессии до достижения порогового значения (значение: день)», padrão `0`) — permite receber um aviso com antecedência.

2.7. LDAP (sincronização e autenticação)

A seção LDAP oferece duas possibilidades: (1) autenticar o login do administrador via LDAP, caso a senha local não corresponda, e (2) sincronizar periodicamente o estado dos clientes (sinalizador VLESS habilitado/desabilitado) a partir do diretório.

Como é utilizado no login: o servidor primeiro verifica o hash bcrypt local da senha. Se ele **não corresponder** e o LDAP estiver habilitado, o painel tenta autenticar o usuário no diretório: se `Bind DN` estiver configurado, é realizado um bind de serviço, em seguida a entrada do usuário é buscada pelo filtro e atributo, e é feita uma

tentativa de bind com o DN encontrado e a senha digitada. O sucesso implica login. (Após uma autenticação LDAP bem-sucedida, se a 2FA estiver habilitada, o código TOTP ainda é verificado.)

Campos da seção:

Campo	Texto	Valor padrão	Descrição
Включить LDAP-синхронизацию	«Включить LDAP-синхронизацию» (<code>enable</code>)	false	Interruptor principal da integração LDAP.
LDAP-хост	«LDAP-хост» (<code>host</code>)	vazio	Endereço do servidor LDAP.
Порт LDAP	«Порт LDAP» (<code>port</code>)	389	Porta. Para LDAPS, normalmente 636.
Использовать TLS (LDAPS)	«Использовать TLS (LDAPS)» (<code>useTls</code>)	false	Quando habilitado, utiliza o esquema <code>ldaps://</code> com verificação do certificado do servidor (sem pular a verificação).
Bind DN	«Bind DN» (<code>bindDn</code>)	vazio	DN da conta de serviço para bind/busca inicial. Se vazio – o bind não é realizado (busca anônima).
Пароль bind	dicas: «Настроено; оставьте пустым, чтобы сохранить текущий пароль.» / «Не настроено.» / «Настроено – введите новое значение для замены»	vazio	Senha para Bind DN . Armazenada separadamente; para manter a senha atual, deixe o campo vazio.
Base DN	«Base DN» (<code>baseDn</code>)	vazio	Raiz da subárvore em que a busca é realizada (busca recursiva em toda a subárvore).
Фильтр пользователя	«Фильтр пользователя» (<code>userFilter</code>)	(<code>objectClass=person</code>)	Filtro LDAP para seleção de contas. Na autenticação, o login é inserido no filtro com escape.
Атрибут пользователя (username/email)	«Атрибут пользователя (username/email)» (<code>userAttr</code>)	<code>mail</code>	Atributo mapeado ao login/identificador do cliente (por exemplo, <code>mail</code> ou <code>uid</code>).
Атрибут VLESS-флага	«Атрибут VLESS-флага» (<code>vlessField</code>)	<code>vless_enabled</code>	Atributo que determina se o acesso VLESS do cliente deve estar habilitado.
Общий атрибут флага (опц.)	«Общий атрибут флага (опц.)» (<code>flagField</code>), dica «Если задано, переопределяет флаг VLESS – напр. <code>shadowInactive</code> .»	vazio	Se definido, é usado em vez de <code>vless_enabled</code> .

Campo	Texto	Valor padrão	Descrição
Truthy-значения	«Truthy-значения» (<code>truthyValues</code>), dica «Через запятую; по умолчанию: <code>true,1,yes,on</code> »	<code>true,1,yes,on</code>	Lista de valores do atributo de sinalizador tratados como «habilitado».
Инвертировать флаг	«Инвертировать флаг» (<code>invertFlag</code>), dica «Включите, когда атрибут означает «отключено» (напр. <code>shadowInactive</code>).»	<code>false</code>	Inverte o significado do sinalizador.
Расписание синхронизации	«Расписание синхронизации» (<code>syncSchedule</code>), dica «Строка типа cron, напр. <code>@every 1m</code> »	<code>@every 1m</code>	Periodicidade da sincronização em formato semelhante a cron.
Теги входящих	«Теги входящих» (<code>inboundTags</code>), dica «Входящие, на которых LDAP-синхронизация может авто-создавать или авто-удалять клиентов.»	vazio	Limita em quais inbound as operações automáticas são permitidas. Se não houver inbound: «Входящие не найдены. Сначала создайте входящий.»
Авто-создание клиентов	«Авто-создание клиентов» (<code>autoCreate</code>)	<code>false</code>	Criar o cliente nos inbound especificados se ele aparecer no diretório.
Авто-удаление клиентов	«Авто-удаление клиентов» (<code>autoDelete</code>)	<code>false</code>	Remover o cliente se ele desaparecer do diretório.
Объём по умолчанию (ГБ)	«Объём по умолчанию (ГБ)» (<code>defaultTotalGb</code>)	<code>0</code>	Limite de tráfego para clientes criados automaticamente (0 = sem limite).
Срок по умолчанию (дни)	«Срок по умолчанию (дни)» (<code>defaultExpiryDays</code>)	<code>0</code>	Prazo de validade para clientes criados automaticamente (0 = sem prazo).
Лимит IP по умолчанию	«Лимит IP по умолчанию» (<code>defaultIpLimit</code>)	<code>0</code>	Limite de IPs simultâneos (0 = sem restrição).

Particularidades da lógica do sinalizador de sincronização: ao ler o atributo de sinalizador (`flagField` , padrão `vless_enabled`), o valor é considerado «habilitado» se estiver na lista de truthy-значения; com a inversão habilitada, o resultado é invertido. O atributo do usuário (`userAttr`) é usado como chave de mapeamento (email/nome) – entradas sem valor desse atributo são ignoradas.

Segurança: recomenda-se habilitar **TLS (LDAPS)** para que as senhas de bind e as senhas verificadas não sejam transmitidas em texto claro, e usar para o `Bind DN` uma conta com permissões mínimas necessárias de leitura.

Exemplo: configuração típica de sincronização LDAP (Active Directory). Preenchimento dos campos da seção para um diretório onde o status de acesso é armazenado em um atributo semelhante a `userAccountControl`, com mapeamento por e-mail:

Campo	Valor
LDAP-хост	<code>ldap.example.com</code>
Порт LDAP	<code>636</code>
Использовать TLS (LDAPS)	<code>habilitado</code>
Bind DN	<code>CN=svc-3xui,OU=Service,DC=example,DC=com</code>
Base DN	<code>OU=Users,DC=example,DC=com</code>
Фильтр пользователя	<code>(objectClass=person)</code>
Атрибут пользователя (username/email)	<code>mail</code>
Атрибут VLESS-флага	<code>vless_enabled</code>
Truthy-значения	<code>true,1,yes,on</code>
Расписание синхронизации	<code>@every 5m</code>

Com essa configuração, a cada 5 minutos o painel percorrerá a subárvore `OU=Users`, mapeará os clientes por `mail` e habilitará/desabilitará o acesso VLESS com base no valor de `vless_enabled`.

3. Visão Geral / Dashboard

O Dashboard («Dashboard», na interface em inglês – *Overview*) é a página inicial do painel. Ela exibe em tempo real o estado do servidor e do processo Xray. Todos os indicadores são fornecidos pelo servidor. O agendador em segundo plano reconstrói o snapshot **a cada 2 segundos** e o envia para todas as abas abertas via WebSocket; uma vez por minuto, as séries de métricas acumuladas são gravadas em disco. O endpoint HTTP `GET /status` retorna o último snapshot em cache.

A seguir, cada indicador e cada elemento de controle da página são detalhados.

3.1. Princípios gerais de coleta de dados

- O snapshot é coletado pela biblioteca `gopsutil`. Se uma medição específica falhar, o campo permanece zerado e um aviso é gravado no log (`get cpu percent failed`, `get uptime failed` etc.) – isso não derruba o dashboard inteiro; o tile correspondente simplesmente exibirá 0/N-A.
- As velocidades «instantâneas» (CPU %, rede, disco I/O) são calculadas como a diferença entre o snapshot atual e o anterior, dividida pelo intervalo em segundos. Por isso, no primeiro carregamento da página, os valores de velocidade podem ser zero até que o segundo snapshot seja coletado.
- O histórico pode ser consultado na seção «History» (*System History*) – os gráficos são construídos com as mesmas séries de dados descritas abaixo (veja item 3.12).

3.2. CPU

O tile «CPU» (*CPU*) exibe a carga atual do processador em percentual, bem como os parâmetros do próprio processador.

Indicador	Descrição
Carga da CPU, %	Fração do tempo de processador utilizado no último intervalo. Suavizada por média exponencial (EMA, coeficiente <code>alpha = 0.3</code>) para evitar oscilações bruscas no indicador. O valor é sempre limitado ao intervalo de 0 a 100 %. Na primeira medição retorna 0 (inicialização do ponto base).
Processadores lógicos	Número de núcleos lógicos – ou seja, incluindo Hyper-Threading.
Núcleos físicos	Número de núcleos físicos.
Frequência	Frequência base do processador em MHz. Solicitada de forma lazy e armazenada em cache: a primeira medição bem-sucedida é salva, novas tentativas são feitas no máximo a cada 5 minutos, e a própria solicitação tem timeout de 1,5 s (em alguns sistemas a consulta de frequência é lenta).

O cálculo da carga de CPU funciona assim: se houver uma implementação nativa para a plataforma, ela é utilizada; caso contrário, o cálculo é feito a partir dos deltas dos contadores de tempo de processador (busy / total). Os tempos Guest e GuestNice são excluídos para não contá-los duas vezes.

3.3. Memória (RAM)

O tile «RAM» (*RAM*) exibe o uso e o total. É apresentado como «usado / total» e/ou percentual de utilização. O histórico registra o percentual.

3.4. Swap

O tile «Swap» (*Swap*) exibe o uso e o total. Se o arquivo/partição de swap não estiver configurado (total = 0), o indicador é zero; na ausência de swap, o valor gravado na série histórica é 0.

3.5. Disco (Storage)

O tile «Disco» (*Storage*) exibe o uso e o total, considerando **apenas a partição raiz** /. O histórico «Uso do disco» (*Disk Usage*) registra o percentual de utilização. Separadamente, é coletado o I/O de disco (leitura / escrita, bytes/s) como delta dos contadores por intervalo – exibido na aba «Disco I/O» do histórico.

3.6. Uptime do sistema

O indicador «Uptime do sistema» (*Uptime*). Representa o tempo desde o boot **de todo o servidor** (em segundos), e não o tempo de execução do painel ou do Xray. O uptime do processo Xray é armazenado separadamente (veja item 3.9), assim como o número de threads do painel (*Threads*).

3.7. Carga média do sistema (Load average)

O bloco «Carga do sistema» (*System Load*) – um array de três números [Load1, Load5, Load15]. A dica de tooltip: «Média de carga do sistema nos últimos 1, 5 e 15 minutos» (*System load average for the past 1, 5, and 15 minutes*). O gráfico histórico é denominado «Média de carga do sistema (1 / 5 / 15 min)». As séries históricas registram os valores separadamente: load1, load5, load15.

Este é o indicador Unix padrão: número médio de processos na fila de execução. Como referência, compare com o número de núcleos: uma carga que supera consistentemente o número de núcleos físicos indica sobrecarga.

3.8. Rede: velocidade e volume total de tráfego

São consideradas **apenas as interfaces físicas**. Interfaces virtuais e de túnel são excluídas: lo / lo0, e tudo que começa com loopback, docker, br-, veth, virbr, tun, tap, wg, tailscale, zt. Os valores são somados para todas as interfaces restantes.

Velocidade geral (*Overall Speed*) – velocidade instantânea, delta dos contadores por intervalo:

Indicador	Descrição
Upload (<i>Upload</i>)	Velocidade de saída, bytes/s.
Download (<i>Download</i>)	Velocidade de entrada, bytes/s.

Volume total de tráfego (*Total Data*) – contadores acumulados desde o início do sistema:

Indicador	Descrição
Enviado (<i>Sent</i>)	Total de bytes enviados.
Recebido (<i>Received</i>)	Total de bytes recebidos.

Adicionalmente, são coletadas as velocidades de pacotes (pacotes/s) e os contadores totais de pacotes — exibidos na aba «Pacotes de rede» (*Network Packets*) do histórico. Séries históricas de rede: `netUp`, `netDown`, `pktUp`, `pktDown`.

3.9. Endereços IP do servidor

O bloco «Endereços IP do servidor» (*IP Addresses*) exibe `IPv4` e `IPv6`. Os endereços externos são determinados por meio de serviços de terceiros (`api4.ipify.org`, `ipv4.icanhazip.com`, `v4.api.ipinfo.io/ip`, `ipv4.myexternalip.com/raw`, `4.ident.me`, `check-host.net/ip` para `IPv4` e análogos para `IPv6`). A lista é percorrida em ordem até o primeiro retorno bem-sucedido; o timeout por requisição é de 3 s.

Particularidades:

- O resultado é **armazenado em cache** pelo tempo de vida do processo: um endereço determinado com sucesso não é consultado novamente.
- Se nenhum serviço responder, o campo permanece `N/A`. Para `IPv6`, ao primeiro `N/A` as requisições `IPv6` são completamente desabilitadas para não desperdiçar tempo em redes sem `IPv6`.
- Há um botão-«olho» para ocultar/exibir os endereços — tooltip «Ocultar ou exibir os endereços IP do servidor» (*Toggle visibility of the IP*). Trata-se apenas de ocultação visual na interface (por exemplo, para capturas de tela), sem efeito sobre os endereços em si.

3.10. Conexões TCP/UDP

O bloco «Estatísticas de conexões» (*Connection Stats*) exibe o número total de conexões TCP e UDP ativas no servidor (em todo o sistema, não apenas do Xray). O gráfico histórico é «Conexões ativas (TCP / UDP)» (*Active Connections*), séries `tcpCount`, `udpCount`.

3.11. Status do Xray e controle do processo

O card «Xray» exibe o estado do processo Xray-core e permite controlá-lo.

Estados

Valor	Label	Tradução	Quando é definido
<code>running</code>	«Iniciado»	<i>Running</i>	O processo Xray está em execução.
<code>stop</code>	«Parado»	<i>Stopped</i>	O processo não está em execução e não há erro de inicialização registrado.
<code>error</code>	«Erro»	<i>Error</i>	O processo não está em execução, mas há um erro de inicialização registrado. O texto do erro é exibido em um popup com o título «Ocorreu um erro ao executar o Xray» (<i>An error occurred while running Xray</i>).
—	«Desconhecido»	<i>Unknown</i>	Exibido enquanto o status ainda não foi recebido.

A **versão do Xray** é exibida ao lado do status.

Botões de controle

- **Stop** (*Stop*). Chama `POST /stopXrayService`. Em caso de sucesso, o painel envia via WebSocket o novo estado `stop` e a notificação «Xray interrompido com sucesso» (*Xray service has been stopped*); em caso de erro – o estado `error` com o texto. Importante: se o painel estiver acessível *através* do próprio Xray, interrompê-lo pode cortar a conexão com o painel – com conexão direta ao painel não há problema.
- **Restart** (*Restart*). Chama `POST /restartXrayService`. Antes da ação é exibida a confirmação «Reiniciar o xray?» com a descrição «Reinicia o serviço xray com a configuração salva». Em caso de sucesso – estado `running` e notificação «Xray reiniciado com sucesso» (*Xray service has been restarted successfully*). O restart aplica a configuração salva atual – utilize-o após alterar as configurações.

Nota. Neste fork, o dashboard conta com controle completo de Start / Stop / Restart para todos os tipos de autenticação; na UI original do 3x-ui não há botão «start» separado – a inicialização é feita via restart.

Botão de visualização de logs do Xray

No card do Xray há um botão de visualização de logs do Xray (*Logs*). Ele aparece apenas quando o access-log está configurado na configuração do Xray: o visualizador integrado lê exatamente esse arquivo, portanto sem o access-log o botão fica oculto. A visibilidade do botão está vinculada a um sinalizador separado `accessLogEnable` e não depende mais do limite de IP – a lista online e o limite de endereços IP continuam funcionando mesmo sem o access-log (veja item 8).

Seleção de versão do Xray

A seção «Seleção de versão» (*Version*) permite alternar o Xray-core para outro release. A lista de versões é carregada via `GET /getXrayVersion`:

- Fonte – API do GitHub do repositório `XTLS/Xray-core` (`/releases`). As requisições são armazenadas em cache por **15 minutos**; em caso de falha do GitHub, é retornada a última lista obtida com sucesso, para que o seletor não fique vazio.
- A lista inclui apenas releases no formato `X.Y.Z` e **não mais antigos que 26.4.25**.

Tooltips: «Selecione a versão desejada» (*Choose the version you want to switch to*) e o aviso «Importante: versões antigas podem não suportar as configurações atuais» (*Choose carefully, as older versions may not be compatible with current configurations*).

Alternância: `POST /installXray/:version`. Fluxo:

Exemplo. Alternar para uma versão específica do Xray-core (o cookie de sessão deve já ter sido obtido via autenticação):

```
curl -X POST 'https://panel.example.com:2053/xpanel/installXray/v25.6.8' \
  -b cookie.txt
```

Aqui `v25.6.8` é a tag da lista retornada por `GET /getXrayVersion`. A versão deve estar presente nessa lista, caso contrário o painel retornará recusa.

1. A versão selecionada é verificada na lista de releases atual (caso contrário – recusa).
2. O Xray é interrompido.
3. O arquivo `Xray-<os>-<arch>.zip` é baixado do GitHub para o SO e arquitetura atuais (suportados: amd64/64, arm64-v8a, arm32-v7a/v6/v5, 386/32, s390x; para Windows – `xray.exe`). O tamanho do arquivo e do binário é limitado a 200 MB.
4. O binário é substituído atômicaamente (via arquivo temporário + renomeação) e marcado como executável.
5. O Xray é reiniciado.

Antes da alternância, é exibido o diálogo «Alternar versão do Xray» (*Do you really want to change the Xray version?*) com a descrição «Isso alterará a versão do Xray para #version#». Em caso de sucesso – notificação «Xray atualizado com sucesso» (*Xray updated successfully*).

3.12. Atualização do painel (3X-UI)

Bloco de verificação de atualizações do painel. Os dados são fornecidos via `GET /getPanelUpdateInfo`:

Campo	Descrição
Versão atual do painel	Versão do painel instalado.
Última versão do painel	Último release do 3x-ui obtido do GitHub.
Atualização disponível	Indicador de que a última versão é mais recente que a atual. Se não houver atualização disponível – é exibido «Painel atualizado».

O botão «**Atualizar painel**» (*Update Panel*) executa `POST /updatePanel`. Tooltip: «Isso atualizará o 3X-UI para o último release e reiniciará o serviço do painel». Antes da execução – confirmação «Você realmente deseja atualizar o painel?» com o texto «Isso atualizará o 3X-UI para a versão #version# e reiniciará o serviço do painel».

Particularidades e limitações:

- A atualização automática é suportada **apenas no Linux** (em outros SOs é retornado um erro).
- O script de atualização é baixado do repositório oficial (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`, limite de 2 MB) e executado via `bash`, preferencialmente de forma isolada via `systemd-run`.
- Em caso de execução bem-sucedida é exibido «Atualização do painel iniciada» (*Panel update started*); se a verificação de atualização falhar – «Falha na verificação de atualização do painel». Durante a instalação é exibido o aviso «Instalação em andamento. Não atualize a página».

3.13. Atualização dos arquivos geográficos (GeoIP / GeoSite)

O botão/diálogo de atualização das bases geo chama `POST /updateGeofile` (todos os arquivos) ou `POST /updateGeofile/:fileName` (um arquivo). A atualização funciona com uma whitelist estrita de nomes e fontes:

Arquivo	Fonte
geoip.dat , geosite.dat	Loyalsoldier/v2ray-rules-dat (latest)
geoip_IR.dat , geosite_IR.dat	chocolate4u/Iran-v2ray-rules (latest)
geoip_RU.dat , geosite_RU.dat	runetfreedom/russia-v2ray-rules-dat (latest)

Comportamento:

- O nome do arquivo é validado: `...`, barras e caminhos absolutos são proibidos; são permitidos apenas `[a-zA-Z0-9._-]+.dat`. Arquivos fora da whitelist não são baixados.
- É utilizada requisição condicional `If-Modified-Since`: se o arquivo no servidor-fonte não foi alterado (HTTP 304), ele não é baixado novamente, apenas o timestamp é atualizado.
- Após o download, o Xray é **reiniciado** (para carregar as novas bases).

Exemplo. Atualizar apenas as bases geográficas russas, sem tocar nos demais arquivos:

```
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geoip_RU.dat' -b cookie.txt
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geosite_RU.dat' -b cookie.txt
```

Para atualizar todos os arquivos da whitelist de uma vez — chame `POST /updateGeofile` sem nome de arquivo.

- Diálogos: «Você realmente deseja atualizar o arquivo geográfico?» com «Isso atualizará o arquivo #filename#» para um único arquivo e «Isso atualizará todos os arquivos geográficos» para o botão «Atualizar todos». Sucesso — «Arquivos geográficos atualizados com sucesso».

3.14. Backup e restauração do banco de dados

O bloco «Backup e restauração» (*Backup & Restore*). O comportamento depende do SGBD em uso (SQLite por padrão ou PostgreSQL).

Exportação do banco (Backup)

O botão «Exportar banco de dados» / «Backup» (*Back Up*) chama `GET /getDb`. O arquivo é entregue como anexo:

- **SQLite**: primeiro é executado o checkpoint (flush do WAL), em seguida o arquivo `x-ui.db` é baixado. Tooltip: «Clique para baixar o arquivo .db contendo o backup do seu banco de dados atual...».
- **PostgreSQL**: é baixado o dump `x-ui.dump` em formato customizado (`pg_dump --format=custom --no-owner --no-privileges`). As ferramentas cliente do PostgreSQL devem estar instaladas no servidor; caso contrário — erro sobre a ausência do `pg_dump`.

Importação do banco (Restauração)

O botão «Importar banco de dados» / «Restauração» (*Restore*) carrega o arquivo via `POST /importDB` (campo de formulário `db`). Tooltip: «Clique para selecionar e carregar o arquivo .db... para restaurar o banco de dados a partir do backup».

O fluxo para **SQLite** é seguro, com rollback:

1. O arquivo é verificado quanto ao formato SQLite e salvo em arquivo temporário, depois a integridade é verificada.
2. O Xray é interrompido, o banco atual é fechado e renomeado para `*.backup` (fallback).
3. O novo arquivo é colocado no lugar do banco ativo, são executados a inicialização e a migração. Se algo der errado — o fallback é restaurado.
4. O Xray é reiniciado.

Para **PostgreSQL**, o `.dump` é carregado (verificação da assinatura `PGDMP`) e aplicado via `pg_restore --clean --if-exists --single-transaction ...`. O tooltip avisa explicitamente: «Isso substituirá todos os dados atuais».

Mensagens: «Banco de dados importado com sucesso», «Ocorreu um erro ao importar o banco de dados», «...ao ler o banco de dados», «...ao obter o banco de dados».

Arquivo de migração (entre SQLite e PostgreSQL)

O botão «Baixar arquivo de migração» (*Download Migration*) chama `GET /getMigration` e gera uma exportação portátil para execução do painel em outro SGBD:

- Em **SQLite**, é baixado `x-ui.dump` (dump SQL em texto).
- Em **PostgreSQL**, é baixado `x-ui.db` — um banco SQLite pronto, construído a partir dos dados do PostgreSQL.

3.15. Elementos adicionais da interface

- **Indicador de clientes online.** O dashboard mantém a série `online` (*Online Clients* / «Clientes online») — número de clientes com conexão ativa. É calculado com o Xray em execução (caso contrário 0) e registrado no histórico no mesmo ciclo de 2 segundos. Gráfico — aba «Online».
- **Histórico do sistema (gráficos).** Botão/seção «Gráficos» → «Histórico do sistema» com abas: «Largura de banda», «Pacotes», «Disco I/O», «Online», «Carga», «Conexões», «Uso do disco». Os dados são obtidos via `GET /history/:metric/:bucket`; os intervalos de agregação permitidos (bucket, seg): **2, 30, 60, 120, 180, 300, 720, 1440, 2880** (os três últimos correspondem aos presets **12h, 24h e 48h** no seletor de intervalo), cada aba recebe até 60 pontos. O buffer circular de métricas armazena dados por até **48 horas**, portanto os gráficos (CPU, RAM, tráfego, pacotes, conexões, disco, online, carga) podem ser visualizados em um período de até dois dias. Métricas permitidas: `cpu, mem, swap, netUp, netDown, pktUp, pktDown, diskRead, diskWrite, diskUsage, tcpCount, udpCount, online, load1, load5, load15`. O label «Últimos 2 minutos» corresponde a `bucket = 2` (modo tempo real).

Exemplo. Obter a série de carga de CPU dos últimos ~2 minutos (bucket = 2 s, até 60 pontos) e a mesma série agregada por 5 minutos (bucket = 300 s):

```
curl 'https://panel.example.com:2053/xpanel/history/cpu/2' -b cookie.txt
curl 'https://panel.example.com:2053/xpanel/history/cpu/300' -b cookie.txt
```

A métrica pode ser substituída por qualquer uma das permitidas (`mem` , `netUp` , `tcpCount` , `load1` etc.). Um bucket fora da lista `2` , `30` , `60` , `120` , `180` , `300` , `720` , `1440` , `2880` será rejeitado.

- **Métricas do Xray** — bloco separado com o consumo de memória e a coleta de lixo do Xray (séries `xrAlloc` , `xrSys` , `xrHeapObjects` , `xrNumGC` , `xrPauseNs`) e o «Observatory» (estado das conexões outbound). Funcionam apenas se o bloco `metrics` estiver configurado na configuração do Xray (`listen 127.0.0.1:11111` , `tag metrics_out`); caso contrário é exibido «Endpoint de métricas do Xray não configurado».

Exemplo de bloco que habilita o tile de métricas do Xray. As seções `metrics` (com tag) e o inbound que escuta essa tag devem estar presentes simultaneamente nas configurações do Xray:

```
{
  "metrics": {
    "tag": "metrics_out"
  },
  "inbounds": [
    {
      "listen": "127.0.0.1",
      "port": 11111,
      "protocol": "dokodemo-door",
      "settings": { "address": "127.0.0.1" },
      "tag": "metrics_out"
    }
  ]
}
```

O endereço `127.0.0.1:11111` é intencionalmente não exposto externamente — o painel o consulta localmente.

- **Alternador de tema escuro.** Localizado no menu geral/cabeçalho, e não no dashboard em si. Opções: «Tema» (*Theme*) com as variantes «Escuro» e «Ultra escuro» (*Ultra Dark*). É uma configuração puramente visual, sem efeito no funcionamento do painel.
- **Outros links** no entorno do dashboard (no menu/rodapé): «Logs», «Configuração» — visualização do JSON final do Xray (`GET /getConfigJson`), «Documentação».

4. Inbounds: criação e parâmetros gerais

A seção «**Entradas**» (ing. *Inbounds*) é a lista de todos os pontos de entrada do Xray pelos quais os clientes se conectam. Cada inbound armazena tanto campos do painel (observação, limite de tráfego, agendamento de redefinição) quanto blocos JSON brutos da configuração do Xray (`settings` , `streamSettings` , `sniffing`).

A criação é feita pelo botão «**Criar conexão**» (*Add Inbound*), e a edição pelo «**Modificar conexão**» (*Modify Inbound*). Ambas as operações são enviadas para os endpoints de API `POST /add` e `POST /update/:id`.

A seguir, são descritos todos os campos do formulário que **não** se referem às configurações de um protocolo específico (clientes, criptografia, REALITY/TLS) e **não** se referem ao transporte/fluxo (abas «**Fluxo**», «**Segurança**») — esses são temas de seções separadas.

4.1. Campos gerais do formulário

Remark (Observação)

Parâmetro	Valor
Campo	<code>remark</code>
Tipo	<code>string</code>
Padrão	vazio

Nome legível por humanos do inbound, exibido na lista e nos cabeçalhos dos diálogos («Excluir conexão "{remark}"?» etc.). O rótulo do campo é «**Observação**». Não afeta o funcionamento do Xray, serve apenas para facilitar a administração; recomenda-se definir nomes únicos e significativos, pois eles são inseridos nos nomes dos arquivos exportados e nas confirmações de operações em massa.

Protocol (Protocolo)

Parâmetro	Valor
Campo	<code>protocol</code>
Rótulo	« Protocolo »
Validação	<code>required,oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun</code>

Lista suspensa do protocolo do inbound. Valores permitidos:

Valor	Observação
<code>vmess</code>	
<code>vless</code>	
<code>trojan</code>	

Valor	Observação
shadowsocks	
wireguard	
hysteria	Hysteria v2 – é hysteria com streamSettings.version = 2 ; não existe um protocolo separado
http	
mixed	socks/http na mesma porta
tunnel	
tun	aceito pelo validador; não existe uma constante de protocolo separada

O campo é obrigatório (required). A escolha do protocolo determina quais campos de configuração de clientes e qual transporte estarão disponíveis (consulte as seções específicas de cada protocolo).

Importante: ao salvar, o serviço normaliza o streamSettings . As configurações de transporte são mantidas apenas para os protocolos vmess , vless , trojan , shadowsocks , hysteria ; para os demais (http , mixed , tunnel , wireguard , tun), o campo streamSettings é **forçosamente limpo**.

Para inbounds do tipo tunnel /TPProxy cujo bloco streamSettings não contém a chave security (variante sem transporte), o formulário abre e salva sem o erro de validação streamSettings.security Invalid input .

Listen IP (IP de escuta)

Parâmetro	Valor
Campo	listen
Tipo	string
Padrão	vazio → o Xray escuta em 0.0.0.0 (todos os IPs)

Endereço IP no qual o inbound aceita conexões. Dica do campo:

«Deixe vazio para escutar em todos os endereços IP».

Ao gerar a configuração do Xray, o valor vazio é substituído por 0.0.0.0 . Além de um IP, o campo também aceita um **caminho de socket Unix** – dica:

«Você também pode especificar o caminho de um socket Unix (por exemplo, /run/xray/in.sock) ou o nome de um socket abstrato com o prefixo @ (por exemplo, @xray/in.sock) para escutar em um socket em vez de uma porta TCP – nesse caso, defina a porta como 0».

Assim, o campo aceita duas formas de socket Unix: caminho no sistema de arquivos (`/run/xray/in.sock`) e nome de socket abstrato com prefixo `@` (`@xray/in.sock`). Em ambos os casos, defina `Port` como `0` .

Este campo é alterado quando é necessário restringir o inbound a uma única interface (por exemplo, `127.0.0.1` para um inbound que funciona apenas como destino de fallback atrás do Nginx) ou quando o inbound escuta em um socket Unix.

Exemplo. Inbound que escuta apenas na interface local (destino de fallback típico atrás do Nginx) e socket Unix:

```
listen = 127.0.0.1  porta = 8443
listen = /run/xray/in.sock  porta = 0
```

Port (Porta)

Parâmetro	Valor
Campo	<code>port</code>
Rótulo	«Porta»
Validação	<code>gte=0,lte=65535</code>
Padrão	— (definido pelo usuário)

Porta TCP/UDP de escuta. Valores permitidos de `0` a `65535` . O valor `0` é usado apenas em combinação com a escuta em socket Unix (veja acima).

Ao salvar, o serviço verifica conflito de porta: dois inbounds não podem ocupar simultaneamente o mesmo `listen:port` para o mesmo transporte (TCP/UDP). O transporte é determinado a partir do protocolo e do `streamSettings / settings` : por exemplo, `hysteria` e `wireguard` sempre ocupam UDP, `kcp / quic` — UDP, e a maioria dos demais — TCP. Em caso de conflito, o salvamento é rejeitado com um erro.

Além disso, o painel não permite ocupar a **porta reservada da API interna do Xray** (tag `api` , padrão `62789` em `127.0.0.1`): um inbound TCP local cujo endereço de escuta coincida com essa porta no loopback é rejeitado com o mesmo erro de conflito de porta. A porta real da API é lida a partir do modelo de configuração do Xray (com valor de fallback `62789`). Em nós (nodes), essa restrição não se aplica — eles têm seu próprio Xray.

A tag Xray (Tag , única) é gerada automaticamente a partir da porta e do transporte no formato `in-<porta>-<tcp|udp|tcpudp|any>` ; para um inbound implantado em um nó, é adicionado o prefixo `n<nodeId>-` . Em caso de colisão, são acrescentados `-2` , `-3` etc. ao final da tag. Normalmente o usuário não edita a tag.

Total traffic (Total de tráfego, GB)

Parâmetro	Valor
Campo	<code>total</code> (em bytes)

Parâmetro	Valor
Rótulo	«Consumo total»
Padrão	0

Limite total de tráfego do inbound. No formulário, o valor é inserido em gigabytes; no banco de dados, é armazenado em bytes. Dica do campo:

«= Sem limite. (unidade: GB)».

Ou seja, **0 significa sem limite**. Este é o limite no nível do inbound inteiro (e não de clientes individuais); o tráfego realmente consumido é armazenado nos campos `up` (enviado) e `down` (recebido) e comparado com `total`.

Expiry date / Duration (Data de expiração / prazo)

Parâmetro	Valor
Campo	<code>expiryTime</code> (timestamp Unix)
Rótulo	«Data de expiração» (ing. <i>Duration</i>)
Padrão	vazio / 0

Período de validade do inbound. Dica:

«Deixe vazio para que seja ilimitado».

O valor vazio (0) significa um inbound sem prazo de expiração. O valor é armazenado como timestamp Unix; o formulário permite definir tanto uma data específica quanto um prazo em dias (contagem relativa a partir do momento atual – rótulo em inglês *Duration*).

Enabled (Ativar)

Parâmetro	Valor
Campo	<code>enable</code>
Rótulo	«Ativar» (ing. <i>Enabled</i>)
Padrão	definido na criação

Indicador de atividade do inbound. A alternância desse sinalizador na lista é tratada por um endpoint «leve» separado `POST /setEnable/:id`, e não por uma atualização completa – isso foi feito propositalmente para evitar a re-serialização de todo o bloco `settings` (de todos os clientes) a cada clique no alternador de um inbound com milhares de clientes. Ao desativar, o inbound é removido do Xray em execução; ao ativar, é adicionado de volta.

Node / Deploy to (Nó / Implantar em)

Parâmetro	Valor
Campo	nodeId
Rótulo	«Implantar em», «Painel local»
Padrão	vazio (painel local)

Seleção de onde o inbound opera fisicamente: no painel local ou em um dos nós registrados. Detalhe de implementação: `nodeId = 0` é normalizado para `nil`, pois `0` não é um id de nó válido, mas sim um artefato do binding do formulário; `nil / 0` significa painel local. Ao salvar um inbound em um nó offline, pode aparecer um toast «a alteração será sincronizada quando o nó se reconectar».

Estratégia de endereço para links (Share address strategy)

Parâmetro	Valor
Campo	estratégia + (opcionalmente) endereço personalizado
Rótulo	«Estratégia de endereço para links» (ing. <i>Share address strategy</i>)
Padrão	«Endereço de escuta do inbound» (<i>Inbound listen</i>)

A lista suspensa determina qual endereço é inserido nos **links de compartilhamento e QR codes exportados** deste inbound. Valores:

Valor	Rótulo	O que é inserido
node	«Endereço do nó» (<i>Node address</i>)	endereço do nó no qual o inbound opera
listen	«Endereço de escuta do inbound» (<i>Inbound listen</i>)	endereço de escuta do próprio inbound
custom	«Personalizado» (<i>Custom</i>)	endereço próprio do campo «Endereço de compartilhamento personalizado» (<i>Custom share address</i>)

Ao selecionar «Personalizado», aparece o campo «Endereço de compartilhamento personalizado»; nele é inserido um host ou IP **sem esquema e sem porta** (o valor é validado). A opção «Endereço do nó» é exibida na lista apenas se existir um nó ativo no qual este inbound possa operar; caso contrário, fica oculta e o valor é revertido para «Endereço de escuta do inbound».

Essa estratégia afeta **apenas** os links de compartilhamento diretos e os QR codes. Ela **não** afeta a geração de assinaturas — lá o endereço ainda é determinado pela lógica habitual do painel.

4.2. Sniffing (Sniffing)

A aba «Sniffing» edita o bloco `sniffing` da configuração do Xray, que é armazenado como JSON bruto. O Sniffing permite que o Xray «inspecione» o nome de domínio/protocolo real dentro de uma conexão para fins de roteamento.

Subcampo	Rótulo	Finalidade
<code>enabled</code>	(alternador da aba)	Ativa/desativa o sniffing para o inbound
<code>destOverride</code>	–	Lista de protocolos para os quais o endereço de destino é interceptado: <code>http</code> , <code>tls</code> , <code>quic</code> , <code>fakedns</code>
<code>metadataOnly</code>	«Somente metadados»	Usar apenas metadados da conexão, sem leitura do payload
<code>routeOnly</code>	«Somente roteamento»	Aplicar o resultado do sniffing apenas para roteamento, sem reescrever o endereço de destino
<code>domainsExcluded</code>	«Domínios excluídos»	Domínios excluídos do sniffing
(IPs excluídos)	«IPs excluídos»	Endereços IP excluídos do sniffing

- **`destOverride`** – conjunto de sniffers: `http` (determina o domínio a partir do cabeçalho HTTP Host), `tls` (a partir do SNI), `quic` (a partir do QUIC ClientHello), `fakedns` (correspondência com o pool FakeDNS). Normalmente, `http` e `tls` são ativados para determinar o domínio.

Exemplo do bloco `sniffing` (determinar domínio por HTTP e TLS, usar o resultado apenas para roteamento, sem tocar na rede local):

```
{
  "enabled": true,
  "destOverride": ["http", "tls"],
  "routeOnly": true,
  "domainsExcluded": ["courier.push.apple.com"]
}
```

- **`metadataOnly`** – quando ativado, o Xray não lê o conteúdo do primeiro pacote e se baseia apenas nos metadados; útil para não interferir em protocolos cujos dados não devem ser «inspecionados».
- **`routeOnly`** – o resultado do sniffing é usado apenas pelas regras de roteamento; o endereço da conexão no outbound não é reescrito para o domínio identificado.

Observação: o painel armazena o `sniffing` como um bloco JSON opaco e não acrescenta nada a ele ao salvar – todos os valores padrão para essas caixas de seleção são formados no lado do aplicativo cliente. Em sua forma bruta, o bloco pode ser editado pela seção «JSON da entrada» (veja abaixo).

4.3. Allocate (estratégia de alocação de portas)

O bloco `allocate` em `streamSettings` controla como o Xray distribui as portas de escuta. Faz parte da configuração do Xray; o painel o armazena e transmite como parte do `streamSettings` /JSON do inbound. Parâmetros (conforme a terminologia do Xray-core):

Subcampo	Finalidade	Valores / padrão
<code>strategy</code>	Estratégia de alocação de portas	<code>always</code> – sempre escutar na porta definida (padrão); <code>random</code> – alterar periodicamente as portas de escuta dentro de um intervalo
<code>refresh</code>	Intervalo de troca de portas (minutos) com <code>random</code>	número inteiro de minutos (recomendado 5; mínimo 2)
<code>concurrency</code>	Quantas portas manter abertas simultaneamente com <code>random</code>	inteiro (padrão 3; no máximo um terço da largura do intervalo de portas)

`strategy: always` mantém o inbound em uma porta fixa (modo padrão). `strategy: random` é usado em cenários anti-bloqueio, quando o inbound «salta» periodicamente entre as portas de um intervalo; nesse caso, `refresh` e `concurrency` fazem sentido. Esses valores devem ser alterados apenas ao usar conscientemente o modo de portas aleatórias.

Exemplo do bloco `allocate` em `streamSettings` (modo de portas aleatórias: manter 3 portas abertas, trocar a cada 5 minutos):

```
{
  "allocate": {
    "strategy": "random",
    "refresh": 5,
    "concurrency": 3
  }
}
```

Para que isso funcione, a `port` do inbound deve ser definida como um intervalo (por exemplo, `20000-20100`).

4.4. External Proxy (Proxy externo)

O campo «**External Proxy**» pertence às configurações de geração de links de convite e é armazenado em `streamSettings` do inbound. Ele define uma lista de endereços externos alternativos (host/porta, opcionalmente com TLS forçado – «**TLS forçado**») que são inseridos nos links dos clientes em vez do `listen:port` real do inbound.

É usado quando os clientes devem se conectar não diretamente ao servidor, mas por meio de um proxy externo/reverso/CDN: nesse caso, os links compartilhados especificam o endereço público desse frontend. Isso não afeta o processo de recepção de conexões pelo Xray – é apenas «cosmético» nos links gerados. Campos relacionados no formulário: «**TLS forçado**», «**Fingerprint**», rótulos de cada registro.

4.5. Fallbacks (Fallbacks)

A seção «**Fallbacks**» define as regras de redirecionamento para conexões que não correspondem a nenhum cliente do inbound. Está disponível para o inbound master com transporte TLS (VLESS/Trojan TCP-TLS). É gerenciado pelos endpoints `GET /:id/fallbacks` / `POST /:id/fallbacks`.

Dica da seção:

«Quando uma conexão neste inbound não corresponde a nenhum cliente, ela é redirecionada para outro destino. Selecione um inbound filho abaixo para que os campos de roteamento (SNI / ALPN / Path / xver) sejam preenchidos automaticamente a partir do seu transporte, ou deixe a seleção vazia e defina Dest diretamente (por exemplo, 8080 ou 127.0.0.1:8080) para redirecionar para um servidor externo, como o Nginx. Cada inbound filho deve escutar em 127.0.0.1 com security=none».

A seção de fallbacks é exibida apenas para inbounds VLESS/Trojan sobre RAW (TCP) com segurança TLS ou REALITY. Um novo inbound começa com `security=none`, portanto a seção pode parecer ausente no início. Nesse estado (VLESS/Trojan, RAW/TCP, segurança ainda não configurada), em vez da seção é exibida uma dica integrada: os fallbacks estarão disponíveis após selecionar TLS ou Reality na aba «Segurança».

Campos de uma linha de fallback

Campo	Padrão	Descrição
(inbound filho)	—	Seleção do inbound filho (rótulo « Selecionar inbound »). Se selecionado, os campos Name/Alpn/Path/Dest podem ser preenchidos automaticamente a partir do seu transporte
Name	vazio (= qualquer)	Condição de correspondência por nome (SNI/nome). Rótulo «qualquer» — « qualquer »
Alpn	vazio	Condição de correspondência por ALPN
Path	vazio	Condição de correspondência por caminho (para transportes WS/HTTP do inbound filho)
Dest	automático	Para onde redirecionar. Placeholder « automático (listen:porta do filho) ». Pode ser uma porta (8080) ou <code>host:port</code> (127.0.0.1:8080)
Xver	0	Versão do protocolo PROXY (« Xver »): 0 — desativado, 1 ou 2 — versão correspondente do PROXY protocol
(ordem)	por posição	Ordem de aplicação das regras; definida pelos botões « Acima »/« Abaixo »

Lógica de salvamento: toda a lista de fallbacks do master é substituída atômica. Uma linha que não tem nem um inbound filho selecionado (`childId <= 0`) nem um Dest definido é **ignorada**. Se o inbound filho selecionado for igual ao id do próprio master, ele é zerado. Na geração do JSON final: se Dest estiver vazio, ele é calculado a partir do inbound filho como `listen:port`, onde `0.0.0.0 / :: / ::0` são substituídos por `127.0.0.1`; campos vazios `name / alpn / path` não são incluídos no JSON de saída; `xver` é adicionado apenas se for maior que 0.

Exemplo do settings.fallbacks final (tráfego com `alpn=h2` vai para o destino WS pelo caminho `/ws`, todo o restante vai para o Nginx local na porta 8080):

```
{
  "fallbacks": [
    { "alpn": "h2", "path": "/ws", "dest": "127.0.0.1:2001", "xver": 1 },
    { "dest": 8080 }
  ]
}
```

A última linha sem `name / alpn / path` é a regra «padrão», que captura todo o restante.

Botões e dicas da seção fallbacks

- «Adicionar fallback» – adiciona uma linha; «Sem fallbacks ainda» – estado vazio.
- «Adicionar rapidamente todos os adequados» / «Adicionar todos» – adiciona uma linha de fallback para cada inbound adequado ainda não conectado. Resultado: «Adicionado(s) {n} fallback(s)» ou «Nenhum inbound adequado novo».
- «Preencher a partir do filho» – reaplicar os campos de roteamento (SNI/ALPN/Path/xver) a partir do transporte do inbound filho selecionado; após a execução – «Preenchido a partir do filho».
- «Modificar campos de roteamento» / «Ocultar avançados» – mostrar/ocultar campos detalhados da linha.
- Os rótulos «Roteia quando» e «Padrão – captura todo o restante» explicam a condição de ativação de cada linha.

Após salvar os fallbacks, o servidor chama a reinicialização do Xray para que os novos `settings.fallbacks` entrem em vigor.

4.6. Redefinição periódica de tráfego

O bloco «Redefinição de tráfego» configura a redefinição automática dos contadores de tráfego do inbound por agendamento. Descrição:

«Redefinição automática do contador de tráfego nos intervalos especificados».

Parâmetro	Valor
Campo	<code>trafficReset</code>
Validação	<code>omitempty,oneof=never hourly daily weekly monthly</code>
Padrão	<code>never</code>
Campo associado	<code>lastTrafficResetTime</code> – timestamp da última redefinição (rótulo «Última redefinição»)

Lista suspensa:

Valor	Rótulo
<code>never</code>	«Nunca»
<code>hourly</code>	«A cada hora»
<code>daily</code>	«Diariamente»
<code>weekly</code>	«Semanalmente»
<code>monthly</code>	«Mensalmente»

Para cada período, está registrado um cron job que é executado no agendamento correspondente (`@hourly` , `@daily` , `@weekly` , `@monthly`). O job seleciona todos os inbounds com o `trafficReset` definido e, para cada um, redefine os contadores do próprio inbound (`up=0` , `down=0`) e o tráfego de todos os seus clientes. Ou seja, a redefinição periódica afeta tanto o inbound quanto seus clientes.

Exemplo do valor do campo. Para que os contadores sejam zerados no primeiro dia de cada mês, seleciona-se «**Mensalmente**» no formulário, o que é salvo como:

```
{ "trafficReset": "monthly" }
```

O valor `never` (padrão) desativa completamente a redefinição automática.

4.7. JSON da entrada (avançado)

A seção «**Seções JSON da entrada**» fornece acesso direto aos blocos JSON brutos do inbound. Descrição:

«JSON completo da entrada e editores separados para settings, sniffing e streamSettings».

Editores disponíveis:

Aba	Rótulo	O que edita
Tudo	«Objeto completo da entrada com todos os campos em um único editor»	o objeto Inbound inteiro
Configurações	«Wrapper do bloco settings do Xray»	campo <code>settings</code>
Sniffing	«Wrapper do bloco sniffing do Xray»	campo <code>sniffing</code>
Stream	«Wrapper do bloco stream do Xray»	campo <code>streamSettings</code>

Esses campos são serializados como objetos JSON aninhados: blocos vazios são retornados como `null`, e um texto que não é JSON válido é encapsulado em uma string para que os dados não sejam perdidos. Erros de parse ao salvar são exibidos com o prefixo «**JSON avançado**».

A janela de visualização «JSON da entrada», assim como a janela de importação de inbound, usa um editor de código completo com realce de sintaxe JSON (em vez de um campo de texto comum): a visualização de configuração – no modo somente leitura com realce, e a importação – no modo editável, o que facilita a leitura e edição.

4.8. Ações com inbound: QR / Edit / Reset / Delete e estatísticas

Na lista e no cartão do inbound estão disponíveis as seguintes ações (menu «**Menu**»):

Estatísticas de tráfego

É exibido o tráfego agregado do inbound: «**Enviado/recebido**» (campos `up / down`), «**Total de tráfego**», «**Total de conexões**». No cartão também – «**Criado**», «**Atualizado**», «**Data de expiração**».

Na lista de inbounds há uma coluna **Speed** com a velocidade atual do tráfego por inbound (upload/download), calculada a partir dos incrementos dos contadores entre pesquisas; a mesma velocidade ao vivo é exibida na janela de estatísticas do inbound. Quando a próxima pesquisa não produz incremento, o valor da velocidade é zerado.

No resumo de clientes na página de inbounds, o status é determinado pela prioridade «esgotado/encerrado»: clientes cujo prazo expirou ou cujo tráfego foi esgotado (e nos quais a tarefa automática removeu o `enable`) pertencem ao status **«Esgotado/Encerrado»** (*Depleted/Ended*), e não ao cinza **«Desativado»** (*Disabled*), e não são contados duas vezes. A classificação coincide com a exibida no cartão do próprio cliente e contabiliza corretamente os clientes vinculados a múltiplos inbounds.

QR code e cópia de links

- **«Detalhes»** — expande os links de conexão e de assinatura.
- QR code do cliente: dica **«Clique no QR code para copiar»**.
- **«Copiar link»** (ing. *Copy URL*), **«Exportar links»**.

Edit (Modificar)

«Modificar conexão» — abre o formulário de edição (`POST /update/:id`). Ao atualizar, o serviço relê a linha existente, transfere os campos alterados, regenera a tag se necessário (caso a tag anterior tenha sido gerada automaticamente) e sincroniza o runtime do Xray. Sucesso — toast **«Conexão atualizada com sucesso»**.

Reset Traffic (Redefinir tráfego)

«Redefinir tráfego» — zera os contadores `up / down` especificamente deste inbound (`POST /:id/resetTraffic` , define `up=0` , `down=0`). Confirmação:

«Redefinir o tráfego de "{remark}"?» / «Redefine os contadores de envio/recebimento desta conexão para 0».

A redefinição de tráfego do inbound **não** afeta os contadores de seus clientes (para eles existem ações separadas «Redefinir tráfego dos clientes»). Após a redefinição, é iniciada a reinicialização do Xray. Sucesso — toast **«Tráfego de entrada redefinido»**. Existe também a variante em massa — **«Redefinir tráfego de todas as conexões»** (`POST /resetAllTraffics`).

Delete (Excluir)

«Excluir conexão» (`POST /del/:id`). Confirmação:

«Excluir conexão "{remark}"?» / «A conexão e todos os seus clientes serão excluídos. Esta ação não pode ser desfeita».

A exclusão remove o inbound do Xray em execução (com reinicialização se necessário). Sucesso — toast **«Conexão excluída com sucesso»**. Exclusão em massa — `POST /bulkDel` , com relatório por elemento e no máximo uma reinicialização do Xray.

Outras ações com clientes do inbound

No menu também estão disponíveis: **«Clonar»** (cópia do inbound com nova porta e lista de clientes vazia), **«Excluir todos os clientes»** (`POST /:id/delAllClients` – exclui todos os clientes, o próprio inbound é mantido), **«Excluir clientes desativados»**, **«Vincular/Desvincular clientes»**, **«Importar»/«Exportar conexões»** (`POST /import`). Os detalhes das operações com clientes pertencem à seção sobre clientes.

5. Protocolos

Ao criar uma conexão de entrada (inbound), o primeiro item a ser escolhido é o **Protocolo** («Protocol»). O protocolo determina qual método de autenticação e criptografia de tráfego o Xray-core aplicará a este inbound, quais campos em `settings` precisarão ser preenchidos, além de quais transportes (`network`) e tipos de segurança (TLS / REALITY) estarão disponíveis.

O campo de protocolo é definido uma única vez na criação do inbound e **não pode ser alterado na edição** (o menu suspenso fica bloqueado no formulário de edição). Para mudar o protocolo, é necessário criar um novo inbound.

5.1. Lista de protocolos suportados

O servidor aceita o seguinte conjunto de valores para o campo `Protocol`:

```
oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun mtproto
```

A partir da versão **3.3.0**, o valor `mtproto` (proxy do Telegram) foi adicionado à lista.

Valor no config	Finalidade	Modelo de cliente
<code>vless</code>	Principal protocolo proxy (padrão na criação de inbound)	Clientes com UUID, suporte a flow e criptografia pós-quântica
<code>vmess</code>	Protocolo proxy clássico do Xray	Clientes com UUID e parâmetro <code>security</code>
<code>trojan</code>	Proxy que se disfarça de HTTPS comum	Clientes com senha
<code>shadowsocks</code>	Proxy Shadowsocks (incluindo SIP022 / 2022-blake3)	Um ou vários usuários (2022)
<code>wireguard</code>	Inbound WireGuard	Peers (não clientes)
<code>hysteria</code>	Inbound Hysteria (padrão versão 2)	Clientes com token <code>auth</code>
<code>http</code>	Proxy HTTP clássico (forward proxy)	Contas user/pass, sem contabilização de tráfego
<code>mixed</code>	Proxy SOCKS + HTTP combinados	Contas user/pass
<code>tunnel</code>	Encaminhador transparente (xray <code>dokodemo-door</code>)	Sem clientes
<code>tun</code>	Interface TUN (apenas renderização de existentes)	Sem clientes
<code>mtproto</code>	Proxy do Telegram (MTPROTO), adicionado em 3.3.0; gerenciado por um processo separado <code>mtg</code> , não pelo Xray	Sem clientes (acesso por segredo)

Observação sobre `tun` : o valor foi mantido na lista para compatibilidade e **exibição** de inbounds salvos anteriormente, mas na versão atual o backend não recomenda sua criação – o suporte foi considerado obsoleto. Criar novos inbounds desse tipo não faz sentido.

Observação sobre Hysteria 2: não existe um protocolo separado «hysteria2». Trata-se do protocolo `hysteria` com o campo `streamSettings.version = 2`. O esquema de link `hysteria2://` é escolhido automaticamente na geração de links de compartilhamento quando a versão do stream é 2.

Nem todos os protocolos suportam distribuição por nós (nodes). Apenas os seguintes podem ser implantados em nós: `vless`, `vmess`, `trojan`, `shadowsocks`, `hysteria`, `wireguard`. Os protocolos `http`, `mixed`, `tunnel`, `tun`, `mtproto` funcionam apenas no painel local.

5.2. Quais protocolos suportam TLS / REALITY / transporte

A possibilidade de habilitar determinada camada de segurança e transporte depende do protocolo e da rede selecionada (`streamSettings.network`):

Funcionalidade	Disponível para protocolos	Redes permitidas (<code>network</code>)
TLS	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> (e sempre para <code>hysteria</code>)	<code>tcp</code> , <code>ws</code> , <code>http</code> , <code>grpc</code> , <code>httpupgrade</code> , <code>xhttp</code>
REALITY	<code>vless</code> , <code>trojan</code>	<code>tcp</code> , <code>http</code> , <code>grpc</code> , <code>xhttp</code>
flow (<code>xtls-rprx-vision</code>)	somente <code>vless</code>	somente <code>tcp</code> , com <code>security = tls</code> ou <code>reality</code>
Stream / transporte (aba «Fluxo»)	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> , <code>hysteria</code>	–

Para os protocolos `http`, `mixed`, `tunnel`, `tun`, `wireguard` a aba de transporte não está disponível – eles não possuem configurações de stream do Xray.

5.3. VLESS

Finalidade: principal protocolo proxy moderno. Suporta XTLS-Vision (`flow`), REALITY, bem como criptografia pós-quântica no próprio nível do VLESS (campos `decryption` / `encryption`). Usado por padrão para novos inbounds.

Campos do bloco `settings` :

Campo	Valor padrão	Descrição
<code>clients</code>	<code>[]</code>	Lista de clientes. Cada um possui: <code>id</code> (UUID), <code>email</code> (obrigatório), <code>flow</code> , limites (<code>limitIp</code> , <code>totalGB</code> , <code>expiryTime</code>), <code>enable</code> , <code>tgId</code> , <code>subId</code> , <code>comment</code> , <code>reset</code>
<code>decryption</code>	<code>none</code>	Parâmetro de descryptografia no lado do servidor. Rótulo na interface: «Descryptografia» (ingl. «Decryption»)
<code>encryption</code>	<code>none</code>	Parâmetro de criptografia correspondente (incluído no link do cliente). Rótulo: «Criptografia» (ingl. «Encryption»)

Campo	Valor padrão	Descrição
<code>fallbacks</code>	<code>[]</code>	Lista de fallbacks (veja a seção sobre fallbacks); disponível quando <code>network = tcp</code> e <code>security = TLS</code> ou <code>REALITY</code>
<code>testseed</code>	(4 números: 900, 500, 900, 256)	«Vision testseed» – 4 inteiros positivos para o padding do XTLS-Vision. Aplicado apenas a clientes com flow <code>xtls-rprx-vision</code> , ignorado nos demais

flow (`xtls-rprx-vision`)

`flow` é definido **no cliente**, não no inbound, e aceita um dos três valores:

Valor	Significado
<code>``</code> (vazio)	Sem XTLS-flow (padrão)
<code>xtls-rprx-vision</code>	XTLS-Vision – modo recomendado para VLESS sobre TCP+TLS/REALITY
<code>xtls-rprx-vision-udp443</code>	O mesmo Vision, mas com processamento de UDP/443 (QUIC)

O campo `flow` fica disponível para seleção somente quando todas as condições são atendidas: protocolo `vless`, `network = tcp` e `security = tls` ou `reality`. O campo **Vision testseed** no formulário é exibido apenas nas mesmas condições.

Exceção para XHTTP: no caso de VLESS sobre `network = xhttp` com autenticação pós-quântica VLESS habilitada (`encryption / decryption, vlessenc`), o flow `xtls-rprx-vision` também é permitido – independentemente da camada de segurança, inclusive com `REALITY`. Nesse caso, o painel transmite corretamente `xtls-rprx-vision` nos links de compartilhamento e assinaturas (incluindo o formato Clash/Mihomo), de modo que o cliente receba a configuração exatamente com o Vision.

Descriptografia / Criptografia (autenticação pós-quântica VLESS)

Os campos `decryption` e `encryption` representam autenticação no nível do próprio VLESS (separada do TLS/REALITY de transporte). Por padrão, ambos são `none`. No formulário há três botões ao lado:

- **Autenticação X25519** (ingl. «X25519 auth») – gera um par `decryption / encryption` baseado em X25519.
- **Autenticação ML-KEM-768** (ingl. «ML-KEM-768 auth») – variante pós-quântica (Post-Quantum).
- **Limpar** – redefine ambos os campos para `none`.

Abaixo dos botões é exibida a linha de status «Selecionado: {auth}», onde o valor é determinado pelo último segmento da string `encryption : vazio/ none` → «None», chave muito longa (>300 caracteres) → ML-KEM-768, curta → X25519, caso contrário «Personalizado».

Tecnicamente os botões acessam `GET /panel/api/server/getNewVlessEnc` (geração de chaves via `xray vlessenc`) e preenchem **ambos** os campos com valores pareados no formato `mlkem768x25519plus.native.<rtt>.<role>` (por exemplo, `decryption = mlkem768x25519plus.native.600s.server-x25519`, `encryption =`

`mlkem768x25519plus.native.0rtt.client-x25519`). O parâmetro `decryption` permanece no servidor; `encryption` vai para o link do cliente.

Importante: ao gerar a configuração do inbound para o Xray, o painel remove o que é desnecessário: se `encryption` (que pertence ao lado do cliente) permanecer em `settings`, ele é **removido** da configuração do servidor. No servidor fica apenas `decryption`.

Quando escolher VLESS: é a opção recomendada por padrão para um novo inbound, especialmente em combinação com REALITY (sem certificado próprio) ou com TLS + XTLS-Vision.

Exemplo: bloco `settings` de um inbound VLESS com um cliente e XTLS-Vision. O campo `flow` fica no cliente; `decryption` permanece no servidor:

```
{
  "clients": [
    {
      "id": "d342d11e-d424-4583-b36e-524ab1f0afa4",
      "email": "user1",
      "flow": "xtls-rprx-vision",
      "limitIp": 2,
      "totalGB": 0,
      "expiryTime": 0,
      "enable": true
    }
  ],
  "decryption": "none"
}
```

Para a combinação com REALITY, o bloco `streamSettings` correspondente (aba «Transport» → Security: REALITY) tem a seguinte aparência:

```
{
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "dest": "www.microsoft.com:443",
    "serverNames": ["www.microsoft.com"],
    "privateKey": "<приватный ключ X25519>",
    "shortIds": ["", "6ba85179e30d4fc2"]
  }
}
```

5.4. VMess

Finalidade: protocolo proxy clássico do Xray. Autenticação por UUID; no cliente é configurado adicionalmente o método de criptografia da carga útil (`security`).

Campos do bloco `settings` :

Campo	Valor padrão	Descrição
clients	[]	Lista de clientes

Para cada cliente VMess (além dos campos comuns `email`, `limites`, `enable`, `tgId`, `subId`, `comment`, `reset`):

Campo do cliente	Valor padrão	Descrição
id	—	UUID do cliente
security	auto	Método de criptografia da carga útil VMess. Valores permitidos: <code>aes-128-gcm</code> , <code>chacha20-poly1305</code> , <code>auto</code> , <code>none</code> , <code>zero</code>

Valores de `security`:

- `auto` — o Xray escolhe a cifra automaticamente conforme a plataforma (recomendado);
- `aes-128-gcm`, `chacha20-poly1305` — cifras AEAD fixas;
- `none` — sem criptografia da carga útil (faz sentido apenas sobre TLS);
- `zero` — sem criptografia e sem autenticação da carga útil.

Compatibilidade histórica: registros antigos podiam armazenar `security: ""` — ao ler, a string vazia é convertida para `auto`. Ao gerar a configuração do servidor, o campo `security` dos clientes VMess é **removido** de `settings`, pois não é necessário para inbound.

Quando escolher VMess: para compatibilidade com clientes antigos ou configurações existentes. Para novas implantações, geralmente VLESS é preferível.

5.5. Trojan

Finalidade: proxy que imita tráfego HTTPS comum. Autenticação por senha. Assim como o VLESS, suporta fallbacks e (com `network = tcp`) REALITY/TLS.

Campos do bloco `settings`:

Campo	Valor padrão	Descrição
clients	[]	Lista de clientes
fallbacks	[]	Lista de fallbacks (disponível com <code>network = tcp</code> e TLS/REALITY)

Para cada cliente Trojan, o campo principal é:

Campo do cliente	Valor padrão	Descrição
password	—	Senha do cliente (obrigatória, mínimo 1 caractere)
email	—	Identificador único do cliente

Os demais campos do cliente são comuns (`limitIp` , `totalGB` , `expiryTime` , `enable` , `tgId` , `subId` , `comment` , `reset`).

Quando escolher Trojan: quando é necessário disfarçar-se de HTTPS na porta 443, inclusive com fallbacks para servidor web (Nginx) para conexões não autorizadas.

Exemplo: bloco `settings` do Trojan com fallback para servidor web local. Conexões não autorizadas (sem senha válida) são encaminhadas ao Nginx que escuta em `127.0.0.1:8080` :

```
{
  "clients": [
    { "password": "S3cret-Pass-1", "email": "user1" }
  ],
  "fallbacks": [
    { "dest": "127.0.0.1:8080" }
  ]
}
```

Para o fallback são necessários `network = tcp` e `Security = TLS` ou `REALITY`; caso contrário, o campo `fallbacks` não está disponível.

5.6. Shadowsocks

Finalidade: proxy Shadowsocks leve. Suporta tanto cifras AEAD legadas quanto os novos métodos SIP022 (`2022-blake3-*`). Pode operar em modo monousuário ou multiusuário.

Campos do bloco `settings` :

Campo	Valor padrão	Descrição
<code>method</code>	<code>2022-blake3-aes-256-gcm</code>	Método de criptografia do inbound. Rótulo na interface: «Método de criptografia» (ingl. «Encryption method»)
<code>password</code>	``	Senha do inbound (para métodos 2022, gerada automaticamente de acordo com o método selecionado)
<code>network</code>	<code>tcp,udp</code>	Transporte. Rótulo: «Rede» (ingl. «Network»). Opções: <code>tcp,udp</code> (TCP,UDP), <code>tcp</code> , <code>udp</code>
<code>clients</code>	<code>[]</code>	Lista de clientes
<code>ivCheck</code>	<code>false</code> (desligado)	Alternância «ivCheck» — proteção contra reutilização de IV

Métodos de criptografia (`method`)

Conjunto permitido:

Método	Categoria
<code>aes-256-gcm</code>	AEAD legado
<code>chacha20-poly1305</code>	AEAD legado

Método	Categoria
chacha20-ietf-poly1305	AEAD legado
xchacha20-ietf-poly1305	AEAD legado
2022-blake3-aes-128-gcm	SS 2022 (recomendado)
2022-blake3-aes-256-gcm	SS 2022 (padrão)
2022-blake3-chacha20-poly1305	SS 2022, monousuário

Lógica do painel por método:

- **Métodos 2022** (2022-blake3-*) são considerados «SS 2022». O método 2022-blake3-chacha20-poly1305 é **monousuário** (multi-usuário não suportado); os demais métodos 2022 permitem vários clientes. O campo de senha (com botão de geração que ajusta o comprimento da chave ao método) é exibido no formulário especificamente para métodos 2022.
- **Cifras legadas** (aes-* , chacha20-*) funcionam pelo esquema clássico «um método + uma senha».

Normalização antes de iniciar o Xray: para cifras legadas, cada cliente deve carregar `method` igual ao método do inbound (caso contrário o Xray falha com «unsupported cipher method:»). Para métodos 2022, ao contrário – o campo `method` do cliente deve estar **vazio** (caso contrário o Xray rejeita o inbound com «users must have empty method»). O painel organiza os dados automaticamente ao trocar o método.

Regeneração de chaves de clientes ao mudar o tamanho da chave: para Shadowsocks-2022, ao trocar o método de criptografia por um método com tamanho de chave diferente (por exemplo entre 2022-blake3-aes-256-gcm e 2022-blake3-aes-128-gcm), o painel regenera automaticamente os PSKs dos clientes para o novo comprimento ao salvar o inbound. Caso contrário, as chaves antigas permaneceriam no tamanho anterior e o Xray as rejeitaria. Consequência: os clientes afetados precisam obter a assinatura novamente – os links anteriores deixarão de funcionar.

Quando escolher Shadowsocks: para implantações simples sem mascaramento TLS; a escolha moderna são os métodos 2022-blake3.

Exemplo: bloco settings do Shadowsocks para método 2022-blake3 (modo multiusuário). O inbound tem sua própria senha (chave base64 do comprimento correto); cada cliente tem sua própria senha; o campo `method` do cliente está **vazio**:

```
{
  "method": "2022-blake3-aes-256-gcm",
  "password": "d2hhdGV2ZXItMzItYnl0ZS1iYXNlNjQta2V5LWhlcU=",
  "network": "tcp,udp",
  "clients": [
    {
      "email": "user1",
      "password": "Y2xpZW50LWtleS0zMilieXRlcy1iYXNlNjQtaGVyZQ==",
      "method": ""
    }
  ]
}
```

Para cifras legadas (`aes-256-gcm` etc.) – ao contrário: uma senha para o inbound, e o `method` do cliente deve coincidir com o método do inbound.

5.7. Dokodemo-door / Tunnel (encaminhador transparente)

Finalidade: encaminhador transparente (no painel – protocolo `tunnel`, que implementa o comportamento do `dokodemo-door`). Aceita tráfego e o redireciona para um endereço/porta especificado, sem autenticação e sem clientes.

Campos do bloco `settings` :

Campo	Valor padrão	Descrição
<code>rewriteAddress</code>	(nenhum)	«Reescrever endereço» (ingl. «Rewrite address») – endereço de destino para onde o tráfego é redirecionado
<code>rewritePort</code>	(nenhum)	«Reescrever porta» (ingl. «Rewrite port») – porta de destino (0-65535)
<code>allowedNetwork</code>	<code>tcp,udp</code>	«Rede permitida» (ingl. «Allowed network»). Opções: <code>tcp,udp</code> , <code>tcp</code> , <code>udp</code>
<code>portMap</code>	<code>{}</code>	«Mapeamento de portas» – mapa porta→porta (Record<string,string>)
<code>followRedirect</code>	<code>false</code> (desligado)	«Seguir redirect» (ingl. «Follow redirect») – usar o endereço de destino original da conexão interceptada

Aba «Transport» para Tunnel: inbounds deste tipo possuem a aba «**Transport**» disponível, limitada à configuração do `sockopt` – suficiente para o modo **TProxy** (proxy transparente/redirect via `sockopt.tproxy`). O menu suspenso de seleção de transporte (`network`) e a aba «Security» para Tunnel estão ocultos, pois TLS/REALITY não são suportados por este tipo.

Quando escolher: para proxy transparente/redirecionamento de portas para serviços internos.

5.8. SOCKS / HTTP (protocolo `mixed`)

Nesta compilação não existe o protocolo separado `socks` – SOCKS e proxy HTTP estão combinados no protocolo `mixed` (SOCKS + HTTP combinados). Além disso, existe um proxy `http` puro separado.

5.8.1. Mixed (SOCKS + HTTP)

Campos do bloco `settings` :

Campo	Valor padrão	Descrição
<code>auth</code>	<code>password</code>	«Auth» – modo de autenticação. Opções: <code>password</code> (por login/senha) ou <code>noauth</code> (sem autorização)
<code>accounts</code>	(opcional)	«Аккаунти» – lista de pares user/pass. Com <code>auth = noauth</code> o campo não é escrito no config
<code>udp</code>	<code>false</code> (desligado)	Alternância «UDP» – suporte a UDP via SOCKS
<code>ip</code>	<code>127.0.0.1</code>	«UDP IP» – endereço local para associações UDP. O campo é exibido apenas quando <code>udp</code> está habilitado

As contas são adicionadas pelo botão «Добавить»; ao adicionar, são gerados login aleatório (8 caracteres) e senha aleatória (12 caracteres), que podem ser editados.

5.8.2. HTTP (proxy puro)

Finalidade: proxy de encaminhamento HTTP clássico. No nível do Xray não rastreia clientes como «faturáveis» (sem email/limites) – existe apenas uma lista de contas.

Campos do bloco `settings` :

Campo	Valor padrão	Descrição
<code>accounts</code>	<code>[]</code>	«Contas» – lista de pares user/pass (ambos os campos são obrigatórios)
<code>allowTransparent</code>	<code>false</code> (desligado)	«Permitir transparente» (ingl. «Allow transparent») – encaminhar requisições com o cabeçalho Host original

Quando escolher SOCKS/HTTP: para acesso proxy local ou de serviço sem mascaramento complexo. `mixed` é conveniente porque uma única porta atende tanto clientes SOCKS quanto HTTP.

5.9. WireGuard (inbound)

Finalidade: inbound WireGuard. Ao contrário dos protocolos proxy, ele não opera com «clientes» – em vez disso são configurados **peers** (dispositivos que o servidor aceita). Transporte e TLS/REALITY não se aplicam a ele.

Campos do bloco `settings` :

Campo	Valor padrão	Descrição
<code>secretKey</code>	–	Chave privada do servidor (obrigatória). Há um botão de geração ao lado; a chave pública é exibida automaticamente (campo somente leitura)
<code>mtu</code>	(opcional)	MTU da interface

Campo	Valor padrão	Descrição
<code>noKernelTun</code>	<code>false</code> (desligado)	«TUN sem kernel» (ingl. «No-kernel TUN») – usar TUN em userspace em vez do kernel
<code>domainStrategy</code>	(opcional)	«Domain Strategy» – estratégia de resolução de domínios: <code>ForceIP</code> , <code>ForceIPv4</code> , <code>ForceIPv4v6</code> , <code>ForceIPv6</code> , <code>ForceIPv6v4</code>
<code>peers</code>	<code>[]</code>	Lista de peers

Campos de cada peer:

Campo do peer	Valor padrão	Descrição
<code>privateKey</code>	(opcional)	Chave privada do cliente – armazenada para que o painel possa renderizar a configuração para o usuário (apenas para peers de inbound)
<code>publicKey</code>	–	Chave pública do peer (obrigatória)
<code>preSharedKey</code> (PSK)	(opcional)	Chave pré-compartilhada adicional
<code>allowedIPs</code>	<code>[]</code>	IPs permitidos. Ao adicionar um novo peer, o painel sugere automaticamente o próximo endereço livre (padrão <code>10.0.0.2/32</code>)
<code>keepAlive</code>	(opcional)	«Keep-alive» – intervalo de manutenção de conexão
<code>comment</code>	(opcional)	«Comment» – rótulo arbitrário do peer; exibido ao lado do cabeçalho «Peer N» e incluído no link de compartilhamento e no <code>remark</code> do arquivo <code>.conf</code>

O botão «Adicionar peer» gera um novo par de chaves e preenche o próximo `allowedIPs` . Cada peer pode ser removido (para o único peer restante a remoção não está disponível).

O campo «Comment» do peer ajuda a distinguir dispositivos: seu texto é exibido no formulário ao lado do cabeçalho «Peer N», além de ser incluído no link de compartilhamento e no `remark` do arquivo `.conf` gerado, facilitando o reconhecimento do dispositivo no aplicativo cliente. Este campo é do painel – o xray-core ignora campos desconhecidos do peer.

Domain Strategy e aba Transport

Além dos peers, o inbound WireGuard possui o campo **Domain Strategy** (estratégia de resolução de domínios: `ForceIP` , `ForceIPv4` , `ForceIPv4v6` , `ForceIPv6` , `ForceIPv6v4`). O campo é opcional e só é escrito no `config` se definido.

O campo **Workers** (`workers` , número de threads de trabalho) foi removido dos formulários WireGuard (tanto inbound quanto outbound): a partir do xray-core v26.6.22, o mecanismo não o utiliza mais e se baseia no mecanismo interno do wireguard-go. Configurações salvas anteriormente funcionam sem alterações – ao analisar, o campo é simplesmente descartado; nenhuma migração é necessária.

Para WireGuard também está disponível a aba «**Transport**» – mas em versão reduzida: nela são configurados apenas `sockopt` e a ofuscação **Finalmask**. O menu suspenso de seleção de transporte (`network`) está oculto,

pois o WireGuard sempre escuta em UDP. Nos registros de ruído (noise), o Finalmask tem um campo separado **Rand Range** (intervalo de bytes 0–255, com validação), e como método de ofuscação para WireGuard e Hysteria está disponível o **Salamander**.

Quando escolher WireGuard: quando é necessário um túnel VPN WireGuard propriamente dito, e não um proxy com mascaramento.

5.10. Hysteria (padrão v2)

Finalidade: inbound Hysteria sobre QUIC. O painel trabalha com a versão 2 por padrão. Cada cliente é autenticado pelo token `auth` em vez de UUID/senha. TLS para Hysteria está sempre disponível (veja a tabela de funcionalidades em 5.2).

Campos do bloco `settings` :

Campo	Valor padrão	Descrição
<code>version</code>	2	Versão do protocolo (mínimo 1; padrão do painel é 2)
<code>clients</code>	<code>[]</code>	Lista de clientes

Para cada cliente, o campo principal é `auth` (token, obrigatório) mais os campos comuns (`email` , limites, `enable` , `tgId` , `subId` , `comment` , `reset`).

Parâmetros adicionais são definidos em `streamSettings.hysteriaSettings` :

Campo	Valor / opções	Descrição
<code>version</code>	fixado em 2 (campo bloqueado)	«Versão» (ingl. «Version»)
<code>udpIdleTimeout</code>	(inteiro ≥ 1, seg.)	«UDP idle timeout (s)» – tempo limite de inatividade UDP
<code>masquerade</code>	desligado por padrão	«Masquerade» – mascaramento como servidor web comum em requisições «não autorizadas»

Ao habilitar `masquerade` , está disponível a seleção do tipo (`type`):

- ```` – padrão (página 404);
- `proxy` – proxy reverso (campos «Upstream URL», «Reescrever Host», «Ignorar TLS verify»);
- `file` – servir diretório (campo «Diretório», por exemplo `/var/www/html`);
- `string` – resposta fixa (campos «Código de status», «Body», «Cabeçalhos»).

Quando escolher Hysteria: quando há necessidade de transporte QUIC e resiliência em canais instáveis/móveis; o mascaramento aumenta a discrição do ponto de entrada.

5.11. MTProto (proxy para Telegram)

Adicionado na versão **3.3.0**. Valor do protocolo – `mtproto`.

MTProto é o protocolo de proxy nativo do Telegram. No 3X-UI, este inbound **não é gerenciado pelo Xray, mas por um processo separado `mtg`**, controlado pelo próprio painel. O painel verifica periodicamente os inbounds MTProto habilitados em relação aos processos `mtg` em execução: inicia os que estão faltando, para os desnecessários e coleta contadores de tráfego das métricas do `mtg`. Por isso, a **contabilização de tráfego** deste inbound funciona como nos protocolos comuns.

Mensagem de ajuda oficial no formulário:

«MTProto é gerenciado por um processo separado `mtg`, não pelo Xray. As configurações de transporte e clientes não se aplicam aqui – compartilhe o link abaixo no Telegram.»

Consequências:

- As abas «**Transport**» (**Stream Settings**) e «**Clients**» **não se aplicam a este inbound** – o acesso é definido por um único segredo, não por uma lista de clientes.
- O inbound MTProto é iniciado **apenas no painel principal**; não é implantado nos nós filhos (nodes) (inbounds com `NodeID` definido são ignorados).
- A aba «**Sniffing**» para MTProto está oculta – este protocolo é gerenciado pelo processo `mtg`, não pelo Xray, portanto o sniffing não se aplica a ele.

Campos. Armazenados em `settings` do inbound:

Campo na interface	Chave	Descrição
Remark	<code>remark</code>	Rótulo do inbound.
Listen IP	<code>listen</code>	IP de escuta (vazio = todas as interfaces).
Port	<code>port</code>	Porta do proxy.
Segredo	<code>settings.secret</code>	Segredo de acesso no formato FakeTLS .
Domínio de disfarce (FakeTLS)	<code>settings.fakeTlsDomain</code>	Domínio cujo tráfego HTTPS o proxy imita.

Formato do segredo (FakeTLS). O painel ajusta automaticamente o segredo para o formato correto: resultado = `ee` + 32 caracteres hex + código hex do domínio de disfarce, ou seja, `ee<hex32><hex(fakeTlsDomain)>`. O prefixo `ee` ativa o modo FakeTLS, e o domínio (por exemplo, um site conhecido) serve para disfarçar o tráfego como HTTPS comum. Basta informar o domínio – o restante o painel constrói automaticamente.

Domain-fronting e opções avançadas do `mtg`

O inbound MTProto possui parâmetros adicionais do processo `mtg`. Os campos **Domain fronting IP**, **Domain fronting port** e **Domain fronting PROXY protocol** definem para onde o `mtg` envia o tráfego não-Telegram (por

exemplo, para um site NGINX falso): se o IP for deixado vazio, o domínio FakeTLS é usado via DNS, a porta padrão é 443. Adicionalmente estão disponíveis **Accept PROXY protocol** (para o listener), **IP preference** (prefer-ipv6 / prefer-ipv4 / only-ipv6 / only-ipv4) e **Debug logging**. Cada valor é gravado no arquivo `mtg-<id>.toml` apenas se estiver definido.

Roteamento do tráfego Telegram pelo Xray

O interruptor «**Route through Xray**» (desligado por padrão) e o campo opcional **Outbound** permitem subordinar o egresso do Telegram ao roteador do Xray. Quando habilitado, o painel insere na configuração do Xray uma ponte SOCKS local com a tag do próprio inbound, e o `mtg` envia o tráfego Telegram através dela. Depois disso, o tráfego pode ser correspondido por regras na aba «Routing» ou forçado para um outbound ou balanceador específico pelo campo **Outbound** (se o campo estiver vazio, as regras de roteamento decidem).

Como compartilhar com o usuário. Para inbounds MTPROTO, o painel gera um link de convite:

Exemplo: segredo FakeTLS e link pronto. Se o domínio de disfarce for `www.cloudflare.com`, o segredo é montado como `ee` + 32 caracteres hex + código hex do domínio, por exemplo:

```
secret = ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

Link de convite pronto (enviado ao usuário junto com o QR code no Telegram):

```
tg://proxy?
server=203.0.113.10&port=443&secret=ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

```
tg://proxy?server=<адрес>&port=<порт>&secret=<секрет>
```

(equivalente — `https://t.me/proxy?server=...&port=...&secret=...`). Este link e o QR code devem ser enviados ao usuário do Telegram — ao abrir, o proxy é adicionado diretamente ao aplicativo. O link também é fornecido pelo servidor de assinaturas.

Quando usar. Método padrão para contornar bloqueios do Telegram; o mascaramento FakeTLS (domínio de disfarce) faz o tráfego parecer uma visita comum ao site indicado.

5.12. Guia rápido de escolha de protocolo

- **VLESS** — escolha padrão; melhor opção com REALITY ou TLS + XTLS-Vision, suporta autenticação pós-quântica.
- **Trojan** — mascaramento como HTTPS com fallbacks para servidor web.
- **VMess** — compatibilidade com clientes antigos.
- **Shadowsocks** — proxy simples sem TLS; a escolha moderna são os métodos `2022-blake3-*`.
- **Hysteria** — QUIC, resiliência em canais ruins.
- **mixed / http** — proxies SOCKS/HTTP de serviço.
- **WireGuard** — túnel VPN completo.
- **tunnel** — redirecionamento transparente de portas.
- **MTPROTO** — proxy para contornar bloqueios do Telegram (FakeTLS); processo separado `mtg`.

6. Transporte (Stream Settings)

O transporte (no campo «**Transporte**» da interface do painel, em inglês *Transmission*) define o modo como o Xray-core transmite os dados dentro de um inbound: qual protocolo de rede é utilizado sobre TLS/Reality e como o tráfego é enquadrado. Esses parâmetros são armazenados no objeto `streamSettings` da configuração do Xray e definidos na aba de transporte do editor de inbound. A criptografia (TLS / Reality) é tratada em uma seção separada – aqui descrevemos apenas a escolha da rede e seus parâmetros.

6.1. Escolha da rede de transmissão

A rede é selecionada na lista suspensa «**Transporte**» (`streamSettings.network`). O valor padrão é `tcp` (exibido na lista como **RAW**). As opções disponíveis são:

Valor na lista	Campo <code>network</code>	Transporte
RAW	<code>tcp</code>	TCP simples (renomeado para RAW nas versões mais recentes do Xray), opcionalmente com ofuscação HTTP
mKCP	<code>kcp</code>	Transporte UDP confiável mKCP
WebSocket	<code>ws</code>	WebSocket sobre HTTP(S)
gRPC	<code>grpc</code>	Túnel gRPC (HTTP/2)
HTTPUpgrade	<code>httpupgrade</code>	HTTP Upgrade
XHTTP	<code>xhttp</code>	XHTTP / SplitHTTP – transporte moderno multiplexado

Ao alterar o valor, o painel limpa o bloco de configurações da rede anterior e preenche o bloco da nova rede com os valores padrão de seu esquema, de modo que cada campo do subformulário sempre possui um valor inicial coerente.

Importante. Nesta versão do painel, **o transporte HTTP/2 (`h2`) não está disponível na lista** – ele foi removido do conjunto de redes; para um túnel bidirecional semelhante ao HTTP/2 use gRPC, e para o transporte moderno mascarado por HTTP use XHTTP. O transporte **Hysteria** (`hysteria`) não é selecionado por esta lista: ele é vinculado diretamente ao protocolo Hysteria e aparece automaticamente quando o próprio inbound é criado com o protocolo Hysteria (veja o item 6.8).

Abaixo, cada rede e cada um de seus campos são descritos individualmente.

6.2. RAW / TCP (`tcpSettings`)

Transporte TCP básico. Por padrão, o tráfego é transmitido «como está»; opcionalmente, pode ser mascarado como uma troca HTTP/1.1 comum.

Campo	Valor padrão	Descrição
Proxy Protocol (<code>acceptProxyProtocol</code>)	<code>false</code> (desativado)	Aceitar o cabeçalho PROXY protocol de um balanceador/ proxy upstream
Ofuscação HTTP (<code>header.type</code>)	<code>none</code> (desativado)	Ativa o mascaramento do tráfego como HTTP/1.1

Proxy Protocol

Alternador «**Proxy Protocol**» (`acceptProxyProtocol`). Quando ativado, o Xray aguarda o cabeçalho PROXY protocol na conexão de entrada e extrai dele o IP real do cliente. Ative apenas se houver um proxy reverso/ balanceador à frente do painel (por exemplo, HAProxy ou nginx com `send-proxy`) que adicione esse cabeçalho. Desativado por padrão.

Ofuscação HTTP (camouflage)

Alternador «**HTTP Ofuscação**». Controla o campo `header` :

- **Desativado** → `header.type = "none"` (o campo `header` simplesmente não está presente no fio). TCP puro.
- **Ativado** → `header.type = "http"` . O tráfego é enquadrado sob a aparência de uma requisição e resposta HTTP/1.1. Ao ativar, o painel preenche imediatamente os sub-objetos `request` e `response` com os valores padrão.

Quando a ofuscação HTTP está ativada, aparecem os campos de configuração da requisição e da resposta simuladas.

Cabeçalho da requisição (`header.request`):

Campo	Chave	Valor padrão	Descrição
Versão da requisição	<code>request.version</code>	<code>1.1</code>	Versão HTTP na linha de início da requisição
Método da requisição	<code>request.method</code>	<code>GET</code>	Método HTTP da requisição simulada
Caminho da requisição	<code>request.path</code>	<code>/</code>	Caminho(s). Inserido como lista de valores separados por vírgula; no fio é um array de strings. Se deixado em branco, substitui-se por <code>/</code>
Cabeçalhos da requisição	<code>request.headers</code>	<code>{}</code> (vazio)	Tabela «Nome/Valor» de cabeçalhos HTTP. Armazenado como mapa <code>nome → [valores]</code> (um nome pode ter múltiplos valores)

Cabeçalho da resposta (`header.response`):

Campo	Chave	Valor padrão	Descrição
Versão da resposta	<code>response.version</code>	1.1	Versão HTTP na linha de início da resposta
Status da resposta	<code>response.status</code>	200	Código de status HTTP da resposta simulada
Razão da resposta	<code>response.reason</code>	OK	Descrição textual do status (reason-phrase)
Cabeçalhos da resposta	<code>response.headers</code>	{} (vazio)	Tabela «Nome/Valor» dos cabeçalhos da resposta (mapa nome → [valores])

Os campos de cabeçalho são editados linha a linha – cada linha define o nome do cabeçalho (Nome) e seu valor (Valor). Esses parâmetros servem apenas para mascarar a aparência do tráfego; não afetam a criptografia. Os valores padrão (GET / HTTP/1.1 , resposta 200 OK) são adequados para a maioria dos cenários – altere-os apenas se precisar imitar um site/serviço específico.

Exemplo de `streamSettings` para RAW com ofuscação HTTP:

```
{
  "network": "tcp",
  "tcpSettings": {
    "acceptProxyProtocol": false,
    "header": {
      "type": "http",
      "request": {
        "version": "1.1",
        "method": "GET",
        "path": ["/"],
        "headers": {
          "Host": ["www.example.com"]
        }
      },
      "response": {
        "version": "1.1",
        "status": "200",
        "reason": "OK"
      }
    }
  }
}
```

Observe que `path` no fio é um array de strings, e cada cabeçalho é um array de valores (`Host` → `["www.example.com"]`).

6.3. mKCP (`kcpSettings`)

mKCP é um transporte confiável sobre UDP. Útil em canais com perda de pacotes e alta latência, mas gera tráfego de controle elevado. Todos os valores padrão coincidem com os recomendados no xray-core.

Campo	Chave	Padrão	Permitido	Descrição
MTU	<code>mtu</code>	1350	576–1460	Tamanho máximo do pacote (bytes). Reduza em caso de problemas de fragmentação
TTI (ms)	<code>tti</code>	20	10–100	Intervalo de transmissão (ms). Menor = menor latência, mas maior overhead
Uplink (MB/s)	<code>uplinkCapacity</code>	5	≥ 0	Capacidade estimada de envio (MB/s)
Downlink (MB/s)	<code>downlinkCapacity</code>	20	≥ 0	Capacidade estimada de recebimento (MB/s)
Multiplicador CWND	<code>cwndMultiplier</code>	1	≥ 1	Multiplicador da janela de congestionamento (congestion window)
Tamanho máx. janela de envio	<code>maxSendingWindow</code>	2097152	≥ 0	Tamanho máximo da janela de envio

Observações sobre os campos:

- **Uplink / Downlink capacity** definem quão agressivamente o mKCP ocupa o canal. Ajuste conforme a largura de banda real: valores muito altos geram tráfego desnecessário, muito baixos subutilizam o canal.
- **TTI** afeta diretamente o compromisso «latência [?] overhead»: valores menores reduzem a latência, mas aumentam o volume de pacotes de controle.
- **MTU** limita o tamanho de um pacote mKCP; reduzi-lo ajuda em canais onde pacotes UDP grandes são fragmentados ou perdidos.

Nesta versão do painel, o campo «seed» (senha de ofuscação do mKCP) e a lista suspensa de **tipo de cabeçalho/ofuscação** (`none` , `srtplib` , `utp` , `wechat-video` , `dtls` , `wireguard`) no subformulário mKCP **não estão disponíveis como campos separados** — a ofuscação de transporte foi movida para o mecanismo geral «FinalMask» (incluindo o modo `mkcp-legacy`), descrito na seção correspondente. O parâmetro «congestion» como alternador independente também não está exposto; o controle de congestionamento é configurado via `cwndMultiplier` e `maxSendingWindow` .

6.4. WebSocket (`wsSettings`)

Transporte WebSocket sobre HTTP(S). Passa bem por CDNs e proxies reversos, mascarando-se como tráfego web comum.

Campo	Chave	Padrão	Descrição
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Aceitar o cabeçalho PROXY protocol de um proxy upstream (veja o item 6.2)
Host	<code>host</code>	<code>""</code> (vazio)	Valor do cabeçalho HTTP <code>Host</code> . Indique ao usar CDN/ domain fronting
Caminho	<code>path</code>	<code>/</code>	Caminho na requisição do handshake WebSocket
Período de heartbeat	<code>heartbeatPeriod</code>	0	Intervalo de envio de quadros heartbeat (em segundos). 0 desativa o heartbeat

Campo	Chave	Padrão	Descrição
Cabeçalhos	headers	{ } (vazio)	Cabeçalhos HTTP adicionais do handshake. Mapa «Nome → Valor» (apenas valores em string, sem arrays)

Observações:

- **Caminho** deve ser igual no servidor (inbound) e no cliente. Com frequência, esse caminho mascara o ponto de entrada no lado do servidor web.
- **Host** faz sentido definir quando o inbound está atrás de uma CDN ou utiliza domain fronting; caso contrário, pode ficar em branco.
- **Período de heartbeat** mantém a conexão «viva» ao passar por proxies/CDNs que encerram sessões inativas agressivamente. Por padrão (0) o heartbeat está desativado.
- Ao contrário do RAW, a tabela de cabeçalhos do WebSocket usa o formato «plano» nome → valor (um valor por cabeçalho).

Exemplo de streamSettings para WebSocket atrás de CDN:

```
{
  "network": "ws",
  "wsSettings": {
    "acceptProxyProtocol": false,
    "host": "cdn.example.com",
    "path": "/ray",
    "heartbeatPeriod": 0,
    "headers": {
      "User-Agent": "Mozilla/5.0"
    }
  }
}
```

Os valores de host e path devem coincidir no cliente; diferentemente do RAW, o valor do cabeçalho aqui é uma string simples, não um array.

6.5. gRPC (grpcSettings)

O transporte com menos parâmetros. Tuneliza o tráfego dentro de chamadas gRPC (sobre HTTP/2); tem boa compatibilidade com CDNs que suportam gRPC. Não há ofuscação de cabeçalhos.

Campo	Chave	Padrão	Descrição
Nome do serviço (Service Name)	serviceName	"" (vazio)	Nome do serviço gRPC (efetivamente o «caminho» do túnel). Deve ser igual no servidor e no cliente
Authority	authority	"" (vazio)	Valor do pseudo-cabeçalho :authority (análogo ao Host para HTTP/2). Indique ao usar CDN/domínio
Multi Mode	multiMode	false (desativado)	Ativa a multiplexação de múltiplos fluxos gRPC paralelos dentro de uma única conexão

Observações:

- **Service Name** é o identificador principal do canal gRPC; deve ser o mesmo em ambos os lados. Valor vazio é permitido, mas normalmente define-se uma string não óbvia para mascaramento.
- **Authority** afeta qual `:authority` é enviado nos quadros HTTP/2; é necessário principalmente ao fazer proxy via CDN.
- **Multi Mode** permite que múltiplos fluxos lógicos passem por uma única conexão física; ative para melhorar o desempenho quando tanto o servidor quanto o cliente suportam isso.

Exemplo de `streamSettings` para gRPC:

```
{
  "network": "grpc",
  "grpcSettings": {
    "serviceName": "GunService",
    "authority": "grpc.example.com",
    "multiMode": false
  }
}
```

`serviceName` (aqui `GunService`) funciona como o «caminho» do túnel e deve ser igual no servidor e no cliente.

6.6. HTTPUpgrade (`httpupgradeSettings`)

Transporte baseado no mecanismo HTTP Upgrade (similar ao WebSocket, mas sem o protocolo WebSocket em si). Também passa bem por proxies e CDNs. O conjunto de campos é o mesmo do WebSocket, mas **sem** o período de heartbeat.

Campo	Chave	Padrão	Descrição
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Aceitar o cabeçalho PROXY protocol de um proxy upstream
Host	<code>host</code>	<code>""</code> (vazio)	Valor do cabeçalho HTTP <code>Host</code>
Caminho	<code>path</code>	<code>/</code>	Caminho da requisição HTTP com o cabeçalho <code>Upgrade</code>
Cabeçalhos	<code>headers</code>	<code>{}</code> (vazio)	Cabeçalhos HTTP adicionais. Mapa «plano» <code>nome → valor</code> (igual ao WebSocket)

O propósito dos campos **Host**, **Caminho** e **Cabeçalhos** é idêntico ao do WebSocket (item 6.4). O heartbeat não está previsto para HTTPUpgrade – essa é uma característica específica do WebSocket.

6.7. XHTTP / SplitHTTP (`xhttpSettings`)

XHTTP (também chamado de SplitHTTP) é um transporte HTTP multiplexado moderno do xray-core. Divide os fluxos de upload e download em requisições HTTP separadas, o que é ideal para CDNs e ambientes com

restrições de duração de conexão. Nem todos os campos são exibidos de uma vez no editor: alguns aparecem dependendo do modo selecionado (`mode`) e dos alternadores ativados.

Campos básicos (sempre visíveis)

Campo	Chave	Padrão	Descrição
Host	<code>host</code>	<code>""</code> (vazio)	Valor do cabeçalho HTTP <code>Host</code>
Caminho	<code>path</code>	<code>/</code>	Caminho base das requisições HTTP
Modo (<code>Mode</code>)	<code>mode</code>	<code>auto</code>	Modo de transmissão (veja abaixo)
Server Max Header Bytes	<code>serverMaxHeaderBytes</code>	<code>0</code>	Limite do tamanho dos cabeçalhos de requisição no servidor (bytes). <code>0</code> – valor padrão do xray-core
Padding Bytes	<code>xPaddingBytes</code>	<code>100–1000</code>	Faixa de padding aleatório (em bytes, formato <code>mín–máx</code>) para dificultar a análise de tamanhos
Cabeçalhos	<code>headers</code>	<code>{}</code> (vazio)	Cabeçalhos HTTP adicionais. Mapa «plano» <code>nome → valor</code>
Método HTTP Uplink	<code>uplinkHTTPMethod</code>	<code>""</code> (Padrão = <code>POST</code>)	Método HTTP das requisições de upload. Opções: vazio (padrão <code>POST</code>), <code>POST</code> , <code>PUT</code> , <code>GET</code> (o último disponível apenas no modo <code>packet-up</code>)
Padding Obfs Mode	<code>xPaddingObfsMode</code>	<code>false</code> (desativado)	Ativa a ofuscação avançada de padding e exibe campos adicionais (veja abaixo)
No SSE Header	<code>noSSEHeader</code>	<code>false</code> (desativado)	Não enviar o cabeçalho <code>Content-Type: text/event-stream</code> (SSE). Ative se ele interferir na passagem por nós intermediários

Campo «Modo» (`mode`)

Lista suspensa com os valores:

Valor	Descrição
<code>auto</code>	Seleção automática de modo (padrão)
<code>packet-up</code>	O fluxo de upload é dividido em requisições HTTP separadas (um pacote por requisição)
<code>stream-up</code>	O fluxo de upload é transmitido em uma única requisição de streaming contínua
<code>stream-one</code>	Uma única requisição de streaming bidirecional compartilhada

A escolha do modo determina quais campos adicionais se tornam visíveis.

Campos visíveis apenas com `mode = packet-up` :

Campo	Chave	Padrão	Descrição
Máx. posts bufferizados no upload	scMaxBufferedPosts	30	Número máximo de requisições POST de upload bufferizadas simultaneamente
Tamanho máx. de upload (bytes)	scMaxEachPostBytes	1000000	Tamanho máximo de uma requisição POST de upload (bytes)
Uplink Data Placement	uplinkDataPlacement	"" (Padrão = body)	Onde posicionar os dados do fluxo de upload: <code>body</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Uplink Data Key	uplinkDataKey	""	Nome da chave/cabeçalho para os dados de uplink. Aparece apenas se <code>uplinkDataPlacement</code> estiver definido e não for <code>body</code>

Campo visível apenas com `mode = stream-up` :

Campo	Chave	Padrão	Descrição
Stream-Up Server	scStreamUpServerSecs	20-80	Faixa de tempo de manutenção da conexão de streaming no servidor (em segundos, formato <code>mín-máx</code>)

Campos de ofuscação de padding (visíveis com `xPaddingObfsMode = ativado`)

Campo	Chave	Padrão	Descrição
Padding Key	xPaddingKey	"" (placeholder <code>x_padding</code>)	Nome da chave para o padding
Padding Header	xPaddingHeader	"" (placeholder <code>X-Padding</code>)	Nome do cabeçalho HTTP pelo qual o padding é transmitido
Padding Placement	xPaddingPlacement	"" (Padrão = <code>queryInHeader</code>)	Onde posicionar o padding: <code>queryInHeader</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Padding Method	xPaddingMethod	"" (Padrão = <code>repeat-x</code>)	Método de geração de padding: <code>repeat-x</code> ou <code>tokenish</code>

Posicionamento de sessão e sequência (sempre visíveis)

Campo	Chave	Padrão	Descrição
Session ID Placement	sessionIDPlacement	"" (Padrão = <code>path</code>)	Onde transmitir o identificador de sessão: <code>path</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Session ID Key	sessionIDKey	"" (placeholder <code>x_session</code>)	Nome da chave de sessão. Aparece apenas se <code>sessionIDPlacement</code> estiver definido e não for <code>path</code>

Campo	Chave	Padrão	Descrição
Session ID Table	<code>sessionIDTable</code>	<code>""</code> (placeholder <code>Base62</code>)	Conjunto de caracteres para geração de identificadores de sessão. É possível escolher um predefinido na lista com autocompletar (<code>ALPHABET</code> , <code>Alphabet</code> , <code>BASE36</code> , <code>Base62</code> , <code>HEX</code> , <code>alphabet</code> , <code>base36</code> , <code>hex</code> , <code>number</code>) ou inserir uma string ASCII personalizada. Vazio – valor padrão do xray-core
Session ID Length	<code>sessionIDLength</code>	<code>""</code> (vazio)	Comprimento ou faixa (por exemplo <code>8-16</code>) dos identificadores gerados. Exibido apenas quando <code>Session ID Table</code> está definido; o mínimo deve ser maior que 0
Sequence Placement	<code>seqPlacement</code>	<code>""</code> (Padrão = <code>path</code>)	Onde transmitir o número de sequência do pacote: <code>path</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Sequence Key	<code>seqKey</code>	<code>""</code> (placeholder <code>x_seq</code>)	Nome da chave de sequência. Aparece apenas se <code>seqPlacement</code> estiver definido e não for <code>path</code>

Os campos de sessão foram renomeados no xray-core v26.6.22: anteriormente chamados **Session Placement** / **Session Key** (`sessionPlacement` / `sessionKey`) – agora são **Session ID Placement** / **Session ID Key** (`sessionIDPlacement` / `sessionIDKey`); o núcleo não reconhece mais os nomes antigos. Inbounds salvos antes da atualização são migrados para as novas chaves automaticamente – não é necessário salvá-los novamente.

Recomendações:

- Para a maioria das instalações, basta manter **Modo = auto** , definir **Caminho/Host** e (ao usar CDN) sincronizá-los com o cliente.
- Os campos de posicionamento (`*Placement` / `*Key`) e de ofuscação de padding são necessários apenas para ajuste fino em cenários específicos de anti-DPI/CDN; quando vazios, são usados os valores padrão do xray-core indicados entre parênteses.
- Parâmetros relativos ao lado do cliente/outbound (por exemplo, intervalos de reenvio de POST, tamanhos de chunk) não são exibidos no formulário de inbound – o servidor ouvinte os ignora. O multiplexador XMUX, por outro lado, está disponível no formulário de inbound (veja abaixo).
- **Valores padrão internos não são gravados.** O painel não grava mais nos arquivos de configuração XHTTP os valores padrão internos `scMaxEachPostBytes` e `scMinPostsIntervalMs` – os valores internos do xray-core são aplicados. Isso elimina a assinatura DPI constante (literal `scMinPostsIntervalMs=30`) pela qual o tráfego poderia ser bloqueado anteriormente. Para inbounds já salvos, valores que coincidem com os padrões do xray-core não são incluídos nos links e assinaturas (não é necessário salvar os inbounds novamente); valores definidos manualmente são preservados.

Exemplo de `streamSettings` para XHTTP (modo `auto`):

```
{
  "network": "xhttp",
  "xhttpSettings": {
    "host": "xhttp.example.com",
    "path": "/yourpath",
    "mode": "auto",
    "xPaddingBytes": "100-1000"
  }
}
```

Para a maioria das instalações, esses quatro campos são suficientes; os campos de posicionamento de sessão/sequência e de ofuscação de padding são deixados em branco – então os valores padrão do xray-core são usados.

XMUX (multiplexação de conexões)

O alternador **XMUX** (`enableXmux`) ativa uma camada de multiplexação que distribui requisições paralelas por um pequeno pool de conexões físicas. Quando ativado, seis campos configuráveis são exibidos (armazenados em `xhttpSettings.xmux`):

Campo	Chave	Padrão	Descrição
Max Concurrency	<code>maxConcurrency</code>	16–32	Máximo de requisições simultâneas por conexão (faixa mín–máx)
Max Connections	<code>maxConnections</code>	0	Máximo de conexões físicas (0 – sem limite)
Max Reuse Times	<code>cMaxReuseTimes</code>	"" (vazio)	Quantas vezes reutilizar uma conexão
Max Request Times	<code>hMaxRequestTimes</code>	600–900	Máximo de requisições por conexão (faixa)
Max Reusable Secs	<code>hMaxReusableSecs</code>	1800–3000	Tempo durante o qual a conexão pode ser reutilizada (segundos, faixa)
Keep Alive Period	<code>hKeepAlivePeriod</code>	"" (vazio)	Período de keep-alive para manter a conexão

Importante. Não é possível definir **Max Connections** e **Max Concurrency** simultaneamente – o xray-core rejeitará essa configuração. Por padrão, ao ativar o XMUX, o painel define `Max Concurrency = 16–32` ; se você definir **Max Connections** (valor maior que 0), o painel removerá o valor padrão de `Max Concurrency` para evitar conflito.

6.8. Transporte Hysteria (`hysteriaSettings`)

O transporte **Hysteria** não é selecionado na lista «Transporte»: ele é ativado automaticamente quando o inbound é criado com o protocolo Hysteria, e fica oculto para outros protocolos (ao sair do protocolo Hysteria, a rede é forçada de volta para `tcp`). Parâmetros:

Campo	Chave	Padrão	Descrição
Versão	<code>version</code>	2 (fixo, campo bloqueado)	Versão do Hysteria. Apenas Hysteria 2 é suportado
UDP idle timeout (s)	<code>udpIdleTimeout</code>	60	Timeout de ociosidade da sessão UDP (segundos). Faixa válida: 2–600; o xray-core rejeita valores fora desse intervalo ao iniciar
Masquerade	<code>masquerade</code>	desativado (ausente)	Ativa o mascaramento do ouvinte como um servidor HTTP/3 durante sondagens

Quando **Masquerade** está ativado, aparece a seleção do tipo (`type`) e os campos dependentes:

- `""` – **default (404 page)**: retorna uma página 404 padrão (sem campos adicionais).
- **proxy (reverse proxy)**: proxy reverso para um site externo.
 - `url` (**Upstream URL**, placeholder `https://www.example.com`) – endereço de destino;
 - `rewriteHost` (**Reescrever Host**, padrão `false`) – substituir o cabeçalho `Host`;
 - `insecure` (**Ignorar TLS verify**, padrão `false`) – não verificar o certificado TLS do upstream.
- **file (serve directory)**: servir arquivos de um diretório.
 - `dir` (**Diretório**, placeholder `/var/www/html`).
- **string (fixed body)**: resposta HTTP fixa.
 - `statusCode` (**Código de status**, padrão `0`, faixa 0–599);
 - `content` (**Body**) – corpo da resposta;
 - `headers` (**Cabeçalhos**) – mapa `nome` → `valor`.

O Masquerade permite que o inbound baseado em Hysteria pareça um servidor HTTP/3 comum durante sondagens ativas, aumentando o sigilo. Por padrão, o mascaramento está desativado.

Exemplo de `hysteriaSettings` com proxy reverso (`masquerade` → `proxy`):

```
{
  "version": 2,
  "udpIdleTimeout": 60,
  "masquerade": {
    "type": "proxy",
    "url": "https://www.example.com",
    "rewriteHost": true,
    "insecure": false
  }
}
```

Aqui, durante sondagens, o ouvinte retorna a resposta de `https://www.example.com`, mascarando-se como um site HTTP/3 comum.

6.9. Parâmetros complementares

Além da seleção de rede, na mesma aba estão disponíveis dois blocos gerais independentes do transporte específico (detalhados nas seções correspondentes):

- **External Proxy** (`externalProxy`) – lista de endereços/portas externos que substituem o endereço do próprio painel nos links de assinatura.
- **Socketopt** (`socketopt`) – opções de socket de baixo nível (TCP Fast Open, mark, estratégia de domínio, proxy transparente etc.).

IP real do cliente (identificação do IP real atrás de CDN/relay)

Quando o inbound está atrás de um intermediário (CDN como Cloudflare, túnel/relay L4 ou outro painel), o Xray enxerga o endereço do intermediário, não o do visitante real. Esse endereço aparece na lista de clientes online e é usado para contar o limite de IPs por cliente, tornando ambos inúteis atrás de um proxy. Para restaurar o IP real, na seção **Socketopt** do formulário de inbound há a seleção de preset **Real client IP**, que combina as configurações `acceptProxyProtocol` e `trustedXForwardedFor`:

Preset	O que faz	Quando usar
Off / direct	Limpa os dois campos.	Inbound acessível diretamente pelos clientes
Cloudflare CDN	Define <code>socketopt.trustedXForwardedFor = ["CF-Connecting-IP"]</code> .	WebSocket / HTTPUpgrade / XHTTP / gRPC atrás da CDN Cloudflare (nuvem laranja)
L4 relay / Spectrum (PROXY)	Ativa <code>acceptProxyProtocol = true</code> .	Túnel/relay L4 à frente do inbound ou Cloudflare Spectrum

Os presets são mutuamente exclusivos: selecionar um limpa o campo do outro, de modo que um `trustedXForwardedFor` desatualizado não sobrescreva um IP restaurado pelo PROXY protocol. Abaixo do preset permanecem visíveis o alternador «raw» **Proxy Protocol** e a lista **Trusted X-Forwarded-For** – o preset apenas os preenche por você e, se necessário, podem ser editados manualmente. Se o preset selecionado não for compatível com o transporte atual (por exemplo, PROXY protocol no mKCP), o formulário exibe um aviso. Esses campos são exclusivos do lado do servidor e **nunca são enviados aos clientes nas assinaturas**.

Use apenas um. `acceptProxyProtocol` lê o IP real do cabeçalho L4 do PROXY protocol, enquanto `trustedXForwardedFor` lê do cabeçalho HTTP da requisição; combine-os manualmente apenas se a sua cadeia de intermediários exigir isso.

- **FinalMask** (`finalmask`) – mecanismo geral de ofuscação de transporte (incluindo a ofuscação legada do mKCP), que substituiu os campos separados «seed»/«header type» dentro dos subformulários de rede.

7. Segurança da conexão: TLS, XTLS e REALITY

Cada inbound que suporta transmissão via fluxo de transporte (VMess, VLESS, Trojan, Shadowsocks, Hysteria) possui a aba «Segurança» no editor. Nela é configurado como o canal de transporte é criptografado e mascarado. Há três modos disponíveis, selecionados por botões de rádio:

Modo	Rótulo na UI	Quando disponível
none	Nenhum	Sempre (exceto Hysteria, onde TLS é obrigatório)
tls	TLS	Para VMess/VLESS/Trojan/Shadowsocks nas redes <code>tcp</code> , <code>ws</code> , <code>http</code> , <code>grpc</code> , <code>httpupgrade</code> , <code>xhttp</code> ; para Hysteria – sempre
reality	Reality	Somente para VLESS/Trojan nas redes <code>tcp</code> , <code>http</code> , <code>grpc</code> , <code>xhttp</code>

O botão **Nenhum** não é exibido quando o protocolo é Hysteria (para o qual TLS é obrigatório). O botão **Reality** aparece somente com uma combinação válida de protocolo e rede (veja a tabela acima).

Ao trocar de modo, o painel reconstrói completamente o bloco `streamSettings`: os campos `tlsSettings` e `realitySettings` do modo anterior são removidos e os valores padrão do modo selecionado são inseridos. Em particular, ao selecionar **Reality**, o painel automaticamente: insere um par aleatório de `target` + `serverNames` (SNI) de uma lista embutida de domínios populares, gera `shortIds` aleatórios e faz uma requisição ao servidor para obter um novo par de chaves X25519 (`privateKey/publicKey`).

7.1. Qual é a diferença: TLS vs XTLS vs REALITY

- **TLS** – criptografia clássica de transporte pelo protocolo TLS 1.2/1.3. O servidor deve ter um certificado válido (domínio próprio + cadeia). O tráfego parece um HTTPS comum, mas para um censor ativo é caracterizado por um handshake TLS reconhecível para o seu domínio; se bloqueado por SNI ou na ausência de um certificado confiável, a conexão é bloqueada/retorna erro.
- **XTLS (Vision)** – não é um modo separado na lista «Segurança», mas um mecanismo de *flow* sobre TLS ou Reality. É ativado no lado do cliente inbound pelo campo **Flow** = `xtls-rprx-vision` (ou `xtls-rprx-vision-udp443`). Vision está disponível para VLESS na rede `tcp` com `security = tls` ou `security = reality`, bem como para VLESS sobre o transporte `xhttp` com criptografia VLESS ativada (`vlessenc` / `ML-KEM`) – nesse caso o campo **Flow** também pode ser definido como `xtls-rprx-vision`, e o valor é incluído corretamente no link `vless://` (`flow=xtls-rprx-vision`). Vision reduz a «dupla criptografia» (TLS-in-TLS), entregando o payload diretamente após o handshake, o que acelera a transferência e melhora o mascaramento. Por isso, a combinação **VLESS + Reality + Flow xtls-rprx-vision** é considerada a configuração moderna recomendada.
- **REALITY** – mecanismo de mascaramento sem certificado próprio. O servidor «toma emprestado» o handshake TLS de um site externo real (`target` / `serverNames`), portanto para um observador a conexão é indistinguível de um acesso a esse site, e nenhum certificado é necessário. A autenticação é baseada em um par de chaves X25519 e um conjunto de `shortIds`. REALITY é resistente a sondagens ativas (`active probing`) e bloqueio por SNI, pois o SNI aponta para um domínio externo real. O preço é a exigência mais rígida de configuração (um `target` correto com porta, sincronização de chaves com o cliente).

Regra rápida de escolha:

- possui domínio próprio e certificado válido, precisa de aparência HTTPS simples → **TLS** (se possível com Vision);
- não tem domínio/certificado ou precisa de máxima invisibilidade contra DPI → **REALITY** (com Vision para VLESS/TCP).

7.2. Modo «Nenhum» (none)

O transporte é transmitido sem camada TLS: os blocos `tlsSettings` e `realitySettings` são excluídos de `streamSettings`. O modo não possui campos adicionais. É adequado quando:

- o inbound escuta apenas em `127.0.0.1` e serve como destino de fallback (segundo a regra do painel, o inbound filho para fallback deve escutar em `127.0.0.1` com `security=none`);
- a criptografia/mascaramento é fornecida por uma camada externa (por exemplo, um proxy reverso Nginx na frente do painel);
- é utilizado um protocolo com criptografia própria (Shadowsocks) em uma rede interna.

Para inbounds acessíveis externamente, o modo «Nenhum» não é recomendado.

Exemplo: bloco `streamSettings` para TLS na rede `tcp` (VLESS/Trojan/VMess). É assim que o resultado fica após selecionar o modo **TLS** e preencher o SNI e os caminhos para o certificado:

```
"streamSettings": {
  "network": "tcp",
  "security": "tls",
  "tlsSettings": {
    "serverName": "vpn.example.com",
    "minVersion": "1.2",
    "maxVersion": "1.3",
    "alpn": ["h2", "http/1.1"],
    "settings": { "fingerprint": "chrome" },
    "certificates": [
      {
        "certificateFile": "/root/cert/vpn.example.com.crt",
        "keyFile": "/root/cert/vpn.example.com.key",
        "ocspStapling": 3600,
        "usage": "encipherment"
      }
    ]
  }
}
```

7.3. Modo TLS

Campos do bloco `tlsSettings`. Os valores padrão são provenientes do esquema do painel.

Parâmetros principais

Campo (rótulo)	Valor padrão	Descrição
SNI (<code>serverName</code>)	"" (vazio)	Server Name Indication – o nome de domínio apresentado no handshake TLS. Deve coincidir com o domínio do certificado. Placeholder em inglês: «Server Name Indication».
Cipher Suites (<code>cipherSuites</code>)	"" → Auto	Lista de conjuntos de cifras permitidos. Por padrão, vazio – a escolha é deixada para o Xray/Go (opção Auto). Altere somente quando precisar restringir explicitamente as cifras.
Versão Mín/Máx (<code>minMaxVersion</code>)	min = 1.2 , max = 1.3	Versões mínima e máxima do TLS. Valores disponíveis: 1.0 , 1.1 , 1.2 , 1.3 . Recomenda-se manter 1.2 – 1.3 ; reduzir o mínimo para 1.0/1.1 não é desejável (versões obsoletas e inseguras).
uTLS (<code>settings.fingerprint</code>)	chrome (no formulário – o item None = "" está disponível)	Impressão digital TLS simulada do client hello (uTLS fingerprint), para que o handshake pareça com o de um navegador popular. Veja a lista abaixo. No TLS, o primeiro item da lista é None (""), que desativa a simulação.
ALPN (<code>alpn</code>)	["h2", "http/1.1"]	Lista de protocolos de camada de aplicação negociados no TLS (seleção múltipla). Valores permitidos: h3 , h2 , http/1.1 . Por padrão, h2 e http/1.1 são oferecidos.

Valores possíveis de **uTLS fingerprint** (os mesmos para TLS e REALITY): chrome , firefox , safari , ios , android , edge , 360 , qq , random , randomized , randomizednoalpn , unsafe . No formulário TLS, o valor vazio **None** também está disponível (a simulação de impressão digital não é aplicada).

Valores disponíveis de **Cipher Suites** (TLS 1.3 e conjuntos ECDHE): TLS_AES_128_GCM_SHA256 , TLS_AES_256_GCM_SHA384 , TLS_CHACHA20_POLY1305_SHA256 , TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA , TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA , TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA , TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA , TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 , TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 , TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 , TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 , TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256 , TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256 .

Alternâncias TLS

Alternância	Padrão	Descrição
Rejeitar SNI desconhecido (<code>rejectUnknownSni</code>)	desligado (false)	Se ativado, o servidor encerra a conexão quando o SNI apresentado pelo cliente não coincide com o nome no certificado. Aumenta a invisibilidade (o servidor não responde a requisições «estranhas»), mas exige correspondência exata do SNI no cliente.
Desativar System Root (<code>disableSystemRoot</code>)	desligado (false)	Desativa o uso do repositório de certificados raiz confiáveis do sistema.
Retomada de sessão (<code>enableSessionResumption</code>)	desligado (false)	Ativa a retomada de sessão TLS (session resumption / session tickets).

Parâmetros adicionais TLS (vcn, curvas, log de chaves, ECH Sockopt)

Campos adicionais estão disponíveis abaixo das configurações principais de TLS.

Campo (rótulo)	Padrão	Descrição
Verify Peer Cert By Name (<code>settings.verifyPeerCertByName</code>)	""	Nomes (separados por vírgula) pelos quais o cliente verifica o certificado do servidor em vez do SNI. É o substituto moderno do campo <code>allowInsecure</code> removido do Xray após 2026-06-01. Valor somente para o painel: não é gravado no config do xray no servidor, mas é transmitido nos links de convite e assinaturas (<code>vcn=...</code>), para que o cliente o aplique por conta própria. Placeholder: <code>example.com</code> .
Curve Preferences (<code>curvePreferences</code>)	""	Restrição e ordem das curvas de troca de chaves TLS, em ordem de preferência (por exemplo, <code>X25519MLKEM768</code> , <code>X25519</code>). Vazio — os valores padrão do Xray-core são usados.
Master Key Log (<code>masterKeyLog</code>)	""	Caminho para registrar as chaves master TLS no formato <code>SSLKEYLOGFILE</code> (para descriptografar o tráfego no Wireshark durante depuração). Placeholder: <code>/path/to/sslkeylog.txt</code> . Em produção, deixe vazio — o arquivo permite descriptografar todo o tráfego.
ECH Sockopt (<code>echSockopt</code>)	desligado	Alternância com parâmetros de socket para a conexão pela qual o Xray solicita a lista de configurações ECH. Quando ativado, estão disponíveis: Dialer Proxy (<code>dialerProxy</code> — roteie a requisição pelo outbound especificado por tag), Domain Strategy (<code>domainStrategy</code>), TCP Fast Open (<code>tcpFastOpen</code>), Multipath TCP (<code>tcpMptcp</code>). Deixe desligado a menos que seja necessário.

Os campos `verifyPeerCertByName`, `curvePreferences`, `masterKeyLog` e `echSockopt` estão no nível superior de `tlsSettings` e sobrevivem ao «corte» dos campos do painel ao salvar a configuração.

Certificados

A seção **Certificado SSL** (com o título «Certificado SSL» na UI) é definida como uma lista: o botão **+** adiciona um novo registro de certificado, o botão **- Remove** o exclui (o botão de remoção está disponível somente quando há mais de um registro). Por padrão, ao ativar o TLS, um registro vazio é criado.

Para cada registro, o seletor de modo de entrada (`useFile`):

- **Caminho do certificado** (valor `useFile = true`, padrão) — especifique os caminhos para os arquivos no servidor:
 - **Chave pública** (`certificateFile`) — caminho para o arquivo do certificado (`.crt` / `.pem`);
 - **Chave privada** (`keyFile`) — caminho para o arquivo da chave privada (`.key`).
- **Conteúdo do certificado** (valor `useFile = false`) — o conteúdo é colado diretamente nos campos (áreas de texto multilinha):
 - **Chave pública** (`certificate`) — conteúdo PEM do certificado;
 - **Chave privada** (`key`) — conteúdo PEM da chave.

Abaixo dos campos do modo «Caminho do certificado» há dois botões:

- **Definir certificado do painel** – preenche os campos com os caminhos para o próprio certificado SSL do painel. Para inbounds no painel central, é utilizado seu certificado (`POST /panel/setting/all` → `webCertFile / webKeyFile`); para inbounds atribuídos a um nó, é utilizado o certificado do próprio nó (`GET /panel/api/nodes/webCert/{nodeId}`), pois os caminhos do painel central não existem no nó. Se o certificado não estiver configurado, é exibido um aviso: «*Nenhum certificado configurado para o painel. Configure-o primeiro nas Configurações.*» (o próprio certificado do painel é definido na seção «Configurações → Segurança»).
- **Limpar** – apaga ambos os caminhos.

Campos adicionais de cada registro de certificado:

Campo	Padrão	Descrição
OCSP Stapling (<code>ocspStapling</code>)	<code>0</code> (desligado)	Intervalo de atualização do OCSP stapling em segundos (mínimo <code>0</code>). Para novos inbounds, está desligado por padrão (<code>0</code>): isso elimina erros nos logs do xray para certificados sem responder OCSP (por exemplo, Let's Encrypt, que abandonou o OCSP). Ative somente para certificados que suportam stapling.
Carregamento único (<code>oneTimeLoading</code>)	desligado (<code>false</code>)	Se ativado, o certificado é lido do disco uma única vez na inicialização e não é relido quando o arquivo é alterado.
Opção de uso (<code>usage</code>)	<code>encipherment</code>	Finalidade do certificado. Valores permitidos: <code>encipherment</code> (criptografia – certificado de servidor comum), <code>verify</code> (verificação), <code>issue</code> (emissão – o servidor assina/emite certificados por conta própria).
Build Chain (<code>buildChain</code>)	desligado (<code>false</code>)	Exibido somente quando <code>usage = issue</code> . Constrói a cadeia de certificados.

Não há um botão separado de certificado autoassinado no editor de inbound: o painel não gera um certificado autoassinado dinamicamente para inbounds. O certificado é especificado por caminho/ conteúdo ou carregado das configurações do painel pelo botão «Definir certificado do painel». A emissão/ obtenção do certificado SSL do próprio painel (incluindo upload de arquivos e vinculação ao domínio) é realizada na seção **Configurações** → **Segurança**; não há endpoints ACME/Let's Encrypt para inbounds aqui.

ECH e fixação de certificado (campos TLS avançados)

Campo	Padrão	Descrição
ECH key (<code>echServerKeys</code>)	<code>""</code>	Chaves de servidor Encrypted Client Hello.
ECH config (<code>settings.echConfigList</code>)	<code>""</code>	Lista de configurações ECH (parte do cliente, incluída no link).

Campo	Padrão	Descrição
SHA-256 do certificado do par (<code>settings.pinnedPeerCertSha256</code>)	[]	Hashes SHA-256 do certificado do par (strings hex, separados por vírgula). Dica literal: « <i>Hashes SHA-256 do certificado do par como string hexadecimal (ex.: e8e2d3...), separados por vírgula. Somente para o painel – não é gravado no config do xray do servidor, mas é incluído nos links de convite para que os clientes possam fixar o certificado.</i> »

Botões: Ao lado do campo **SHA-256 do certificado do par** há dois botões de preenchimento automático:

- **Fill from this inbound's certificate** (ícone de escudo) – preenche com o hash SHA-256 do certificado deste próprio inbound (obtido localmente via endpoint `getCerHash`).
- **Fetch the hash by pinging the SNI (xray tls ping)** (ícone de download) – obtém o hash do certificado ativo do servidor realizando uma conexão TLS ao SNI especificado (no servidor é chamado `getRemoteCertHash`). O campo **SNI** (`serverName`) deve estar preenchido – caso contrário, é exibida a dica «*Set the SNI (serverName) first to ping the remote certificate.*»

Os hashes obtidos são adicionados ao campo (separados por vírgula) e incluídos nos links de convite para que o cliente possa fixar o certificado.

- **Obter novo certificado ECH** – solicita ao servidor um novo par ECH para o SNI atual (`POST /panel/api/server/getNewEchCert` , no servidor é executado `xray tls ech --serverName <SNI>`); preenche os campos **ECH key** e **ECH config**.
- **Limpar** – zera ambos os campos ECH.

7.4. Modo REALITY

Campos do bloco `realitySettings` . REALITY não utiliza certificado SSL: em seu lugar – um handshake TLS emprestado de um domínio externo e um par de chaves X25519.

Parâmetros de mascaramento

Campo (rótulo)	Valor padrão	Descrição
Mostrar (<code>show</code>)	desligado (<code>false</code>)	Saída de depuração do REALITY nos logs do Xray. Normalmente deixado desligado.
Xver (<code>xver</code>)	0	Versão do protocolo PROXY transmitida ao backend (0 – desligado). Mínimo 0 .
uTLS (<code>settings.fingerprint</code>)	chrome	Impressão digital TLS simulada (mesma lista do TLS, mas sem a opção vazia None).
Alvo (<code>target</code>)	"" (o painel preenche aleatoriamente ao ativar)	Campo obrigatório. O domínio real cujo handshake TLS é emprestado pelo REALITY. Dica literal: « <i>Obrigatório. Deve conter a porta (ex.: example.com:443). Sem a porta, o Xray-core não inicia.</i> » A validação no painel verifica a presença e a validade da porta; caso contrário, são exibidos os erros «Alvo REALITY é obrigatório» / «Alvo REALITY deve conter a porta...» / «O alvo REALITY possui uma porta inválida». O botão de atualização ao lado preenche um par aleatório da lista embutida.

Campo (rótulo)	Valor padrão	Descrição
SNI (<code>serverNames</code>)	<code>[]</code> (preenchido junto com o alvo)	Lista de SNIs permitidos (entrada múltipla por tags). Deve corresponder ao domínio em Alvo . O botão de atualização preenche o SNI junto com o alvo aleatório.
Diferença máx. de tempo (ms) (<code>maxTimediff</code>)	<code>0</code>	Divergência máxima permitida de relógio entre cliente e servidor em milissegundos (<code>0</code> – sem limite). Mínimo <code>0</code> .
Versão mín. do cliente (<code>minClientVer</code>)	<code>""</code>	Versão mínima do cliente Xray (placeholder <code>25.9.11</code>). Vazio – sem restrição.
Versão máx. do cliente (<code>maxClientVer</code>)	<code>""</code>	Versão máxima do cliente Xray. Vazio – sem restrição.
Short IDs (<code>shortIds</code>)	<code>[]</code> (gerados ao ativar)	Lista de identificadores curtos (hex) que distinguem os clientes. Entrada múltipla por tags; o botão de atualização gera um conjunto aleatório.
SpiderX (<code>settings.spiderX</code>)	<code>/</code>	Caminho do «spider» (parte do cliente do REALITY), usado ao simular o acesso ao site externo. Incluído no link de convite.

Alvo (`target`) e **SNI** (`serverNames`) ao ativar o REALITY e ao pressionar o botão de atualização são preenchidos com um par aleatório da lista embutida do painel: `www.amazon.com`, `aws.amazon.com`, `www.oracle.com`, `www.nvidia.com`, `www.amd.com`, `www.intel.com`, `www.sony.com` (cada um com a porta `:443`). Escolha um site HTTPS externo robusto e estável que não esteja atrás do seu próprio servidor.

Exemplo: bloco `streamSettings` para REALITY na rede `tcp` (VLESS). Nenhum certificado é necessário – em vez disso, um domínio emprestado e um par de chaves X25519:

```
"streamSettings": {
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "show": false,
    "xver": 0,
    "dest": "www.nvidia.com:443",
    "serverNames": ["www.nvidia.com"],
    "privateKey": "YOUR_X25519_PRIVATE_KEY",
    "shortIds": [ "", "6ba85179e30d4fc2" ],
    "settings": {
      "publicKey": "YOUR_X25519_PUBLIC_KEY",
      "fingerprint": "chrome",
      "spiderX": "/"
    }
  }
}
```

Aqui o campo **Alvo** (`target`) do painel corresponde a `dest` na configuração final do Xray. Se um inbound REALITY foi criado com o destino na chave `dest` (por versões antigas do painel, via API ou ferramentas externas), o painel ao analisar normaliza `dest` → `target` quando `target` está vazio – portanto, tal inbound é carregado corretamente, o campo **Alvo** não fica vazio e o salvamento não quebra o REALITY em funcionamento.

Chaves REALITY (X25519)

Campo	Padrão	Descrição
Chave pública (<code>settings.publicKey</code>)	""	Chave pública X25519 (o cliente a insere em sua configuração/link).
Chave privada (<code>privateKey</code>)	""	Chave privada X25519 (armazenada somente no servidor).

Botões abaixo das chaves:

- **Obter novo certificado** – solicita ao servidor um novo par de chaves (GET `/panel/api/server/getNewX25519Cert` ; no servidor é executado `xray x25519`), preenche **Chave privada** e **Chave pública**. Esse mesmo par é gerado automaticamente ao ativar o modo REALITY pela primeira vez.

Exemplo: obter um par de chaves X25519 via API (fora do formulário, por exemplo em um script). A requisição retorna as chaves privada e pública:

```
curl -s -b cookie.txt https://your-panel:2053/panel/api/server/getNewX25519Cert
# Resposta:
# {"success":true,"obj":{"privateKey":"...","publicKey":"..."}}
```

`cookie.txt` – arquivo de cookie de sessão obtido após o login via `POST /login`.

- **Limpar** – zera ambas as chaves.

Assinatura pós-quântica ML-DSA-65 (mldsa65)

Camada adicional (opcional) de autenticação pós-quântica do REALITY:

Campo	Padrão	Descrição
mldsa65 Seed (<code>mldsa65Seed</code>)	""	Seed da chave ML-DSA-65 do servidor.
mldsa65 Verify (<code>settings.mldsa65Verify</code>)	""	Valor de verificação (parte do cliente, incluído no link).

Botões:

- **Obter novo Seed** – solicita um novo par (GET `/panel/api/server/getNewmldsa65` ; no servidor é executado `xray mldsa65`), preenche **mldsa65 Seed** e **mldsa65 Verify**.
- **Limpar** – zera ambos os campos.

Limite de velocidade de fallback e log de chaves REALITY

Nas configurações do REALITY há um limite de velocidade para o tráfego de fallback – ele impede que sondagens ativas usem o servidor como canal gratuito para o domínio emprestado. A configuração é definida separadamente para dois sentidos – **Limit Fallback Upload** e **Limit Fallback Download** (`limitFallbackUpload` / `limitFallbackDownload`), cada um com o mesmo conjunto de campos:

Campo (rótulo)	Padrão	Descrição
After Bytes (afterBytes)	0	Quantos bytes permitir em velocidade máxima antes de iniciar a limitação. 0 – limitar desde o primeiro byte.
Bytes Per Sec (bytesPerSec)	0	Teto de velocidade do tráfego de fallback em bytes por segundo após o limite. 0 – sem limitação (desativa este sentido).
Burst Bytes Per Sec (burstBytesPerSec)	0	Reserva para picos breves acima da velocidade constante (tamanho do token-bucket). Se menor que Bytes Per Sec , é elevado até seu valor.

Nessa mesma seção há o campo **Master Key Log** (masterKeyLog) – caminho para registrar as chaves master TLS no formato `SSLKEYLOGFILE` para depuração no Wireshark; em produção, deixe vazio.

7.5. Recomendações práticas de configuração

1. **VLESS + Reality (recomendado):** crie um inbound VLESS na rede `tcp`, na aba «Segurança» selecione **Reality** – o painel preencherá automaticamente `target /SNI`, `shortIds` e gerará as chaves X25519. Se necessário, clique em «Obter novo certificado» para o seu próprio par de chaves. Para clientes VLESS, ative **Flow** = `xtls-rprx-vision` (XTLS Vision) – isso proporcionará máximo desempenho e invisibilidade.

Exemplo: link de cliente final VLESS + Reality + Vision. É assim que o link de convite gerado pelo painel para tal inbound é exibido (os valores de chaves/ID são ilustrativos):

```
vless://uuid-клиента@1.2.3.4:443?
type=tcp&security=reality&pbk=ПУБЛИЧНЫЙ_КЛЮЧ&fp=chrome&sni=www.nvidia.com&sid=6ba85179e3
0d4fc2&spx=%2F&flow=xtls-rprx-vision#my-reality
```

Aqui `pbk` é a chave pública X25519, `sni` é o domínio emprestado de **Alvo**, `sid` é um dos **Short IDs**, `flow=xtls-rprx-vision` é o XTLS Vision ativado. 2. **TLS com domínio próprio:** selecione **TLS**, preencha **SNI** com o nome do domínio, adicione o certificado (por caminho de arquivos ou por conteúdo), ou clique em «Definir certificado do painel» se o domínio e o certificado já estiverem configurados em «Configurações → Segurança». Mantenha **Versão Mín/Máx** = `1.2 – 1.3` e **uTLS** = `chrome` para simular um navegador comum. 3. Não deixe o modo **Nenhum** para inbounds expostos externamente – use-o somente para destinos de fallback locais (`127.0.0.1`) ou quando o TLS é fornecido por um proxy externo. 4. Dica da interface: para a maioria dos campos avançados há a dica «*Recomenda-se manter as configurações padrão*» – altere-os somente se compreender as consequências.

8. Clientes

Cliente é a conta de usuário VPN: um conjunto de credenciais (UUID ou senha), vinculado a um ou mais inbound, com cota de tráfego própria, prazo de validade e limite de conexões simultâneas. Neste fork, o cliente é uma entidade independente (tabela `clients`): o mesmo cliente pode ser vinculado a vários inbound ao mesmo tempo, preservando o UUID/senha comum e o contador de tráfego compartilhado. A seção **Clientes** exibe todas as contas do painel independentemente do inbound, com pesquisa, filtros, ordenação e operações em massa.

8.1. Campos do cliente

A seguir, cada campo do editor **Adicionar cliente** / **Editar cliente** é detalhado.

O formulário do cliente está dividido em duas abas: **Principal** (email, vinculação ao inbound, limites, prazo, grupo, comentário, tag reversa) e **Credenciais** (UUID/senha/auth, Flow, VMess Security). Nos rótulos dos campos, a cota é indicada como **Limite de tráfego (GB)**, e os prazos como **Duração (dias)** e **Renovação automática (dias)**; os campos **Limite de tráfego (GB)** e **Limite de IP** possuem dicas explicando que `0` significa "sem restrições". Ao editar um cliente já existente, o botão de geração de email aleatório fica oculto, e o botão de log de IP fica diretamente ao lado do campo **Limite de IP**, exibindo o número de endereços registrados.

Campo	Chave JSON	Padrão	Descrição
Email	<code>email</code>	– (obrigatório)	Identificador único do cliente
UUID	<code>id</code>	gerado automaticamente	Identificador para VMess/VLESS
Senha	<code>password</code>	gerada automaticamente	Senha para Trojan/Shadowsocks
Autorização	<code>auth</code>	gerada automaticamente	Senha para Hysteria
Flow	<code>flow</code>	vazio	Flow control (XTLS), somente VLESS
VMess Security	<code>security</code>	<code>auto</code>	Método de criptografia VMess
Limite de IP	<code>limitIp</code>	<code>0</code> (sem limite)	Máximo de IPs simultâneos
Total enviado/recebido (GB)	<code>totalGB</code>	<code>0</code> (sem limite)	Cota de tráfego
Prazo de validade	<code>expiryTime</code>	<code>0</code> (sem prazo)	Data de expiração
Renovação automática	<code>reset</code>	<code>0</code> (desativado)	Período de redefinição de tráfego, em dias
ID do usuário Telegram	<code>tgId</code>	<code>0</code> (nenhum)	ID numérico do Telegram
ID da assinatura	<code>subId</code>	gerado automaticamente	Identificador da assinatura
Grupo	<code>group</code>	vazio	Rótulo lógico de agrupamento
Comentário	<code>comment</code>	vazio	Observação livre
Habilitado	<code>enable</code>	<code>true</code>	Se a conta está ativa

Email (identificador)

O campo **Email** é o identificador principal e obrigatório do cliente. Apesar do nome, não precisa ser um endereço de e-mail: qualquer rótulo de texto serve (nome de usuário, número). O valor deve ser **único** dentro do painel; tentar criar um segundo cliente com um email já em uso é rejeitado (`email already in use`), exceto quando o `subId` também coincide (o que é interpretado como vinculação do mesmo cliente).

O Email **não pode ser deixado em branco** (`client email is required`) e **não pode conter espaços, os caracteres / , \ nem caracteres de controle** ("Email não pode conter espaços, '/', ' ou caracteres de controle"). O email participa da contabilidade de tráfego, do log de IP, da lista de usuários online e dos nomes das operações – não é recomendado alterá-lo retroativamente.

UUID / Senha / Autorização (credenciais)

O campo de credenciais específico depende do protocolo do inbound ao qual o cliente é vinculado. Os valores são preenchidos automaticamente se o campo for deixado em branco:

- **UUID** (campo `id`) – para os protocolos **VMess** e **VLESS**. Se não for definido, um UUID v4 aleatório é gerado.
- **Senha** (campo `password`) – para **Trojan** e **Shadowsocks**. Para Trojan, por padrão, é gerado um UUID sem hífen. Para Shadowsocks, é gerada uma chave no comprimento adequado em Base64 conforme o método de criptografia do inbound: 16 bytes para `2022-blake3-aes-128-gcm`, 32 bytes para `2022-blake3-aes-256-gcm` e `2022-blake3-chacha20-poly1305`; para outros métodos – UUID sem hífen. Se uma chave inserida manualmente não for compatível com o método 2022-blake3, ela será substituída por uma gerada automaticamente.
- **Autorização** (campo `auth`) – senha para **Hysteria**. Por padrão, UUID sem hífen.

Como um único cliente pode ser vinculado a inbound de protocolos diferentes, o registro do cliente pode conter simultaneamente UUID, senha e auth – cada protocolo usa seu próprio campo.

Exemplo: como as credenciais do cliente aparecem em settings de diferentes inbound. O mesmo cliente em um inbound VLESS é identificado por `id`, em Trojan por `password`, em Shadowsocks por `password` (chave Base64):

```
// fragmento de settings.clients de um inbound VLESS
{ "id": "b831381d-6324-4d53-ad4f-8cda48b30811", "email": "user-a", "flow": "xtls-rprx-vision" }

// o mesmo cliente em um inbound Trojan
{ "password": "b831381d63244d53ad4f8cda48b30811", "email": "user-a" }

// o mesmo cliente em um inbound Shadowsocks (método 2022-blake3-aes-256-gcm)
{ "password": "GPY0aA3f7C00az53eaQ8eqMfRDjmBl0h7v1u3+Z+pHk=", "email": "user-a" }
```

Flow

Flow (campo `flow`) – controle de fluxo XTLS. Aplicável **somente ao VLESS** e somente quando o inbound estiver configurado para XTLS Vision: VLESS sobre transporte **TCP** com security **tls** ou **reality**. O valor

permitido é `xtls-rprx-vision` (assim como o histórico `xtls-rprx-vision-udp443`); um valor vazio indica ausência de flow.

Se o inbound não suportar XTLS-flow, o flow definido é **silenciosamente zerado** ao salvar o cliente: para um mesmo cliente vinculado a vários inbound, o flow é aplicado apenas onde for permitido. Altere apenas se estiver usando intencionalmente o VLESS-Vision.

VMess Security

VMess Security (campo `security`) – método de criptografia da carga útil para VMess. O valor padrão é `auto` (o Xray escolhe a cifra automaticamente). Os valores permitidos são os padrão do VMess: `auto`, `aes-128-gcm`, `chacha20-poly1305`, `none`, `zero`. Para outros protocolos, o campo não é utilizado.

Limite de IP (conexões simultâneas)

Limite de IP (campo `limitIp`) – número máximo de **endereços IP diferentes** a partir dos quais o cliente pode estar conectado simultaneamente. O valor padrão é `0`, o que significa **sem restrição**. Com um valor positivo, o painel rastreia os IPs ativos do cliente e, ao ultrapassar o limite, desativa a conta por meio de uma tarefa em segundo plano. (A partir da versão **3.3.1**, a contagem de IPs é feita via a API online-stats do núcleo Xray e **não requer** log de acesso; em versões mais antigas do núcleo, o painel volta a ler o log de acesso, que deve estar habilitado.) Use para impedir que uma única assinatura seja compartilhada em muitos dispositivos: por exemplo, `2` – permite dois dispositivos.

O Limite de IP é aplicado por meio do **Fail2ban**, portanto o campo **Limite de IP** só estará ativo se o Fail2ban estiver instalado e funcionando (o painel verifica o status via `GET /panel/api/server/fail2banStatus`). Se o Fail2ban não estiver instalado, o campo no editor do cliente (e no formulário de adição em massa) fica bloqueado e, ao passar o cursor, exibe uma dica sugerindo instalar o Fail2ban pelo menu bash `x-ui` ("Fail2ban is not installed, so the IP limit cannot be enforced. Install Fail2ban from the x-ui bash menu to enable this option."); no Windows, a dica informa que o Fail2ban não está disponível lá ("Fail2ban is not available on Windows, so the IP limit cannot be enforced."), e se a funcionalidade estiver desabilitada no servidor – "The IP limit feature is disabled on this server.". Ao atualizar o painel, o limite de IP salvo de clientes em servidores sem Fail2ban é zerado por uma migração única, pois ele não era aplicado de qualquer forma.

Exemplos de valores. `limitIp: 0` – sem restrição; `limitIp: 1` – estritamente um dispositivo por vez; `limitIp: 3` – até três IPs diferentes. No quarto IP ativo, a tarefa em segundo plano desativará o cliente (`enable = false`) até que você execute **Redefinir limite de IP**.

Operações relacionadas: **Log de IP** exibe a lista de IPs registrados do cliente; cada registro contém, além do IP em si, o horário do último acesso e o rótulo do nó (`@ nome_do_nó`) pelo qual a conexão foi detectada – em configuração multi-painel é possível ver por qual nó o cliente se conectou. **Redefinir limite de IP** limpa o log de IP acumulado para que o cliente possa se conectar novamente sem aguardar o vencimento natural das entradas.

Total enviado/recebido (GB) – cota de tráfego

Total enviado/recebido (GB) (campo `totalGB`) – cota total de tráfego (envio + recebimento). O valor padrão – `0` – significa **sem limite**. Ao atingir a cota (`up + down >= total`), o cliente é considerado **esgotado**.

(depleted) e desativado. Na interface, normalmente é inserido em gigabytes; no banco de dados é armazenado em bytes.

Na lista de clientes, a coluna **Tráfego** exibe uma barra colorida de uso: volume de tráfego consumido, o rótulo do limite (ou o símbolo ∞ para sem limite) e uma dica ao passar o cursor com o detalhamento de enviado/recebido e saldo restante. O mesmo indicador compacto é exibido nos cartões de clientes no celular.

Prazo de validade (Expiry)

Prazo de validade (campo `expiryTime`) define o momento de expiração da conta. O campo tem três modos:

- **Sem prazo** – `0`. O cliente nunca expira por tempo.
- **Data específica** – Unix-timestamp positivo (em milissegundos). Ao chegar (`expiryTime <= agora`), o cliente é considerado expirado (expired) e desativado. Na interface, normalmente é definido pela seleção de data ou pela duração em dias (**Duração**, unidade – **Dias**).
- **Início após o primeiro uso** – valor **negativo**, codificando a duração. Enquanto o cliente não transmitir nenhum byte, o prazo permanece negativo ("início adiado"). No primeiro ciclo de contagem de tráfego, o painel o converte para uma data absoluta: `agora + |duração|`. Isso permite vender, por exemplo, "30 dias a partir da primeira conexão", sem saber de antemão quando o cliente será ativado. A conversão é feita uma vez por email, de forma que todos os inbound vinculados recebam o mesmo prazo.

Exemplo de codificação do prazo. Data fixa 1º de março de 2026, 00:00 UTC → `expiryTime: 1772323200000` (timestamp positivo em milissegundos). "30 dias a partir da primeira conexão" → `expiryTime: -2592000000` (valor negativo, $30 \times 24 \times 60 \times 60 \times 1000$); no primeiro byte de tráfego o painel o substituirá por `agora + 2592000000`. Sem prazo → `expiryTime: 0`.

Renovação automática (período de redefinição de tráfego do cliente)

O campo **Renovação automática** (campo `reset`) é o período de renovação/redefinição automática em dias. Dica: "Renovação automática após o vencimento. (0 = desativado) (unidade: dia)".

- `0` – renovação automática **desativada** (valor padrão). Ao expirar o prazo, o cliente simplesmente fica esgotado.
- `> 0` – uma tarefa em segundo plano ao expirar o prazo **zera os contadores de tráfego** (`up = down = 0`), **avança o prazo** em `reset` dias (se necessário, em vários períodos, até que o novo prazo fique no futuro) e, se necessário, **reativa** o cliente. Isso implementa uma assinatura periódica (por exemplo, mensal). A renovação automática **não se aplica a inbound em nós-node** (`node_id IS NOT NULL`).

Consequência importante: clientes com `reset > 0` são **excluídos** do conceito de "esgotado" nas operações de exclusão em massa – seu tráfego/prazo é zerado pela renovação automática conforme esperado, e não torna a conta candidata à exclusão.

ID do usuário Telegram

ID do usuário Telegram (campo `tgId`) – identificador numérico do Telegram do usuário para vinculação ao bot Telegram integrado ao painel (notificações, visualização autônoma de estatísticas). Dica: "ID numérico do usuário Telegram (0 = nenhum)". O valor padrão `0` – sem vinculação. Este campo suporta filtragem (**Tem / Não tem**).

ID da assinatura (subId)

ID da assinatura (campo `subId`) — identificador pelo qual o cliente é incluído na **assinatura** (subscription). Todos os clientes com o mesmo `subId` são entregues por um único link de assinatura. Se o campo for deixado em branco na criação, o painel **gera automaticamente** um `subId` aleatório (UUID). O valor deve ser **único** entre clientes com emails diferentes (`subId already in use`) e está sujeito às mesmas restrições de caracteres que o email ("O ID da assinatura não pode conter espaços, '/', '' ou caracteres de controle").

Sem `subId`, o link de assinatura para o cliente não está disponível ("Este cliente não possui subId, o link de compartilhamento não está disponível").

Aba Links (links externos e assinaturas)

Além das abas **Principal** e **Credenciais**, o editor do cliente possui uma terceira aba **Links** (dica: "Add third-party share links and remote subscription URLs to include in this client's subscription."). Nela, o botão **Add External Link** adiciona links de compartilhamento de terceiros (`vless://`, `vmess://`, `trojan://`, `ss://`, `hysteria2://`, `wireguard://`), e o botão **Add External Subscription** adiciona URLs de assinaturas remotas (por exemplo, `https://provider.example/sub/...`).

Tudo isso é mesclado na resposta de assinatura deste cliente (formatos raw, JSON e Clash): os links são adicionados como estão, e as assinaturas remotas são baixadas periodicamente pelo painel (com cache e timeout curto) e suas configurações são unidas às próprias. Assim, em um único link de assinatura do cliente, é possível entregar seus próprios servidores junto com configurações externas.

Grupo

Grupo (campo `group`) — rótulo lógico para agrupar clientes relacionados. Dica: "Rótulo lógico para agrupar clientes relacionados (ex.: equipe, cliente, região). Filtrável pela barra de ferramentas.", placeholder — "ex.: customer-a". Campo opcional (vazio por padrão). É possível filtrar a lista por grupo e realizar operações em massa; para remover o rótulo de um cliente, use a ação **Desagrupar**.

É possível remover o grupo diretamente no editor de um cliente: ao limpar o campo **Grupo** e salvar, o rótulo é corretamente removido e o cliente deixa de aparecer no grupo anterior.

Comentário

Comentário (campo `comment`) — observação de texto livre para o administrador (vazio por padrão). O conteúdo é incluído na pesquisa e disponível para filtragem (**Tem** / **Não tem** comentário).

Habilitado

Habilitado (campo `enable`) — flag de atividade da conta. Por padrão **habilitado** (`true`); ao criar, mesmo que o flag não seja enviado, o painel força `true`. Um cliente desabilitado (`enable = false`) não pode se conectar e no resumo pertence à categoria de **inativos** (deactive). O painel desativa automaticamente os clientes que esgotaram a cota, expiraram ou ultrapassaram o limite de IP.

Campos somente leitura

No cartão do cliente também são exibidos campos de serviço: **Criado em** (`created_at`) e **Atualizado em** (`updated_at`) – marcações de data/hora de criação e última alteração, preenchidas automaticamente e não editáveis. O campo **Tag reversa** (`reverse`) – tag Reverse opcional para proxy reverso simples VLESS ("Tag Reverse opcional").

8.2. Vinculação ao inbound

Cada cliente deve estar vinculado a pelo menos um inbound – ao criar, é necessário no mínimo um (`at least one inbound is required`). No editor, este campo é **Entradas vinculadas** com a dica **Selecione uma ou mais entradas**.

- **Vincular** – adicionar o cliente aos inbound selecionados (mesmo UUID/senha e tráfego compartilhado). As vinculações existentes são mantidas.
- **Desvincular** – remover o cliente dos inbound selecionados. O próprio registro do cliente é mantido (para exclusão completa, use **Excluir**). Pares em que o cliente não estava vinculado são ignorados silenciosamente.

Ao salvar um cliente vinculado a vários inbound, campos incompatíveis com o protocolo/transporte específico (por exemplo, Flow fora do VLESS-Vision) são automaticamente ajustados para valores válidos para cada inbound.

Acima da lista de seleção de inbound (no formulário do cliente, ao adicionar clientes em massa e nas janelas de vinculação/desvinculação em massa) há os botões **Selecionar todos** e **Limpar**. Nessas listas, cada inbound é rotulado com sua observação (remark), se definida, caso contrário – com a tag do inbound.

8.3. Operações sobre o cliente

Para um cliente individual (via cartão **Informações do cliente** ou menu de contexto **Ações**) estão disponíveis:

Visualização de informações, QR code e link

- **Informações do cliente** – cartão com todos os campos, tráfego utilizado/restante (**Saldo**), prazo de validade e inbound vinculados.

A consulta de cliente via API (`GET /panel/api/clients/get/:email`) junto aos campos `client` e `inboundIds` também retorna `usedTraffic` – tráfego efetivamente consumido (enviado + recebido, incluindo dados dos nós), facilitando a comparação do consumo com a cota `totalGB`.

- **QR code e Link** – link de configuração do cliente para importação no aplicativo cliente. É gerado a partir de todos os inbound vinculados com protocolo suportado (`GET /links/:email`). Se não houver links compatíveis: "Nenhum link de compartilhamento disponível – vincule o cliente a uma entrada com protocolo suportado primeiro."
- **Link de assinatura** – URL de assinatura pelo `subId` (`GET /subLinks/:subId`). Disponível somente se o cliente tiver `subId` e o serviço de assinatura estiver habilitado em **Configurações do painel** → **Assinatura** (caso contrário "Serviço de assinatura desabilitado."). Adicionalmente é fornecida a **URL de assinatura JSON**.

Redefinir tráfego

Redefinir tráfego (`POST /resetTraffic/:email`) zera os contadores de envio/recebimento (`up` , `down`) do cliente específico. A cota (`totalGB`) e o prazo de validade **não são afetados** – apenas o volume consumido é zerado. Toast: "Tráfego redefinido". Se o cliente não estiver vinculado a nenhum inbound: "Vincule este cliente a uma entrada primeiro".

O botão **Redefinir tráfego** também está disponível no formulário de edição do cliente – no painel inferior, ao lado de **Cancelar** / **Salvar** (uma confirmação é solicitada antes da redefinição). Se o cliente foi desativado por esgotamento de tráfego, a redefinição (tanto individual quanto em massa) automaticamente o **reativa** (`enable = true`) e propaga imediatamente essa alteração para os nós – não é necessário reativar o cliente manualmente no master e nos nós.

Redefinir limite de IP

Limpa o log de IP acumulado do cliente (`POST /clearIps/:email`), para remover o bloqueio temporário por exceder o limite de conexões simultâneas. Toast: "Log foi limpo".

Excluir

Excluir (`POST /del/:email`) – exclusão completa do cliente. Diálogo de confirmação: título "Excluir cliente {email}?", texto "O cliente será removido de todas as entradas vinculadas e seu registro de tráfego será destruído. Esta ação não pode ser desfeita.". A exclusão remove o cliente de **todos** os inbound e destrói seu registro de tráfego. Toast: "Cliente excluído".

8.4. Operações em massa

Na lista de clientes é possível marcar vários registros (**Selecionar todos**, **Limpar seleção**); o contador exibe "{count} selecionados". Para os selecionados estão disponíveis:

- **Excluir ({count})** (`POST /bulkDel`) – exclusão em grupo. Confirmação: "Excluir {count} clientes?", "Cada cliente selecionado é removido de todas as entradas vinculadas, seu registro de tráfego é destruído. Esta ação não pode ser desfeita.". Toast: "Clientes excluídos: {count}", em caso de falha parcial – "Excluídos: {ok}, falhos: {failed}".
- **Editar ({count}) / Ajustar** (`POST /bulkAdjust`) – alteração em massa de prazo e/ou cota. Diálogo "Editar {count} clientes" com dica "Valores positivos adicionam, negativos reduzem. Clientes com prazo ou tráfego ilimitado são ignorados no campo correspondente.". Campos: **Adicionar dias** e **Adicionar tráfego (GB)**.

Lógica:

- **Prazo:** clientes com prazo ilimitado (`expiryTime == 0`) são ignorados ("unlimited expiry"); para clientes com data, o prazo é ajustado pelo número de dias informado; para clientes no modo "após o primeiro uso" (prazo negativo), a duração de espera é corrigida. Uma redução que exceda o saldo restante é ignorada ("reduction exceeds remaining time/delay window").
- **Tráfego:** clientes sem limite (`totalGB == 0`) são ignorados ("unlimited traffic"); caso contrário, a cota é alterada pelo volume informado, não ficando abaixo de zero.
- Se nem dias nem tráfego forem informados: "Informe dias ou tráfego antes de aplicar.". Toast: "Editados: {count}" / "Editados: {ok}, ignorados: {skipped}".

Exemplo: prorrogar os clientes selecionados por 30 dias e adicionar 50 GB. No diálogo **Editar**, informe **Adicionar dias** = 30 , **Adicionar tráfego (GB)** = 50 . Para, ao contrário, retirar uma semana e reduzir a cota em 10 GB, insira valores negativos: **Adicionar dias** = -7 , **Adicionar tráfego (GB)** = -10 (clientes com prazo ilimitado ou sem limite de tráfego no campo correspondente serão ignorados).

- **Vincular ({count}) / Desvincular ({count})** (POST /bulkAttach / bulkDetach) – vinculação/desvinculação em massa dos clientes selecionados aos inbound selecionados. Os alvos são apenas inbound multiusuário. Resultado da desvinculação: "Desvinculados {detached}, ignorados {skipped}."
- **Links de assinatura ({count})** – tabela resumida de URLs de assinatura e assinatura JSON dos clientes selecionados com o botão **Copiar todos**. Se nenhum tiver subId: "Nenhum dos clientes selecionados possui ID de assinatura."
- **Adicionar ao grupo e Desagrupar** – atribuição e remoção do rótulo de grupo.

Redefinição de tráfego e exclusão por status

- **Redefinir tráfego de todos os clientes** (POST /resetAllTraffics) – zera os contadores up / down de todos os clientes do painel. Confirmação: "Redefinir tráfego de todos os clientes?" e "Os contadores de envio/recebimento de todos os clientes são zerados. Cotas e prazos de validade não são afetados. Esta ação não pode ser desfeita.". Toast: "Tráfego de todos os clientes redefinido".
- **Excluir esgotados** (POST /delDepleted) – exclui todos os clientes cuja **cota está esgotada** (total > 0 and up + down >= total) ou cujo prazo expirou (expiry_time > 0 and expiry_time <= agora), desde que reset = 0 (clientes com renovação automática não são afetados). Confirmação: "Excluir clientes esgotados?", "São excluídos todos os clientes cuja cota de tráfego está esgotada ou cujo prazo expirou. Esta ação não pode ser desfeita.". Toast: "Clientes esgotados excluídos: {count}".

Exportação, importação e exclusão de clientes sem vínculo

Quando nada está selecionado, no menu **Mais** da página **Clientes** estão disponíveis três operações.

Exportar clientes (GET /clients/export) abre um visualizador com a lista JSON de todos os clientes no formato {client, inboundIds} com botões de cópia e download (arquivo clients-export.json).

Importar clientes (POST /clients/import) abre um editor no qual se cola esse mesmo JSON e clica em **Import**: clientes com inboundIds são criados e vinculados aos inbound, clientes sem vínculos são restaurados como registros "soltos" independentes, e emails já existentes **nunca são sobrescritos** – eles vão para a lista de ignorados. Toasts: "{count} clients imported", "{ok} imported, {failed} skipped".

Excluir clientes sem vínculo (POST /clients/delOrphans) – operação perigosa: exclui todos os clientes não vinculados a nenhum inbound, juntamente com seu registro de tráfego, log de IP e links externos. Confirmação: "Delete clients without an inbound?", "Removes every client that is not attached to any inbound, along with its traffic record. This cannot be undone.". Toast: "{count} unattached clients deleted". A ação é irreversível.

8.5. Pesquisa, filtros e ordenação

Acima da lista há uma barra de pesquisa ("Pesquisar email, comentário, sub ID, UUID, senha, auth...") – ela busca por email, comentário, subId, UUID, senha e auth. Contador de resultados: "Exibindo {shown} de {total}".

A lista de clientes é atualizada automaticamente: o painel busca a página atual a cada poucos segundos, de modo que clientes recém-conectados e alterações na ordem de classificação aparecem sem atualização manual (o indicador de carregamento não pisca durante a consulta em segundo plano).

O painel **Filtrar clientes** permite filtrar por status (categoria), protocolo, inbound vinculado, intervalo de prazo de validade, intervalo de tráfego utilizado, presença de renovação automática (**Tem/Não tem**), presença de ID Telegram e comentário, além de por grupo. Em painéis com nós, aparece um multiselect **Nós**: é possível restringir a lista a clientes dos nós selecionados; um item separado **Painel local** filtra clientes de inbound sem vínculo a nó (o filtro só é visível quando há nós). Ordenação: **Mais antigos/recentes primeiro, Atualizados recentemente, Online recentemente, Email A→Z / Z→A, Mais tráfego, Maior saldo, Expirando em breve**.

8.6. Ícones e status

Prioridade de status: esgotado/expirado → inativo → expirando em breve → ativo.

- **Online / Offline** – cliente com conexão ativa (presente na lista de usuários online atual) e **habilitado**. A lista de usuários online é atualizada por requisições separadas (`/onlines` , `/onlinesByGuid`).
 - **Esgotado** (depleted) – cota consumida (`up + down >= totalGB`) **ou** prazo expirado (`expiryTime <= agora`). Tal cliente é automaticamente desativado e fica sujeito à ação **Excluir esgotados**.
 - **Expirando em breve** (expiring) – cliente habilitado com menos que o intervalo limite até a expiração do prazo **ou** menos que o volume limite até o esgotamento da cota (os limites são definidos nas configurações do painel). Não se aplica se o cliente já estiver esgotado/desativado.
 - **Inativo** (deactive) – cliente com `enable = false` (desativado manualmente ou por tarefa em segundo plano).
 - **Ativo** (active) – habilitado, não esgotado, prazo não expirado e ainda longe dos limites.
-

9. Grupos de clientes

Esta é uma funcionalidade exclusiva deste fork do 3X-UI. No projeto original 3x-ui (MHSanaei) não existe o conceito de "grupo de clientes" – aqui foram adicionados uma tabela separada de grupos, a página **Grupos** no menu do painel e os métodos de API correspondentes. Se você migrar a configuração para o 3x-ui original, o rótulo de grupo simplesmente não será considerado em nenhum lugar.

9.1. O que é um grupo de clientes e para que serve

Grupo é um rótulo lógico nomeado (label) que pode ser atribuído a um ou mais clientes. Ele não cria uma nova forma de conexão e não é um inbound nem um nó – trata-se de um marcador puramente organizacional, útil para filtrar e processar clientes em massa.

A ideia central do modelo de clientes neste fork: **o cliente é uma entidade de nível superior, identificada pelo email** (o campo `email` na tabela `clients` possui índice único). Um mesmo cliente (um email com as mesmas credenciais) pode estar associado simultaneamente a vários inbound e até a vários nós, inclusive com protocolos diferentes. O rótulo de grupo é armazenado **uma única vez por cliente** e, portanto, é propagado automaticamente para todas as suas associações a inbound de uma só vez.

O rótulo de grupo é um marcador lógico de agrupamento:

Camada	Onde é armazenado	Campo
Registro do cliente (BD)	tabela <code>clients</code>	<code>group_name</code> (por padrão string vazia <code>' '</code>)
Cadastro de grupos (BD)	tabela <code>client_groups</code>	<code>name</code> (índice único, não vazio)
Configurações do inbound (Xray)	JSON <code>settings.clients[].group</code>	duplicado em cada objeto de cliente de cada inbound ao qual o cliente pertence

Para que serve na prática:

- **Um cliente em vários inbound/nós.** Se um cliente é "vendido" como acesso a vários inbound simultaneamente (por exemplo, protocolos diferentes ou nós diferentes), o grupo permite gerenciá-lo como uma unidade: zerar o tráfego, excluir, renomear o rótulo – em uma única operação em todos os seus inbound.
- **Operações em massa e filtragem.** Na página **Clientes**, a lista pode ser filtrada por grupo; na página **Grupos** estão disponíveis ações em massa sobre todos os membros do grupo.
- **Organização de um grande número de clientes.** Rótulos como `vip`, `trial`, `team-A` ajudam a distribuir milhares de clientes em categorias lógicas sem precisar criar inbound separados.

9.2. Relação do grupo com clientes, inbound, nós e protocolos

Esta é a subseção mais importante para compreensão, pois a sincronização do rótulo não é trivial.

O grupo descreve o cliente, não o inbound. O rótulo reside no registro do cliente (`clients.group_name`). Quando um cliente está associado a vários inbound, a qualquer mudança de grupo o painel percorre **todos** os

inbound nos quais esse cliente está presente e insere/remove o campo `group` dentro das configurações Xray deles (`settings.clients[]`). Tecnicamente isso funciona assim: pelo email do cliente são localizados todos os inbound dos quais ele faz parte, depois o objeto de cliente com esse email é editado no JSON de configurações de cada um desses inbound. Portanto:

- O grupo **não depende do protocolo**. Um mesmo email pode ser cliente VLESS em um inbound e cliente Hysteria em outro – o rótulo de grupo continuará sendo um só e será aplicado a ambos (as credenciais de cada protocolo são mantidas separadamente).
- O grupo **abrange os nós**. Os inbound pertencentes a nós se diferenciam dos inbound do painel principal pelo campo `nodeId` (nos inbound do painel principal ele é `null / 0`). O rótulo de grupo é propagado para os objetos de cliente nos inbound independentemente de ser um inbound principal ou de nó – desde que o cliente com esse email esteja presente nele.

O rótulo de grupo é resistente à sincronização com nós e à reconstrução de configurações. Esse comportamento foi projetado intencionalmente:

- Quando um nó envia um snapshot de tráfego, seus dados **não sobrescrevem** os campos locais `group_name` e `comment` do cliente no BD do painel. O grupo e o comentário são considerados campos "locais do painel" – o nó não os gerencia.
- Na reconstrução das configurações do inbound, um valor vazio de `group` nos dados recebidos **não redefine** o rótulo já salvo. O grupo é gerenciado exclusivamente pela página **Grupos** (e não pela edição das configurações Xray do inbound), portanto "grupo vazio" em uma reconstrução normal é interpretado como "não alterar", e não como "limpar".

Conclusão prática: as únicas operações que **intencionalmente limpam** o rótulo são a exclusão do grupo e a remoção explícita do cliente do grupo (veja abaixo). Edições comuns do inbound ou sincronização em segundo plano com o nó não perderão o grupo.

9.3. Cadastro de grupos e grupos "vazios"

A lista de grupos na página é formada pela união de duas fontes:

1. **Grupos derivados (derived)** – todos os valores não vazios de `group_name` efetivamente presentes nos clientes, com contagem do número de clientes.
2. **Grupos salvos (stored)** – registros da tabela `client_groups`.

Essa união gera um efeito importante: um grupo pode existir **sem nenhum cliente**. Esse grupo é criado pelo botão "Adicionar grupo" (registro em `client_groups`) e é exibido na lista com o contador `0`. Esses registros são os chamados **grupos vazios**. A lista é sempre ordenada por nome sem distinção de maiúsculas/minúsculas.

Contadores de resumo na página:

Campo	O que mostra
Total de grupos	Número total de grupos (salvos e derivados juntos).
Clientes com grupo	Quantos clientes possuem um rótulo de grupo não vazio.
Grupos vazios	Quantos grupos existem sem clientes (contador <code>0</code>).
Clientes no grupo	Número de clientes em um grupo específico (coluna da tabela).

9.4. Campos e colunas do grupo

O registro do grupo na tabela `client_groups` contém:

Campo	Tipo	Padrão	Descrição
<code>Id</code>	int	auto-incremento	Chave primária do registro do grupo.
<code>Name</code>	string	— (obrigatório)	Nome do grupo. Índice único, não pode ser vazio. Na UI — coluna Nome .
<code>CreatedAt</code>	int64 (ms)	hora de criação	Momento de criação do registro do grupo.
<code>UpdatedAt</code>	int64 (ms)	hora de alteração	Momento da última modificação.

Na tabela da página são exibidas no mínimo as colunas **Nome** e **Clientes no grupo**, além dos botões de ação (veja abaixo).

9.5. Criação de grupo

Botão **Adicionar grupo**.

Passos:

1. Clique em **Adicionar grupo**.
2. Insira o nome do grupo.
3. Confirme.

Comportamento do backend (`POST /panel/api/clients/groups/create` , corpo `{"name": "..."}`):

- O nome é cortado nos espaços das extremidades. Um nome vazio é rejeitado com o erro «group name is required».
- Se já existir um grupo com esse nome — erro «group already exists».
- Em caso de sucesso, é criado um registro em `client_groups` (inicialmente sem clientes — trata-se de um grupo vazio).

Mensagem de sucesso: «Grupo **"{name}"** criado.»

Exemplo: criar um grupo vazio via API. Prepare um conjunto de rótulos com antecedência, antes de preenchê-los com clientes:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/create' \
  -H 'Content-Type: application/json' \
  -b cookie.txt \
  -d '{"name": "vip"}'
```

Resposta em caso de sucesso:

```
{ "success": true, "msg": "Группа «vip» создана.", "obj": null }
```

Uma chamada repetida com o mesmo nome retornará `"success": false` e a mensagem `group already exists`.

Criar um grupo vazio com antecedência é conveniente quando você quer preparar um conjunto de rótulos e depois preenchê-los com clientes usando "Adicionar clientes..."

9.6. Renomear grupo

Botão **Renomear**, título do diálogo — «**Renomear {name}**».

Comportamento (`POST /panel/api/clients/groups/rename` , corpo `{"oldName": "...", "newName": "..."} :`

- Ambos os nomes são cortados nos espaços. Nome antigo vazio — erro «old group name is required», nome novo vazio — «new group name is required».
- Se o nome novo for igual ao antigo — nada é feito (0 clientes afetados).
- Caso contrário, a renomeação é executada atômica:
 - o registro em `client_groups` é renomeado;
 - em todos os clientes com `group_name = oldName` o campo é atualizado para `newName` ;
 - em **todos os inbound** dos quais os clientes afetados fazem parte (incluindo os de nós), o valor de `group` nas configurações Xray é corrigido do antigo para o novo.
- Após a renomeação, o painel marca o Xray como necessitando de reinicialização e envia notificação sobre alteração de clientes.

Mensagens:

- Sucesso: «**Grupo renomeado para {count} cliente(s).**»
- Conflito de nomes na UI: «**Já existe um grupo com o nome "{name}"**».

9.7. Adição de clientes ao grupo

Botão **Adicionar clientes...**, título — «**Adicionar clientes ao grupo "{name}"**».

Dica literal no diálogo:

«Selecione os clientes para adicionar a este grupo. As associações existentes a inbound são mantidas; somente o rótulo de grupo é alterado. Clientes já pertencentes a este grupo não são exibidos.»

Se não houver ninguém a adicionar, é exibida a mensagem «**Não há outros clientes para adicionar.**»

Comportamento (`POST /panel/api/clients/groups/bulkAdd` , corpo `{"emails": [...], "group": "..."} :`

- O nome do grupo é obrigatório (caso contrário, erro «group name is required»); lista de emails vazia — a operação não faz nada.
- Se esse grupo ainda não existir em `client_groups` nem entre os derivados — ele será criado automaticamente.

- Para os emails selecionados, o campo `group_name` dos clientes é definido como `group`; **as associações dos clientes a inbound não são alteradas** – somente o rótulo é afetado. Em seguida, o campo `group` é definido em todos os inbound desses clientes.
- É retornado o número de registros de clientes afetados; o Xray é marcado para reinicialização.

Mensagem de sucesso: «**{count} cliente(s) adicionado(s) a {name}.**»

Exemplo: marcar vários clientes com um grupo em uma única requisição. Os clientes são especificados por email e as associações a inbound não são alteradas:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/bulkAdd' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"emails": ["alice@example.com", "bob@example.com"], "group": "vip"}'
```

Se o grupo `vip` ainda não existir, ele será criado automaticamente. Após a requisição, o campo `group_name` = `"vip"` será definido no registro desses clientes, e o objeto de cliente em cada um dos seus inbound nas configurações Xray receberá o campo `"group": "vip"`:

```
{ "id": "6f1b...", "email": "alice@example.com", "group": "vip", "enable": true }
```

9.8. Remoção de clientes do grupo (sem excluir os clientes em si)

Botão **Remover clientes...**, título – «**Remover clientes do grupo "{name}"**».

Dica literal:

«Selecione os membros para remover deste grupo. Os próprios clientes são mantidos (use "Excluir clientes do grupo" para exclusão completa).»

Comportamento (`POST /panel/api/clients/groups/bulkRemove`, corpo `{"emails": [...]}`): tecnicamente é o mesmo que "Adicionar ao grupo" com um grupo vazio. O campo `group_name` dos clientes selecionados é limpo e o campo `group` é removido das configurações Xray dos seus inbound. Os próprios clientes e suas associações a inbound são mantidos.

Mensagem de sucesso: «**{count} cliente(s) removido(s) de {name}.**»

9.9. Zerar tráfego do grupo

Botão **Zerar tráfego**.

Diálogo de confirmação:

- Título: «**Zerar tráfego do grupo {name}?**»
- Texto: «**Isso zerará up/down de todos os {count} cliente(s) neste grupo.**»

Comportamento: para todos os emails dos membros do grupo, os campos `up` e `down` na tabela de tráfego são zerados e o campo `enable` é definido como `true` (o cliente é habilitado). A operação é executada em lotes dentro de uma transação.

Mensagem de sucesso: «Tráfego zerado para {count} cliente(s).»

9.10. Exclusão do grupo e exclusão dos clientes do grupo

Na página existem **duas operações de exclusão fundamentalmente diferentes** — é fácil confundi-las, por isso a distinção é crítica.

9.10.1. Excluir grupo (manter clientes)

Botão «**Excluir grupo (manter clientes)**».

Diálogo:

- Título: «**Excluir grupo {name}?**»
- Texto: «**Isso exclui o grupo e limpa seu rótulo em {count} cliente(s). Os próprios clientes não são excluídos.**»

Comportamento (`POST /panel/api/clients/groups/delete` , corpo `{"name": "..."}`): o registro do grupo é excluído de `client_groups` , o campo `group_name` de todos os seus clientes é limpo e o campo `group` é removido dos seus inbound. **Os clientes, suas conexões e o tráfego são mantidos.** O Xray é marcado para reinicialização.

Mensagem de sucesso: «**Grupo limpo em {count} cliente(s).**»

9.10.2. Excluir clientes do grupo (exclusão completa)

Botão «**Excluir clientes do grupo**».

Diálogo:

- Título: «**Excluir todos os clientes em {name}?**»
- Texto: «**Isso exclui permanentemente {count} cliente(s) junto com seus registros de tráfego. O rótulo do grupo também é limpo. Essa ação não pode ser desfeita.**»

Esta é uma operação destrutiva: ela exclui os próprios clientes (por meio de exclusão em massa por email, endpoint `POST /panel/api/clients/bulkDel`), incluindo seus registros de tráfego, removendo-os assim de todos os inbound.

Mensagens:

- Sucesso: «**{count} cliente(s) excluído(s).**»
- Resultado parcial: «**{ok} excluído(s), {failed} ignorado(s)**»

Se o grupo estiver vazio, as ações sobre seus membros não estão disponíveis — é exibida a mensagem «Este grupo ainda não possui clientes.»

9.11. Relação com a página "Clientes"

O rótulo de grupo é visível e utilizado também fora da página **Grupos**:

- No registro compacto do cliente existe o campo `group`, portanto a pertinência ao grupo é exibida na lista de clientes.
- A lista de clientes (`/panel/api/clients/list/paged`) aceita o parâmetro de filtro `group`: é possível passar um nome ou vários separados por vírgula. A correspondência é feita com lógica "OU" dentro do campo, sem distinção de maiúsculas/minúsculas. Caso especial: um elemento vazio na lista de grupos do filtro significa "clientes sem grupo" (cujo `group` está vazio).
- Na resposta da página de clientes é retornado o array `groups` — lista completa dos nomes de grupos existentes, para que a UI possa construir a lista suspensa de filtro.

Exemplo: filtrar clientes por grupos. A requisição retorna apenas clientes com os rótulos `vip` ou `trial` (vários nomes — separados por vírgula, lógica "OU"):

```
GET /panel/api/clients/list/paged?group=vip,trial
```

Para obter clientes **sem** grupo, passe um elemento vazio na lista — por exemplo, o valor de filtro `group=` (string vazia) ou `group=vip,` (rótulo `vip` mais clientes sem grupo).

9.12. Resumo dos endpoints de API

Todas as rotas de grupos estão montadas sob `/panel/api/clients`:

Método e caminho	Finalidade	Corpo da requisição
GET <code>/panel/api/clients/groups</code>	Lista de grupos com contadores de clientes	—
GET <code>/panel/api/clients/groups/:name/emails</code>	Emails de todos os membros do grupo (ordenados por email)	—
POST <code>/panel/api/clients/groups/create</code>	Criar grupo vazio	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/rename</code>	Renomear grupo	<code>{"oldName", "newName"}</code>
POST <code>/panel/api/clients/groups/delete</code>	Excluir grupo mantendo clientes (limpeza do rótulo)	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/bulkAdd</code>	Adicionar clientes ao grupo (por email)	<code>{"emails": [...], "group"}</code>
POST <code>/panel/api/clients/groups/bulkRemove</code>	Remover clientes do grupo (limpeza do rótulo)	<code>{"emails": [...]}</code>

Método e caminho	Finalidade	Corpo da requisição
POST /panel/api/clients/ bulkDel	Exclusão completa de clientes (usada por "Excluir clientes do grupo")	<code>{"emails": [...], "keepTraffic"}</code>

Exemplo: cenário típico de ciclo de vida de um grupo via API.

```
# 1. Criar o rótulo trial
curl -s .../panel/api/clients/groups/create -d '{"name":"trial"}'

# 2. Atribuí-lo a dois clientes
curl -s .../panel/api/clients/groups/bulkAdd -d '{"emails": ["u1@example.com", "u2@example.com"], "group":"trial"}'

# 3. Zerar o tráfego de todos os membros (por email de /groups/trial/emails)
curl -s .../panel/api/clients/groups/bulkRemove -d '{"emails":["u2@example.com"]}'

# 4. Excluir o grupo, mas manter os clientes (somente limpeza do rótulo)
curl -s .../panel/api/clients/groups/delete -d '{"name":"trial"}'
```

O passo 4 exclui o registro do grupo e limpa `group_name` nos seus clientes, mas os próprios clientes, suas conexões e o tráfego são mantidos. Para a exclusão permanente dos próprios clientes, usa-se `bulkDel` em vez disso.

As operações que alteram o rótulo nos clientes (`rename` , `delete` , `bulkAdd` , `bulkRemove`) marcam o Xray como necessitando de reinicialização e enviam notificação sobre alteração de clientes.

9.13. Tráfego por grupo

Novidade da versão 3.3.0: na seção **Grupos** (página "Clientes", aba de gerenciamento de grupos) a tabela de grupos agora exibe não apenas o número de clientes em cada grupo, mas também o tráfego total consumido pelo grupo. A coluna é intitulada «**Tráfego utilizado**».

O que a coluna mostra

Para cada linha de grupo é exibida a soma do tráfego de todos os clientes pertencentes a esse grupo – ou seja, `up + down` (tráfego enviado + recebido) somados de todos os seus membros. Isso fornece uma resposta rápida à pergunta "quanto o grupo todo baixou/enviou no total", sem precisar abrir os clientes um por um e somar manualmente.

Ao lado na tabela de grupos são exibidas:

Coluna	O que significa
Nome	Nome do grupo
Clientes	Quantos clientes estão marcados com este grupo (antes a coluna se chamava "Clientes no grupo")
Enviado	Total de <code>up</code> (tráfego enviado) de todos os clientes do grupo
Recebido	Total de <code>down</code> (tráfego recebido) de todos os clientes do grupo

Coluna	O que significa
Tráfego utilizado	Total de <code>up</code> + <code>down</code> de todos os clientes do grupo

O tráfego enviado e recebido são exibidos em colunas separadas **Enviado** e **Recebido**, enquanto a coluna **Tráfego utilizado** mostra a soma dos dois. A coluna de número de clientes se chama simplesmente **Cientes**.

O resumo acima da tabela exibe adicionalmente agregados de todos os grupos – «**Total de grupos**» e «**Cientes com grupo**», e o tráfego total é dividido em dois cartões: «**Total enviado / recebido**» (com setas para cima/baixo – tráfego enviado e recebido separadamente de todos os grupos) e «**Tráfego total**» (com ícone de diagrama – soma total dos dois).

Como é calculado

O cálculo é feito por uma única consulta SQL à tabela de clientes com junção (`LEFT JOIN`) da tabela de contabilização de tráfego:

- pelo campo de rótulo do grupo (`group_name`) os clientes são agrupados e seu número é contado – isso é o "Clientes no grupo";
- o tráfego é obtido como a soma de `up` + `down` da tabela `client_traffics` anexada. Ou seja, são somados os bytes enviados (`up`) e recebidos (`down`) de cada cliente;
- como o email é único tanto na tabela de clientes quanto na tabela de tráfego, a junção não duplica o tráfego de um único cliente.

Particularidades dos valores:

- **Cientes sem registro de tráfego** são contados no contador de membros, mas adicionam 0 à soma, portanto um grupo recém-criado exibe tráfego 0 .
- **Grupos vazios** (criados, mas sem clientes) também estão presentes na lista com contador zero e tráfego zero: além dos grupos "derivados" dos rótulos dos clientes, os grupos explicitamente salvos são mesclados ao resultado, após o que a lista é ordenada por nome sem distinção de maiúsculas/minúsculas.
- Clientes sem rótulo de grupo (`group_name` vazio) não entram no cálculo.

Ações relacionadas

A partir da tabela de grupos ainda estão disponíveis ações sobre o grupo inteiro, incluindo «**Zerar tráfego**» – zera `up` / `down` de todos os clientes do grupo selecionado. Após esse reset, a coluna "Tráfego utilizado" para esse grupo exibe 0 .

10. Assinaturas (Subscription)

Uma assinatura (subscription) é um mecanismo que permite fornecer ao cliente um único link permanente (URL), pelo qual o cliente VPN baixa e atualiza periodicamente de forma automática o conjunto completo de configurações. Em vez de enviar manualmente ao usuário um link separado para cada inbound, é passado um endereço único no formato `https://domínio:porta/sub/<subId>`. Por esse endereço, o painel monta em tempo real todas as configurações vinculadas ao cliente e as entrega no formato que o cliente necessita. Quando as configurações do servidor mudam (novo endereço, rotação de chaves Reality, adição de inbound), o cliente recebe a configuração atualizada na próxima atualização automática, sem exigir nenhuma ação do usuário.

A assinatura é servida por um servidor HTTP/HTTPS separado dentro do painel, que é iniciado de forma independente do painel web e escuta em sua própria porta. Isso foi feito por razões de segurança: a porta de assinatura pode ser aberta para o exterior sem precisar expor a porta do próprio painel.

10.1. O que é subId e como o link é formado

Cada cliente em um inbound possui o campo `subId` (na interface – «ID da assinatura»). Esse valor é a chave da assinatura: o painel busca em todos os inbounds os clientes cujo `subId` coincide com o solicitado e une suas configurações em uma única resposta.

- Se vários clientes (em um ou em diferentes inbounds) tiverem o mesmo `subId`, suas configurações serão incluídas em uma única assinatura. Essa é a maneira padrão de fornecer a um usuário vários servidores/protocolos por um único link.

Exemplo: um usuário – dois servidores por um único link. Suponha que existam dois inbounds (VLESS no servidor A e Trojan no servidor B). Para entregar ao usuário ambas as configurações por um único link, atribua o mesmo `subId` aos dois clientes:

```
Inbound 1 (VLESS): email = ivan@vpn, subId = ivan2025
Inbound 2 (Trojan): email = ivan@vpn, subId = ivan2025
```

Então, pelo endereço `https://sub.example.com:2096/sub/ivan2025`, o painel entregará ambas as configurações de uma vez. Se você adicionar posteriormente um terceiro inbound com o mesmo `subId`, ele aparecerá para o usuário na próxima atualização automática da assinatura, sem necessidade de enviar um novo link.

- Se o campo `subId` do cliente estiver vazio, não é possível compartilhar o link de acesso geral. Na interface, isso é indicado pela mensagem: «Este cliente não possui subId, o link de acesso compartilhado não está disponível.»

Links externos e assinaturas do cliente (aba «Links»)

No formulário do cliente há uma aba «**Links**», onde para um cliente específico é possível anexar fontes adicionais de configurações que são misturadas à assinatura desse cliente (formatos RAW, JSON e Clash):

- **Add External Link** – link de compartilhamento externo (`vless://` , `trojan://` , `ss://` etc.). É adicionado à resposta como está; para JSON/Clash, é adicionalmente convertido em configuração.
- **Add External Subscription** – endereço de assinatura externa. O painel a baixa por conta própria (com cache e timeout curto) e incorpora as configurações obtidas à lista geral do cliente.

Isso é conveniente para fornecer ao cliente servidores adicionais além dos seus inbounds pelo mesmo link único. Se a resposta da assinatura remota for muito grande, ela não é mais truncada silenciosamente: o painel retorna um erro e continua usando o último valor armazenado em cache com sucesso.

- O valor de `subId` não pode ser definido arbitrariamente: ao salvar, é verificado que ele não contém espaços, os caracteres `/` , `\` ou caracteres de controle. A mensagem de validação correspondente: «O ID da assinatura não pode conter espaços, `/`, `\` ou caracteres de controle».

O link final é construído como `<base>/<subPath>/<subId>` (consulte a seção sobre configurações do servidor de assinaturas e o campo «URI de proxy reverso»). Se nenhum cliente for encontrado pelo `subId` (o cliente foi excluído, o `subId` não existe), o servidor retorna HTTP 404 sem corpo. Em caso de erro interno – HTTP 500. Os clientes VPN se orientam apenas pelo código de resposta, portanto o corpo do erro é intencionalmente vazio.

Ordem dos links inbound na assinatura

Cada inbound possui o campo «**Ordem na assinatura**» (`subSortIndex`) – um número a partir de 1, que define a posição dos links desse inbound na resposta da assinatura. Valores menores aparecem primeiro; em caso de valores iguais, a ordem original de criação (por id) é preservada. A ordem se aplica a todos os formatos de resposta – texto simples, página de assinatura, JSON e Clash. Esse campo não afeta a ordem dos inbounds no próprio painel.

O campo é editado no formulário do inbound ao lado das configurações do endereço no link (share address) e é sincronizado com os nós pelas regras normais. Se ao menos um inbound tiver uma ordem diferente de 1, uma coluna compacta «**Ordem**» aparece na lista de Inbounds.

10.2. Configurações do servidor de assinaturas

Todos os parâmetros de assinatura estão na seção de configurações do painel na aba «**Assinatura**». Abaixo, cada parâmetro é explicado; entre parênteses está a chave interna da configuração e o valor padrão.

A própria seção está dividida em abas: «**Configurações do painel**», «**Informações**», «**Perfil**», «**Certificados**», «**Happ**» e «**Clash / Mihomo**». Os campos de título da assinatura, URL de suporte, página de perfil, anúncio e diretório de tema estão na aba «**Perfil**»; as regras de roteamento Happ e Clash/Mihomo estão nas abas de mesmo nome; o intervalo de atualização da assinatura está na aba «**Informações**».

Parâmetros principais

Campo (UI)	Chave	Valor padrão	Descrição
Habilitar assinatura	subEnable	true (habilitado)	Inicia um servidor de assinaturas separado. Dica: «Função de assinatura com configuração separada». Se desativado, o servidor de assinaturas não é iniciado e nenhum dos links funciona.
IP de escuta	subListen	vazio	Endereço IP no qual o servidor de assinaturas aceita conexões. Dica: «Deixe vazio para usar o padrão e monitorar todos os endereços IP».
Porta de assinatura	subPort	2096	Porta TCP do servidor de assinaturas. Dica: «O número da porta para o serviço de assinatura não deve estar em uso no servidor» – a porta deve estar livre e não deve conflitar com o painel ou o Xray.
Caminho URI	subPath	/sub/	Caminho pelo qual as assinaturas comuns são entregues. Dica: «Deve começar com '/' e terminar com '/'».
Domínio de escuta	subDomain	vazio	Domínio pelo qual o acesso à assinatura é permitido (validação do Host). Dica: «Deixe vazio para usar o padrão e escutar todos os domínios e endereços IP». Se definido, requisições com outro Host são rejeitadas.

Importante sobre segurança: o caminho padrão `/sub/` (e `/json/` para JSON) é amplamente conhecido e fácil de adivinhar. O painel exibe um aviso: «O caminho de assinatura padrão `/sub/` é amplamente conhecido – altere-o.» e uma mensagem similar para JSON. Recomenda-se definir um caminho próprio não óbvio.

TLS / certificado

Campo (UI)	Chave	Padrão	Descrição
Caminho para o arquivo de chave pública do certificado de assinatura	subCertFile	vazio	Caminho completo para o arquivo de certificado (<code>.crt</code> / <code>fullchain</code>). Dica: «Insira o caminho completo começando com '/'».
Caminho para o arquivo de chave privada do certificado de assinatura	subKeyFile	vazio	Caminho completo para o arquivo de chave privada. Dica: «Insira o caminho completo começando com '/'».

Se ambos os caminhos forem definidos e o certificado for carregado com sucesso, o servidor de assinaturas opera via **HTTPS**. Se os campos estiverem vazios ou o certificado não puder ser lido, o servidor retorna ao **HTTP** (o erro é registrado no log). A presença de TLS válido também afeta a formação da URL base: com a porta 443 e TLS, ou porta 80 sem TLS, o número da porta é omitido no link.

Intervalo de atualização

Campo (UI)	Chave	Padrão	Descrição
Intervalos de atualização da assinatura	subUpdates	12	Com que frequência (em horas) o aplicativo cliente deve solicitar novamente a assinatura. Dica: «Intervalo entre atualizações no aplicativo cliente (em horas)».

O valor é transmitido ao cliente no cabeçalho HTTP `Profile-Update-Interval`; os clientes modernos o utilizam como período de atualização automática da configuração.

Formato e informações na resposta

Campo (UI)	Chave	Padrão	Descrição
Codificar	<code>subEncrypt</code>	<code>true</code>	Dica: «Criptografar as configurações retornadas na assinatura». Tecnicamente, não é criptografia, mas codificação Base64 de todo o corpo da assinatura comum (formato esperado pela maioria dos clientes). Quando desativado, os links são entregues em texto simples, um por linha.
Mostrar informações de uso	<code>subShowInfo</code>	<code>true</code>	Dica: «Exibir o tráfego restante e a data de expiração após o nome da configuração». Quando ativado, marcadores de tráfego restante () e prazo de validade (por exemplo, <code>5D, 3H</code>) são adicionados ao nome (remark) de cada configuração; para clientes expirados/ indisponíveis, é exibido <code>N/A</code> .
Incluir Email no nome	<code>subEmailInRemark</code>	<code>true</code>	Dica: «Incluir o email do cliente no nome do perfil de assinatura». Adiciona o email do cliente ao remark do perfil.

Modelo de remark (Remark Template)

O nome exibido (remark) de cada configuração na assinatura é formado de acordo com o **modelo de remark** – campo «**Modelo de observação**» (`remarkTemplate`) na aba «**Informações**» das configurações de assinatura. O antigo construtor de modelo de observação (seleção separada de partes de inbound/email/proxy externo e caractere separador) foi removido da interface; agora você escreve um formato de nome personalizado e insere variáveis nele. O valor padrão é `{{INBOUND}} | {{TRAFFIC_LEFT}} | {{DAYS_LEFT}}D`. Se o campo for deixado vazio, o modelo de remark anterior (não configurável pela interface) é usado.

As variáveis são agrupadas nas seções **Client**, **Traffic** e **Time & status** e são exibidas ao lado do campo como chips clicáveis `{{VAR}}` com dica ao passar o mouse; clicar insere o token no modelo, com pré-visualização ao vivo disponível. Cada variável é substituída individualmente para o cliente específico no momento da geração da assinatura. A notação simplificada entre chaves simples também é aceita (`{DATA_LEFT}`, `{EXPIRE_DATE}`, `{PROTOCOL}`, `{TRANSPORT}` etc.) – o painel a converte automaticamente para o formato interno `{{...}}`.

Variáveis disponíveis:

- **Identificação do cliente:** `{{EMAIL}}`, `{{INBOUND}}` (remark do próprio inbound), `{{HOST}}` (remark do host), `{{ID}}` (UUID), `{{SHORT_ID}}` (primeiros 8 caracteres do UUID), `{{SUB_ID}}`, `{{COMMENT}}`, `{{TELEGRAM_ID}}`, `{{PROTOCOL}}`, `{{TRANSPORT}}`.
- **Tráfego:** `{{TRAFFIC_USED}}`, `{{TRAFFIC_LEFT}}`, `{{TRAFFIC_TOTAL}}` (e suas variantes `*_BYTES` em bytes exatos), `{{UP}}`, `{{DOWN}}`, `{{USAGE_PERCENTAGE}}`.
- **Prazo e status:** `{{DAYS_LEFT}}`, `{{TIME_LEFT}}`, `{{EXPIRE_DATE}}` (AAAA-MM-DD), `{{JALALI_EXPIRE_DATE}}` (data no calendário jalali), `{{EXPIRE_UNIX}}`, `{{CREATED_UNIX}}`, `{{RESET_DAYS}}`, `{{STATUS}}` (active / expired / disabled / depleted), `{{STATUS_EMOJI}}`.

O modelo pode ser dividido em segmentos pela barra vertical `|`. Um segmento no qual a variável produz um valor «ilimitado» (∞) – por exemplo `{{TRAFFIC_LEFT}}` ou `{{DAYS_LEFT}}` para um cliente sem limites – é

automaticamente ocultado. Além disso, o bloco de uso de tráfego e prazo é exibido uma vez, no primeiro link do cliente, para não se repetir em cada configuração.

Exemplo. O modelo `{{EMAIL}} | {{TRAFFIC_LEFT}} | {{DAYS_LEFT}}D` dará para um cliente com 42 GB restantes e 7 dias um nome como `ivan@vpn 42.00GB 7D`, e para um cliente sem limite — apenas `ivan@vpn` (os segmentos com `∞` são omitidos). | Modelo de remark | `remarkTemplate | {{INBOUND}} | {{TRAFFIC_LEFT}} | {{DAYS_LEFT}}D` | Modelo livre do nome exibido (remark) de cada configuração com substituição de variáveis `{{VAR}}`. É aplicado individualmente a cada cliente na geração da assinatura. O antigo construtor de «modelo de observação» (seleção de inbound/email/proxy externo e separador) foi removido da interface e é usado apenas como alternativa de fallback se o campo for deixado vazio. Para mais detalhes, veja «Modelo de remark (Remark Template)» abaixo. |

Metadados do perfil (cabeçalhos de resposta)

Essas strings são transmitidas ao cliente nos cabeçalhos HTTP da resposta e são exibidas no cliente VPN como metadados do perfil. Todas são vazias por padrão.

Campo (UI)	Chave	Cabeçalho	Descrição
Título da assinatura	<code>subTitle</code>	<code>Profile-Title</code> (em Base64)	«Nome da assinatura visível para o cliente no cliente VPN». Para Clash, também é usado como nome do perfil importado via <code>Content-Disposition</code> .
URL de suporte	<code>subSupportUrl</code>	<code>Support-Url</code>	«Link para suporte técnico exibido no cliente VPN».
URL do perfil	<code>subProfileUrl</code>	<code>Profile-Web-Page-Url</code>	«Link para o seu site exibido no cliente VPN». Se não definido, a URL real da requisição de assinatura é usada.
Anúncio	<code>subAnnounce</code>	<code>Announce</code> (em Base64)	«Texto do anúncio exibido no cliente VPN».

Além disso, cada resposta inclui o cabeçalho `Subscription-Userinfo` com dados agregados de tráfego do cliente: `upload`, `download`, `total` e `expire` (momento de expiração em segundos). Com isso, o cliente exibe o tráfego restante e o prazo de validade.

Roteamento (apenas para o cliente Happ)

Campo (UI)	Chave	Padrão	Descrição
Habilitar roteamento	<code>subEnableRouting</code>	<code>false</code>	«Configuração global para habilitar o roteamento no cliente VPN. (Apenas para Happ)». Transmitido no cabeçalho <code>Routing-Enable</code> .
Regras de roteamento	<code>subRoutingRules</code>	vazio	«Regras de roteamento globais para o cliente VPN. (Apenas para Happ)». Transmitido no cabeçalho <code>Routing</code> .

| Ocultar configurações do servidor | `subHideSettings` | `false` | «Ocultar as configurações do servidor na assinatura (apenas para Happ)». Quando ativado, o cliente Happ oculta a possibilidade de visualizar e alterar os parâmetros do servidor. A opção funciona apenas para o cliente Happ. |

URI de proxy reverso

Campo (UI)	Chave	Padrão	Descrição
URI de proxy reverso	subURI	vazio	«Alterar o URI base da URL de assinatura para uso por trás de servidores proxy».

Se o campo estiver vazio, o painel constrói o endereço base do link a partir do domínio e porta de assinatura (levando em conta o TLS). Se a assinatura for distribuída por um proxy reverso/CDN externo em outro domínio ou caminho, o URI base final é definido nesse campo, e todos os links serão construídos a partir dele. Campos separados equivalentes existem para JSON (subJsonURI) e Clash (subClashURI).

Se apenas o subURI geral for definido, e os campos individuais para JSON e Clash forem deixados vazios, os links desses formatos na página de assinatura herdam o esquema e o host do subURI (e não a porta do servidor sub e http) – assim eles coincidem com o endereço do proxy reverso.

Exemplo: assinatura por trás de um proxy reverso. A assinatura em si escuta na porta 2096, mas externamente está disponível via nginx/CDN em https://cfg.example.com/u/. Para que os links na resposta sejam construídos a partir do endereço externo, e não do interno domínio:2096, defina o URI base final no campo «Reverse proxy URI»:

```
Reverse proxy URI: https://cfg.example.com/u
```

Então o link final terá o formato https://cfg.example.com/u/ivan2025. Para os formatos JSON e Clash, se necessário, preencha os campos separados subJsonURI e subClashURI da mesma forma.

10.3. Formatos de saída

A assinatura pode ser entregue em três formatos independentes, cada um com seu próprio endpoint, que pode ser habilitado/desabilitado separadamente.

Endereço do servidor e nós na resposta

O endereço do servidor nos links de assinatura é substituído pela mesma estratégia de endereço no link usada para links comuns e QR-codes no painel: «listen» – endereço de vinculação roteável, «custom» – endereço personalizado definido pelo usuário (shareAddr), «node» (padrão) – endereço do nó. Para inbounds sem estratégia explicitamente definida, a saída da assinatura não muda. Isso permite que um inbound de nó, vinculado a um IP público específico, entregue aos clientes um endereço acessível. A estratégia se aplica aos formatos raw, JSON e Clash.

O nome do nó (Node) não é adicionado ao nome (remark) do perfil na assinatura: no aplicativo cliente é exibido apenas o remark do inbound definido pelo administrador, sem o sufixo interno como @nome-do-nó. Para distinguir entradas com o mesmo nome em uma assinatura multi-nó, defina remarks diferentes manualmente ou use hosts gerenciados (Hosts) com seus próprios Remark.

Se, devido a uma dessincronização entre nós, o mesmo cliente acabar no inbound JSON de serviço duas vezes, a saída da assinatura elimina automaticamente esses duplicados por email em todos os três formatos, de modo que perfis repetidos não aparecem na saída.

Hosts gerenciados (Hosts)

A seção **Hosts** (item do menu lateral; página de resumo com quantidade Total/Enabled/Disabled e lista) define substituições de endereço para links de assinatura. Para cada inbound, é possível adicionar um ou mais **hosts** – endpoints que são substituídos nos links de assinatura entregues ao cliente **no lugar do endereço, porta e parâmetros TLS do próprio inbound**. Isso é conveniente para distribuir tráfego via CDN ou relay sem modificar o inbound em si.

Cada host possui:

- **Remark** e descrição (Description), vinculação ao **Inbound** específico, interruptor **Enable** e atribuição a nós (**Nodes**).
- **Address** (vazio – herda o endereço do inbound) e **Port** (0 – herda a porta do inbound); **Tags** (consideradas apenas na assinatura RAW).
- Aba **Security** – same / tls / none / reality com SNI, fingerprint, ALPN, certificado fixado (pinned-cert), allowInsecure e ECH.
- Aba **Advanced** – cabeçalho Host, Path, rota VLESS, Mux, Sockopt, Final Mask e exclusão do host de formatos específicos de assinatura (raw / json / clash).
- Aba **Clash (mihomo)** – versão IP, Mihomo X25519, embaralhamento de hosts (Shuffle host).

Os hosts são ordenados dentro do seu inbound e suportam habilitação, desabilitação e exclusão em massa. Os hosts gerenciados substituem o antigo array External Proxy.

Links comuns (SUB) – Base64 / texto simples

Formato básico, endpoint `subPath` (padrão `/sub/`). Sempre habilitado (quando a assinatura está habilitada globalmente). Retorna uma lista de links Xray (`vless://`, `vmess://`, `trojan://`, `ss://` etc.) – um por linha. Com a opção «Codificar» (`subEncrypt`) ativada, toda a lista é codificada em Base64; quando desativada, é entregue em texto simples. Esse formato é suportado por praticamente todos os clientes (v2rayNG, V2RayTun, Sing-box, NekoBox, Streisand, Shadowrocket, Happ e outros).

Exemplo: corpo da resposta com «Codificar» desativado. Com `subEncrypt = false`, o endpoint `/sub/` entrega texto simples – um link por linha:

```
vless://3c8f...@a.example.com:443?security=reality&...#srvA-ivan
trojan://p4ss@b.example.com:443?security=tls&...#srvB-ivan
```

Com `subEncrypt = true` (padrão), a mesma lista é inteiramente codificada em Base64 e entregue como uma única string – exatamente o formato esperado pela maioria dos clientes.

Assinatura JSON (sing-box e compatíveis)

Endpoint `subJsonPath` (padrão `/json/`), habilitado por uma opção separada.

Campo (UI)	Chave	Padrão	Descrição
Assinatura JSON	<code>subJsonEnable</code>	<code>false</code>	«Habilitar/desabilitar o endpoint JSON da assinatura de forma independente.».

Retorna uma configuração JSON completa (formato compatível com sing-box e clientes derivados – Podkop, OpenWRT sing-box, Karing, NekoBox). Para esse formato, parâmetros adicionais estão disponíveis (aba `subFormats`):

- **Mux** (`subJsonMux` , padrão vazio) – configurações JSON de multiplexação (Mux) que são injetadas no outbound de cada fluxo de assinatura JSON. «Transmissão de vários fluxos de dados independentes em uma única conexão.».
- **Final Mask** (`subJsonFinalMask` , padrão vazio) – «Máscaras finalmask xray (TCP/UDP) e configurações QUIC adicionadas a cada fluxo de assinatura JSON. Requer uma versão recente do xray no cliente.».
Configurado através de subcampos: «Pacotes» (`packets`), «Comprimento» (`length`), «Intervalo» (`interval`), «Divisão máx.» (`maxSplit`), «Ruídos» (`noises` : «Tipo»/ `type` , «Pacote»/ `packet` , «Atraso (ms)»/ `delayMs` , «Aplicar a»/ `applyTo` , botão «+ Ruído»), além de «Simultaneidade» (`concurrency`), «Simultaneidade xudp» (`xudpConcurrency`) e «xudp UDP 443» (`xudpUdp443`).
- **Regras de roteamento** (`subJsonRules` , padrão vazio) – regras globais adicionadas à configuração JSON.

Assinatura Clash / Mihomo (YAML)

Endpoint `subClashPath` (padrão `/clash/`), habilitado por uma opção separada.

Campo (UI)	Chave	Padrão	Descrição
Assinatura Clash / Mihomo	<code>subClashEnable</code>	<code>false</code>	Habilita a geração de configuração YAML para clientes Clash e Mihomo.
Habilitar roteamento	<code>subClashEnableRouting</code>	<code>false</code>	«Adicionar regras de roteamento globais Clash/Mihomo às assinaturas YAML geradas.».
Regras de roteamento globais	<code>subClashRules</code>	vazio	«Regras Clash/Mihomo adicionadas ao início de cada assinatura YAML antes de MATCH,PROXY.».

A resposta é entregue com o tipo `application/yaml; charset=utf-8` . Se o «Título da assinatura» (`subTitle`) estiver definido, ele também é transmitido no cabeçalho `Content-Disposition` (`attachment; filename*=UTF-8''<title>`), para que o cliente Clash nomeie o perfil importado com esse nome.

O formato dos links e YAML gerados é mantido atualizado para clientes modernos: Shadowsocks-2022 (SS2022) não codifica mais userinfo em Base64; links Shadowsocks com ofuscação http são entregues no formato SIP002 com o plugin `obfs-local` ; para assinaturas Clash/Mihomo, um conjunto completo de campos XHTTP está implementado. Nenhuma configuração adicional é necessária – os links simplesmente são reconhecidos de forma mais correta pelos clientes.

Observação: nesta versão, são suportados exatamente três formatos – links comuns (Base64/texto), JSON (compatível com sing-box) e Clash/Mihomo (YAML). Não há um formato Outline separado no servidor de assinaturas.

10.4. Página de informações da assinatura e QR-codes

Se você abrir o link da assinatura em um navegador (ou adicionar explicitamente o parâmetro `?html=1` ou `?view=html` à URL, ou enviar o cabeçalho `Accept: text/html`), o servidor, em vez da resposta «bruta»,

entrega uma **página visual de informações da assinatura** («Informações da assinatura»). Os clientes VPN continuam recebendo a resposta em formato legível por máquina, pois não solicitam HTML.

A página (aplicativo de página única construído com Vite) exibe:

- **Informações sobre a assinatura** (bloco Descriptions):
 - «ID da assinatura» – valor de `subId` ;
 - «Status» – «Ativa», «Inativa» ou «Ilimitado». O status «inativa» é definido se o cliente estiver desabilitado, tiver esgotado o limite de tráfego ou o prazo tiver expirado;
 - «Download» e «Upload» – volumes de tráfego;
 - «Limite total» – limite de tráfego ou ∞ se não houver limite;
 - «Prazo de validade» – data de expiração ou «Sem prazo»;
 - tráfego restante e hora do último acesso online.
 - As datas são exibidas no calendário gregoriano ou jalali, dependendo da configuração «Calendar Type» do painel (`datepicker` , padrão `gregorian`).
- **Links da assinatura**: para cada formato habilitado – uma linha separada com tag colorida (verde **SUB**, roxo **JSON**, dourado **CLASH**), botão de cópia e botão de **QR-code** (janela pop-up, tamanho 240 px). A linha com JSON e CLASH aparece apenas se o formato correspondente estiver habilitado nas configurações.
- **Links individuais** («Copiar link»): lista completa das configurações individuais incluídas na assinatura, cada uma com sua tag de protocolo, botão de cópia e QR-code (para links post-quantum, o QR-code não é gerado).
- **Botão «Copiar todas as configurações»** (acima da lista de links individuais): com um clique, copia para a área de transferência todos os links de configuração de uma vez (cada um em uma nova linha), sem a necessidade de copiá-los um a um; após a conclusão, é exibida a notificação «Todas as configurações copiadas».
- **Botões de importação rápida em aplicativos** (menus suspensos por plataforma): para Android – `v2box`, `v2rayNG` (deep-link `v2rayng://install-config?url=...`), `Sing-box`, `V2RayTun`, `NPV Tunnel`, `Happ` (`happ://add/...`); para iOS – `Shadowrocket` (via parâmetro `flag=shadowrocket`), `v2box` (`v2box://install-sub?url=...&name=...`), `Streisand` (`streisand://import/...`), `V2RayTun`, `NPV Tunnel`, `Happ`. Esses botões abrem o deep-link do aplicativo desejado com o endereço da assinatura já preenchido, ou copiam o link para a área de transferência.

A página de informações é entregue com cabeçalhos de desativação de cache (`Cache-Control: no-cache`), para que o cliente sempre veja os dados atualizados de tráfego e prazo de validade.

10.5. Modelos personalizados da página de assinatura

A partir da versão 3.3.0, é possível substituir a página de landing padrão da assinatura por um template HTML próprio. Por padrão, o endereço da assinatura entrega a página integrada, mas se um diretório com seu próprio template for especificado, o painel o renderizará e inserirá nele os dados atualizados do cliente (tráfego, prazo de validade, links etc.).

Importante: o painel **não fornece** templates prontos. No repositório há apenas o diretório `sub_templates/` com um arquivo de instruções `sub_templates/README.md` ; o seu próprio tema precisa ser criado do zero.

Onde é habilitado

O diretório do tema é definido nas configurações do painel:

Configurações → **Assinatura** → seção «Informações da assinatura», campo «Diretório do tema da assinatura» (`subThemeDir`).

Descrição do campo na interface: «Caminho absoluto para a pasta com o template personalizado (index.html/sub.html) para a página de assinatura (por exemplo, /etc/3x-ui/sub_templates/my-theme/). Deixe vazio para usar a página padrão.»

No mesmo bloco estão as configurações relacionadas, cujos valores estão disponíveis no template:

Na descrição do campo «Diretório do tema da assinatura» há um link «**Guia de templates** ↗» para a documentação sobre como criar seus próprios templates de estilo para a página de assinatura.

- «**Título da assinatura**» (`subTitle`) – nome visível ao cliente;
- «**URL de suporte**» (`subSupportUrl`) – link para suporte técnico.

Parâmetro de configuração

Parâmetro	Valor padrão	Finalidade
<code>subThemeDir</code>	"" (vazio)	Caminho absoluto para o diretório com seu template HTML. Vazio = página padrão integrada.

Como inserir seu próprio template

1. Crie no servidor uma pasta para o tema (em qualquer lugar), por exemplo `/etc/3x-ui/sub_templates/my-theme/`.
2. Coloque dentro um arquivo HTML com o nome `index.html` ou `sub.html`.

Exemplo: caminho para o tema. Layout final no servidor e valor do campo nas configurações:

```
/etc/3x-ui/sub_templates/my-theme/
└─ index.html      (ou sub.html – ele tem prioridade)
```

```
Configurações → Assinatura → «Diretório do tema da assinatura»:
/etc/3x-ui/sub_templates/my-theme/
```

O caminho deve ser **absoluto** (começar com `/`). Se a pasta não contiver nem `index.html` nem `sub.html`, o painel entregará a página integrada. 3. No painel, abra **Configurações** → **Assinatura** e insira o caminho **absoluto** para essa pasta no campo «Diretório do tema da assinatura». 4. Salve as configurações.

Comportamento de seleção de arquivo e renderização:

- Se o diretório contiver `sub.html`, ele será usado; caso contrário, `index.html` será utilizado. Ou seja, `sub.html` tem prioridade sobre `index.html`.

- O template é renderizado pelo motor padrão do Go `html/template`.
- O template analisado é **armazenado em cache** e relido do disco apenas quando o tempo de modificação do arquivo mudar. Portanto, edições no template são aplicadas sem reiniciar o painel, mas sem a sobrecarga de leitura/análise a cada requisição.
- A resposta é formada em um buffer completo e só então entregue ao cliente: se o template falhar durante a execução, uma página parcialmente gerada (corrompida) não será enviada ao usuário.

Comportamento padrão e fallback

- Campo vazio → a página SPA integrada é entregue (os dados são injetados em `window.__SUB_PAGE_DATA__`).
- O caminho não existe ou não é um diretório → a página padrão é usada.
- O diretório não contém nem `index.html` nem `sub.html` → o aviso «subThemeDir set but no usable template found» é registrado no log, e a página padrão é entregue.
- O arquivo de template existe, mas não pode ser analisado → o erro «custom template parse failed» é registrado no log, e a página padrão é entregue.
- Erro durante a execução do template → «custom template execution failed» é registrado no log, e a página padrão é entregue.

Ou seja, qualquer problema com o template personalizado não «quebra» a assinatura — o painel sempre retorna à página integrada. Todas as páginas de assinatura (tanto a personalizada quanto a padrão) são entregues com cabeçalhos de desativação de cache (`Cache-Control: no-cache, no-store, must-revalidate`), para que os clientes sempre recebam dados atualizados de tráfego e prazo.

Variáveis disponíveis no template

O contexto do template recebe um conjunto de dados do cliente da assinatura. O acesso é feito via `{{ .nome }}`:

Variável	Tipo	Descrição
<code>{{ .sId }}</code>	string	ID da assinatura (UUID).
<code>{{ .enabled }}</code>	bool	Se o cliente/assinatura está habilitado.
<code>{{ .download }}</code>	string	Volume de download formatado (ex.: «2.5 GB»).
<code>{{ .upload }}</code>	string	Volume de upload formatado.
<code>{{ .total }}</code>	string	Limite total de tráfego formatado.
<code>{{ .used }}</code>	string	Tráfego usado formatado (download + upload).
<code>{{ .remained }}</code>	string	Tráfego restante formatado.
<code>{{ .expire }}</code>	int64	Prazo de validade — Unix timestamp em segundos (<code>0</code> = sem prazo). Para <code>Date</code> do JS, multiplique por 1000.
<code>{{ .lastOnline }}</code>	int64	Hora do último acesso online — Unix timestamp em milissegundos (<code>0</code> = nunca).
<code>{{ .downloadByte }}</code>	int64	Download em bytes exatos.
<code>{{ .uploadByte }}</code>	int64	Upload em bytes exatos.

Variável	Tipo	Descrição
<code>{{ .totalByte }}</code>	int64	Limite total em bytes exatos.
<code>{{ .subUrl }}</code>	string	URL da página de assinatura.
<code>{{ .subJsonUrl }}</code>	string	URL da configuração JSON da assinatura.
<code>{{ .subClashUrl }}</code>	string	URL da configuração Clash/Mihomo.
<code>{{ .subTitle }}</code>	string	Título da assinatura das configurações (pode estar vazio).
<code>{{ .subSupportUrl }}</code>	string	URL de suporte das configurações (pode estar vazio).
<code>{{ .links }}</code>	[]string	Lista de strings de configuração (VMess, VLESS etc.). Iteração: <code>{{ range .links }}</code> ... <code>{{ end }}</code> .
<code>{{ .emails }}</code>	[]string	Lista de emails pertencentes à assinatura.
<code>{{ .datepicker }}</code>	string	Formato de calendário atual do painel: <code>gregorian</code> ou <code>jalali</code> (obtido da configuração «Tipo de calendário»; se vazio — <code>gregorian</code>).

Exemplo mínimo do corpo do template usando algumas variáveis:

```
<h1>{{ .subTitle }}</h1>
<p>Usado: {{ .used }} de {{ .total }} (restante {{ .remained }})</p>
{{ range .links }}<div>{{ . }}</div>{{ end }}

**Exemplo: data de expiração a partir de `expire`.** O campo `{{ .expire }}` é um Unix
timestamp em **segundos**, portanto para JavaScript deve-se multiplicar por 1000; o
valor `0` significa «sem prazo»:

```html
<script>
 var exp = {{ .expire }};
 document.write(exp === 0
 ? 'Sem prazo'
 : 'Até ' + new Date(exp * 1000).toLocaleDateString());
</script>
```

Observe que `{{ .lastOnline }}` já é fornecido em **milissegundos** — não é necessário multiplicar por 1000.

----

## ## 11. Xray: roteamento, outbounds, DNS e extensões

A seção **"Configurações do Xray"** é um editor de modelo de configuração do Xray-core, com base no qual o painel gera o `config.json` final para iniciar o núcleo. A dica da seção de modelo: **"O arquivo de configuração do Xray é criado com base no modelo."** Ao contrário dos inbounds (que são armazenados separadamente no banco de dados e inseridos no modelo durante a montagem da configuração), todo o restante – logs, roteamento, outbounds, DNS, políticas, estatísticas – é definido aqui.

> Importante: o valor do modelo é armazenado no banco de dados com a chave `xrayTemplateConfig`. Ao salvar, o painel aplica uma série de transformações automáticas (consulte [11.10](#1110-salvamento-reinicialização-e-transformações-automáticas)). Qualquer JSON com sintaxe inválida será rejeitado com o erro `"xray template config invalid"`.

### #### Localização no menu: "Saídas" e "Roteamento"

**"Saídas" (Outbounds)** e **"Roteamento" (Routing)** são itens separados no menu lateral (imediatamente abaixo de "Hosts", acima de "Configurações do Painel"), cada um com seu próprio endereço: `/outbound` e `/routing`. Links diretos para essas páginas e o recarregamento da página funcionam como esperado. No submenu **"Configurações do Xray"** permanecem apenas: Principal, Balanceador, DNS e Modelo Avançado. Na descrição abaixo, as seções [11.3](#113-regras-de-roteamento-routing) e [11.4](#114-outbounds-conexões-de-saída) correspondem às páginas "Roteamento" e "Saídas".

### ### 11.1. Estrutura do editor: abas/modos

O editor oferece vários modos de exibição do modelo (filtros por seções JSON):

Modo	O que exibe
Principal	Seções básicas (Log, roteamento básico, configurações principais)
Modelo Avançado	JSON completo do modelo Xray
Todos	Todas as seções simultaneamente

Grupos lógicos de configurações dentro do editor:

- **Configurações principais** (dica: **"Esses parâmetros descrevem as configurações gerais"**).
- **Log** (consulte [11.9](#119-logs-e-estatísticas-stats-metrics)).
- **Conexões básicas**: bloqueios e rotas diretas.
- **Entradas** (dica: **"Alterar o modelo de configuração para conectar determinados clientes"**).
- **Saídas** (consulte [11.4](#114-outbounds-conexões-de-saída)).
- **Balanceador** (consulte [11.5](#115-balanceadores-balancers)).
- **Roteamento** (dica: **"A prioridade de cada regra é importante!"**, consulte [11.3](#113-regras-de-roteamento-routing)).
- **DNS / Fake DNS** (consulte [11.6](#116-dns)).

### ### 11.2. Configurações principais (General)

#### #### Freedom Protocol Strategy

Campo	Rótulo	Descrição	Padrão
-------	--------	-----------	--------

```
|---|---|---|---|
| `FreedomStrategy` | **Configuração da estratégia do protocolo Freedom** | Estratégia
de saída de rede para o outbound direto (freedom). Dica: *"Configuração da estratégia de
saída de rede no protocolo Freedom"*. Controla o campo `domainStrategy` dentro de
`settings` do outbound com protocolo `freedom`. | No modelo de referência,
`domainStrategy` para o freedom-outbound `direct` é **`AsIs`** (o endereço não é
resolvido, é passado na forma original). |
```

`domainStrategy` para freedom (valores do Xray-core): `AsIs` (não resolve o domínio no lado do servidor), bem como a família `UseIP` / `UseIPv4` / `UseIPv6` e suas variantes "forçadas" `ForceIP\*`, que obrigam o servidor de saída a resolver o domínio e conectar-se ao IP obtido. Mude para `UseIPv4` se o servidor de saída não tiver IPv6 ou se precisar forçar o uso apenas de IPv4.

#### #### Freedom Happy Eyeballs (IPv4/IPv6)

```
| Campo | Rótulo | Descrição |
|---|---|---|
| `FreedomHappyEyeballs` | **Freedom Happy Eyeballs (IPv4/IPv6)** | Dica: *"Conexão de
pilha dupla para saída direta (freedom) – útil em servidores de saída com IPv4 e IPv6."*
Ativa o algoritmo Happy Eyeballs (tentativa simultânea em ambas as famílias de
endereços) para o freedom-outbound. |
| try delay | (dica) | *"Milissegundos antes de tentar outra família de endereços. 150–
250 ms é um bom ponto de partida."* Atraso antes de mudar para a família de endereços
alternativa. O intervalo recomendado é de 150–250 ms. |
```

#### #### Overall Routing Strategy

```
| Campo | Rótulo | Descrição | Padrão |
|---|---|---|---|
| `RoutingStrategy` | **Configuração de roteamento de domínios** | Estratégia geral de
resolução DNS para roteamento. Dica: *"Configuração da estratégia geral de roteamento de
resolução DNS"*. Controla o campo `routing.domainStrategy`. | No modelo de referência,
`routing.domainStrategy` = **`AsIs`**.
```

`routing.domainStrategy` define como as regras de roteamento de IP são combinadas com solicitações de domínio: `AsIs` (apenas regras de domínio, sem resolução), `IPIfNonMatch` (se o domínio não corresponder às regras – resolver e verificar regras de IP), `IPOnDemand` (resolver imediatamente ao encontrar uma regra de IP). Para que as regras de IP (por exemplo, `geoip:\*`) funcionem com solicitações de domínio, normalmente é necessário `IPIfNonMatch`.

#### #### Outbound Test URL

```
| Campo | Rótulo | Descrição | Padrão |
|---|---|---|---|
| `outboundTestUrl` | **URL para teste de saída** | URL para verificar a conectividade
ao testar outbound. Dica: *"URL para verificar a conexão de saída"*. Armazenado
separadamente do modelo, com a chave `xrayOutboundTestUrl`. | **`https://www.google.com/
generate_204`** |
```

O valor passa por sanitização. Durante o teste de outbound, ele é verificado adicionalmente como uma URL pública – isso é uma proteção contra SSRF: o usuário não pode fornecer uma URL arbitrária (incluindo interna) através do cliente; a URL de teste é sempre obtida das configurações do servidor. Um valor vazio ao salvar/testar é substituído pelo `generate\_204` padrão.

#### #### Block BitTorrent

Campo	Rótulo	Descrição
`Torrent`	**Bloquear BitTorrent**	Adiciona a `routing.rules` uma regra que envia o tráfego com `protocol: ["bittorrent"]` para o outbound `blocked`. No modelo de referência, essa regra está presente por padrão.

#### #### Limites de conexão (Connection Limits)

Dica: \*"Políticas de nível de conexão para usuários de nível 0. Deixe o campo vazio para usar o valor padrão do Xray."\* Esses parâmetros são gravados em `policy.levels.0`.

Campo	Rótulo	Descrição	Padrão
`connIdle`	**Tempo limite de inatividade** (segundos)	*"Fecha a conexão após inatividade pelo número especificado de segundos. Reduzir o valor libera memória e descritores de arquivo mais rapidamente em servidores com carga alta (padrão do Xray: 300)."	vazio → padrão do Xray **300**
`bufferSize`	**Tamanho do buffer** (KB)	*"Tamanho do buffer interno por conexão em KB. Defina como 0 para minimizar o uso de memória em servidores com pouca RAM (o valor padrão do Xray depende da plataforma)." Placeholder: **"auto"**.   vazio → depende da plataforma; `0` – minimizar	

\*\*Exemplo (`policy.levels.0`)\*\*. Os campos desse grupo são gravados na política de nível 0. Em um servidor com alta carga e pouca RAM, você pode acelerar a liberação de recursos assim:

```
```json
{
  "policy": {
    "levels": {
      "0": {
        "connIdle": 120,
        "bufferSize": 0
      }
    }
  }
}
```

Aqui a conexão é encerrada após 120 s de inatividade (em vez dos 300 padrão), e `bufferSize: 0` minimiza o consumo de memória em buffers. Um campo deixado vazio no formulário simplesmente não entrará no JSON – e o Xray aplicará seu valor padrão.

11.3. Regras de roteamento (routing)

Lista de regras `routing.rules`. **A ordem é crítica** ("A prioridade de cada regra é importante!"): as regras são avaliadas de cima para baixo, a primeira correspondência é aplicada. Dica: "Arraste para alterar a ordem". Botões de controle de ordem: **Primeiro**, **Último**, **Mover para cima**, **Mover para baixo**.

Cada regra tem `type: "field"`. Botões: **Criar regra**, **Editar regra**. Dica para campos de lista: "Itens separados por vírgula".

Na página "Roteamento", os botões **"Importar regras"** e **"Exportar regras"** estão agrupados no menu suspenso **"mais"** (more) – assim como na página "Saídas". O botão **"Exportar regras"** não baixa o arquivo imediatamente, mas abre uma janela modal com pré-visualização do JSON e os botões **"Copiar"** e **"Baixar"**: o conteúdo pode ser revisado antes de salvar. A exportação de saídas na página "Saídas" funciona da mesma forma.

Route Tester (testador de rota)

Na aba Routing há uma subaba **Route Tester** — ela pergunta ao Xray em execução qual outbound processaria uma conexão específica, sem enviar tráfego real. Informe o domínio ou IP, porta, rede (TCP/UDP) e, se necessário, o inbound e o protocolo interceptado (`http` / `tls` / `quic` / `bittorrent`), em seguida clique em **Test Route**. A decisão é obtida diretamente do mecanismo de roteamento ativo.

A resposta mostra o outbound selecionado e, ao usar um balanceador, também a tag do balanceador. Se nenhuma regra corresponder, o testador informa que o tráfego vai para o outbound padrão (o primeiro na lista `outbounds`). Isso é útil para verificar a ordem das regras antes de depender delas.

Ativação e desativação de uma regra individual

Uma regra de roteamento individual pode ser temporariamente **desativada** por um interruptor, sem excluí-la. Na tabela de regras há uma coluna **"Ativar"** com um interruptor (Switch), e no formulário de regra o campo **"Ativar"** — também um interruptor. Uma regra desativada não entra na configuração final do Xray, mas é salva no modelo e pode ser reativada a qualquer momento.

A regra de serviço de estatísticas (`inboundTag: ["api"]` → `outboundTag: "api"`) não pode ser desativada — seu interruptor está bloqueado para não quebrar a contabilidade de tráfego do painel (consulte [11.10](#)).

Campos do formulário de regra

Campo do formulário	Rótulo	Campo JSON	Descrição
Origem	Origem	<code>source</code>	Endereços IP/sub-redes de origem. Lista separada por vírgula.
Porta de origem	Porta de origem	<code>sourcePort</code>	Porta(s) de origem.
Destino	Destino	<code>domain + ip + port</code>	Domínios, IPs e portas de destino. Domínios suportam prefixos <code>domain:</code> , <code>full:</code> , <code>regexp:</code> , <code>keyword:</code> , bem como <code>geosite:*</code> ; IPs — <code>geoip:*</code> e CIDR.
Rede	—	<code>network</code>	<code>tcp</code> , <code>udp</code> ou <code>tcp,udp</code> .
Protocolo	—	<code>protocol</code>	<code>http</code> , <code>tls</code> , <code>bittorrent</code> (determinado por sniffing).
Usuário	Usuário	<code>user</code>	Filtro por e-mail/identificador do usuário.
Atributos / Valor	Atributos / Valor	<code>attrs</code>	Atributos de cabeçalhos HTTP para correspondência.
VLESS route	VLESS route	—	Roteamento pelo campo route para VLESS.
Tags de entradas	Tags de entradas	<code>inboundTag</code>	Uma ou mais tags de inbound às quais a regra se aplica (incluindo as internas <code>api</code> e a tag DNS das configurações de DNS). Nas listas de inbound é exibido como "tag (remark)" se o inbound tiver uma observação separada, caso contrário — apenas a tag; nas regras salvas, apenas as tags continuam sendo armazenadas.

Campo do formulário	Rótulo	Campo JSON	Descrição
Tag de saída	Tag de saída / Conexão de saída	<code>outboundTag</code>	Para onde direcionar o tráfego correspondente.
Tag do balanceador	Tag do balanceador / Balanceador	<code>balancerTag</code>	Dica: "Direciona o tráfego por meio de um dos balanceadores de carga configurados".

Exclusão mútua entre `outboundTag` e `balancerTag`: "Não é possível usar `balancerTag` e `outboundTag` ao mesmo tempo. Se usados simultaneamente, apenas o `outboundTag` funcionará." Em uma regra, defina a tag de saída ou a tag do balanceador, nunca ambas.

Regras integradas do modelo de referência

No `config.json` padrão, a seção `routing` contém três regras (nessa ordem):

1. `inboundTag: ["api"] → outboundTag: "api"` – regra de serviço para a API gRPC de estatísticas do painel.
2. `ip: ["geoip:private"] → outboundTag: "blocked"` – bloqueio de intervalos privados.
3. `protocol: ["bittorrent"] → outboundTag: "blocked"` – bloqueio de BitTorrent.

A regra `api → api` é sempre automaticamente promovida para a posição 0 ao salvar (consulte [11.10](#)), para que a solicitação de estatísticas não seja "capturada" por uma regra catch-all superior.

Exemplo de regra. Enviar todo o tráfego para sites russos e redes privadas diretamente (sem proxy), e o restante para um balanceador. A ordem importa: a regra "direcionar diretamente" deve estar acima do catch-all. Em `routing.rules`:

```
{
  "type": "field",
  "domain": ["geosite:category-ru", "domain:example.ru"],
  "ip": ["geoip:ru", "geoip:private"],
  "outboundTag": "direct"
}
```

Para que as regras de IP (`geoip:ru`) funcionem também para solicitações de domínio, normalmente é necessário `routing.domainStrategy: "IPIfNonMatch"` no nível superior de roteamento (consulte [11.2](#)).

Grupos de roteamento pré-configurados (Conexões básicas)

No modo "Conexões básicas", o painel ajuda a montar regras típicas a partir de listas prontas:

Grupo	Campos	Dica
Bloqueio por protocolos/sites	—	"Configure para que os clientes não tenham acesso a determinados protocolos"
Bloqueio por países	IPs bloqueados, Domínios bloqueados	"Esses parâmetros bloquearão o tráfego dependendo do país de destino."
Conexões diretas	IPs diretos, Domínios diretos	"Conexão direta significa que determinado tráfego não será redirecionado por outro servidor."
Regras IPv4	—	"Esses parâmetros permitirão que os clientes sejam roteados para os domínios de destino apenas por IPv4"
Regras WARP	—	"Essas opções direcionarão o tráfego dependendo do destino específico via WARP"
Roteamento NordVPN	—	"Essas opções direcionarão o tráfego dependendo do destino específico via NordVPN."

MTProto-inbound: roteamento do tráfego do Telegram pelo Xray

O MTProto-inbound tem um interruptor **"Route through Xray"** (desativado por padrão) e uma seleção opcional de **Outbound**. Quando ativado, o painel adiciona uma ponte SOCKS de loopback com a tag do próprio inbound à configuração do Xray, e o mtg direciona o tráfego do Telegram por ela. Depois disso, o tráfego de saída do Telegram é controlado pelo roteador: pode ser correspondido por regras comuns na aba Routing pela tag de inbound ou forçosamente direcionado para o outbound ou balanceador selecionado por meio do campo **Outbound**. Deixe **Outbound** vazio para que as regras de roteamento tomem a decisão.

11.4. Outbounds (conexões de saída)

Lista de outbounds. Botões: **Criar conexão de saída, Editar conexão de saída**. Dica: "Alterar o modelo de configuração para definir as conexões de saída deste servidor".

No modelo de referência existem dois outbounds obrigatórios:

- `protocol: "freedom", tag: "direct"` — saída direta para a internet (com `domainStrategy: "AsIs"` e `finalRules: [{action: "allow"}]`);
- `protocol: "blackhole", tag: "blocked"` — "buraco negro" para o tráfego bloqueado.

Campos gerais do formulário de outbound

Campo	Rótulo	Descrição
Tag	Tag (dica: "Tag única")	Identificador único do outbound. Placeholder: "tag-única". Validação: "A tag é obrigatória", "A tag já está sendo usada por outra saída".
Protocolo	—	Tipo de saída (veja abaixo).
Endereço / Porta	Endereço / Porta	Destino da conexão. Endereço e porta são obrigatórios.
Enviar via	Enviar via	Endereço IP local da interface de saída (<code>sendThrough</code>). Placeholder: "IP local".

Campo	Rótulo	Descrição
Dialer proxy (cadeia)	—	Dica: "Conecte esta saída por meio de outra saída (pela tag) para construir uma cadeia de proxy. Deixe vazio para conexão direta." Placeholder: "Selecione a saída para encadeamento". Implementado via <code>streamSettings.sockopt.dialerProxy</code> .

Protocolos de outbound suportados

Protocolos suportados pelo formulário:

- **freedom** — saída direta. Campos `settings.domainStrategy`, `finalRules` (veja abaixo), Happy Eyeballs. Não pode ser testado ("Outbound has no testable endpoint").
- **blackhole** — descarta o tráfego. Campo **Tipo de resposta**. Não pode ser testado.
- **socks**, **http** — lista `settings.servers[]` com `address / port`; campo **Senha de autorização**.
- **vmess** — `settings.vnext[]` (`address / port`).
- **vless** — `settings.address / settings.port`.
- **trojan**, **shadowsocks** — `settings.servers[]`.
- **wireguard** — `settings.peers[]` com `endpoint`, além de chaves (consulte 11.7).
- **hysteria** — `settings.address / settings.port` (transporte UDP).

Para outbound do tipo **loopback** está disponível o bloco **Sniffing** com os mesmos parâmetros que o inbound: ativação, **destOverride**, **Metadata Only**, **Route Only** e lista de **domínios excluídos**.

Na máscara **UDP** (FinalMask) para **Hysteria2** estão disponíveis modos adicionais. A máscara **Salamander** tem um seletor **Mode** com os valores **Salamander** e **Gecko**: o modo Gecko adiciona preenchimento aleatório de pacotes com campos **Min/Max** de tamanho (`packetSize`, intervalo 1–2048, padrão 512–1200) — isso protege contra fingerprinting pelo comprimento dos pacotes. A máscara **Realm** (UDP hole-punching) recebeu um bloco opcional **TLS Config** com os campos **Server Name** (SNI), **ALPN** (`h3 / h2 / http/1.1`), **Fingerprint** (uTLS) e o interruptor **Allow Insecure**.

Exemplo: encadeamento via SOCKS upstream. O outbound `upstream` conecta-se a um proxy SOCKS5 externo, enquanto `chained` envia seu tráfego por ele (`dialerProxy`), formando uma cadeia. Em `outbounds`:

```
[
  {
    "tag": "upstream",
    "protocol": "socks",
    "settings": {
      "servers": [{ "address": "203.0.113.10", "port": 1080 }]
    }
  },
  {
    "tag": "chained",
    "protocol": "freedom",
    "streamSettings": {
      "sockopt": { "dialerProxy": "upstream" }
    }
  }
]
```

Agora uma regra de roteamento com `outboundTag: "chained"` enviará o tráfego para a internet por meio de `upstream`.

Importar outbound de um link de compartilhamento

Um outbound pode ser importado de um link de compartilhamento (`vless://`, `vmess://` etc.). Durante a importação, as configurações do multiplexador **xmux** (XHTTP) transmitidas no bloco `extra=` do link também são salvas: após a importação, seus valores são preenchidos no subformulário **XMUX** do outbound criado.

Campos Mux (multiplexação)

Máx. paralelismo, Máx. conexões, Máx. reutilizações, Máx. solicitações, Máx. segundos de reutilização, Período keep alive. Esses parâmetros configuram o comportamento mux/XUDP da saída.

Sockopts (configurações de socket)

Grupo **Sockopts**: **Intervalo keep alive, Mark (fwmark), Interface, Apenas IPv6, Aceitar proxy protocol, Proxy protocol, TCP user timeout (ms), TCP keep-alive idle (s).** Aqui também é definido o dialer-proxy da cadeia.

Freedom finalRules (substituição do bloqueio de IPs privados)

Para o freedom-outbound está disponível o grupo **Regras finais**:

Campo	Rótulo	Descrição
<code>overrideXrayPrivateIp</code>	Substituir o bloqueio padrão de IPs privados no Xray	Remove a proibição interna do Xray para saídas para IPs privados.
<code>action</code>	Ação	<code>allow</code> (como no modelo de referência: <code>finalRules: [{action: "allow"}]</code>), <code>redirect</code> (Redirect) ou outros.
<code>blockDelay</code>	Atraso de bloqueio (ms)	Atraso antes de descartar a conexão.
<code>redirect / fragment</code>	Redirect / Fragment	Ações de redirecionamento e fragmentação de tráfego.

Máscara fragment: Lengths e Delays por fragmento

Na máscara **fragment** (tipo fragment em FinalMask, para TCP), os campos únicos Length e Delay foram substituídos por listas **Lengths** e **Delays**: para cada segmento é possível definir um intervalo de comprimento separado (por exemplo `100-200`) e atrasos em milissegundos (por exemplo `10-20` ou `0`). As linhas das listas podem ser adicionadas e removidas; valores únicos salvos anteriormente são transferidos automaticamente para um array de um elemento.

Outros campos do formulário

- **UDP over TCP e Versão UoT** — para protocolos semelhantes ao shadowsocks.
- **Sem cabeçalho gRPC, Tamanho do chunk Uplink** — parâmetros de transporte gRPC.

- Campos TLS/uTLS: **Verificar nome do peer**, **Pinned SHA256**, **Short ID**, **Vision testpre**, placeholder "nome do servidor".

Teste de saídas

Botões: **Testar**, **Testar todas**. Estados: **Testando conexão...**, **Teste bem-sucedido**, **Teste falhou**, **Não foi possível testar a conexão de saída**. Resultado: **Resultado do teste**, latência em milissegundos.

Dois modos (dica: "TCP: probe rápido apenas de dial. HTTP: solicitação completa via xray."):

- **TCP** (`mode=tcp`) – simples dial para `host:port` , executado em paralelo para todos os endpoints, ~timeout de 5 s. Verifica apenas a acessibilidade TCP, não valida o protocolo de proxy. Para `freedom / blackhole /tag blocked` retornará *"Outbound has no testable endpoint"*.
- **HTTP** (`mode=http` ou vazio) – inicia uma instância temporária do Xray, executa uma solicitação HTTP real (probe URL = `outboundTestUrl` do servidor), mede a latência real. Modo autoritativo mas custoso: serializado por um bloqueio global (*"Another outbound test is already running, please wait"*). O timeout de uma tentativa é de 10 s, a janela de espera pelo resultado é de 15 s (aumentados para que outbounds saudáveis em canais lentos ou tunelados não sejam marcados como "Failed"). Em caso de falha, a causa real (erro de DNS, connection refused, expiração do deadline, erro de TLS etc.) é registrada no log do painel/Xray, para o qual apontam as mensagens gerais de timeout.

Protocolos UDP (`wireguard` , `hysteria`) e transportes UDP (`kcp` , `quic` , `hysteria`) são **sempre** testados no modo HTTP, mesmo que TCP seja solicitado – um dial UDP puro não distingue um endpoint "ativo" de um "inativo". Para `wireguard`, `noKernelTun: true` é definido forçosamente na configuração de teste.

Verificação em lote e divisão por etapas

Testar e **Testar todas** no modo HTTP iniciam uma única instância temporária compartilhada do Xray para o lote de outbounds, criam um inbound SOCKS de loopback com regra para cada um e enviam paralelamente uma solicitação HTTP real por ele; **Testar todas** verifica os outbounds em lotes. **Testar todas** também verifica os outbounds obtidos de assinaturas (tabela "de assinaturas", somente leitura) – suas linhas também são destacadas com o resultado do teste. Nesse caso, os outbounds `freedom` ("direct") e `dns` não são testados em nenhum modo (não são proxies): o botão de teste fica indisponível para eles, **Testar todas** os ignora, e a proteção do servidor proíbe seu teste HTTP mesmo em chamada direta à API. Além do sucesso/erro, o pop-up de resultado mostra o status HTTP da resposta e a divisão do tempo por etapas: **Proxy connect** (conexão ao proxy), **TLS via outbound** (TLS via outbound) e **First byte** (tempo até o primeiro byte) – isso ajuda a entender em qual etapa ocorreu o atraso ou a falha.

Estatísticas de tráfego dos outbounds

O painel mantém contadores de tráfego por tags (`up / down / total`). O botão de reset zera os contadores de uma tag específica ou de todas (`tag = "-alltags-` "). Os campos **Informações da conta** e **Status da conexão de saída** mostram um resumo.

11.5. Balanceadores (Balancers)

Lista de `routing.balancers`. Botões: **Criar balanceador**, **Editar balanceador**.

Na aba Balancers há colunas de estado ao vivo: **Live Target** mostra o destino ativo atual do balanceador no Xray em execução, e **Override** permite substituir manualmente a seleção do destino (o valor **Auto (strategy)** retorna a seleção pela estratégia). O estado é atualizado por um botão separado. Se o balanceador ainda não estiver ativo no Xray em execução, o painel sugerirá salvar as alterações primeiro ou iniciar o Xray.

Campo	Rótulo	Descrição
Tag	Tag (dica: "Tag única")	Identificador único. Placeholder: "tag única do balanceador". Validação: "A tag é obrigatória", "A tag já está sendo usada por outro balanceador".
Seletores	Seletores	Lista de tags de outbound (por substring) entre as quais o balanceador seleciona a saída. Pelo menos um deve ser selecionado: "Selecione pelo menos uma saída".
Fallback	Fallback	Tag de outbound de reserva, se nenhum seletor corresponder.
Estratégia	Estratégia	Algoritmo de seleção (veja abaixo).

Estratégia e parâmetros de observação

A estratégia (`strategy.type`) determina como o balanceador seleciona o outbound entre os seletores. Valores do Xray-core: `random` (aleatório), `roundRobin` (em rodízio), `leastPing` (menor latência com base nos resultados do observatory), `leastLoad` (menor carga). Para `leastLoad` / `leastPing` são usados os parâmetros de `strategy.settings` :

Campo	Rótulo	Descrição
<code>expected</code>	Esperado	Placeholder: "número ideal de nós" – número alvo de nós ativos.
<code>maxRtt</code>	Máx. RTT	Limite superior do RTT aceitável na seleção de candidatos.
<code>tolerance</code>	Tolerância	Tolerância ao comparar latências/carga.
<code>baselines</code>	Baselines	Valores limite de latência para agrupamento de nós.
<code>costs</code>	Costs	Coefficientes de peso (cost) para tags individuais.

Exemplos de estratégias. O bloco `strategy` fica dentro do balanceador (no JSON – ao lado de `tag` e `selector`):

```
"strategy": { "type": "random" }      // seleção aleatória entre os seletores
"strategy": { "type": "roundRobin" }  // em rodízio, alternadamente
"strategy": { "type": "leastPing" }   // menor latência (requer observador)
```

Para `leastLoad`, os parâmetros são definidos em `settings` :

```

"strategy": {
  "type": "leastLoad",
  "settings": {
    "expected": 2,
    "maxRTT": "1s",
    "tolerance": 0.05,
    "baselines": ["500ms", "1s", "2s"],
    "costs": [
      { "regex": false, "match": "proxy-premium", "value": 0.1 },
      { "regex": true, "match": "^proxy-cheap-.+$", "value": 5 }
    ]
  }
}

```

Como funciona (exemplo). Suponha que o observador mediu latências para as saídas: A = 250 ms , B = 280 ms , C = 700 ms , D = 1500 ms . Com as configurações acima, a seleção ocorre assim:

1. **maxRTT: "1s"** – saídas com latência acima de 1 s são descartadas: D (1500 ms) é eliminado. Restam A , B , C .
2. **baselines + expected** – as saídas são agrupadas por limites de latência, e é escolhido o **menor** limite que contenha pelo menos **expected** saídas. O limite **500ms** já contém A e B – isso é 2 (= **expected**), então o grupo { A , B } é selecionado. C (700 ms) não entra na seleção enquanto houver saídas rápidas suficientes (ele é uma "reserva quente").
3. **tolerance: 0.05** – dentro do grupo selecionado, as saídas cujas latências diferem em no máximo 5% são consideradas equivalentes e a carga é dividida igualmente entre elas. A (250) e B (280) diferem em ~12% (> 5%), portanto em igualdade de condições a preferência vai para a mais rápida A ; se a diferença estivesse dentro de 5% – o tráfego fluiria por A e B .
4. **costs** – antes da comparação, ajustam o "custo" das saídas individuais: um **value** menor torna a saída mais atraente, maior – o oposto. No exemplo, **proxy-premium** recebe **0.1** (torna-se "mais barato" e é preferido), e todos os **proxy-cheap-*** (por expressão regular, **regex: true**) – **5** (tornam-se "mais caros" e são usados por último). Assim é possível priorizar suavemente as saídas sem excluí-las rigidamente.

Resultado: o tráfego fluirá principalmente por A (com latências semelhantes – igualmente com B), C permanecerá como reserva, D está excluído até que seu RTT caia abaixo de **maxRTT** .

Observador: **observatory** e **burstObservatory** (medições para **leastPing** / **leastLoad**)

As estratégias **leastPing** e **leastLoad** não medem nada por si mesmas – elas precisam de dados sobre a latência e disponibilidade de cada outbound. Esses dados são coletados pelo **observador** (**observatory**): ele "pinga" periodicamente cada outbound monitorado e registra o tempo de resposta e a disponibilidade. Os mesmos dados são exibidos na aba "**Observatory**" (estados **Ativo** / **Indisponível**, "**Última atividade**", "**Última tentativa**").

Não há formulário separado para o observador no painel – o bloco é adicionado **manualmente** no editor de configuração do Xray, no nível superior do config (ao lado de **routing** e **outbounds**), e em seguida é necessário **reiniciar o Xray**.

Duas variantes disponíveis:

- **observatory** – simples: `subjectSelector` + `probeURL` + `probeInterval`.
- **burstObservatory** – avançado, com configuração detalhada de ping via `pingConfig`; conveniente para múltiplas saídas.

Exemplo de bloco `burstObservatory`:

```
{
  "subjectSelector": ["WS-SE", "WS-FR", "WS-PL"],
  "pingConfig": {
    "destination": "https://www.google.com/generate_204",
    "interval": "1m",
    "connectivity": "http://connectivitycheck.platform.hicloud.com/generate_204",
    "timeout": "5s",
    "sampling": 2
  }
}
```

Finalidade dos campos:

Campo	O que define
<code>subjectSelector</code>	Lista de prefixos de tags de outbound para monitoramento. O Xray seleciona todos os outbounds cujas tags começam com as strings especificadas. No exemplo, são monitoradas as saídas <code>WS-SE...</code> , <code>WS-FR...</code> , <code>WS-PL...</code> . Essas tags devem corresponder ao que está selecionado nos Seletores do balanceador.
<code>pingConfig.destination</code>	URL solicitada por cada outbound para medir a latência. Use uma página "leve" com resposta <code>204</code> sem corpo – por exemplo <code>https://www.google.com/generate_204</code> . O tempo até a resposta é a latência medida.
<code>pingConfig.interval</code>	Com que frequência pingar cada outbound. String de duração: <code>"1m"</code> – uma vez por minuto, também <code>"30s"</code> , <code>"5m"</code> etc. Com mais frequência – dados mais frescos, mas mais tráfego em segundo plano.
<code>pingConfig.connectivity</code>	(opcional) URL para verificar a conectividade básica do próprio servidor. Se estiver inacessível – significa que o problema está na rede do servidor, e o observador não marca o outbound como indisponível (proteção contra falsos positivos em caso de falha local). Normalmente também um endpoint com resposta <code>204</code> .
<code>pingConfig.timeout</code>	Quanto tempo aguardar a resposta de um ping antes de considerar a tentativa malsucedida (por exemplo <code>"5s"</code>).
<code>pingConfig.sampling</code>	Quantas medições recentes armazenar e calcular a média por outbound. <code>2</code> – considerar os dois últimos pings (suaviza picos aleatórios).

Como conectar tudo:

1. No editor do Xray, adicione o bloco `burstObservatory` com os `subjectSelector` desejados.
2. Crie um balanceador: **Estratégia** = `leastPing`, nos **Seletores** especifique as mesmas tags de outbound (`WS-SE`, `WS-FR`, `WS-PL`).
3. Direcione o tráfego para ele por uma regra de roteamento (campo **Tag do balanceador**, consulte [11.3](#)).

4. Reinicie o Xray. Na aba "**Observatory**" aparecerão os estados das saídas, e o balanceador começará a selecionar a mais rápida dentre as ativas.

Em uma regra não é possível definir `balancerTag` e `outboundTag` simultaneamente – apenas o `outboundTag` funcionará.

11.6. DNS

Seção `dns`. Ativação: **Ativar DNS** (dica: "Ativar servidor DNS integrado").

Parâmetros gerais do DNS

Campo	Rótulo	JSON	Descrição / dica
<code>tag</code>	Nome da tag DNS	<code>dns.tag</code>	"Esta tag estará disponível como tag de entrada nas regras de roteamento." Permite rotear as próprias solicitações DNS via <code>inboundTag</code> .
<code>clientIp</code>	IP do cliente	<code>dns.clientIp</code>	"Usado para notificar o servidor sobre a localização do IP especificada durante solicitações DNS" (EDNS Client Subnet).
<code>strategy</code>	Estratégia de consulta	<code>dns.queryStrategy</code>	"Estratégia geral de resolução de nomes de domínio". Valores: <code>UseIP</code> , <code>UseIPv4</code> , <code>UseIPv6</code> .
<code>disableCache</code>	Desativar cache	<code>dns.disableCache</code>	"Desativa o cache DNS".
<code>disableFallback</code>	Desativar DNS de fallback	<code>dns.disableFallback</code>	"Desativa as solicitações DNS de fallback".
<code>disableFallbackIfMatch</code>	Desativar DNS de fallback ao corresponder	<code>dns.disableFallbackIfMatch</code>	"Desativa as solicitações DNS de fallback ao corresponder à lista de domínios do servidor DNS".
<code>enableParallelQuery</code>	Ativar consultas paralelas	–	"Ativar consultas DNS paralelas a vários servidores para resolução mais rápida".
<code>useSystemHosts</code>	Usar hosts do sistema	<code>dns.useSystemHosts</code>	"Usar o arquivo hosts do sistema instalado".

Exemplo de bloco `dns`. As solicitações para domínios Google são resolvidas por meio do servidor DoH da Cloudflare, todo o restante – por `1.1.1.1`; para respostas do Google, apenas IPs não privados são esperados. No nível superior do config:

```
"dns": {
  "tag": "dns-inbound",
  "queryStrategy": "UseIPv4",
  "servers": [
    {
      "address": "https://cloudflare-dns.com/dns-query",
      "domains": ["geosite:google"],
      "expectIPs": ["geoip:!private"]
    },
    "1.1.1.1"
  ]
}
```

Uma string de servidor ("1.1.1.1") sem campos é o servidor padrão para todos os outros domínios. A tag `dns-inbound` pode então ser usada como `inboundTag` nas regras de roteamento para direcionar as próprias solicitações DNS pelo outbound desejado.

Cache de registros expirados

Campo	Rótulo	Descrição
<code>serveStale</code>	Usar expirados	"Retornar resultados expirados do cache enquanto a atualização ocorre em segundo plano".
<code>serveExpiredTTL</code>	TTL de expirados	"Prazo de validade (segundos) dos registros de cache expirados; 0 = sem limite".

Servidores DNS (lista `dns.servers`)

Botões: **Criar DNS**, **Editar DNS**, **Excluir todos** (confirmação: "Todos os servidores DNS serão removidos da lista. Esta ação não pode ser desfeita."). Modelos: **Usar modelo**, janela **Modelos DNS**, incluindo o preset **Família**.

Ao clicar em **Editar DNS** em um registro de servidor DNS (assim como em um registro Fake DNS), a janela de edição preenche os valores salvos do servidor, não os valores padrão.

Campos do servidor DNS:

Campo	Rótulo	Descrição
<code>address</code>	—	Endereço DNS (IP, URL DoH, <code>localhost</code> , <code>fakedns</code> etc.).
<code>domains</code>	Domínios	Lista de domínios para os quais este servidor é usado.
<code>expectIPs</code>	IPs esperados	Aceitar a resposta somente se o IP estiver na lista.
<code>unexpectIPs</code>	IPs não esperados	Descartar respostas com os IPs especificados.
<code>skipFallback</code>	Ignorar Fallback	Não usar este servidor como fallback.
<code>finalQuery</code>	Consulta final	Marca o servidor como final na cadeia.
<code>timeoutMs</code>	Timeout (ms)	Timeout de solicitação ao servidor.

Hosts (registros estáticos)

Grupo **Hosts** (`dns.hosts`). Botão **Adicionar Host**; estado vazio **Hosts não definidos**. Campos: domínio (placeholder: "Domínio (ex. domain:example.com)") e valores (placeholder: "IP ou domínio – digite e pressione Enter").

Logs DNS

Consulte [11.9](#): sinalizador **Logs DNS** (`dnsLog`) na seção de log.

11.7. Fake DNS

Seção `fakedns` . Botões: **Criar Fake DNS**, **Editar Fake DNS**.

Campo	Rótulo	Descrição
<code>ipPool</code>	Sub-rede do pool de IPs	Intervalo CIDR do qual são atribuídos IPs fictícios (por exemplo <code>198.18.0.0/15</code>).
<code>poolSize</code>	Tamanho do pool	Quantos endereços manter no pool circular.

O Fake DNS é usado em conjunto com sniffing no inbound: o núcleo fornece ao cliente um IP fictício, memoriza a correspondência domínio↔IP e restaura o domínio durante o roteamento. Para que o Fake DNS funcione, um servidor DNS com endereço `fakedns` deve ser adicionado à lista de servidores DNS.

Exemplo: combinação Fake DNS + servidor DNS. Primeiro definimos o pool de endereços fictícios, depois adicionamos o servidor DNS `fakedns` para que as solicitações de domínio recebam IPs desse pool:

```
"fakedns": [
  { "ipPool": "198.18.0.0/15", "poolSize": 65535 }
],
"dns": {
  "servers": [
    { "address": "fakedns", "domains": ["geosite:geolocation-!cn"] },
    "1.1.1.1"
  ]
}
```

Adicionalmente, no inbound é necessário ativar sniffing com `destOverride: ["fakedns"]` , caso contrário o núcleo não terá de onde obter o domínio real para restauração.

11.8. WireGuard / WARP / NordVPN

Campos WireGuard (`wireguard`)

Campo	Rótulo	Descrição
<code>secretKey</code>	Chave secreta	Chave privada da interface local.
<code>publicKey</code>	Chave pública	Chave pública do peer.

Campo	Rótulo	Descrição
psk	Chave compartilhada	PreShared Key (opcional).
allowedIPs	IPs permitidos	Intervalos roteados para o túnel.
endpoint	Ponto de extremidade	host:port do peer.
domainStrategy	Estratégia de domínio	Estratégia de resolução para o WireGuard-outbound.

Cloudflare WARP (warp)

A integração usa a API `https://api.cloudflareclient.com/v0a4005` (client-version `a-6.30-3596`). Ações do controlador (`/xray/warp/:action`): `config`, `reg`, `license`, `data`, `del`.

Passo a passo:

1. **Criar conta WARP** → `reg`: o painel gera/aceita chaves privada (`privateKey`) e pública (`publicKey`), registra o dispositivo na Cloudflare e salva `access_token`, `device_id`, `license_key`, `private_key` (além de `client_id`) na configuração `warp`.
2. **Chave de licença WARP / WARP+** → `license`: configuração da chave WARP+ de 26 caracteres (placeholder: *"Chave WARP+ de 26 caracteres"*). Em caso de erro: *"Não foi possível definir a licença WARP"*. Se a configuração ainda não foi obtida: *"Primeiro obtenha a configuração WARP"*.
3. **Informações da conta: Nome do dispositivo, Modelo do dispositivo, Dispositivo ativado, Tipo de conta, Função, WARP+ data, Cota, Uso.**
4. **Adicionar saída** — cria um WireGuard-outbound com as chaves e o endpoint da Cloudflare obtidos.
5. **Excluir conta** → `del`: limpa os dados WARP salvos.

NordVPN (nord / nordvpn)

A integração usa NordLynx (= WireGuard). Ações do controlador (`/xray/nord/:action`): `countries`, `servers`, `reg`, `setKey`, `data`, `del`.

Passo a passo:

1. **Token de acesso** → `reg`: o painel solicita ao `api.nordvpn.com` as credenciais NordLynx e extrai `nordlynx_private_key`. Salva `private_key` e `token` na configuração `nord`. Alternativa — `setKey`: inserir a **Chave privada** diretamente (não pode estar vazia).
2. **País** → `countries` carrega a lista de países; **Cidade** (ou **Todas as cidades**).
3. **Servidor** → `servers` carrega os servidores do país selecionado (`countryId` é validado como número — proteção contra injeções). Filtro: são exibidos apenas servidores com **Carga** > 7%. Se não houver servidores: *"Nenhum servidor encontrado para o país selecionado"*. Se o servidor não tiver chave pública NordLynx: *"O servidor selecionado não informa a chave pública NordLynx"*.
4. Criação/atualização da saída: notificações *"Saída NordVPN adicionada"* / *"Saída NordVPN atualizada"*.

Prioridade IPv4 e TUN no espaço do usuário

Os WireGuard-outbounds gerados pelos assistentes WARP e NordVPN usam `domainStrategy`: `"ForceIPv4v6"` (prioridade IPv4 com fallback para IPv6 em hosts somente-v6) em vez de `ForceIP` — isso

elimina o "travamento" do handshake em hosts com IPv6 parcialmente configurado, quando um registro AAAA do endpoint da Cloudflare é selecionado. Além disso, o TUN no espaço do usuário (`noKernelTun: true`) é ativado para eles em vez do kernel TUN: este último requer permissões e roteamento por fwmark e falha silenciosamente em muitos VPS, enquanto a verificação de conexão integrada do painel sempre testa via TUN no espaço do usuário – agora o tráfego real e a verificação seguem o mesmo caminho. A alteração aplica-se apenas aos outbounds recém-adicionados ou redefinidos; modelos já salvos mantêm suas configurações.

11.9. Proxy reverso e TUN

Reverse (proxy reverso)

Seção `reverse` da configuração do Xray. No formulário de outbound há um interruptor para o tipo **Proxy reverso**. Botões: **Criar proxy reverso**, **Editar proxy reverso**.

Campo	Rótulo	Descrição
Tipo	Tipo	Bridge ou Portal – dois papéis do proxy reverso Xray.
Domínio	Domínio	Domínio de serviço para o par bridge[?]portal.
Tag / Conexão	Tag / Conexão	Tags para vincular bridge e portal.
Reverse Tag	Tag do proxy reverso	Dica: "Tag da conexão de saída para proxy reverso VLESS simples. Deixe vazio para desativar." Placeholder: "tag de saída (vazio = desativado)". Implementa um VLESS reverse simplificado.

No formulário de outbound também estão presentes campos de fluxo reverso: **Sniffing reverso**, **Workers**, **Reservado**, **Intervalo mín. de carregamento (ms)**, **Tamanho máx. de carregamento (bytes)**.

TUN (tun)

Campo	Rótulo	Descrição	Padrão
name	—	"Nome da interface TUN."	xray0
mtu	—	"Unidade máxima de transmissão. Tamanho máximo dos pacotes de dados."	1500
<code>userLevel</code>	Nível do usuário	"Todas as conexões estabelecidas por meio desta entrada usarão este nível de usuário."	0

11.10. Logs e estatísticas (Stats, metrics)

Log (log)

Dica: "Os logs podem diminuir a velocidade do servidor. Ative apenas os tipos de log necessários quando precisar!"

Seção `log` do modelo de referência: `access: "none"`, `error: ""`, `loglevel: "warning"`, `dnsLog: false`, `maskAddress: ""`.

Campo	Rótulo	JSON	Descrição	Padrão
logLevel	Nível de logs	logLevel	"Nível de log para logs de erros..." Valores: debug , info , warning , error , none .	warning
accessLog	Logs de acesso	access	"Caminho para o arquivo de log de acesso. O valor especial 'none' desativa os logs de acesso."	none
errorLog	Logs de erros	error	"Caminho para o arquivo de logs de erros. O valor especial 'none' desativa os logs de erros."	"" (padrão)
dnsLog	Logs DNS	dnsLog	"Ativar logs de solicitações DNS"	false
maskAddress	Mascaramento de endereço	maskAddress	"Quando ativado, o endereço IP real é substituído por um endereço mascarado nos logs."	"" (desativado)

Estatísticas (stats / policy)

Grupo **Estatísticas**. Ativa os contadores em `policy.system` e `policy.levels`. No modelo de referência: `statsInboundUplink: true`, `statsInboundDownlink: true`, `statsOutboundUplink: false`, `statsOutboundDownlink: false`; para o nível 0 – `statsUserUplink: true`, `statsUserDownlink: true`.

Campo	Rótulo	Descrição	Padrão
statsInboundUplink	Estatísticas de uplink de entrada	"Ativa a coleta de estatísticas para o tráfego de saída de todos os proxies de entrada."	true
statsInboundDownlink	Estatísticas de downlink de entrada	"Ativa a coleta de estatísticas para o tráfego de entrada de todos os proxies de entrada."	true
statsOutboundUplink	Estatísticas de uplink de saída	"Ativa a coleta de estatísticas para o tráfego de saída de todos os proxies de saída."	false
statsOutboundDownlink	Estatísticas de downlink de saída	"Ativa a coleta de estatísticas para o tráfego de entrada de todos os proxies de saída."	false

As estatísticas por clientes e inbounds (uplink/downlink) são a base para a exibição de tráfego no painel e nos clientes; não é recomendado desativá-las. As estatísticas de outbound estão desativadas por padrão e são necessárias apenas se você rastrear o tráfego por tags de saída.

Metrics

No modelo de referência há uma seção `metrics` (`listen: "127.0.0.1:11111"`, `tag: "metrics_out"`) e a API correspondente `metrics_out`. O painel usa este listener para coletar métricas e snapshots do observatory: ele analisa `metrics.listen` do modelo, consulta `/debug/vars` e agrega o histórico de latências por tags. Se você alterar o endereço/porta de `metrics.listen`, o painel passará a usar o novo endereço; remover a seção `metrics` desativará a coleta de gráficos do observatory.

O teste de outbound no modo HTTP inicia uma **instância temporária separada** do Xray com seu próprio listener `metrics` em uma porta aleatória — não é o mesmo listener que está no config principal.

11.11. Salvamento, reinicialização e transformações automáticas

Botões

Botão	Ação
Salvar	POST <code>/xray/update</code> : valida e salva o modelo + <code>outboundTestUrl</code> .
Reiniciar Xray	Recarrega o serviço com a configuração salva. Confirmação: "Reiniciar xray?" / "Recarrega o serviço xray com a configuração salva."

Notificações: sucesso — "Xray reiniciado com sucesso", "Xray parado com sucesso"; erros — "Ocorreu um erro ao reiniciar o Xray", "Ocorreu um erro ao parar o Xray." A janela **Saída de reinicialização do Xray** exibe o diagnóstico do núcleo.

Aplicação a quente das alterações (sem reinicialização completa)

As alterações em inbounds, outbounds e regras de roteamento são aplicadas "ao vivo": ao clicar em **Salvar**, o painel calcula a diferença entre a configuração antiga e a nova e aplica apenas as partes alteradas via API gRPC do Xray (`HandlerService/RoutingService`), sem reiniciar o processo. A reinicialização completa é executada automaticamente apenas quando seções sem API de recarga a quente são alteradas (`log`, `dns`, `policy`, `observatory` etc.). Por isso, na página do Xray não é necessário um botão separado "Reiniciar" — **Salvar** aplica as alterações por si mesmo. A reinicialização do núcleo quando necessário ainda ocorre automaticamente (veja também o recarregamento automático durante a atualização de assinaturas e a rotação do WARP).

Restauração do modelo padrão

O endpoint `GET /xray/getDefaultJsonConfig` retorna o modelo de referência (`config.json`, integrado ao binário). Com ele é possível redefinir a configuração para os valores de fábrica.

Transformações automáticas ao salvar

Ao salvar as configurações do Xray, o painel executa (nesta ordem):

- Remoção de invólucros** — remove invólucros do tipo `{ "xraySetting": <config>, "inboundTags": ..., "outboundTestUrl": ... }`, caso tenham entrado no valor por acidente (caso contrário, as camadas se acumulariam a cada salvamento). Até 8 camadas são removidas.
- Verificação da configuração** — o JSON é analisado na estrutura de configuração do Xray; em caso de erro — rejeição com `"xray template config invalid"`.
- Garantia da regra de estatísticas** — a regra `inboundTag: ["api"]` → `outboundTag: "api"` é forçosamente promovida para a posição 0 em `routing.rules` (ou adicionada se ausente). Isso garante que a solicitação gRPC de estatísticas do painel não seja interceptada por uma regra catch-all superior (caso contrário, os clientes podem aparecer offline com tráfego zero mesmo com o proxy funcionando).

Por causa do item 3, não tente remover ou mover a regra `api → api` — o painel a restaurará ao salvar. Esta é a infraestrutura de serviço de estatísticas, não uma rota de usuário.

11.12. Outbound de assinatura (com atualização automática)

A partir da versão 3.3.0, o painel pode importar `outbound s` diretamente de uma URL de assinatura — no mesmo formato que os provedores de VPN fornecem para aplicativos clientes. As assinaturas são relidas regularmente em segundo plano, portanto o conjunto de `outbound s` no servidor é mantido atualizado sem edição manual do modelo de configuração.

Na interface em português, a seção é chamada **"Assinaturas de saídas"**, descrição: "Importar saídas de URLs de assinaturas remotas (vmess/vless/trojan/ss/...). As tags permanecem inalteradas para uso em balanceadores e regras de roteamento. A atualização é realizada automaticamente." A seção está localizada na página Xray, acima do painel de configuração de `outbound s`.

Como funciona

As assinaturas são armazenadas separadamente do modelo de configuração do Xray. O modelo **nunca é sobrescrito**: os `outbound s` obtidos das assinaturas são adicionados à configuração final durante cada geração de config do Xray.

Adicionando uma assinatura

No formulário "Adicionar assinatura" estão disponíveis os seguintes campos:

Campo	Chave	Padrão	Finalidade
URL da assinatura	<code>url</code>	— (obrigatório)	Endereço da assinatura. Placeholder: "https://..." (lista de links em base64)". Apenas HTTP(S) é aceito; o endereço é verificado por segurança.
Observação	<code>remark</code>	vazio	Rótulo arbitrário (placeholder "ex. nós HK").
Prefixo de tag	<code>tagPrefix</code>	<code>subN-</code>	Prefixo com o qual começam as tags dos <code>outbound s</code> importados. Se deixado vazio, o painel escolherá automaticamente o menor número livre no formato <code>sub1-</code> , <code>sub2-</code> etc.
Intervalo de atualização	<code>updateInterval</code>	600 segundos (10 minutos)	Com que frequência a assinatura é relida. Na interface é definido em horas/minutos.
Ativado	<code>enabled</code>	sim (<code>true</code>)	Apenas assinaturas ativadas entram no config e são atualizadas automaticamente.
Permitir endereços privados	<code>allowPrivate</code>	não (<code>false</code>)	Permite URLs em localhost, LAN e IPs privados. Desativado por padrão para proteção contra SSRF — ative apenas para uma fonte local confiável.

Campo	Chave	Padrão	Finalidade
Antes das saídas manuais	prepend	não (false)	Se ativado, os <code>outbound</code> s desta assinatura são colocados antes dos <code>outbound</code> s manuais do modelo, e um deles pode se tornar o <code>outbound</code> padrão. Caso contrário, são adicionados depois .

O botão "**Pré-visualizar**" (`POST /outbound-subs/parse`) permite baixar e analisar a URL antes de salvar e ver quais `outbound` s e tags serão gerados; nada é gravado no banco de dados nesse momento. Se nada for reconhecido pela URL, é exibido "Nenhuma saída encontrada nesta URL."

A ordem de várias assinaturas na lista geral de `outbound` s é definida pela prioridade (`priority`) e alterada pelas setas para cima/baixo (`POST /outbound-subs/:id/move`).

Quais formatos de assinatura são aceitos

O corpo da resposta da URL é processado assim:

- O conteúdo é primeiro testado como **base64** (variantes padrão e URL-safe, com complemento automático de padding e remoção de espaços/quebras de linha). Se for base64 — é decodificado; caso contrário, é usado como está.
- Em seguida, o corpo é dividido em linhas. Cada linha não vazia que não comece com `#` é analisada como um link. Linhas não reconhecidas (comentários, protocolos não suportados) são silenciosamente ignoradas.
- Esquemas de links suportados: `vmess://` , `vless://` , `trojan://` , `ss://` (Shadowsocks), `hysteria2://` / `hy2://` , `wireguard://` / `wg://` .

Ou seja, é compatível com a assinatura padrão no formato "lista de links codificada em base64", como a maioria dos provedores usa.

Tags estáveis

Para cada link é calculada uma "identidade" estável (URI principal sem fragmento de observação; para `vmess` — JSON interno sem o campo `ps`). A correspondência "identidade → tag" é salva e, na próxima atualização, o mesmo servidor recebe a mesma tag, mesmo que a observação ou parâmetros secundários tenham sido alterados. Isso foi feito especificamente para que balanceadores e regras de roteamento continuem funcionando após as atualizações:

- Uma tag exata em um balanceador/regra continuará apontando para o mesmo servidor.
- Um seletor de prefixo/wildcard (por exemplo, `hk-*`) capturará automaticamente novos servidores que a assinatura retornar posteriormente — esta é a forma recomendada de "assinar um pool".
- Se um servidor desaparecer da assinatura, sua tag simplesmente desaparece do array final de `outbound` s; se o balanceador tiver `fallbackTag` , o Xray o usará.
- Se o provedor alterar o UUID/host/credenciais do servidor, a identidade muda — isso é considerado um novo `outbound` com uma nova tag.

Dentro de uma exportação, as tags são deduplicadas com o sufixo `-N` . As tags de assinaturas preservam caracteres não ASCII (por exemplo, cirílico) e permanecem legíveis: letras e dígitos Unicode são mantidos no slug, e a pontuação é substituída por hífen — tags de nomes cirílicos não são mais reduzidas a apenas números.

Como funciona a atualização automática

- A tarefa de atualização de assinaturas em segundo plano é executada **a cada 5 minutos**.
- Em cada execução, ela percorre todas as assinaturas ativadas e atualiza apenas aquelas cujo próprio intervalo expirou: uma assinatura é atualizada se ainda não foi atualizada nenhuma vez ou se passaram pelo menos `updateInterval` desde a última atualização. Assim, a tarefa verifica as assinaturas frequentemente, mas cada assinatura específica é relida não mais do que seu `updateInterval` (padrão de 10 minutos). Isso está refletido na dica correspondente na interface.
- Atualização: a URL é verificada novamente quanto à segurança como pública (endereços privados são bloqueados, a menos que a assinatura tenha `allowPrivate` definido), a solicitação vai pelo cliente proxy do painel com o cabeçalho `User-Agent: 3x-ui-outbound-sub/1.0`. A cadeia de redirecionamentos é limitada a 10 saltos, e cada salto também é verificado quanto à privacidade (proteção contra SSRF). Espera-se HTTP 200; caso contrário, um erro é registrado.
- Após o parsing bem-sucedido, o resultado é salvo, o horário da última atualização é registrado e o erro é limpo. Em caso de erro, seu texto fica visível na interface como "Último erro", e os `outbound`s obtidos anteriormente permanecem válidos.
- Se pelo menos uma assinatura foi realmente atualizada, a tarefa marca o Xray para reinicialização e envia uma invalidação de UI para que a interface carregue os novos `outbound`s. A reinicialização real do Xray ocorre no próximo ciclo de 30 segundos do gerenciador.

A atualização manual de uma assinatura é feita pelo botão **"Atualizar agora"** (`POST /outbound-subs/:id/refresh`); ele também marca o Xray para reinicialização. Adicionar, editar ou excluir uma assinatura também ativa o sinalizador de reinicialização do Xray (ao excluir, seus `outbound`s saem do config na próxima reinicialização). A interface informa: "Após adicionar ou atualizar, reinicie o Xray (ou aguarde a próxima reinicialização automática) para que as saídas se tornem ativas."

Como isso entra na configuração do Xray

A cada geração de configuração do Xray, os `outbound`s ativos das assinaturas são divididos em dois grupos — `prepend` (sinalizador "Antes das saídas manuais") e os demais — e mesclados com o modelo: `[prepend de assinaturas] + [outbound's do modelo] + [restantes das assinaturas]`. Dentro de cada grupo, as assinaturas vão por prioridade. Os `outbound`s manuais do modelo não são afetados; se o array de `outbound`s do modelo não puder ser analisado por algum motivo, os `outbound`s das assinaturas não são misturados a ele (para não perder os manuais).

Os `outbound`s importados são exibidos adicionalmente no próprio painel de `outbound`s em um bloco separado **"De assinaturas de saídas (somente leitura)"** — não é possível editá-los lá; o gerenciamento é feito apenas pela seção "Assinaturas de saídas".

11.13. Rotação de IP no WARP

No 3X-UI é possível configurar um WARP-outbound — uma conexão WireGuard de saída para o Cloudflare WARP (tag `warp` na configuração do Xray). O painel registra automaticamente nos servidores da Cloudflare uma conta de dispositivo, obtém as chaves e endereços WireGuard e os insere no outbound com a tag `warp`. Por meio desse outbound, o tráfego sai para a internet com um endereço IP do Cloudflare WARP. A novidade da versão 3.3.0 é a possibilidade de alterar esse IP de saída manualmente ou por agendamento, sem recriar a conta WARP manualmente.

O gerenciamento está localizado na seção **Xray** no cartão WARP (após clicar em "Criar conta WARP" e obter o config; antes disso as ações estão indisponíveis – o painel informará "Primeiro obtenha a configuração WARP").

O que acontece ao trocar o IP

O botão **"Trocar IP"** inicia a troca de IP. Lógica:

1. Um novo par de chaves WireGuard é gerado.
2. Com a nova chave, o dispositivo WARP é registrado novamente nos servidores da Cloudflare (novo `device_id`, `access_token`, endereços e dados do peer).
3. Os novos dados são gravados no WARP-outbound da configuração do Xray: são atualizados `secretKey`, `address` (v4 /32 e v6 /128), `reserved` (de `client_id`), bem como `publicKey` e `endpoint` do peer.
4. Se uma chave de licença WARP+ foi definida anteriormente (com pelo menos 26 caracteres), ela é automaticamente reinstalada na nova conta. Em caso de falha, isso é apenas um aviso nos logs – a troca de IP não é cancelada.
5. Após a troca bem-sucedida, o Xray é marcado como precisando de reinicialização para que o novo outbound entre em vigor.

Em caso de sucesso, a interface exibe "Endereço IP do WARP alterado com sucesso!".

Rotação automática por agendamento

No cartão WARP há um interruptor **"Atualização automática de endereço IP"** e o campo **"Intervalo (dias)"**. Dica: "0 – desativar. Altera automaticamente o endereço IP."

Parâmetro	Valor
Configuração no banco de dados	<code>warpUpdateInterval</code> (inteiro, ≥ 0)
Valor padrão	0 (rotação automática desativada)
Unidade de medida	dias
0	desativa a troca automática
> 0	trocar o IP a cada N dias

Salvar o intervalo armazena `warpUpdateInterval` e, se o valor for maior que 0, redefine o "horário da última atualização" para o momento atual – caso contrário, o agendador trocaria o IP no próximo tick.

O agendamento é executado por uma tarefa em segundo plano que roda a cada hora – ou seja, o painel verifica a cada hora se é hora de fazer a rotação. Algoritmo de verificação:

- se o intervalo ≤ 0 – não faz nada;
- se o "horário da última atualização" for 0 (por exemplo, o intervalo foi definido diretamente no banco de dados) – esta é a primeira execução: a tarefa apenas registra o marcador de tempo base e NÃO troca o IP imediatamente;
- se passaram pelo menos `intervalo × 24 × 3600` segundos desde a última atualização – é executada a mesma troca de IP, o marcador de tempo é atualizado e a reinicialização do Xray é agendada.

Detalhe importante: a troca manual pelo botão "Trocar IP" também redefine o marcador de tempo da última atualização. Portanto, após uma rotação manual, a contagem do intervalo automático começa do zero e a troca planejada não ocorrerá imediatamente em seguida.

"Via proxy do painel"

Alterado na versão 3.3.1. A configuração separada "Proxy de rede do painel" (`panelProxy`) foi removida. O tráfego de saída do próprio painel (incluindo solicitações à WARP API) agora é direcionado pelo **outbound de tráfego do painel** selecionado – um Xray-outbound ou balanceador (consulte a seção 13). A descrição abaixo refere-se às versões anteriores à 3.3.1.

Todas as solicitações à API do Cloudflare WARP (registro, obtenção de config, definição de licença, troca de IP) não vão diretamente, mas pelo cliente HTTP do painel com timeout de 15 segundos. Esse cliente respeita a configuração **"Proxy de rede do painel"** (`panelProxy`) das configurações do painel.

Da descrição da configuração: o proxy roteia as próprias solicitações de saída do painel (atualizações de bases geo, verificações de versão do Xray/painel, Telegram e agora também as solicitações ao WARP) – para contornar a filtragem do servidor. São aceitos endereços no formato `socks5://` ou `http(s)://`, por exemplo um inbound SOCKS local do próprio Xray. Se o campo estiver vazio ou o proxy estiver configurado incorretamente – a conexão direta é usada (o comportamento não é quebrado).

Benefício para o WARP: se o servidor não conseguir acessar diretamente o `api.cloudflareclient.com`, o registro e a rotação falhavam antes. Agora, informando um proxy funcional em `panelProxy` (incluindo o próprio inbound do Xray), é possível garantir a disponibilidade da WARP API e o funcionamento tanto do botão manual quanto da rotação planejada.

Quando isso é útil

- Troca regular do IP de saída para o outbound que usa WARP – reduz o risco de bloqueios e rastreamento por um único endereço.
- "Renovar" o IP manualmente, se o endereço atual da Cloudflare foi bloqueado ou está funcionando lentamente.
- Servidores sem acesso direto à API do Cloudflare WARP: o roteamento das solicitações via `panelProxy` torna o registro e a rotação operacionais.

12. Nós (multipainel, master/slave)

A seção **Nós** transforma uma instalação comum do 3X-UI em um **painel central (master)**, que monitora remotamente outros painéis 3X-UI (filhos) e os gerencia. Cada nó é uma instalação separada do 3X-UI em seu próprio servidor; o master acessa-o por meio de sua própria API HTTP, consulta seu estado e sincroniza para ele os inbounds e clientes atribuídos. Isso é a funcionalidade de **multipainel**: em vez de acessar cada painel individualmente, você vê todos os servidores em uma única lista e os gerencia de forma centralizada.

Princípio importante: **o nó não é um agente, mas um painel 3X-UI completo**. O master não "instala" nada nele — ele apenas se conecta à sua API por token. Remover um nó da lista interrompe apenas o monitoramento; o painel remoto em si não é afetado (dica: «Isso interromperá o monitoramento do nó. O painel remoto em si não será afetado»).

12.1. Resumo no topo da lista

Acima da tabela de nós são exibidos contadores agregados:

Campo	Descrição
Total de nós	Número total de nós na lista.
Online	Quantos nós têm o status <code>online</code> .
Offline	Quantos nós têm o status <code>offline</code> .
Latência média	Latência média (ping) para os nós, em milissegundos.

12.2. Adicionando e editando um nó

Os botões **Adicionar nó** e **Editar nó** abrem um formulário com os campos do nó.

São obrigatórios (dica: «Nome, endereço, porta e token de API são obrigatórios») os campos **Nome**, **Endereço**, **Porta** e **Token de API**.

Ao clicar em "Salvar" (tanto ao adicionar quanto ao editar), o painel **primeiro verifica a acessibilidade do nó** com um timeout de 6 segundos. Se o nó não responder, o registro não é salvo e um erro é exibido. Ou seja, não é possível adicionar um nó reconhecidamente inacessível.

Campos do formulário

Campo	Padrão	Valores permitidos	Descrição
Nome	— (obrigatório)	string não vazia, única	Nome interno do nó. A coluna de nome tem restrição de unicidade — dois nós com o mesmo nome não podem ser criados. Placeholder da dica: <code>ex: de-frankfurt-1</code> . Ao salvar, espaços nas extremidades são removidos.
Observação	vazio	qualquer string	Anotação/descrição opcional do nó. Não afeta o funcionamento.

Campo	Padrão	Valores permitidos	Descrição
Esquema	<code>https</code>	<code>http</code> / <code>https</code>	Protocolo de conexão com o painel remoto. Se deixado vazio ou especificado com valor inválido, a normalização definirá <code>https</code> . Se o nó responder por HTTP comum mas o esquema estiver configurado como <code>https</code> , o painel retornará uma dica compreensível: «the server speaks HTTP, not HTTPS; set the node scheme to http».
Endereço	– (obrigatório)	host ou IP	Endereço do painel remoto. Placeholder: <code>panel.example.com</code> ou <code>1.2.3.4</code> . O endereço é normalizado; por padrão, endereços privados/loais são proibidos como proteção contra SSRF – veja «Permitir endereço privado».
Porta	– (obrigatório)	inteiro 1–65535	Porta do painel web do nó remoto. Valores fora do intervalo são rejeitados («node port must be 1-65535»).
Caminho base	<code>/</code>	string de caminho	Caminho base (web base path) do painel remoto, se definido. É normalizado: começa e termina com <code>/</code> garantidamente (valor vazio → <code>/</code>). O painel concatena <code>panel/api/server/status</code> a ele ao consultar.
Token de API	– (obrigatório)	token do painel remoto	Bearer token para acesso à API do nó. Transmitido no cabeçalho <code>Authorization: Bearer <token></code> . Placeholder: «Token da página de Configurações do painel remoto». Dica: «O painel remoto exibe seu token de API na seção Configurações → Token de API». Ou seja, o token deve ser criado no próprio nó (Configurações → Token de API) e colado aqui.
Habilitado	<code>true</code>	sim/não	Habilita o monitoramento e a sincronização do nó. Nós desabilitados não são consultados pelas tarefas em segundo plano (heartbeat e traffic-sync os ignoram) e não participam da atualização em massa do painel.
Permitir endereço privado	<code>false</code>	sim/não	Remove a proteção SSRF e permite conectar ao nó por endereço privado/local. Dica: «Habilitar somente para nós em rede privada ou VPN». Habilite apenas quando o nó realmente estiver em uma rede privada ou acessível via VPN.

Obtenção e regeneração do token no lado do nó

O token é obtido no painel remoto, na seção **Configurações** → **Token de API**. Lá também é possível reemiti-lo: o botão **Gerar novo token** com o aviso: «Regenerar invalidará o token atual. Qualquer painel central que o utilize perderá o acesso até que seja atualizado. Continuar?». Após a regeneração, o token antigo no painel master deixará de funcionar – é necessário atualizá-lo no formulário do nó.

Conexão de saída (Connection outbound)

O campo **Connection outbound** (Conexão de saída, `outboundTag`) define como o tráfego das requisições do master para a API deste nó sai do servidor. Se um tag de Xray outbound for selecionado, as requisições do painel ao nó não ocorrerão diretamente, mas passarão pelo outbound especificado; o painel adicionará automaticamente à configuração ativa um inbound de bridge no loopback e aplicará a alteração em tempo real, sem reinicialização. Dica: «Route this node's panel API traffic through the selected Xray outbound. A loopback bridge inbound is added to the running config automatically and applied live. Leave empty for a direct connection».

O seletor funciona como o seletor de outbound do painel: os tags são agrupados em **Outbounds** (saídas comuns) e **Balancers** (balanceadores), e os outbounds do tipo blackhole são ocultados da lista. Valor vazio (placeholder «Direct connection») = conexão direta com o nó.

Importação de inbound (seleção de inbounds a sincronizar)

No formulário do nó há a configuração **Importar inbound** (`inboundSyncMode`) com dois modos: **Todos os inbounds** (`all` , padrão) e **Selecionados** (`selected`). Por padrão, o master sincroniza para o nó todos os inbounds que têm este nó selecionado; os nós existentes continuam operando no modo «Todos os inbounds».

No modo **Selecionados**, um campo de múltipla seleção de tags de inbound aparece abaixo do campo. Clique em **Carregar inbounds** – o master, com os parâmetros de conexão digitados (ainda não salvos), solicitará ao nó a lista de seus inbounds (endpoint `POST /panel/api/nodes/inbounds`) e exibirá seus tags; marque os desejados. O painel sincronizará e implantará no nó apenas os tags marcados, enquanto os demais inbounds existentes diretamente no nó serão deixados intocados – o master não os exclui nem os gerencia.

Exemplo: solicitar a lista de inbounds do nó para importação seletiva. No corpo são passados os parâmetros de conexão ainda não salvos; na resposta – os tags dos inbounds disponíveis no nó:

```
POST /panel/api/nodes/inbounds
Content-Type: application/json

{ "name": "de-fra-1", "scheme": "https", "address": "node1.example.com",
  "port": 2053, "basePath": "/", "apiToken": "abcdef..." }
```

12.3. Verificação de TLS (para nós https)

O grupo de campos define como o master verifica o certificado HTTPS do nó. Essas configurações são **relevantes apenas para o esquema https** ; para nós `http` elas são ignoradas.

Verificação de TLS – lista suspensa, dica: «Como o painel verifica o certificado HTTPS do nó. Fixação ou Ignorar – para certificados autoassinados (somente nós https)».

Modo	Valor	Padrão	Descrição
Verificar (CA padrão)	<code>verify</code>	sim (padrão)	Verificação normal da cadeia de certificados por uma CA confiável. Adequado para nós com certificado público/Let's Encrypt. Também é usado para todos os nós <code>http</code> .
Fixar certificado (SHA-256)	<code>pin</code>	–	A cadeia de CA não é verificada, mas o SHA-256 do certificado folha do nó é comparado com a impressão digital salva (comparação em tempo constante). Mantém proteção contra MITM para certificados autoassinados . Requer o preenchimento do campo de impressão digital.
Ignorar verificação	<code>skip</code>	–	A verificação do certificado é completamente desativada. Aviso: «Ignorar a verificação remove a proteção contra ataques de homem no meio – o token de API pode ser interceptado. É melhor fixar o certificado».

Aos três modos acima, na versão 3.4.0 foi adicionado um quarto – **Mutual TLS (certificado de cliente)** (`mtls`), disponível, como os demais, apenas para o esquema `https` .

Modo	Valor	Padrão	Descrição
Mutual TLS (certificado de cliente)	mtls	—	Além de verificar o certificado do nó, o master também se autentica para o nó com um certificado de cliente emitido pela sua própria CA. Para o nó neste modo, o token de API torna-se opcional — o nó reconhece o master pelo certificado. Ao selecionar este modo, uma dica é exibida: «This node authenticates the panel with a client certificate. Copy this panel's CA from the Node mTLS section onto the node, set its Trusted parent CA, then restart it».

Para habilitar o TLS mútuo para um nó: no lado do nó, configure o modo **Mutual TLS**, copie a CA do painel de controle da seção **Node mTLS** (veja abaixo), registre-a no nó como **CA pai confiável** e reinicie o nó.

Se qualquer valor diferente de `skip`, `pin` ou `mtls` for selecionado, a normalização forçará `verify`.

Fixação de certificado

Ao selecionar **Fixar certificado**, os seguintes campos aparecem:

- **SHA-256 do certificado fixado** — campo de entrada. Aceita a impressão digital em **base64** (formato `pinnedPeerCertSha256` do Xray) ou em **hex** com ou sem dois-pontos (estilo `openssl -fingerprint`). Dica: «SHA-256 do certificado do nó em base64 ou hex. Clique em 'Obter' para lê-lo do nó agora». Placeholder: «SHA-256 em base64 ou hex». Ao selecionar `pin`, uma impressão digital vazia ou incorreta causa erro de validação ao salvar.

Exemplo: a mesma impressão digital em dois formatos. O campo aceita qualquer um dos formatos — ambos representam o mesmo certificado:

```
# base64 (formato pinnedPeerCertSha256 do Xray)
607TNg3l2k0pq8R1sT2uV3wX4yZ5a6B7c8D9e0F1g2=

# hex com dois-pontos (estilo openssl x509 -fingerprint -sha256)
E8:E2:D3:60:DE:5D:9A:4D:29:AB:CF:11:B2:7C:34:...
```

Se a impressão digital ainda não for conhecida, clique em **Obter** — o master a lê do nó via HTTPS e a insere no campo.

- Botão **Obter** — conecta-se ao nó via HTTPS sem verificar o certificado e lê o SHA-256 do certificado folha atual (endpoint `POST /certFingerprint`), inserindo-o no campo. Após sucesso — «Certificado atual do nó obtido»; em caso de falha — «Falha ao obter o certificado». Disponível apenas para nós https.

Node mTLS (autenticação TLS mútua entre painéis)

Na página **Nós** há uma seção separada **Node mTLS** — configuração de autenticação TLS mútua, que adiciona ao token de API um segundo fator (certificado de cliente) para chamadas «painel → nó». O TLS mútuo é opcional; se os campos da seção estiverem vazios, os nós funcionam pelo esquema anterior — **somente com token de API** (dica: «Mutual TLS adds a client-certificate factor on top of the API token for node-to-node calls. It is opt-in: leave it empty to keep token-only auth»). A seção tem duas operações:

- **Copiar CA deste painel** (`POST /panel/api/nodes/mtls/ca`) — copia o certificado raiz (CA) deste painel para a área de transferência. Essa CA deve ser transmitida aos nós gerenciados para que eles confiem no

certificado de cliente do painel; nos próprios nós, em seguida, define-se o modo de verificação TLS **Mutual TLS** (dica: «Hand this CA to the nodes this panel manages, then set their TLS verification to Mutual TLS»). Após a cópia – «CA certificate copied to clipboard».

- **CA pai confiável** (`Trusted parent CA` , `POST /panel/api/nodes/mtls/trustCA`) – campo usado quando este painel atua como nó para um painel superior (de controle). Cole aqui a CA do painel de controle para exigir seu certificado de cliente e clique em **Save trust CA**. A alteração requer **reinicialização do painel** (dica: «When this panel is itself a node, paste the managing panel's CA here to require its client certificate. Restart the panel to apply»).

12.4. O que é exibido para cada nó

Colunas da tabela e campos do cartão do nó (estado observado, preenchido a cada consulta de heartbeat):

Campo	Descrição
Status	<code>online</code> / <code>offline</code> / <code>unknown</code> – veja abaixo.
CPU	Utilização do processador do servidor remoto em porcentagem.
Memória	Uso de RAM em porcentagem (calculado como <code>current/total*100</code>).
Uptime	Tempo de operação contínua do servidor (em segundos).
Latência	Tempo de resposta do nó na última consulta (ms).
Último ping	Hora do último heartbeat bem-sucedido (segundos Unix; <code>0</code> = «nunca»; valor recente é exibido como «agora pouco»).
Versão do Xray	Versão do Xray-core em execução no nó.
Versão do painel	Versão do 3X-UI no nó – comparada com a atual para o indicador de atualização.
(inbounds)	Quantos inbounds estão fisicamente hospedados neste nó.
(clientes)	Número de clientes nos inbounds do nó.
(online)	Quantos clientes do nó estão atualmente conectados.
(esgotados)	Quantos clientes do nó expiraram ou esgotaram o limite de tráfego . Clientes desabilitados manualmente não entram neste contador.
(velocidade)	Velocidade atual (ao vivo) de transferência nos inbounds hospedados no nó.

Os contadores de inbounds/clientes/online são vinculados ao nó pelo seu GUID estável (`panelGuid`), não pelo id local – para que um cliente em um subnó seja contabilizado exatamente sob o subnó, e não sob o nó intermediário através do qual ele é sincronizado.

Para inbounds hospedados no nó, a página exibe clientes online, contadores e **velocidade de transferência atual**. A vinculação por GUID estável distingue corretamente até mesmo nós "clonados" com o mesmo `panelGuid` .

Status do nó

Status	Quando é definido
online	O nó respondeu com <code>success=true</code> à consulta <code>panel/api/server/status</code> .
offline	O nó não respondeu, retornou erro HTTP, <code>success=false</code> ou uma resposta irreconhecível.
unknown	Valor inicial, enquanto o nó ainda não foi consultado nenhuma vez.

Quando uma consulta falha, o texto do erro é salvo e exibido de forma compreensível, o que ajuda a diagnosticar a causa do «offline».

12.5. Ações sobre o nó

- **Testar conexão** (`POST /test`) – no formulário do nó, testa a conectividade com os parâmetros digitados (ainda não salvos) com timeout de 6 s. Resultado: «Conexão OK ({ms} ms)» ou «Falha ao conectar». Útil para depurar endereço/porta/token/TLS antes de salvar.
- **Verificar agora** (botão «Verificar agora», `POST /probe/:id`) – consulta não planejada de um nó já salvo; atualiza imediatamente o status e as métricas (CPU/memória/uptime/latência/versões) e registra o heartbeat. Em caso de falha – «Verificação falhou».

Exemplo: testar e consultar um nó via API do master. «Testar conexão» experimenta parâmetros ainda não salvos do formulário:

```
POST /panel/api/nodes/test
Content-Type: application/json

{ "scheme": "https", "address": "de-frankfurt-1.example.com", "port": 2053,
  "basePath": "/", "apiToken": "eyJhbGciOi...", "tlsMode": "verify" }
```

Consulta não planejada de um nó já salvo com id 7:

```
POST /panel/api/nodes/probe/7
```

- **Atualizar painel** (`POST /updatePanel` com corpo `{ids:[...]}`) – inicia no nó seu atualizador automático padrão: o nó baixa o último release do 3X-UI e reinicia com ele. O botão **Atualizar selecionados ({count})** faz isso para vários nós marcados de uma vez. Próximo ao nó é exibido um indicador: **Atualização disponível** ou **Atualizado**, com base na comparação da versão do painel do nó com a mais recente.

Exemplo: atualizar vários nós em uma única solicitação. No corpo são passados os ids dos nós marcados; apenas os habilitados e `online` serão atualizados, os demais serão retornados como ignorados.

```
POST /panel/api/nodes/updatePanel
Content-Type: application/json

{ "ids": [3, 7, 12] }
```

Resposta do tipo «Atualização iniciada em 2 nós, 1 falhou»: o nó 12, por exemplo, pode ter estado offline e por isso foi ignorado.

- Confirmação: «Atualizar {count} nós para a versão mais recente? Cada nó selecionado baixará o último release e reiniciará. Somente nós habilitados e online serão atualizados».
- **Apenas nós habilitados com status online são atualizados.** Um nó desabilitado nos resultados é marcado como «node is disabled», offline — «node is offline». Resultado: «Atualização iniciada em {ok} nós, {failed} falhou». Se nenhum nó adequado for selecionado — «Selecione pelo menos um nó habilitado e online».
- **Set Cert from Panel** (auxiliar, GET /webCert/:id) — ao criar um inbound no nó, permite substituir os caminhos para o **próprio** certificado TLS web do nó (e não do painel central), para que os arquivos existam exatamente no nó. Requer que o nó esteja habilitado e acessível.
- **Excluir nó** (POST /del/:id) — confirmação: «Excluir nó "{name}"? Isso interromperá o monitoramento do nó. O painel remoto em si não será afetado». Exclui o registro do nó e suas estatísticas de tráfego acumuladas; o painel remoto continua funcionando normalmente. **Um nó só pode ser excluído depois que todos os inbounds forem removidos dele.** Se pelo menos um inbound ainda estiver vinculado ao nó (via node_id), o painel rejeitará a exclusão com um erro do tipo «cannot delete node: N inbound(s) still attached to it; detach or delete them first» — primeiro desassocie ou exclua esses inbounds, depois exclua o nó. Isso elimina inbounds «órfãos» com referência pendente a um nó excluído.

12.6. Histórico de métricas

O botão/gráfico de histórico acessa GET /history/:id/:metric/:bucket. Métricas disponíveis: **cpu** e **mem** — acumuladas a cada heartbeat bem-sucedido. O tamanho do intervalo de agregação (bucket, em segundos) é limitado por uma lista branca:

Exemplo: solicitação de histórico. Gráfico de utilização de CPU do nó 7 com agregação em intervalos de 60 segundos (retorna até 60 pontos):

```
GET /panel/api/nodes/history/7/cpu/60
```

Para memória e modo «tempo real» (2 s) — respectivamente .../7/mem/60 e .../7/cpu/2. Valores fora da lista branca são rejeitados («invalid metric» / «invalid bucket»).

Bucket (s)	Uso
2	Modo tempo real
30	Intervalos de 30 s
60	Intervalos de 1 min
120	Intervalos de 2 min
180	Intervalos de 3 min
300	Intervalos de 5 min

Retorna até 60 pontos. Métrica ou bucket inválidos são rejeitados («invalid metric» / «invalid bucket»).

12.7. Como inbounds e clientes são sincronizados

Um inbound «pertence» a um nó através do campo `node_id` (no editor de inbound, o nó é selecionado):

Exemplo: token no formulário do nó. O token é obtido no painel filho (Configurações → Token de API) e colado no campo **Token de API** do master. A cada consulta, o master o envia no cabeçalho:

```
GET https://panel.example.com:2053/panel/api/server/status
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.abc123...
```

Se o painel filho tiver um **caminho base** (web base path) configurado, por exemplo `/secret/`, o master o inserirá automaticamente antes de `panel/api/server/status` → `https://panel.example.com:2053/secret/panel/api/server/status`.

1. **Implantação de configuração (reconcile).** A qualquer alteração em um inbound/cliente vinculado a um nó, o nó é marcado como «sujo». A tarefa em segundo plano, para cada nó habilitado **com status onLine** e com alterações pendentes, implanta no nó seus inbounds (por `node_id`) e em seguida limpa o flag de estado «sujo». Um nó que está desabilitado, offline ou «sujo» é considerado «pendente» — a implantação é adiada até que a conexão seja restaurada.
2. **Coleta de tráfego.** A mesma tarefa solicita ao nó um snapshot de tráfego e o mescla nas estatísticas locais. Com base no tráfego mesclado, a verificação de esgotamento de limites/prazos é executada e, se necessário, os clientes são desconectados; o contador «esgotados» por nó reflete exatamente isso. Se o nó estiver inacessível, seus clientes online são limpos.

Para um cliente vinculado a vários painéis ao mesmo tempo, o master, na mesma tarefa, distribui adicionalmente para os nós o consumo de tráfego **total de todos os painéis** desse cliente (em uma tabela separada no nó, chave — GUID do master; sobrescrita a cada envio, portanto uma redefinição no lado do master também é propagada). No nó, o tráfego do cliente exibe o maior dos dois valores — o local ou o recebido —, e ao exceder a cota total, o cliente é desconectado **localmente no próprio nó** (pelo mesmo mecanismo de reinicialização do Xray no desligamento automático, que interrompe as conexões já estabelecidas). Isso elimina a situação em que o nó via apenas sua parcela de tráfego, subestimava o consumo e continuava atendendo um cliente que já havia esgotado o limite total. Ao redefinir o tráfego, renovar automaticamente ou excluir o cliente, os contadores enviados são limpos.

3. **Heartbeat.** Uma tarefa separada em segundo plano consulta periodicamente todos os nós **habilitados** (com limite de paralelismo) via `panel/api/server/status`, atualiza status/métricas/versões e, se houver clientes web, distribui a árvore atualizada de nós via WebSocket.

12.8. Cadeias de nós (subnós / nós transitivos)

A topologia pode não ser plana: um nó pode ser master para seus próprios nós. Esses painéis subordinados aparecem para você como **Subnós** — estas são **projeções somente leitura**, obtidas do nó direto.

- Dica: «Somente leitura: nó subordinado acessível via {pai}. Gerencie-o a partir do próprio painel {pai}». Ou seja, o subnó não pode ser editado, excluído ou atualizado aqui — todas as operações sobre ele são realizadas a partir do painel de seu pai direto.

- A identidade do subnó é determinada pelo seu GUID; graças a isso, os clientes online e inbounds são contabilizados exatamente sob o nó físico que os hospeda, mesmo em uma cadeia `Nó1 → Nó2 → Nó3` (o master «avança» um nível para dentro a cada nó direto).
- Se o nó direto ficar inacessível, seu cache de subnós é limpo, e os subnós desaparecem da árvore até que a conexão seja restaurada.

12.9. Nós: novidades na versão 3.3.0

Na versão 3.3.0, a seção **Nós** recebeu três melhorias notáveis: atribuição correta de tráfego e clientes online em topologias multi-hop, sincronização de client-IP entre nós e um indicador de status separado para o caso em que o painel do nó está ativo, mas o núcleo Xray nele travou.

1. Multi-hop: atribuição correta de tráfego na cadeia de subnós

Anteriormente, os contadores (número de inbounds, clientes online, esgotados) eram calculados no nível do nó «direto». Se você tiver uma cadeia do tipo `Master → Nó1 → Nó2 → Nó3`, tudo o que fisicamente reside em `Nó2 / Nó3` era erroneamente atribuído ao `Nó1`, através do qual chegou ao master. Na versão 3.3.0, a atribuição é feita pelo fonte real.

Como funciona:

- **Os subnós se tornam visíveis como linhas separadas.** Cada painel publica a lista de seus nós diretos; são incluídos apenas os nós com `Guid` conhecido — uma identidade estável é necessária para atribuir o nó exatamente um «salto» acima. O master periodicamente (a partir da tarefa de heartbeat) busca essas listas e as armazena em cache, então adiciona aos nós diretos os subnós «transitivos».
- **Nós transitivos são somente leitura.** Na UI eles são marcados como **«Subnó»** com a dica: *«Somente leitura: nó subordinado acessível via {pai}. Gerencie-o a partir do próprio painel {pai}»*. Não há botões de controle nessa linha — o nó é gerenciado a partir do painel de seu pai imediato.
- **Hierarquia via GUID.** O `ParentGuid` do nó direto é o GUID do próprio master; o do nó transitivo é o GUID de seu nó pai. Assim é construída a árvore.
- **A fonte de verdade para os contadores é `origin_node_guid` no inbound.** Este é o `panelGuid` do nó que hospeda fisicamente esse inbound. É definido ao sincronizar o inbound do nó e **preservado como está nos saltos subsequentes**, portanto um inbound profundamente aninhado é atribuído ao nó real, não ao intermediário. Com base nesse GUID, os contadores de número de inbounds, clientes online e clientes esgotados são recalculados. Lógica de seleção da chave:

Estado do inbound	Atribuído a
<code>origin_node_guid</code> definido	este GUID (nó-fonte real)
vazio, mas <code>node_id</code> definido	GUID sintético do nó (build antigo, ainda não reportou seu <code>panelGuid</code>)
vazio e <code>node_id</code> vazio	GUID próprio do master (inbound no Xray local)

Os clientes online também são agrupados por GUID, portanto cada linha de nó exibe apenas os que estão realmente conectados a ele.

O que o usuário vê: em uma topologia plana (nós diretamente sob o master), nada muda — os contadores por GUID e por `id` coincidem. Mas assim que uma cadeia de nós aparece, linhas de «Subnós» surgem na lista, e os números de inbounds/online/esgotados de cada nó agora refletem exatamente sua própria carga, e não a soma de tudo que passou por ele em trânsito.

2. Sincronização de client-IP do access.log entre nós

O limite por IP (`limitIp` do cliente) depende dos endereços que o Xray grava em seu `access.log`. Anteriormente, cada nó via apenas as conexões consigo mesmo, portanto a restrição «no máximo N IPs por cliente» não funcionava no cluster: o cliente podia se conectar a nós diferentes e contornar o limite. Na versão 3.3.0, os IPs observados são sincronizados em todo o cluster.

Como funciona:

- Em cada nó, uma tarefa em segundo plano analisa o `access.log`, extraíndo de cada linha o IP, o e-mail do cliente e o timestamp, e os armazena em uma tabela local (um registro por e-mail, IPs armazenados como array JSON `{ip, timestamp}`). Endereços locais `127.0.0.1` e `::1` são descartados.
- A sincronização **a cada 10 segundos** realiza uma troca bidirecional para cada nó habilitado e online: busca os IPs do nó e os mescla na tabela local, depois envia para o nó o quadro geral do master.
- A mesclagem combina observações antigas e recebidas **sem contagem dupla** de um mesmo IP visto em vários nós, e **sem ressuscitar** registros desatualizados: aplica-se o mesmo limiar de antiguidade que na tarefa local — **30 minutos**. Para cada IP é salvo o timestamp mais recente. Registros de outros nós recebem um novo id local (os espaços de id dos nós são independentes); a inserção concorrente do mesmo e-mail é protegida contra duplicatas.
- Ao calcular o limite, um IP é considerado «ativo» se foi observado no scan local atual ou tem um timestamp muito recente na base sincronizada (**dentro de 2 minutos**). É exatamente isso que faz o limite funcionar em escala de todo o cluster, mesmo se o endereço foi observado em outro nó. Ao exceder o limite, os IPs «ativos» mais antigos são enviados para o log do fail2ban, e as conexões são forçadamente interrompidas (remove/re-add do cliente via Xray API).

O que o usuário vê: o limite por número de IPs agora se aplica a todo o cluster, e não a cada nó individualmente; no painel, para o cliente são visíveis os IPs observados em qualquer nó (dentro da janela de 30 minutos). Não há botão/configuração separado para isso — a sincronização ocorre automaticamente em segundo plano, desde que o `access.log` do nó esteja habilitado e acessível (para o próprio limite, o Fail2Ban também é necessário no nó).

3. Indicador de status separado: painel do nó online, mas Xray travou

Anteriormente, o status do nó era essencialmente «online / offline». Se o painel do nó respondia, o nó era considerado online — mesmo quando o núcleo Xray nele não estava funcionando, e os clientes efetivamente não podiam se conectar. Na versão 3.3.0, a saúde do painel e a saúde do núcleo Xray são separadas.

Como funciona:

- Ao consultar o nó, o master obtém da resposta do `/panel/api/server/status` remoto os campos `xray.state` e `xray.errorMsg` e os salva no nó. Esses campos são preenchidos mesmo em um ping bem-sucedido do painel, quando o núcleo não está saudável — exatamente para distinguir a acessibilidade do painel do estado do Xray.

- Valores de `xray.state` : "running" (em execução), "stop" (parado), "error" (erro).
- Esses valores são traduzidos em status do nó. Aos status habituais foram adicionados novos:

Chave de status	Descrição	Quando é exibido
<code>online</code>	«Online»	o painel responde, o Xray está funcionando (<code>running</code>)
<code>offline</code>	«Offline»	o painel está inacessível / ping falhou
<code>unknown</code>	«Desconhecido»	o estado ainda não foi determinado
<code>xrayError</code>	«Erro do Xray»	o painel está online, mas o núcleo Xray está no estado <code>error</code> (há <code>errorMsg</code>)
<code>xrayStopped</code>	«Parado»	o painel está online, mas o Xray está parado (<code>stop</code>)

- Para esse estado, a UI usa um **indicador roxo separado** (cor diferente do verde «online» e do vermelho «offline»). O roxo sinaliza diretamente: é possível alcançar o nó, o problema está no próprio núcleo Xray, não na rede ou no painel em si.

O que o usuário vê: em vez de um enganoso «verde» com o núcleo caído, o nó é destacado em **roxo** com o status «**Erro do Xray**» ou «**Parado**». Isso mostra imediatamente que é preciso consertar o Xray no nó (reiniciar o núcleo, verificar o `errorMsg`), e não investigar a acessibilidade do próprio nó. O mesmo `xrayState` / `xrayError` é propagado para os subnós transitivos (veja item 1), portanto um estado incorreto do núcleo é visível em toda a cadeia.

13. Configurações do Pannel

A seção "Configurações" (título da página — **Configurações**, em inglês *Panel Settings*) controla o comportamento da própria interface web 3X-UI: em qual endereço e porta ela escuta, como é protegida, como interage com o bot do Telegram e serviços externos, e em qual fuso horário executa as tarefas agendadas. Cada parâmetro é armazenado na tabela `settings` do banco de dados como um par "chave — valor"; se o valor não estiver no banco, o valor padrão é aplicado.

Importante — aplicação das alterações. Qualquer alteração nesta página deve ser salva pelo botão **Salvar** (*Save*) e, em seguida, o painel deve ser reiniciado para que as alterações entrem em vigor. A dica exibida é: "Salve as alterações e reinicie o painel para aplicá-las." Ao salvar, é exibida a notificação "Configurações alteradas".

13.1. Salvar e reiniciar o painel

Elemento	Finalidade
Salvar (<i>Save</i>)	Grava todos os campos do formulário no banco de dados (<code>POST /panel/setting/update</code>). Antes de gravar, os valores são validados — endereços, portas ou caminhos incorretos serão rejeitados e o painel retornará um erro.
Reiniciar painel (<i>Restart Panel</i>)	Reinicia o servidor web do painel (<code>POST /panel/setting/restartPanel</code>). A reinicialização ocorre com um atraso de 3 segundos. Dica: "Tem certeza de que deseja reiniciar o painel? Confirme e o painel será reiniciado em 3 segundos. Se o painel ficar indisponível, verifique o log do servidor." Em caso de sucesso — "Painel reiniciado com sucesso."
Restaurar configurações padrão (<i>Reset to Default</i>)	Exclui todas as configurações salvas no banco de dados; em seguida, o painel passa a usar os valores padrão. As credenciais do administrador não são redefinidas por esta operação.

A reinicialização é feita enviando ao processo do painel o sinal `SIGHUP` (ou por meio de um hook de reinicialização registrado). No Windows, a reinicialização automática via sinal não é suportada. **As alterações nos parâmetros de escuta (IP, porta, caminho, domínio, certificados, fuso horário) são aplicadas somente após reiniciar o painel.**

13.2. Configurações gerais (aba "Painel" / *General*)

Idioma da interface (*Language*)

Idioma da interface web do painel. Idiomas disponíveis: `en-US` (inglês), `ru-RU` (russo), `zh-CN`, `zh-TW`, `fa-IR`, `ar-EG`, `es-ES`, `id-ID`, `ja-JP`, `pt-BR`, `tr-TR`, `uk-UA`, `vi-VN`. Esta é uma configuração de exibição e não afeta o funcionamento do Xray.

Tipo de calendário (*Calendar Type*)

- **Chave:** `datepicker`
- **Valor padrão:** `gregorian` (gregoriano).

- **Finalidade:** tipo de calendário utilizado na seleção de datas (por exemplo, ao definir a data de validade dos clientes). Dica: "As tarefas agendadas serão executadas de acordo com este calendário." O valor alternativo é o calendário persa (jalali), muito utilizado pelo público iraniano do painel.

Tamanho da paginação (*Pagination Size*)

- **Chave:** `pageSize`
- **Valor padrão:** `25`
- **Valores permitidos:** inteiro de `0` a `1000`.
- **Finalidade:** número de linhas por página nas tabelas (listas de conexões/inbound). Dica: "Define o tamanho da página para a tabela de conexões. Defina 0 para desativar" — quando `0`, a paginação é desativada e todos os registros são exibidos em uma única lista.
- **Reinicialização do painel não é necessária** (configuração de exibição).

Reiniciar Xray após desativação automática (*Restart Xray After Auto Disable*)

- **Chave:** `restartXrayOnClientDisable`
- **Valor padrão:** `true`
- **Finalidade:** quando um cliente é desativado automaticamente (por expiração do prazo de validade ou por atingir o limite de tráfego), o Xray é reiniciado para encerrar as conexões já estabelecidas desse cliente. Dica: "Quando um cliente é desativado automaticamente por expiração do prazo ou limite de tráfego, reiniciar o Xray." A função em si não mudou — o interruptor apenas reside na aba "Painel" (*General*), junto com as demais configurações gerais.

Modelo de observação e caractere separador (*Remark Model & Separation Character*)

- **Chave:** `remarkModel`
- **Valor padrão:** `-ieo`
- **Finalidade:** define como o nome (remark) da configuração é formado na assinatura. A string consiste no **primeiro caractere** — o separador — seguido de uma **sequência de letras de ordem**:
 - `i` — observação do inbound (*inbound remark*);
 - `e` — e-mail do cliente;
 - `o` — rótulo adicional (*extra*).

Com o valor padrão `-ieo`, o separador é `-` e a ordem das partes é: inbound → e-mail → extra (por exemplo, `MyInbound-user@mail-extra`). Partes vazias são omitidas. O campo "Exemplo de observação" (*Sample Remark*) na interface mostra uma pré-visualização do nome gerado. A inclusão do e-mail no nome depende adicionalmente do parâmetro "Incluir e-mail no nome" nas configurações de assinatura (seção sobre assinaturas).

Exemplo: como o valor de `remarkModel` afeta o nome da configuração. Suponha que o inbound se chame `VLESS-Reality`, o e-mail do cliente seja `alex@vpn` e o rótulo adicional seja `RU`. Então:

Valor do campo	Nome resultante (remark)
-ieo (padrão)	VLESS-Reality-alex@vpn-RU
_ie	VLESS-Reality_alex@vpn
-ei	alex@vpn-VLESS-Reality
o (espaço como separador, apenas rótulo)	RU

O primeiro caractere da string é sempre o separador; as demais letras definem quais partes e em que ordem comporão o nome.

13.3. Acesso ao painel: IP, porta, caminho, domínio, certificado

Este grupo define o ponto de entrada de rede do painel. **Todas as alterações aqui são aplicadas somente após reiniciar o painel.**

Campo	Chave	Valor padrão	Descrição
Endereço IP para gerenciar o painel (<i>Listen IP</i>)	webListen	"" (vazio)	IP em que a interface web escuta. Vazio = escutar em todos os IPs. Dica: "Deixe em branco para conexão de qualquer IP". Se definido, deve ser um endereço IP válido (caso contrário, o salvamento é rejeitado).
Domínio do painel (<i>Listen Domain</i>)	webDomain	"" (vazio)	Nome de domínio do painel para verificação de requisição por domínio. Vazio = aceitar conexões de quaisquer domínios e IPs. Dica: "Deixe em branco para conexão de quaisquer domínios e IPs."
Porta do painel (<i>Listen Port</i>)	webPort	2053	Porta em que o painel opera. Dica: "Porta em que o painel opera". Permitido de 1 a 65535. A porta deve estar livre; o painel e o serviço de assinatura não podem usar simultaneamente o mesmo par IP:porta.
Caminho URI (<i>URI Path</i>)	webBasePath	/	Caminho base de URL do painel (basePath). Dica: "Deve começar com '/' e terminar com '/'". Ao salvar, o painel adiciona automaticamente as barras inicial e final, caso estejam ausentes. Caracteres proibidos no caminho são rejeitados.

Certificado do painel (TLS / HTTPS)

Campo	Chave	Valor padrão	Descrição
Caminho para o arquivo de chave pública do certificado do painel (<i>Public Key Path</i>)	webCertFile	""	Caminho completo para o arquivo de certificado (cadeia). Dica: "Insira o caminho completo começando com '/'".
Caminho para o arquivo de chave privada do certificado do painel (<i>Private Key Path</i>)	webKeyFile	""	Caminho completo para o arquivo de chave privada. Dica: "Insira o caminho completo começando com '/'".

Se **ao menos um** dos caminhos de certificado/chave for definido, o painel tenta carregar o par "certificado + chave" ao salvar; em caso de erro (arquivo inexistente, incompatibilidade entre chave e certificado), o

salvamento é rejeitado. Quando ambos os caminhos corretos são definidos, o painel passa a usar HTTPS. Ambos os campos vazios = o painel opera via HTTP simples.

Avisos de segurança (*Security warnings*). O painel exibe o bloco "Seu painel pode estar exposto:" com avisos quando detecta uma configuração insegura:

- operação via HTTP simples – "configure o TLS para produção";
- porta padrão 2053 – "altere-a para uma porta aleatória";
- caminho base padrão / – "altere-o para um valor aleatório";
- caminho de assinatura padrão /sub/ e assinatura JSON /json/ – "altere-o". São recomendações, não bloqueios.

13.4. Sessão, proxy do painel e proxies confiáveis (aba "Proxy e servidor" / *Proxy and Server*)

Duração da sessão (*Session Duration*)

- **Chave:** `sessionMaxAge`
- **Valor padrão:** 360 (minutos, ou seja, 6 horas).
- **Valores permitidos:** de 1 a 525600 minutos (1 ano).
- **Finalidade:** por quanto tempo o administrador permanece autenticado sem precisar fazer login novamente. A unidade é **minuto**. Dica: "Duração da sessão no sistema (valor: minuto)".

Outbound para tráfego do painel (*Panel Traffic Outbound*)

- **Chave:** `panelOutbound`
- **Valor padrão:** "" (vazio = conexão direta).
- **Finalidade:** define o **outbound** Xray pelo qual o painel envia **suas próprias requisições** – verificações de versão e download do painel/Xray, chamadas ao Telegram, atualização regular de arquivos geo – para contornar a filtragem de servidor do GitHub/Telegram. O campo é uma **lista suspensa**: nela estão listados os outbounds do template de configuração do Xray, os outbounds de assinaturas de outbound, bem como os **balanceadores** de rota (em grupo separado). Outbounds do tipo `blackhole` são excluídos da lista – roteá-los para um "buraco negro" não faz sentido. Dica literal: "Roteia as próprias requisições do painel – verificações de versão e downloads do painel/Xray, Telegram e atualização regular de arquivos geo – por este outbound Xray para contornar a filtragem de servidor do GitHub/Telegram. Um inbound de bridge loopback local é adicionado automaticamente à configuração em execução e aplicado em tempo real. A atualização automática de Geodata integrada ao Xray não é afetada; ela tem seu próprio outbound para download. Deixe em branco para conexão direta."

Como funciona. Ao selecionar um outbound, o painel adiciona automaticamente à configuração em execução um inbound loopback de serviço (bridge SOCKS com a tag `panel-egress`) e uma regra de roteamento que direciona o próprio tráfego HTTP do painel para o outbound selecionado. Se um balanceador for selecionado, o `balancerTag` é inserido na regra e o tráfego do painel é distribuído entre seus participantes. O bridge e a regra são aplicados **em tempo real**, sem reinicialização completa do

painel. Deixe o campo vazio para conexão direta. A atualização automática de geo-dados integrada ao Xray **não é afetada** por esta configuração – ela tem seu próprio outbound dentro do roteamento do Xray.

- **Formato:** `socks5://` (ou `socks5h://`) ou `http(s)://`, com autenticação se necessário no formato `socks5://user:pass@host:port`. Os esquemas suportados são estritamente: `socks5`, `socks5h`, `http`, `https` – outros esquemas são considerados inválidos e o painel volta à conexão direta. O exemplo típico é um inbound SOCKS local do próprio Xray.
- Dica literal: "Roteia as requisições de saída do próprio painel (atualizações de geo, verificações de versão do Xray/painel, Telegram) por este proxy para contornar a filtragem de servidor do GitHub/Telegram. Aceita `socks5://` ou `http(s)://`, ex.: inbound SOCKS local do Xray. Deixe em branco para conexão direta."
- Um proxy inválido não gera erro ao salvar – o painel simplesmente usa conexão direta e registra um aviso no log.

Exemplo de valores do campo. Se o servidor já possui um inbound SOCKS local do Xray na porta `10808`, direcione as próprias requisições do painel por ele:

```
socks5://127.0.0.1:10808
```

Para um proxy HTTP externo com autenticação:

```
http://user:pass@proxy.example.com:8080
```

Após salvar e reiniciar, o painel buscará atualizações de geo-bases, verificará versões e acessará o Telegram pelo proxy especificado.

CIDRs de proxy confiáveis (*Trusted proxy CIDRs*)

- **Chave:** `trustedProxyCIDRs`
- **Valor padrão:** `127.0.0.1/32, ::1/128` (apenas host local).
- **Formato:** lista de endereços IP ou sub-redes CIDR separados por vírgula (por exemplo, `10.0.0.0/8, 192.168.1.5`). Cada elemento é validado como IP ou CIDR – um valor incorreto é rejeitado ao salvar.
- **Finalidade:** lista as origens com permissão para definir os cabeçalhos `X-Forwarded-Host`, `X-Forwarded-Proto` e o cabeçalho de IP real do cliente. Dica literal: "IP/CIDR separados por vírgula com permissão para definir os cabeçalhos de host encaminhado, proto e IP do cliente." Deve ser configurado quando o painel opera por trás de um proxy reverso (nginx, Caddy, etc.) para identificar corretamente os IPs dos clientes e o esquema.

Exemplo: painel atrás de um proxy reverso. Se o nginx está no mesmo host e encaminha requisições ao painel, mantenha a confiança apenas ao host local (valor padrão):

```
127.0.0.1/32, ::1/128
```

Se o proxy está em um servidor separado na rede interna `10.0.0.0/8`, adicione sua sub-rede; caso contrário, o painel ignorará os cabeçalhos enviados por ele e verá o IP do proxy em vez do cliente real:

```
127.0.0.1/32,::1/128,10.0.0.0/8
```

Exemplo do bloco nginx correspondente, que encaminha o IP real e o esquema:

```
proxy_set_header X-Real-IP      $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $host;
```

13.5. Bot do Telegram (aba "Bot do Telegram" / *Telegram Bot*)

Ativar bot do Telegram (*Enable Telegram Bot*)

- **Chave:** `tgBotEnable`
- **Tipo/padrão:** booleano, `false`.
- **Finalidade:** ativa o funcionamento do bot do Telegram. Dica: "Acesso às funções do painel pelo bot do Telegram".

Token do Telegram (*Telegram Token*)

- **Chave:** `tgBotToken`
- **Padrão:** `""`.
- **Finalidade:** token do bot. Dica: "É necessário obter o token junto ao gerenciador de bots do Telegram @botfather".
- **Característica de segurança:** o token é um valor secreto. Na resposta do painel à leitura das configurações, ele não é retornado (o campo é limpo e apenas o flag "configurado/não configurado" é enviado). Se o campo for deixado em branco ao salvar, o token anteriormente salvo **é mantido** (não é apagado).

Idioma do bot do Telegram (*Telegram Bot Language*)

- **Chave:** `tgLang`
- **Padrão:** `en-US`.
- **Finalidade:** idioma das mensagens do bot (independentemente do idioma da interface web). A lista de idiomas disponíveis coincide com os idiomas do painel.

ID de usuário do administrador do bot (*Admin Chat ID*)

- **Chave:** `tgBotChatId`
- **Padrão:** `""`.
- **Formato:** um ou mais Telegram User IDs numéricos **separados por vírgula**.
- **Finalidade:** destinatários de notificações e administradores com permissão para gerenciar o painel pelo bot. Dica: "Um ou mais User IDs do(s) administrador(es) do bot do Telegram. Para obter o User ID, use @userinfobot ou o comando '/id' no bot."

Frequência de notificações (*Notification Time*)

- **Chave:** `tgRunTime`
- **Padrão:** `@daily` (uma vez por dia).
- **Formato:** string no formato **Crontab** (são suportadas tanto expressões cron padrão quanto abreviações como `@daily`, `@hourly`, `@every 1h`). Dica: "Especifique o intervalo de notificações no formato Crontab". Controla os relatórios periódicos do bot.

Exemplos de valores do campo.

Valor	Quando o bot envia o relatório
<code>@daily</code>	uma vez por dia à meia-noite (padrão)
<code>@hourly</code>	a cada hora
<code>@every 6h</code>	a cada 6 horas
<code>0 9 * * *</code>	diariamente às 09:00
<code>30 8 * * 1</code>	toda segunda-feira às 08:30

O horário é calculado no fuso horário definido na configuração "Fuso horário" (item 13.6).

Proxy SOCKS (*SOCKS Proxy*)

- **Chave:** `tgBotProxy`
- **Padrão:** `""`.
- **Finalidade:** proxy SOCKS5 específico para a conexão do bot ao Telegram. Dica: "Se você precisa de um proxy Socks5 para se conectar ao Telegram, configure seus parâmetros conforme o guia." Aplica-se especificamente ao tráfego do bot (diferente do "Proxy de rede do painel" geral do item 13.4).

Servidor de API do Telegram (*Telegram API Server*)

- **Chave:** `tgBotAPIServer`
- **Padrão:** `""` (usar o servidor padrão `api.telegram.org`).
- **Formato:** URL `http(s)://...`; ao salvar, passa por validação de URL — endereços inválidos são rejeitados. Dica: "Servidor de API do Telegram utilizado. Deixe em branco para usar o servidor padrão." Necessário para um servidor de Telegram Bot API implantado de forma independente.

Notificações do bot (grupo "Notificações" / *Notifications*)

Campo	Chave	Padrão	Descrição
Backup do banco de dados (<i>Database Backup</i>)	<code>tgBotBackup</code>	<code>false</code>	Enviar ao Telegram o arquivo de backup do banco de dados junto com o relatório. Dica: "Enviar notificação com o arquivo de backup do banco de dados".
Notificação de login (<i>Login Notification</i>)	<code>tgBotLoginNotify</code>	<code>true</code>	Notificar quando houver tentativa de login no painel. Dica: "Exibe o nome de usuário, endereço IP e horário quando alguém tenta acessar seu painel."

Campo	Chave	Padrão	Descrição
Antecedência da notificação de expiração (<i>Expiration Date Notification</i>)	<code>expireDiff</code>	<code>0</code>	Com quantos dias de antecedência ao vencimento do cliente enviar a notificação. <code>0</code> – desativado. Permitido <code>>= 0</code> . Dica: "Receber notificação de expiração da sessão antes de atingir o valor limite (valor: dia)".
Limite de tráfego para notificação (<i>Traffic Cap Notification</i>)	<code>trafficDiff</code>	<code>0</code>	Limite de tráfego restante para notificação. Dica: "Receber notificação de esgotamento de tráfego antes de atingir o limite (valor: GB)". Permitido <code>0–100</code> .
Limite de carga de CPU (<i>CPU Load Notification</i>)	<code>tgCpu</code>	<code>80</code>	Notificar os administradores se o uso de CPU ultrapassar o limite (em %). Permitido <code>0–100</code> . Dica: "Notificar os administradores no Telegram se a carga de CPU ultrapassar este limite (valor: %)".

13.6. Data e hora (aba "Data e hora" / *Date and Time*)

Fuso horário (*Time Zone*)

- **Chave:** `timeLocation`
- **Valor padrão:** `Local` (fuso horário do sistema do servidor).
- **Formato:** nome de zona da base IANA tz (por exemplo, `Europe/Moscow`, `UTC`, `Asia/Tehran`).
- **Finalidade:** fuso horário no qual o painel executa as tarefas agendadas (relatórios do bot, redefinição/verificação de tráfego, expiração de prazos). Dica: "As tarefas agendadas são executadas de acordo com o horário neste fuso horário".
- **Validação:** ao salvar, a zona é verificada – uma zona inexistente é rejeitada. Se posteriormente um valor incorreto estiver no banco de dados, o painel em tempo de execução voltará para `Local` e, se este também estiver indisponível, para `UTC`.

13.7. Tráfego externo e comportamento do Xray (aba "Tráfego externo" / *External Traffic*)

Campo	Chave	Padrão	Descrição
Informar tráfego externo (<i>External Traffic Inform</i>)	<code>externalTrafficInformEnable</code>	<code>false</code>	Notificar uma API externa a cada atualização de tráfego. Dica: "Notificar uma API externa a cada atualização de tráfego."
URI de informação de tráfego externo (<i>External Traffic Inform URI</i>)	<code>externalTrafficInformURI</code>	<code>""</code>	URL para o qual o painel envia atualizações de tráfego. Passa por validação de URL ao salvar. Dica: "As atualizações de tráfego são enviadas para este URI".
Reiniciar Xray após desativação automática (<i>Restart Xray After Auto Disable</i>)	<code>restartXrayOnClientDisable</code>	<code>true</code>	Reiniciar o Xray quando um cliente é desativado automaticamente por expiração ou por exceder o limite de tráfego. Dica: "Quando um cliente é desativado automaticamente por expiração do prazo ou limite de tráfego, reiniciar o Xray." O interruptor está na aba "Painel" (General) – veja o item 13.2; aqui é apresentado por completude.

13.8. Outros: template de configuração do Xray e URL de verificação

Template de configuração do Xray (*xrayTemplateConfig*)

- **Chave:** `xrayTemplateConfig`
- **Padrão:** template JSON integrado (embedded), fornecido com a build.
- **Finalidade:** template JSON base da configuração do Xray-core, sobre o qual o painel constrói inbound/outbound. Este valor **não é retornado** na resposta comum de todas as configurações e é editado em uma página de configuração separada do Xray, não na lista geral de campos de configurações do painel. O template padrão está disponível via `GET /panel/setting/getDefaultJsonConfig`.

URL de verificação de saída (*xrayOutboundTestUrl*)

- **Chave:** `xrayOutboundTestUrl`
- **Padrão:** `https://www.google.com/generate_204`
- **Finalidade:** URL utilizado ao verificar a funcionalidade das conexões de saída (outbound). Ao ser definido, passa por sanitização como URL HTTP(S).

13.9. Conta do administrador e tokens de API

Esses parâmetros estão na aba adjacente ("Conta" / *Authentication*) e são detalhados na seção sobre segurança; aqui – um breve resumo das chaves.

- **Alteração de credenciais** (campos "Login atual", "Senha atual", "Novo login", "Nova senha") é salva por uma requisição separada `POST /panel/setting/updateUser`. O login e a senha atuais corretos são necessários; o novo login e a nova senha não devem estar em branco. Mensagens: "Você alterou com sucesso as credenciais do administrador." / "Nome de usuário ou senha incorretos".
- **Autenticação de dois fatores (2FA)** – chaves `twoFactorEnable` (padrão `false`) e o secreto `twoFactorToken`. O token é secreto: com 2FA ativado, deixar o campo em branco ao salvar não apaga o token existente. Na **primeira** ativação do 2FA, o painel invalida as sessões atuais (a "época de login" é incrementada).
- **Tokens de API** são gerenciados por endpoints separados (`/panel/setting/apiTokens...`): listagem, criação (`apiTokens/create`), exclusão, ativação/desativação. O próprio token é exibido **apenas uma vez na criação** e não é armazenado em formato legível: "Copie este token agora. Por razões de segurança, ele não é armazenado em formato legível e não será exibido novamente."

Os detalhes sobre 2FA, senhas, sincronização LDAP e formatos de assinatura (JSON/Clash, fragmentation, noises, mux) estão nas seções específicas correspondentes do manual.

13.10. Alterações de API na versão 3.3.0 (importante para integrações)

Na versão 3.3.0, a estrutura dos caminhos da API do servidor foi alterada. Se você possui integrações externas (scripts, bots, painéis centrais, tarefas de CI) que acessam o painel via HTTP, elas **precisam ser atualizadas**, caso contrário deixarão de funcionar.

⚠ BREAKING CHANGE: os endpoints `/panel/setting/*` e `/panel/xray/*` foram movidos para `/panel/api`

Anteriormente, o gerenciamento de configurações do painel e da configuração do Xray ficava separado, nos caminhos `/panel/setting/*` e `/panel/xray/*`. Agora ambos os conjuntos estão registrados dentro do grupo de API comum `/panel/api`. Os caminhos antigos foram **removidos completamente** – uma requisição a eles retornará 404.

Por que isso foi feito: todo o grupo `/panel/api` passa por uma verificação de acesso unificada, ou seja, esses endpoints agora aceitam o mesmo cabeçalho `Authorization: Bearer <token>` que o restante da API. O token de API representa acesso completo de administrador, e assim toda a superfície da API se tornou uniforme.

O que NÃO mudou: as páginas da interface web (rotas SPA) `/panel/settings` e `/panel/xray` permanecem no lugar – a mudança é apenas nos endpoints de API do servidor.

Tabela de correspondência de caminhos (antigo → novo)

O prefixo para todos os caminhos abaixo – simplesmente `api/` foi adicionado após `/panel/`.

Antes (≤ 3.2.x)	Depois (3.3.0)	Método
<code>/panel/setting/all</code>	<code>/panel/api/setting/all</code>	POST
<code>/panel/setting/defaultSettings</code>	<code>/panel/api/setting/defaultSettings</code>	POST
<code>/panel/setting/update</code>	<code>/panel/api/setting/update</code>	POST
<code>/panel/setting/updateUser</code>	<code>/panel/api/setting/updateUser</code>	POST
<code>/panel/setting/restartPanel</code>	<code>/panel/api/setting/restartPanel</code>	POST
<code>/panel/setting/getDefaultJsonConfig</code>	<code>/panel/api/setting/getDefaultJsonConfig</code>	GET
<code>/panel/setting/apiTokens</code>	<code>/panel/api/setting/apiTokens</code>	GET
<code>/panel/setting/apiTokens/create</code>	<code>/panel/api/setting/apiTokens/create</code>	POST
<code>/panel/setting/apiTokens/delete/:id</code>	<code>/panel/api/setting/apiTokens/delete/:id</code>	POST
<code>/panel/setting/apiTokens/setEnabled/:id</code>	<code>/panel/api/setting/apiTokens/setEnabled/:id</code>	POST
<code>/panel/xray/</code>	<code>/panel/api/xray/</code>	POST
<code>/panel/xray/update</code>	<code>/panel/api/xray/update</code>	POST
<code>/panel/xray/getDefaultJsonConfig</code>	<code>/panel/api/xray/getDefaultJsonConfig</code>	GET
<code>/panel/xray/getXrayResult</code>	<code>/panel/api/xray/getXrayResult</code>	GET
<code>/panel/xray/getOutboundsTraffic</code>	<code>/panel/api/xray/getOutboundsTraffic</code>	GET
<code>/panel/xray/resetOutboundsTraffic</code>	<code>/panel/api/xray/resetOutboundsTraffic</code>	POST

Antes (≤ 3.2.x)	Depois (3.3.0)	Método
<code>/panel/xray/testOutbound</code>	<code>/panel/api/xray/testOutbound</code>	POST
<code>/panel/xray/warp/:action</code>	<code>/panel/api/xray/warp/:action</code>	POST
<code>/panel/xray/nord/:action</code>	<code>/panel/api/xray/nord/:action</code>	POST
<code>/panel/xray/outbound-subs</code> (e <code>/outbound-subs/*</code>)	<code>/panel/api/xray/outbound-subs</code> (e <code>/outbound-subs/*</code>)	GET/POST/ DELETE

Os sub-caminhos, corpos de requisição e formatos de resposta não foram alterados — apenas o **prefixo** mudou.

Como corrigir integrações existentes

1. Encontre em seus scripts/configurações todas as ocorrências de `/panel/setting/` e `/panel/xray/`.
2. Substitua o prefixo: adicione `api/` imediatamente após `/panel/` (por exemplo, `/panel/setting/all` → `/panel/api/setting/all`).
3. Corpos de requisição, parâmetros e formato de resposta não precisam ser alterados — apenas a URL muda.
4. Como as configurações e a configuração do Xray agora estão sob `/panel/api`, elas podem (e devem) ser acessadas com o mesmo token de API `Authorization: Bearer <token>` que `/panel/api/inbounds/*` e demais endpoints. Não se esqueça do CSRF-middleware, que está ativo em todo o grupo `/panel/api`.

Exemplo: leitura de todas as configurações via API. Antes (≤ 3.2.x):

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/setting/all" \
-H "Authorization: Bearer <token>"
```

Agora (3.3.0) — adicionado `api/` após `/panel/`:

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/api/setting/all" \
-H "Authorization: Bearer <token>"
```

Da mesma forma para reinicialização do painel: `POST /panel/api/setting/restartPanel`. O caminho antigo `/panel/setting/restartPanel` agora retornará 404.

API tipada: esquemas e documentação (Swagger / OpenAPI)

Na versão 3.3.0, a especificação OpenAPI passou a ser totalmente tipada. Antes, as respostas tipadas eram descritas por um objeto vazio `{}`; agora os componentes e esquemas (`components.schemas`) são gerados diretamente a partir dos modelos de dados. Com isso:

- O Swagger UI exibe os modelos de dados reais, e não stubs sem conteúdo.
- Geradores externos (`openapi-generator` e similares) podem gerar clientes prontos na linguagem desejada a partir da especificação.
- Cada resposta tipada possui um `$ref` para um modelo específico e exemplos de resposta incluídos.

Onde consultar a documentação da API:

- **Página Swagger integrada.** No menu do painel – item **"Documentação da API"** (rota SPA `/panel/api-docs`). Aqui todos os endpoints estão listados de forma interativa, com descrições, corpos de requisição e exemplos de resposta.
 - **Especificação OpenAPI 3.0 bruta** disponível no endereço `/panel/api/openapi.json`. Esta URL pode ser inserida diretamente no Postman, Insomnia ou `openapi-generator`. A especificação está integrada ao binário na etapa de build; quando o painel opera com um `webBasePath` não padrão, o campo `servers` na especificação é automaticamente reescrito para o caminho base atual, de modo que o botão "Try it out" e os geradores externos apontem para o prefixo correto.
-

14. Bot do Telegram

O painel 3X-UI possui um bot do Telegram integrado, por meio do qual é possível receber notificações sobre o estado do servidor e dos clientes, além de gerenciar clientes individuais diretamente pelo mensageiro. O bot opera por meio de long polling (consulta contínua ao Telegram), portanto não requer um domínio externo nem uma porta aberta – basta acesso de saída aos servidores do Telegram.

O bot distingue dois tipos de interlocutores:

- **Administrador** – usuário cujo Telegram User ID está especificado nas configurações do bot (campo «User ID do administrador do bot»). Tem acesso a todas as funções: estatísticas do servidor, backup, gerenciamento de clientes, reinicialização do Xray.
- **Cliente** – qualquer outro usuário cujo Telegram User ID esteja vinculado a um cliente de conexão inbound específico (campo `tgId` do cliente). Visualiza apenas informações sobre suas próprias assinaturas.

Exemplo: vincular um cliente ao Telegram. Para que o usuário receba estatísticas de sua assinatura, seu Telegram User ID numérico é registrado no campo `tgId` do cliente. Nas configurações JSON do cliente, isso aparece assim:

```
{
  "email": "ivan",
  "id": "6f1e6b1a-0c3d-4f2a-9b7e-1a2b3c4d5e6f",
  "tgId": "123456789",
  "enable": true,
  "limitIp": 2,
  "totalGB": 53687091200,
  "expiryTime": 0
}
```

Depois disso, o usuário com User ID `123456789` poderá solicitar ao bot `/usage ivan` e ver suas estatísticas. O mesmo ID pode ser definido pelo administrador por meio do botão  Definir usuário do Telegram» no cartão do cliente – não é necessário editar o JSON manualmente.

14.1. Ativação e configuração do bot

Todos os parâmetros do bot são definidos no painel em **Configurações** → **Bot do Telegram**. Após alterar as configurações, é necessário salvá-las e reiniciar o painel – o bot é inicializado na inicialização do servidor web.

Campo (UI)	Chave de configuração	Valor padrão	Descrição
Habilitar bot do Telegram	<code>tgBotEnable</code>	<code>false</code>	Chave principal. Dica: «Acesso às funções do painel via bot do Telegram». Enquanto desativado, o bot não é iniciado e as tarefas de notificação não são agendadas.
Token do Telegram	<code>tgBotToken</code>	(vazio)	Token do bot. Dica: «É necessário obter o token do gerenciador de bots do Telegram <code>@botfather</code> ». Sem um token válido, o bot não inicia.

Campo (UI)	Chave de configuração	Valor padrão	Descrição
Proxy SOCKS	tgBotProxy	(vazio)	Proxy para conexão ao Telegram. Dica: «Se você precisar de um proxy Socks5 para conectar ao Telegram, configure seus parâmetros conforme o guia».
Servidor de API do Telegram	tgBotAPIServer	(vazio)	Servidor de API alternativo do Telegram. Dica: «Servidor de API do Telegram a ser utilizado. Deixe em branco para usar o servidor padrão».
User ID do administrador do bot	tgBotChatId	(vazio)	Um ou mais Telegram User IDs de administradores separados por vírgula. Dica: «Para obter o User ID, use @userinfobot ou o comando /id no bot».
Frequência de notificações para administradores do bot	tgRunTime	@daily	Agendamento do relatório periódico em formato crontab. Dica: «Especifique o intervalo de notificações no formato Crontab».
Backup do banco de dados	tgBotBackup	false	Dica: «Enviar notificação com arquivo de backup do banco de dados». Anexa o backup ao relatório periódico.
Notificação de login	tgBotLoginNotify	true	Dica: «Exibe o nome de usuário, endereço IP e horário quando alguém tenta acessar o painel».
Limite de carga da CPU para notificação	tgCpu	80	Limite de uso de CPU em porcentagem (validação 0–100). Dica: «Notifica os administradores no Telegram se a carga da CPU exceder este limite (valor: %»». Com valor 0, a verificação de CPU é desabilitada.
Idioma do bot do Telegram	—	—	Idioma no qual o bot formata todas as mensagens.

Obtendo o token via @BotFather

1. Abra no Telegram a conversa com **@BotFather**.
2. Envie o comando `/newbot` e siga as instruções (nome do bot e `username` único terminando em `bot`).
3. O BotFather fornecerá um token no formato `123456789:AA...`. Copie-o para o campo **Token do Telegram**.

Obtendo o User ID do administrador

O User ID é o identificador numérico da conta (não o username). Há duas formas de obtê-lo:

- Escrever para o bot **@userinfobot**.
- Iniciar o bot já configurado e enviar o comando `/id` — ele retornará seu ID.

Insira o número obtido no campo **User ID do administrador do bot**. Para designar vários administradores, liste seus IDs separados por vírgula (por exemplo, `11111111,22222222`). Cada ID é validado como número inteiro; um valor inválido resultará em erro na inicialização do bot.

Exemplo: valor do campo «User ID do administrador do bot». Um único administrador — apenas um número:

123456789

Dois administradores separados por vírgula (espaços são opcionais):

123456789,987654321

Cada valor deve ser um número inteiro. Entradas como `@username` ou `123 456` (com espaço dentro do número) não são aceitas – o bot não iniciará.

Proxy

São suportados os esquemas `socks5://`, `http://` e `https://`. Se o campo de proxy for deixado em branco, o bot tentará usar o proxy geral do painel (se estiver configurado e seu esquema for suportado). URLs com esquema não suportado ou sintaxe inválida são ignoradas – o bot conecta diretamente. O proxy é útil quando o acesso direto à API do Telegram a partir do servidor está bloqueado.

Notificações por e-mail (SMTP)

Além do Telegram, os mesmos eventos podem ser recebidos por e-mail. O canal é configurado em **Configurações** → **Email** na aba **SMTP Settings**:

Campo (UI)	Chave de configuração	Valor padrão	Descrição
Enable Email Notifications	<code>smtpEnable</code>	<code>false</code>	Chave principal das notificações por e-mail via SMTP.
SMTP Host	<code>smtpHost</code>	(vazio)	Host do servidor SMTP (por exemplo, <code>smtp.gmail.com</code>).
SMTP Port	<code>smtpPort</code>	<code>587</code>	Porta do servidor SMTP.
SMTP Username	<code>smtpUsername</code>	(vazio)	Nome de usuário para autenticação SMTP. Também utilizado como endereço do remetente (From).
SMTP Password	<code>smtpPassword</code>	(vazio)	Senha para autenticação SMTP. Armazenada de forma oculta; se já estiver definida, o campo mostra o indicador «configurado» e pode ser deixado em branco para preservar o valor atual.
Recipients	<code>smtpTo</code>	(vazio)	Lista de destinatários separados por vírgula (por exemplo, <code>admin@example.com, ops@example.com</code>).
Encryption	<code>smtpEncryptionType</code>	<code>starttls</code>	Tipo de criptografia da conexão: <code>none</code> (sem criptografia), <code>starttls</code> (STARTTLS) ou <code>tls</code> (TLS implícito).

O botão **Send Test Email** envia um e-mail de teste e exibe o resultado por etapas: **Connection** (conexão), **Authentication** (autenticação) e **Send** (envio). Em caso de problema, o diagnóstico indica em qual etapa ocorreu o erro (por exemplo, «Authentication failed – check username and password» ou «Server requires STARTTLS – change encryption type»), facilitando o ajuste dos parâmetros.

Na segunda aba (**Notifications**) são selecionados os eventos sobre os quais serão enviados e-mails – com os mesmos grupos de cartões utilizados para o Telegram (veja «Barramento de eventos e seleção de notificações» na seção 14.5).

Servidor de API do Telegram

Por padrão, o bot se conecta à API oficial do Telegram. No campo **Servidor de API do Telegram** é possível especificar o endereço de um servidor Bot API próprio (`telegram-bot-api`). A URL é verificada quanto à segurança; endereços bloqueados ou inválidos são descartados e o servidor padrão é utilizado.

14.2. Menu principal e botões

O menu é ativado pelo comando `/start` . Os botões formam um teclado inline anexado à mensagem; o conjunto de botões depende de você ser administrador ou cliente.

Menu do administrador

Botão	Ação
 Relatório de uso de tráfego ordenado	Lista todos os clientes ordenados por tráfego, com o consumo de cada um; e-mails sem dados são marcados com « Sem resultados».
 Estado do servidor	Resumo do servidor (veja a seção 14.5). O botão « Atualizar» recarrega os dados.
 Redefinir todo o tráfego	Zera os contadores de tráfego de todos os clientes. Solicita confirmação («Tem certeza? »), depois exibe « Sucesso» ou « Falha» para cada cliente, e ao final « Redefinição de tráfego concluída para todos os clientes».
 Backup do BD	Envia o arquivo do banco de dados e o <code>config.json</code> (veja a seção 14.6).
 Log de banimentos	Envia os arquivos de log de endereços IP banidos por excesso de limite de IP.
 Conexões inbound	Resumo de todos os inbounds: Remark, porta, tráfego, número de clientes, data de expiração.
 Expirando em breve	Lista de inbounds e clientes cujo tráfego ou prazo está próximo do limite (veja a seção 14.5).
 Comandos	Exibe a ajuda de comandos do administrador.
 Online	Quantidade e lista de clientes online; clicar no e-mail abre o cartão do cliente. Botão « Atualizar».
 Todos os clientes	Abre a seleção de inbound e depois a lista de seus clientes – para visualização/gerenciamento.
 Novo cliente	Inicia o assistente de adição de cliente (seleção de inbound → rascunho → confirmação).
Configurações de assinatura / links individuais / QR-code	Seleção de inbound e cliente para obter link de assinatura, links individuais ou QR-codes.

Menu do cliente

O cliente tem acesso a um conjunto reduzido de botões:

Botão	Ação
Estatísticas do cliente	Exibe dados de todas as assinaturas vinculadas ao Telegram User ID do cliente.
Comandos	Exibe a ajuda de comandos do cliente.
Configurações de assinatura	Seleção do próprio cliente → link de assinatura.
Links individuais	Seleção do próprio cliente → links individuais.
QR-code	Seleção do próprio cliente → QR-codes.

Se o usuário não tiver nenhum cliente com seu Telegram User ID, o bot responde: « Sua configuração não foi encontrada!  Por favor, peça ao administrador que use seu Telegram User ID na configuração.  Seu User ID: ...». Esse ID deve ser repassado ao administrador para que ele o insira no campo do cliente.

14.3. Comandos do bot

O bot tem quatro comandos registrados, visíveis no menu «/» do Telegram:

Comando	Descrição (do menu)	Acesso	O que faz
/start	Exibir o menu principal	todos	Saudação; ao administrador exibe adicionalmente «  Bem-vindo ao bot de gerenciamento !» e o menu principal.
/help	Ajuda do bot	todos	Exibe a saudação geral e a sugestão de escolher um item do menu.
/status	Verificar o status do bot	todos	Responde «  O bot está funcionando normalmente».
/id	Exibir seu Telegram ID	todos	Retorna «  Seu User ID: ... ». Útil para obter o próprio User ID.

Além dos registrados, três comandos com argumento são processados (não aparecem no menu «/», mas funcionam):

- **/usage [Email]** – busca um cliente por e-mail.
 - Para o **administrador**, exibe o cartão completo do cliente (com botões de gerenciamento).
 - Para o **cliente**, exibe apenas sua própria assinatura com o e-mail especificado (pela vinculação do Telegram User ID). Sem argumento, o bot solicita o e-mail: « Por favor, especifique um e-mail para pesquisar».
- **/inbound [nome da conexão]** – somente para o administrador. Busca o inbound pelo Remark e exibe seus parâmetros com estatísticas de todos os clientes. Sem argumento (ou para o cliente) – « Comando desconhecido».
- **/restart** – somente para o administrador. Reinicia o Xray Core. Possíveis respostas: « O núcleo Xray foi reiniciado com sucesso», « O Xray Core não está em execução» (se o núcleo não estiver ativo), « Erro ao reiniciar o xray-core. ». Quaisquer argumentos após **/restart** resultam em mensagem de comando desconhecido com a dica **/restart**.

Em chats em grupo, um comando no formato **/comando@botusername** é processado apenas se o username corresponder ao nome do bot atual.

Ajuda do administrador (botão «Comandos»):

Para reiniciar o Xray Core: /restart
 Para buscar um cliente por e-mail: /usage [Email]
 Para buscar conexões inbound (com estatísticas de clientes): /inbound [nome da conexão]
 Seu Telegram User ID: /id

Ajuda do cliente:

Para visualizar informações sobre sua assinatura: /usage [Email]
 Seu Telegram User ID: /id

14.4. Gerenciamento de clientes (somente administrador)

Ao abrir o cartão de um cliente (via «Todos os clientes», «Online», «Expirando em breve» ou /usage), o administrador vê os dados do cliente (e-mail, inbounds vinculados, status «Ativo», status de conexão, data de expiração, consumo de tráfego) e botões inline de gerenciamento:

Botão	Finalidade
 Atualizar	Recarregar o cartão do cliente.
 Redefinir tráfego	Zerar o contador de tráfego do cliente. Requer confirmação «  Confirmar redefinição de tráfego?».
 Limite de tráfego	Definir o limite de tráfego. Valores predefinidos:  Ilimitado (0), 1/5/10/20/30/40/50/60/80/100/150/200 GB ou «  Personalizado» — entrada numérica via teclado digital integrado (botões 0–9, «  » — redefinir para 0, «  » — apagar o último dígito, «  Confirmar: N»). O valor é definido em gigabytes.
 Alterar data de expiração	Opções predefinidas:  Ilimitado, «  Personalizado», adicionar 7/10/14/20 dias, 1/3/6/12 meses. Um número positivo prorroga o prazo (adiciona dias à data de expiração atual ou a «agora», se o prazo já tiver expirado); 0 remove a restrição de prazo.
 Log de IP	Exibe os endereços IP registrados do cliente (com marcas de tempo, se disponíveis). No log há «  Atualizar» e «  Limpar IP» (com confirmação «  Confirmar limpeza de IP?»).
 Limite de IP	Limite de IPs simultâneos. Opções:  Ilimitado (0), 1–10 ou «  Personalizado» (teclado digital).
 Definir usuário do Telegram	Exibe o Telegram User ID atualmente vinculado ao cliente; permite remover o vínculo («  Remover usuário do Telegram» com confirmação). A vinculação de um novo usuário é feita via seleção de contato do Telegram do sistema.
 Ativar/Desativar	Ativa ou desativa o cliente. Requer confirmação «  Confirmar ativar/desativar usuário?».

Todas as operações que alteram a configuração (limite de tráfego/IP, data de expiração, vinculação/desvinculação do usuário do Telegram, ativar/desativar) marcam o Xray para reinicialização quando necessário, para que as alterações entrem em vigor. Após uma operação bem-sucedida, o bot exibe uma confirmação no formato «  : ...» e reexibe o cartão.

Qualquer entrada numérica nos assistentes é limitada a valores < 9999999.

14.5. Notificações e relatórios

As notificações são enviadas a todos os administradores (todos os User IDs de `tgBotChatId`).

Barramento de eventos e seleção de notificações

As notificações são construídas sobre um barramento de eventos unificado, com dois canais de entrega – **Telegram** e **e-mail (SMTP)**. Para cada canal, os eventos a notificar são selecionados separadamente. Em **Configurações** → **Telegram**, isso é feito na aba **Notifications**; em **Configurações** → **Email** – na aba de mesmo nome.

Os eventos são agrupados em cartões; cada grupo tem um interruptor principal com contador de eventos habilitados (n/total) e estado intermediário quando apenas alguns estão selecionados. Os grupos disponíveis são:

- **Outbound** – «Down» (`outbound.down`) e «Up» (`outbound.up`): queda e recuperação do outbound.
- **Xray Core** – «Crash» (`xray.crash`): encerramento inesperado do núcleo Xray.
- **Nodes** – «Down» (`node.down`) e «Up» (`node.up`): nó ficou indisponível ou se recuperou.
- **System** – «CPU high (%)» (`cpu.high`) e «Memory high (%)» (`memory.high`): alta utilização de CPU e RAM. Ambos os eventos possuem um campo inline de limite percentual ao lado.
- **Security** – «Login attempt» (`login.attempt`): tentativa de acesso ao painel.

O conjunto de eventos habilitados é armazenado separadamente: para Telegram – em `tgEnabledEvents`, para e-mail – em `smtpEnabledEvents`. Por padrão, em ambos os canais estão habilitados «Login attempt» e «CPU high» (valor `login.attempt,cpu.high`).

Notificação de login no painel

Controlada pela opção **Notificação de login** (`tgBotLoginNotify`, habilitada por padrão). A cada tentativa de acesso ao painel web, os administradores recebem uma mensagem:

- Em caso de sucesso: « Login no painel bem-sucedido.» + host, nome de usuário, IP, horário.
- Em caso de falha: « Erro de login no painel.» + host, **motivo** (por exemplo, «Erro 2FA» em caso de segundo fator incorreto), nome de usuário, IP, horário.

Excesso de carga de CPU e memória

A cada minuto, o painel verifica a utilização de CPU e RAM. Se o limite `tgCpu` > 0 e a carga média de CPU no último minuto o exceder, os administradores recebem: « A carga do processador está em N%, o que excede o limite de M%». Da mesma forma, a utilização de RAM é verificada contra o limite `tgMemory` (padrão 80%) – evento «Memory high (%)».

Ambos os limites são definidos por campos inline ao lado dos eventos «CPU high (%)» e «Memory high (%)» no grupo **System** da aba Notifications (veja «Barramento de eventos e seleção de notificações» acima). Para o canal de e-mail, existem chaves separadas `smtpCpu` e `smtpMemory`. Com o valor do limite igual a 0, a verificação correspondente é desabilitada.

Relatório periódico (por agendamento)

Agendado pela expressão cron do campo **Frequência de notificações** (`tgRunTime` , padrão `@daily`). Se o valor estiver vazio ou for inválido, é usado `@daily` . O relatório inclui:

Construtor de agendamento

O campo **Frequência de notificações para administradores do bot** é definido não por entrada manual de texto, mas por meio de um construtor de agendamento. Primeiro, o modo é selecionado em uma lista suspensa:

- **@every – repetir em intervalo** – aparecem um campo numérico e a seleção da unidade (**Segundos / Minutos / Horas**); o resultado é montado em uma expressão como `@every 6h` .
- **@hourly – a cada hora, @daily – todos os dias às 00:00, @weekly – toda semana, @monthly – todo mês** – predefinições prontas, salvas como o macro correspondente (`@hourly` , `@daily` , `@weekly` , `@monthly`).
- **Personalizado (crontab)** – campo para uma expressão crontab própria. O agendador do painel opera com segundos habilitados, portanto a expressão personalizada consiste em **6 campos**: segundo, minuto, hora, dia do mês, mês, dia da semana (por exemplo, `0 30 8 * * *` – todos os dias às 08:30:00). Ao alternar para este modo, o campo é preenchido com o equivalente crontab da seleção atual, como ponto de partida.

Exemplo: valores do campo «Frequência de notificações» (`tgRunTime`). São suportados tanto atalhos prontos quanto o formato crontab completo:

Valor	Quando dispara
<code>@daily</code>	uma vez por dia à meia-noite (valor padrão)
<code>@hourly</code>	a cada hora
<code>@every 6h</code>	a cada 6 horas
<code>0 9 * * *</code>	todos os dias às 09:00
<code>0 9 * * 1</code>	toda segunda-feira às 09:00
<code>0 */12 * * *</code>	a cada 12 horas (às 00:00 e 12:00)

Ordem dos campos no crontab: minuto, hora, dia do mês, mês, dia da semana.

1. A linha « Relatórios agendados: » e a data/hora atual.
2. **Estado do servidor** (veja abaixo).
3. Bloco «Expirando em breve» por inbound e clientes.
4. Notificações pessoais para clientes com Telegram User ID vinculado – cada cliente não administrador recebe a lista de suas assinaturas com tráfego ou prazo próximo do limite (incluindo as desativadas).
5. Se **Backup do banco de dados** (`tgBotBackup`) estiver habilitado – backup do BD para os administradores.

Estado do servidor contém: host, versão do 3X-UI e Xray, IPv4/IPv6, tempo de atividade (em dias), carga média (Load1/2/3), RAM (atual/total), número de clientes online, contadores de conexões TCP/UDP, tráfego de rede total (↑/↓) e status do Xray.

«Expirando em breve» exibe:

- por inbound: número de desativados e número de «prestes a expirar», seguido da listagem desses inbounds (Remark, porta, tráfego, data de expiração);
- por clientes: o mesmo, mais os cartões dos clientes e botões com seus e-mails (clicar abre o cartão do cliente).

Os limites de «prestes a expirar» são obtidos das configurações gerais do painel: margem de tráfego (em GB) e margem de prazo (em dias). Um inbound/cliente é considerado «próximo do limite» quando o tráfego restante até o limite é menor que a margem OU a data de expiração está dentro da margem.

14.6. Backup e logs

- **Backup do BD** (botão « Backup do BD» ou opção no relatório periódico): o bot envia o horário do backup, o arquivo do banco de dados (`x-ui.db` , ou `x-ui.dump` para PostgreSQL) e o arquivo de configuração do Xray `config.json` .
- **Log de banimentos** (botão « Log de banimentos»): envia os arquivos de log atual e anterior dos endereços IP banidos por excesso de limite de IP. Arquivos vazios não são enviados.

14.7. Particularidades de funcionamento

- **Mensagens longas** são divididas em partes (limite de ~2000 caracteres), e o teclado inline é anexado à última parte.
- **Paralelismo**: comandos e cliques em botões são processados de forma concorrente (pool de até 10 manipuladores simultâneos).
- **Confiabilidade de envio**: em caso de erros de conexão, as mensagens são reenviadas com backoff exponencial (1s/2s/4s, até 3 tentativas).
- **Cache**: os dados de «Estado do servidor» são armazenados em cache para que cliques frequentes em «Atualizar» não sobrecarreguem o sistema.
- **Reinicialização do bot**: ao salvar as configurações e reiniciar o painel, o ciclo de polling anterior é encerrado corretamente e um novo é iniciado – apenas uma instância de recebimento de atualizações funciona por vez.

15. Bases geográficas (geoip / geosite e personalizadas)

As bases geográficas são arquivos binários `.dat` que o Xray-core utiliza para roteamento e filtragem de tráfego com base na localização geográfica (faixas de IP) ou na categoria de domínios. O painel é capaz de baixar e atualizar tanto o conjunto padrão de arquivos geo quanto fontes personalizadas arbitrárias especificadas por URL. Todos os arquivos são armazenados no diretório `bin` ao lado do binário do Xray (caminho padrão `bin`, substituível pela variável de ambiente `XUI_BIN_FOLDER`).

15.1. O que são geoip.dat e geosite.dat

- **geoip.dat** – base de correspondências "endereço IP → código de país/região". É usada em regras de roteamento na forma `geoip:<código>`, por exemplo `geoip:ru`, `geoip:cn`, bem como para marcadores especiais `geoip:private` (redes privadas/locais). Em essência, responde à pergunta "em qual país este IP está localizado".
- **geosite.dat** – base de correspondências "domínio → categoria/lista". É usada na forma `geosite:<categoria>`, por exemplo `geosite:category-ads-all` (domínios de anúncios), `geosite:google`, `geosite:ru`. Em essência, são listas agrupadas de domínios.

Esses arquivos são necessários para construir regras do tipo "todo o tráfego para IPs/domínios russos – diretamente, o restante – pelo outbound" e similares. As próprias regras são definidas na seção de roteamento do Xray; as bases geográficas apenas fornecem dados para elas. Sem arquivos geo atualizados, as regras que referenciam `geoip:` / `geosite:` não funcionarão ou dependerão de listas desatualizadas.

Exemplo: regra "domínios e IPs russos – diretamente". Essa regra na seção de roteamento direciona todo o tráfego para recursos russos para o outbound com a tag `direct`:

```
{
  "type": "field",
  "domain": ["geosite:category-ru"],
  "ip": ["geoip:ru"],
  "outboundTag": "direct"
}
```

15.2. Arquivos geo padrão e sua atualização

O painel contém uma lista de permissões (allowlist) fixa com seis arquivos padrão com fontes de download definidas de forma rígida. A atualização é realizada via

POST `/panel/api/server/updateGeofile/:fileName` (ou sem nome de arquivo – para atualizar todos de uma vez).

Exemplo: atualização de um arquivo e de todos ao mesmo tempo via API. Atualizar apenas `geoip_RU.dat`:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile/
geoip_RU.dat' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Atualizar todos os seis arquivos padrão com uma única requisição (sem indicar o nome do arquivo):

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Resposta de sucesso:

```
{ "success": true, "msg": "Geofile updated successfully", "obj": null }
```

Nome do arquivo	Fonte (repositório de releases)
geoup.dat	github.com/Loyalsoldier/v2ray-rules-dat (geoup.dat)
geosite.dat	github.com/Loyalsoldier/v2ray-rules-dat (geosite.dat)
geoup_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geoup.dat)
geosite_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geosite.dat)
geoup_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geoup.dat)
geosite_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geosite.dat)

Particularidades da atualização dos arquivos padrão:

- **Botão de atualização de um único arquivo.** Antes do download, é exibida uma confirmação: "Você realmente deseja atualizar o arquivo geo?" com a explicação "Isso atualizará o arquivo #filename#." (em inglês *Do you really want to update the geofile? This will update the #filename# file.*). Em caso de sucesso, aparece a notificação "Arquivos geo atualizados com sucesso" (em inglês *Geofile updated successfully*).
- **Botão "Atualizar todos"** (em inglês *Update all*) baixa todos os seis arquivos. Confirmação: "Isso atualizará todos os arquivos geo." (em inglês *This will update all geofiles.*).
- **Download condicional.** Se o arquivo local já existir, o cabeçalho `If-Modified-Since` com o horário de modificação do arquivo é adicionado à requisição. A resposta `304 Not Modified` do servidor significa que o arquivo não foi alterado – ele não é baixado novamente, apenas o carimbo de tempo do arquivo é atualizado.
- **Segurança do nome do arquivo.** Somente nomes da allowlist são aceitos; o nome é verificado quanto à ausência de `..`, separadores de caminho `/` e `\`, caminhos absolutos, e deve corresponder ao padrão `^[a-zA-Z0-9._-]+\..dat$`. Qualquer nome fora da lista é rejeitado com o erro "Invalid geofile name".
- **Reinicialização do Xray.** Após o download dos arquivos geo, o Xray-core é reiniciado para que ele releia as bases atualizadas. Se a reinicialização falhar, a mensagem de erro incluirá uma linha correspondente.

Atualização das bases geo pela linha de comando (x-ui)

As bases geo também podem ser atualizadas sem o painel – pelo menu interativo `x-ui` (item de atualização de arquivos geo) ou pelo comando não interativo `x-ui update-all-geofiles`. Para cada arquivo do conjunto (geoup/geosite, incluindo os conjuntos IR e RU) é exibido um status separado: "atualizado", "já está atualizado" ou "erro de download". Em caso de falha no download, nenhuma mensagem de sucesso falsa é exibida. A reinicialização do Xray (e consequentemente a interrupção das conexões ativas) ocorre somente se pelo menos um arquivo foi de fato atualizado; se nenhum arquivo foi alterado (todos retornaram `304 Not Modified`), o painel e o Xray não são reiniciados.

15.3. Atualização automática de geo-dados pelo Xray (Geodata Auto-Update)

Fontes `.dat` adicionais por URL arbitrário não são adicionadas pelas ferramentas do painel, mas sim pela seção nativa `geodata` do Xray-core. A seção correspondente está disponível na janela modal de atualizações do Xray (Dashboard → atualizações do Xray, `xrayUpdates`) – esta é a aba "Atualização Automática de Geodata" (em inglês *Geodata Auto-Update*). O painel aqui apenas edita a chave `geodata` no template de configuração do Xray; o download, a verificação e o recarregamento a quente dos arquivos são realizados pelo próprio núcleo do Xray.

Na parte superior da seção é exibida uma dica: "O Xray baixa esses arquivos de acordo com o agendamento e os recarrega sem reinicialização. As URLs devem ser HTTPS. O arquivo já deve existir na pasta bin antes que o Xray possa atualizá-lo." (em inglês *Xray downloads these files on schedule and hot-reloads them without a restart. URLs must be HTTPS. Each file must already exist in the bin folder once before Xray can update it.*).

Campos da seção

- **Agendamento (cron)** (em inglês *Schedule (cron)*) – string cron com 5 campos; valor padrão `0 4 * * *` (diariamente às 04:00). Ao salvar, é verificado que a string contém exatamente 5 campos, caso contrário é exibido o erro "O cron deve conter 5 campos, ex.: `0 4 * * *`".
- **Baixar via outbound (opcional)** (em inglês *Download through outbound (optional)*) – lista suspensa com as tags dos outbounds disponíveis (incluindo outbounds de inscrições), pelo qual o Xray baixará os arquivos; outbounds com protocolo `blackhole` não aparecem na lista. O campo pode ser deixado vazio – nesse caso é usada a conexão direta. Esta seleção é independente do outbound para as próprias requisições do painel (ver §11): a atualização automática do geodata tem seu próprio outbound separado para download.
- **Lista de arquivos** – cada linha define um par "URL + Nome do arquivo" (em inglês *File name*). A URL deve começar com `https://` (caso contrário "Uma URL HTTPS é necessária para cada arquivo."). O nome do arquivo é informado de forma simples, sem caminhos ou separadores – apenas caracteres `^[A-Za-z0-9._-]+$` (caso contrário "O nome do arquivo deve ser simples, ex.: `geosite_custom.dat` (sem caminhos)"). Ao inserir a URL, o painel tenta preencher o nome do arquivo automaticamente a partir do último segmento do caminho. O botão "Adicionar arquivo" (em inglês *Add file*) adiciona uma linha, o botão de lixeira a remove.

Se a lista estiver vazia, é exibida a dica: "Nenhum arquivo configurado. Referencie os arquivos nas regras de roteamento como `ext:geosite_custom.dat:categoria`." (em inglês *No files configured. Reference files in routing rules as `ext:geosite_custom.dat:category`*).

Salvamento

O botão "Salvar e reiniciar o Xray" (em inglês *Save & Restart Xray*) exibe a confirmação "Salvar configurações de geodata?" com a explicação "O template de configuração do Xray será atualizado e o Xray será reiniciado." (em inglês *Save geodata settings? This updates the Xray config template and restarts Xray*). Após salvar, a chave `geodata` é gravada no template de configuração (`POST /panel/api/xray/update`) e o Xray é reiniciado (`POST /panel/api/server/restartXrayService`). Se a lista de arquivos estiver vazia, a chave `geodata` é removida do template.

Particularidades importantes:

- **O arquivo já deve existir em `bin`**. O Xray atualiza apenas os arquivos `.dat` que já estão presentes na pasta `bin` no momento da inicialização. Portanto, um novo arquivo personalizado deve primeiro ser colocado em `bin` manualmente (ou pelo menos uma versão vazia/desatualizada deve ser criada com o nome correto), e só então o Xray começará a mantê-lo atualizado de acordo com o agendamento.
- **Recarregamento a quente**. Após o download programado, o Xray relê as bases atualizadas sem reinicialização completa do processo.
- **Compatibilidade**. Os arquivos geo baixados anteriormente (tanto os padrão quanto os personalizados) continuam funcionando nas regras de roteamento com a sintaxe `ext:` sem alterações.

Se a lista estiver vazia, é exibida a dica: "Nenhuma fonte geo personalizada ainda – clique em 'Adicionar' para criar uma" (em inglês *No custom geo sources yet – click Add to create one*).

Colunas da tabela e campos da fonte

Campo (UI)	JSON	Valor padrão	Descrição
Tipo (<i>Type</i>)	<code>type</code>	– (obrigatório)	Tipo do recurso: apenas <code>geosite</code> ou <code>geoip</code> . Determina o nome do arquivo resultante.
Apelido (<i>Alias</i>)	<code>alias</code>	– (obrigatório)	Identificador curto da fonte. O nome do arquivo é construído a partir dele e do tipo.
URL (<i>URL</i>)	<code>url</code>	– (obrigatório)	Link direto para o arquivo <code>.dat</code> (http/https).
Ativado (<i>Enabled</i>)	–	–	Indicador de atividade da fonte na lista.
Atualizado (<i>Last updated</i>)	<code>lastUpdatedAt</code>	<code>0</code>	Horário da última atualização bem-sucedida (tempo Unix; <code>0</code> – ainda não atualizado).
Roteamento (ext:...) (<i>Routing (ext:...)</i>)	–	–	String pronta para regras de roteamento: <code>ext:<arquivo.dat>:tag</code> .
Ações (<i>Actions</i>)	–	–	Botões "Editar", "Excluir", "Atualizar agora".

Adicionalmente, o banco de dados armazena campos de serviço: `localPath` (caminho real do arquivo no diretório `bin`), `lastModified` (valor do cabeçalho `Last-Modified` retornado pelo servidor, usado para download condicional), `createdAt` e `updatedAt`.

Nomenclatura dos arquivos

O nome do arquivo resultante é formado automaticamente a partir do tipo e do apelido:

- tipo `geoip` → `geoip_<alias>.dat`;
- tipo `geosite` → `geosite_<alias>.dat`.

Por exemplo, uma fonte com tipo `geosite` e apelido `myads` criará o arquivo `geosite_myads.dat`.

Exemplo: adição de uma fonte via API. Adicionar uma lista personalizada de domínios de anúncios como recurso `geosite` com o apelido `myads`:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/customGeo/add' \
-H 'Cookie: 3x-ui=<session-cookie>' \
-H 'Content-Type: application/json' \
-d '{
  "type": "geosite",
  "alias": "myads",
  "url": "https://example.com/lists/myads.dat"
}'
```

O painel baixará o arquivo no diretório `bin` como `geosite_myads.dat`, salvará o registro e reiniciará o Xray.

Botões e ações

- **Adicionar** (em inglês *Add*) – abre o formulário "Adicionar fonte" (em inglês *Add custom geo*). O botão de salvamento é "Salvar" (em inglês *Save*). API: `POST /add`.
- **Editar** (em inglês *Edit*) – formulário "Editar fonte" (em inglês *Edit custom geo*). API: `POST /update/:id`. Ao alterar o tipo ou o apelido, o arquivo antigo é excluído e o novo é baixado novamente.
- **Excluir** (em inglês *Delete*) – confirmação "Excluir esta fonte personalizada?" (em inglês *Delete this custom geo source?*). Remove o registro do banco de dados e o próprio arquivo `.dat`. API: `POST /delete/:id`. Em caso de sucesso: "Arquivo geo personalizado " excluído".
- **Atualizar agora** (em inglês *Update now*) – baixa novamente a fonte específica e atualiza o carimbo de tempo. API: `POST /download/:id`. Em caso de sucesso: "Arquivo geo " atualizado".
- **Atualizar todos** – atualiza todas as fontes personalizadas de uma vez. API: `POST /update-all`. Em caso de sucesso total: "Todas as fontes geo personalizadas foram atualizadas" (em inglês *All custom geo sources updated*). Se pelo menos uma fonte não for atualizada, a operação é considerada parcialmente malsucedida com a mensagem "Falha ao atualizar uma ou mais fontes geo personalizadas" (em inglês *One or more custom geo sources failed to update*), e a resposta lista as fontes bem-sucedidas e as com falha.

Após qualquer uma das ações (adição, edição, exclusão, atualização, atualização de todos com pelo menos um sucesso) o Xray-core é reiniciado.

Passo a passo: adição de uma fonte

1. Clique em "Adicionar".
2. No campo "Tipo", selecione `geosite` ou `geoip`.
3. No campo "Apelido", insira um identificador (apenas letras latinas minúsculas, dígitos, `-` e `_`; dica de placeholder: `a-z 0-9 _ -`).
4. No campo "URL", informe o link direto para o arquivo `.dat` (deve começar com `http://` ou `https://`).
5. Clique em "Salvar". O painel baixará imediatamente o arquivo no diretório `bin`, salvará o registro e reiniciará o Xray.

15.4. Validação e restrições

Ao criar e editar uma fonte, verificações rigorosas são realizadas. Mensagens de erro:

Condição	Mensagem (PT)	Mensagem (EN)
Tipo diferente de <code>geosite</code> / <code>geoip</code>	O tipo deve ser geosite ou geoip	<i>Type must be geosite or geoip</i>

Condição	Mensagem (PT)	Mensagem (EN)
Apelido vazio	Informe o apelido	<i>Alias is required</i>
Caracteres inválidos no apelido (não corresponde a <code>^[a-z0-9_-]+\$</code>)	O apelido contém caracteres inválidos	<i>Alias must match allowed characters</i>
Apelido reservado	Este apelido está reservado	<i>This alias is reserved</i>
URL vazia	Informe a URL	<i>URL is required</i>
URL inválida (não pode ser analisada)	URL inválida	<i>URL is invalid</i>
Esquema diferente de http/https	A URL deve usar http ou https	<i>URL must use http or https</i>
Host vazio/inválido ou bloqueado pela proteção SSRF	Host da URL inválido	<i>URL host is invalid</i>
Duplicata "tipo + apelido"	Este apelido já está em uso para este tipo	<i>This alias is already used for this type</i>
Fonte não encontrada	Fonte não encontrada	<i>Custom geo source not found</i>
Erro de download	Falha no download	<i>Download failed</i>

Dicas no formulário (validação no cliente): "Apelido: apenas a-z, dígitos, - e _" (*Alias may only contain lowercase letters, digits, - and _*) e "A URL deve começar com http:// ou https://" (*URL must start with http:// or https://*).

Restrições técnicas adicionais:

- **Apelidos reservados.** Não é possível usar apelidos que conflitem com os arquivos padrão. São reservados (comparação sem distinção de maiúsculas/minúsculas, hífen equiparado a sublinhado): `geoip`, `geosite`, `geoip_ir`, `geosite_ir`, `geoip_ru`, `geosite_ru`. Por exemplo, `geosite-ru` será rejeitado como `geosite_ru`.
- **Proteção SSRF.** O host da URL é resolvido para um IP, e se ele apontar para um endereço privado/interno, o download é bloqueado (o usuário verá "Host da URL inválido"). Isso impede o uso do painel para acessar serviços internos.
- **Proteção contra path traversal.** O caminho final do arquivo deve estar dentro do diretório `bin` (com resolução de links simbólicos); qualquer tentativa de sair desse diretório é rejeitada.
- **Tamanho mínimo do arquivo.** O arquivo baixado é considerado válido apenas se tiver no mínimo 64 bytes; um arquivo muito pequeno é rejeitado com erro de download.
- **Proxy e download condicional.** Se um proxy estiver configurado nas definições do painel, o download é feito por meio dele; caso contrário, é usada a conexão direta com transporte seguro contra SSRF. Assim como para os arquivos padrão, é aplicado `If-Modified-Since / 304 Not Modified` (um arquivo não alterado não é baixado novamente). O timeout de download é de 10 minutos; a verificação de disponibilidade da URL (HEAD, e se necessário – GET parcial) é de 12 segundos.

15.5. Verificação automática na inicialização do painel

Ao iniciar, o painel percorre todas as fontes personalizadas e verifica a existência e integridade do arquivo local para cada uma (arquivo ausente, sendo um diretório, ou menor que 64 bytes). Se o arquivo estiver ausente ou corrompido, a fonte é testada e uma tentativa de novo download é realizada. Isso garante que, após reinstalação ou perda do diretório `bin`, os arquivos geo personalizados sejam restaurados automaticamente.

15.6. Uso das bases geográficas nas regras de roteamento

Nas regras de roteamento do Xray, as bases geográficas são usadas em campos como `domain / ip` por meio de prefixos:

- **geoip**: para bases de IP — `geoip:<código>`. Exemplos: `geoip:ru`, `geoip:cn`, `geoip:private`. Obtido de `geoip.dat` (ou `geoip_RU.dat` etc., se a regra aponta para um arquivo específico).
- **geosite**: para bases de domínios — `geosite:<categoria>`. Exemplos: `geosite:category-ads-all`, `geosite:google`, `geosite:ru`. Obtido de `geosite.dat`.

Exemplo: bloqueio de anúncios via geosite. Regra que envia todos os domínios de anúncios para um "buraco negro" (pressupõe-se um outbound com a tag `blocked` e protocolo `blackhole`):

```
{
  "type": "field",
  "domain": ["geosite:category-ads-all"],
  "outboundTag": "blocked"
}
```

Para arquivos **personalizados**, é usada a sintaxe de arquivo externo `ext:`. A dica na UI diz: "Nas regras de roteamento, use o valor como `ext:arquivo.dat:tag` (substitua tag)." (em inglês *In routing rules use the value column as `ext:file.dat:tag` (replace tag)*). Formato:

```
ext:<nome_do_arquivo.dat>:<tag>
```

onde `<nome_do_arquivo.dat>` é `geoip_<alias>.dat` ou `geosite_<alias>.dat`, e `<tag>` é a lista/ categoria específica dentro do arquivo. O painel na coluna "Roteamento (ext:...)" sugere um template pronto no formato `ext:geosite_myads.dat:tag` — basta substituir `tag` pela tag desejada. O nome desse arquivo é definido na seção "Atualização Automática de Geodata" (ver §15.3) no campo "Nome do arquivo" — por exemplo `geosite_custom.dat`; é referenciado nas regras como `ext:geosite_custom.dat:category`.

Exemplo: regra baseada em arquivo personalizado. Se uma fonte do tipo `geosite` com apelido `myads` foi adicionada, e a lista dentro do arquivo `.dat` está marcada com a tag `ads`, a regra de roteamento fica assim:

```
{
  "type": "field",
  "domain": ["ext:geosite_myads.dat:ads"],
  "outboundTag": "blocked"
}
```

Para uma fonte de IP (tipo `geoip`, apelido `mycorp`, tag `office`), o campo será `"ip": ["ext:geoip_mycorp.dat:office"]`.

16. Operação: backups, logs, atualização, CLI

Esta seção abrange a manutenção diária do painel: criação e restauração de backups do banco de dados, visualização de logs do painel e do Xray, reinicialização e parada de serviços, atualização do Xray e do próprio painel, tarefas periódicas (cron) e remoção do painel. Parte das operações é realizada pela interface web (abas na página «Dashboard» e «Configurações do Painel»), e parte pelo menu de console `x-ui` no servidor.

16.1. Backup e restauração do banco de dados

Todos os dados do painel (inbounds, clientes, grupos, nós, configurações) são armazenados em um único banco de dados. O gerenciamento de backups está disponível na página «Dashboard», na aba «Backup», com o cabeçalho «Backup e Restauração».

O painel suporta dois mecanismos de banco de dados, e o comportamento do backup depende disso:

- **SQLite** (padrão) — os dados ficam em um arquivo `x-ui.db`.
- **PostgreSQL** — se o painel estiver configurado com PostgreSQL, o bloco exibe um aviso:

«Este painel está rodando em PostgreSQL. "Backup" baixa um arquivo `pg_dump` (.dump), e "Restaurar" o envia de volta via `pg_restore`. As ferramentas cliente do PostgreSQL (`pg_dump` e `pg_restore`) devem estar instaladas no servidor.»

Exportação (criação de cópia)

O botão «Exportar banco de dados» (`Back Up`) baixa o arquivo de backup para o seu dispositivo.

Mecanismo de BD	Nome do arquivo	O que acontece no servidor
SQLite	<code>x-ui.db</code>	Primeiro é feito um checkpoint WAL para que o arquivo contenha as entradas mais recentes; depois o arquivo é lido integralmente e enviado para download
PostgreSQL	<code>x-ui.dump</code>	O <code>pg_dump</code> é executado e o arquivo resultante é enviado para download

Dicas na interface:

- **SQLite**: «Clique para baixar o arquivo `.db` contendo o backup do seu banco de dados atual para o seu dispositivo.»
- **PostgreSQL**: «Clique para baixar o dump PostgreSQL (.dump) do banco de dados atual para o seu dispositivo.»

Tecnicamente, a exportação é uma requisição `GET /panel/api/server/getDb`. O nome do anexo é definido pelo servidor (`Content-Disposition`) de acordo com o mecanismo em uso.

O nome do arquivo de backup é formado com base no endereço do servidor, e não com o nome fixo `x-ui.db` / `x-ui.dump`. Ao baixar pelo navegador, o nome é retirado do endereço do painel na barra de endereços (host da requisição); caso contrário, usa-se o domínio web configurado, ou, na ausência deste, o IP público do servidor (primeiro IPv4, depois IPv6), com fallback para `x-ui`. Isso facilita distinguir backups de servidores diferentes. A

extensão permanece `.db` para SQLite e `.dump` para PostgreSQL; os backups enviados via Telegram também são nomeados conforme o domínio/IP.

Exemplo: download de backup via API. A mesma exportação pode ser obtida via linha de comando – por exemplo, para um script de backup automatizado. É necessária uma sessão autenticada (cookie de login):

```
# 1) Fazemos login e salvamos o cookie da sessão
curl -s -c cookies.txt \
  -d 'username=admin&password=admin' \
  https://panel.example.com:2053/panel/login

# 2) Baixamos o arquivo do banco (o nome é definido pelo servidor: x-ui.db ou x-ui.dump)
curl -s -b cookies.txt -OJ \
  https://panel.example.com:2053/panel/api/server/getDb
```

Se o painel estiver acessível por um caminho base (Web Base Path), ele deve ser adicionado ao URL: `...:2053/<base_path>/panel/api/server/getDb`.

Importação (restauração)

O botão «**Importar banco de dados**» (Restore) abre um seletor de arquivo e envia o arquivo para o servidor para restauração (POST `/panel/api/server/importDB`, campo do formulário `db`).

Dicas na interface:

- SQLite: «Clique para selecionar e enviar um arquivo `.db` do seu dispositivo para restaurar o banco de dados a partir do backup.»
- PostgreSQL: «Clique para selecionar e enviar um arquivo `.dump` para restaurar o banco de dados PostgreSQL. Isso substituirá todos os dados atuais.»

Processo de importação para SQLite (importante entender que é atômico e com rollback):

1. O arquivo enviado é verificado quanto ao formato – deve ser um banco SQLite válido; caso contrário, retorna o erro «Invalid db file format».
2. O arquivo é salvo como `x-ui.db.temp` temporário e passa por verificação de integridade.
3. **O Xray é parado** antes da substituição do banco.
4. O banco atual é renomeado para `x-ui.db.backup` (fallback).
5. O arquivo temporário é movido para o lugar do banco de trabalho, são executadas a inicialização, as migrações de esquema e depois a migração de inbound.
6. **Se qualquer etapa falhar** – é feito rollback: o banco anterior é restaurado a partir de `x-ui.db.backup`, e o Xray é reiniciado com os dados antigos.
7. Em caso de sucesso, o arquivo de fallback é excluído e **o Xray é reiniciado automaticamente** com os dados restaurados.

Mensagens da interface ao final:

Resultado	Texto
Sucesso	«Banco de dados importado com sucesso»

Resultado	Texto
Erro na importação	«Ocorreu um erro ao importar o banco de dados»
Erro na leitura do arquivo	«Ocorreu um erro ao ler o banco de dados»

A restauração substitui completamente os dados atuais. Como o Xray é parado brevemente durante o processo, as conexões ativas dos clientes são interrompidas durante a importação.

Arquivo de migração entre mecanismos (SQLite ⇌ PostgreSQL)

Separado do backup comum, existe a função «**Baixar arquivo de migração**» (`Download Migration`, requisição `GET /panel/api/server/getMigration`). Ela gera um arquivo portátil para migração para outro mecanismo de banco de dados:

Mecanismo atual	O que é baixado	Nome do arquivo	Finalidade
SQLite	Dump SQL portátil (texto)	<code>x-ui.dump</code>	Alimentar o PostgreSQL com seus dados
PostgreSQL	Banco SQLite construído a partir dos dados do PostgreSQL	<code>x-ui.db</code>	Retornar o painel para SQLite

Dicas:

- No SQLite: «Clique para baixar uma exportação portátil `.dump` (texto SQL) do seu banco de dados SQLite.»
- No PostgreSQL: «Clique para baixar um banco de dados SQLite (`.db`) construído a partir dos seus dados PostgreSQL e pronto para executar o painel em SQLite.»

A conversão `.db` ⇌ `.dump` para SQLite também pode ser feita pela CLI com o comando `x-ui migrateDB [file]` (ver seção 16.7).

Backup via bot do Telegram

Se o bot do Telegram estiver configurado (ver seção sobre notificações), ele pode enviar um backup diretamente no chat do administrador. O backup via Telegram inclui **dois arquivos**: o próprio banco de dados (`x-ui.db`, ou `x-ui.dump` no PostgreSQL) e a configuração do Xray `config.json`. A mensagem é precedida pela linha « Hora do backup: ...».

Há duas formas de receber o backup no Telegram:

1. **Sob demanda.** O botão « **Backup do BD**» no menu do bot – o bot envia os arquivos imediatamente no chat atual.
2. **Automaticamente com o relatório.** Nas configurações do bot há um interruptor «**Backup do banco de dados**» (`Database Backup`) com a descrição «Enviar notificação com o arquivo de backup do banco de dados». Quando habilitado, a cada envio periódico do relatório, o bot envia o backup após o relatório para todos os administradores. O período de envio do relatório é definido pelo agendamento cron do bot (ver

seção 16.6). Entre os arquivos e entre os administradores, o bot faz pausas para não exceder os limites do Telegram.

O backup via bot é enviado somente se o bot estiver em execução; no PostgreSQL, também requer `pg_dump` instalado no servidor.

16.2. Visualização de logs

O painel possui dois visualizadores de logs independentes, ambos acessíveis pela aba «**Logs**» no «Dashboard». Cada janela pode ser atualizada (ícone de «atualizar» no cabeçalho) e permite baixar o conteúdo exibido como arquivo `x-ui.log` (botão com ícone de download).

Logs do painel (aplicação / syslog)

Janela de logs do painel (`POST /panel/api/server/logs/{count}`). Controles:

Elemento	Valor padrão	Descrição
Número de linhas	20	Lista suspensa: 10 / 20 / 50 / 100 / 500
Nível	Info	Nível mínimo: Debug / Info / Notice / Warning / Error
SysLog (checkbox)	desativado	De onde buscar os logs: do buffer da aplicação ou do journal do sistema

O comportamento depende do checkbox **SysLog**:

- **Desativado (padrão):** os logs são retirados do buffer circular interno do painel, filtrados pelo nível selecionado. Os registros são exibidos com o nível (DEBUG / INFO / NOTICE / WARNING / ERROR) e a fonte: `X-UI:` – mensagens do próprio painel, `XRAY:` – mensagens encaminhadas do Xray.
- **Ativado:** o painel executa no servidor `journalctl -u x-ui --no-pager -n <count> -p <level>`, ou seja, exibe o journal do sistema do serviço `x-ui`. O número de linhas permitido é de 1 a 10000; o nível aceita valores syslog (`emerg/0`, `alert/1`, `crit/2`, `err/3`, `warning/4`, `notice/5`, `info/6`, `debug/7`). No Windows, o modo SysLog não é suportado – será exibido um aviso para desmarcar o checkbox e usar os logs da aplicação. Se o `systemd` /serviço não estiver disponível, aparecerá uma mensagem de erro ao iniciar o `journalctl`.

Exemplo: o mesmo journal a partir do console do servidor. Quando o painel está inacessível (por exemplo, não inicia), o journal do sistema pode ser lido diretamente – é exatamente o comando que o painel executa no modo SysLog:

```
# últimas 100 linhas de nível warning e acima
journalctl -u x-ui --no-pager -n 100 -p warning

# acompanhar o journal em tempo real
journalctl -u x-ui -f
```

O nível nesta janela filtra a **saída**. Qual é o nível mínimo registrado no console/syslog é determinado pelo nível de logging do painel (variável de ambiente, padrão `Info` ; em arquivo, o painel sempre escreve no nível `DEBUG`).

Logs do Xray (log de acesso)

Janela separada para o access-log do Xray (`POST /panel/api/server/xraylogs/{count}`). Ela analisa as linhas do log de acesso do Xray e as exibe em tabela: **Date, From, To, Inbound, Outbound, Email**.

Elemento	Valor padrão	Descrição
Número de linhas	<code>20</code>	10 / 20 / 50 / 100 / 500
Filtro	vazio	Busca textual por substring (aplicada ao pressionar Enter)
Direct (checkbox)	ativado	Exibir conexões diretas (tráfego via freedom-outbound)
Blocked (checkbox)	ativado	Exibir conexões bloqueadas (tráfego para blackhole-outbound)
Proxy (checkbox)	ativado	Exibir tráfego proxiado

O tipo de evento é determinado automaticamente pela tag de outbound na linha de log: correspondência com tags freedom → «DIRECT» (verde), blackhole → «BLOCKED» (vermelho), todo o resto → «PROXY» (azul). Linhas `api` → `api` e linhas vazias são ignoradas.

Para que esta janela exiba registros, o Xray deve ter o **log de acesso** habilitado com um caminho de arquivo (não `none`) – veja abaixo. Se o access-log estiver desativado ou o arquivo inacessível, a janela ficará vazia («No Record...»).

16.3. Nível e configuração de logging do Xray

Os parâmetros de logging do próprio Xray são definidos na página «**Configurações do Xray**», no bloco «**Log**» (Log) com o aviso:

«Os logs podem desacelerar o servidor. Habilite somente os tipos de log necessários quando precisar!»

Campo	Tradução	Valor padrão	Descrição
Nível de logs (<code>logLevel</code>)	Log Level	<code>warning</code>	Nível de detalhamento do log de erros do Xray. Valores permitidos: <code>debug</code> , <code>info</code> , <code>notice</code> , <code>warning</code> , <code>error</code> . Dica: «Nível do log para os logs de erros, indicando quais informações devem ser registradas.»
Logs de acesso (<code>accessLog</code>)	Access Log	<code>none</code>	Caminho para o arquivo de log de acesso. O valor especial <code>none</code> desativa os logs de acesso. Dica: «Caminho para o arquivo de log de acesso. O valor especial "none" desativa os logs de acesso.»

Campo	Tradução	Valor padrão	Descrição
Logs de erros (<code>errorLog</code>)	Error Log	vazio (caminho padrão)	Caminho para o arquivo de log de erros; <code>none</code> os desativa. Dica: «Caminho para o arquivo de log de erros. O valor especial "none" desativa os logs de erros.»
Logs de DNS (<code>dnsLog</code>)	DNS Log	<code>false</code> (desativado)	Habilitar logging de requisições DNS. Dica: «Habilitar logs de requisições DNS.»
Mascaramento de endereço (<code>maskAddress</code>)	Mask Address	vazio (desativado)	Quando ativado, o endereço IP real é automaticamente substituído por um endereço mascarado nos logs. Dica: «Quando ativado, o endereço IP real é substituído por um endereço mascarado nos logs.»

Como por padrão «**Logs de acesso**» = `none`, a janela «Logs do Xray» (seção 16.2) está inicialmente vazia. Para que ela funcione, defina aqui um caminho para o access-log e reinicie o Xray.

Observe: um access-log vazio afeta apenas esta janela. A lista de clientes online no «Dashboard» e o limite de quantidade de IPs no formulário do cliente **não dependem** do access-log – o painel determina os clientes online e conta seus endereços IP via online-stats API do núcleo Xray (estatísticas de conexões). Em versões antigas do núcleo que não possuem essa API, o painel automaticamente volta ao método anterior (leitura do access-log), e nesse caso o caminho para o access-log aqui ainda é necessário para o limite de IPs.

Limite por quantidade de IPs e fail2ban. A restrição em si por quantidade de IPs do cliente (campo «IP Limit» no formulário do cliente e ao adicionar em massa) só é aplicada no servidor se o **fail2ban** estiver instalado – é ele que bane os endereços que excedem o limite. O painel verifica a presença do fail2ban (`GET /panel/api/server/fail2banStatus`); se não estiver presente, o campo «IP Limit» fica indisponível com uma dica explicativa (no Windows – com uma mensagem separada), e os limites previamente definidos nesses servidores são automaticamente zerados, pois de qualquer forma não teriam efeito. O bloqueio do fail2ban se aplica tanto ao TCP quanto ao UDP. Em servidores comuns, o fail2ban agora é instalado e configurado automaticamente durante a instalação e atualização do painel (ver seção 16.5).

Exemplo: bloco `log` que fará a janela «Logs do Xray» começar a exibir registros. Na configuração JSON do Xray, fica assim:

```
{
  "log": {
    "loglevel": "warning",
    "access": "./access.log",
    "error": "",
    "dnsLog": false,
    "maskAddress": ""
  }
}
```

O principal é substituir `"access": "none"` por um caminho de arquivo (por exemplo, `"./access.log"`). Após salvar, reinicie o Xray e a tabela na janela «Logs do Xray» será preenchida com registros.

16.4. Gerenciamento do Xray: parada e reinicialização

O estado do Xray é gerenciado pelo card do Xray no «Dashboard». O estado atual é exibido com um dos seguintes valores: **Executando** (`Running`), **Parado** (`Stopped`), **Desconhecido** (`Unknown`), **Erro** (`Error`). Em caso de erro, um tooltip «Erro ao iniciar o Xray» fica disponível.

Botão	Tradução	Endpoint	Ação
Parar	Stop	<code>POST /panel/api/server/stopXrayService</code>	Para o processo do Xray. Em caso de sucesso – notificação de aviso «Xray service has been stopped».
Reiniciar	Restart	<code>POST /panel/api/server/restartXrayService</code>	Reinicia (ou inicia) o Xray com a configuração atual. Em caso de sucesso – notificação «Xray service has been restarted successfully».

Após qualquer uma das operações, o painel transmite o novo estado via WebSocket, de modo que o status no «Dashboard» é atualizado sem recarregar a página. Se a operação falhar, o estado do Xray torna-se «Erro» e o texto do erro aparece na notificação.

Além da reinicialização manual, o painel verifica automaticamente se é necessário reiniciar o Xray (tarefa em background a cada 30 s) e se o processo caiu (verificação a cada segundo) – ver seção 16.6.

16.5. Reinicialização e atualização do painel

Reinicialização do painel

Na página «**Configurações do Paine!**» há a ação «**Reiniciar Paine!**» (`Restart Panel` , `POST /panel/api/setting/restartPanel`). Ao confirmar, o painel é reiniciado **após 3 segundos**.

Mensagens:

- Confirmação: «Tem certeza de que deseja reiniciar o painel? Confirme e a reinicialização ocorrerá em 3 segundos. Se o painel ficar inacessível, verifique o log do servidor.»
- Sucesso: «Paine! reiniciado com sucesso».

Tecnicamente, no Linux a reinicialização é feita enviando o sinal `SIGHUP` ao processo do painel (ou via hook registrado). No Windows, o envio de `SIGHUP` não é suportado.

Atualização automática do painel (Update Panel)

No «Dashboard» está disponível a função «**Atualizar Paine!**» (`Update Panel`) – atualização do 3X-UI para o último lançamento diretamente pela interface web.

Antes da atualização, o painel compara as versões (`GET /panel/api/server/getPanelUpdateInfo`), consultando o último release do 3x-ui no GitHub:

Campo	Tradução
Versão atual do painel	Current panel version
Última versão do painel	Latest panel version
Painel atualizado / «Atualizado»	Panel is up to date / Up to date – exibido quando não há nova versão

Iniciar a atualização – `POST /panel/api/server/updatePanel`. Diálogo de confirmação:

- «Você realmente deseja atualizar o painel?»
- «Isso atualizará o 3X-UI para a versão #version# e reiniciará o serviço do painel.»

Após iniciar – mensagem popup «Atualização do painel iniciada» (`Panel update started`); em caso de falha na verificação de versão – «Falha na verificação de atualização do painel» (`Panel update check failed`).

O que acontece no servidor: a atualização automática é suportada **apenas no Linux** (em outros sistemas operacionais, retornará o erro «panel web update is supported only on Linux installations»). O painel baixa o script oficial `update.sh` do GitHub (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`) e o executa em um processo separado: preferencialmente via `systemd-run` em uma unidade separada (`x-ui-web-update-<timestamp>`), e na ausência do `systemd` – como um processo separado desconectado. Ao concluir, o script atualiza os componentes e reinicia o serviço do painel. É necessário `bash` para a execução.

Se durante a atualização o script gerar um novo caminho base aleatório da web do painel (Web Base Path), o serviço `x-ui` é reiniciado automaticamente para que o novo caminho funcione imediatamente. (Sem reinicialização, o servidor continuaria servindo o caminho antigo enquanto a interface exibia o novo, e o novo endereço ficaria inacessível até uma reinicialização manual.)

Nos nós (nodes), o painel deste mesmo 3x-ui é atualizado de forma centralizada via `POST /panel/api/nodes/updatePanel` – ver seção sobre nós.

Instalação automática do fail2ban

Para que o limite por quantidade de IPs dos clientes (seção 16.3) funcione imediatamente, durante a instalação e atualização do painel em um servidor comum, o `fail2ban` agora é instalado e configurado automaticamente (antes, isso acontecia somente na imagem Docker). O comportamento é controlado pela variável de ambiente `XUI_ENABLE_FAIL2BAN`: a configuração é executada se a variável não estiver definida ou for igual a `true`. A execução manual está disponível pelo comando `x-ui setup-fail2ban`. Uma falha na configuração do `fail2ban` não interrompe a instalação ou atualização do painel.

Instalação e atualização em hosts somente IPv6

Os scripts `install.sh` e `update.sh` agora funcionam corretamente em servidores apenas com IPv6: o download do release, do script `x-ui.sh` e dos arquivos de serviço não usa mais IPv4 forçado (`curl -4`), utilizando o protocolo disponível. Portanto, o painel pode ser instalado e atualizado também em hosts sem endereço IPv4.

Redefinição da porta do painel pela variável `XUI_PORT`

A porta de escuta do painel web pode ser redefinida pela variável de ambiente `XUI_PORT` — ela age apenas durante a execução do processo atual e **não altera** o valor salvo `webPort` no banco de dados. São permitidos valores de `1` a `65535` ; um valor vazio, incorreto ou fora do intervalo é ignorado (usa-se `webPort`) com um aviso no log. Isso é útil durante a implantação, principalmente no Docker: ao usar rede bridge, a porta publicada do container deve coincidir com `XUI_PORT` — por exemplo, `XUI_PORT=8080` e `ports: "8080:8080"` .

Atualização e troca de versão do Xray-core

Também no «Dashboard» é possível gerenciar a versão do Xray-core separadamente do painel.

- **Atualizações do Xray** (`Xray Updates`) / **Selecionar versão** (`Version`) — lista suspensa com versões disponíveis. Dicas: «Selecione a versão desejada» e aviso «Importante: versões antigas podem não suportar as configurações atuais».
- Instalação/troca de versão — `POST /panel/api/server/installXray/{version}` . Diálogo: «Trocar versão do Xray» / «Tem certeza de que deseja trocar a versão do Xray?». Em caso de sucesso — «Xray atualizado com sucesso».

Exemplo: troca de versão do Xray-core via requisição à API. A versão é especificada pela tag de release do XTLS/Xray-core (com prefixo `v`). Por exemplo, mudar para `v1.8.24` :

```
curl -s -b cookies.txt -X POST \
  https://panel.example.com:2053/panel/api/server/installXray/v1.8.24
```

(`cookies.txt` — arquivo com cookie do exemplo na seção 16.1.) Após a instalação, o Xray será reiniciado automaticamente com a versão selecionada.

No servidor, ao trocar a versão, o Xray é primeiro parado, o arquivo da versão desejada é baixado do GitHub (XTLS/Xray-core), o binário é descompactado e substituído, e depois o Xray é reiniciado com verificação dos tamanhos do arquivo/binário.

16.6. Tarefas periódicas (cron)

O painel registra uma série de tarefas em background ao iniciar. Seus agendamentos são fixos (não configuráveis na UI, exceto o agendamento do relatório do Telegram e a sincronização LDAP). Abaixo estão as tarefas relacionadas à operação.

Tarefa	Agendamento	Finalidade
Verificação de funcionamento do Xray	a cada 1 s	Controlar se o processo do Xray está em execução
Verificação de necessidade de reinicialização do Xray	a cada 30 s	Reinicializar se a configuração foi marcada como alterada
Coleta de tráfego do Xray	a cada 5 s (inicia 5 s após a inicialização)	Contabilização de tráfego de inbounds/clientes

Tarefa	Agendamento	Finalidade
Verificação de IPs dos clientes	a cada 10 s	Controle do limite de IPs pelo log
Heartbeat e sincronização de tráfego dos nós	a cada 5 s	Comunicação com os nós (nodes)
Limpeza de logs	diariamente (@daily)	Limpa os logs de limite de IPs e o access-log persistente, rotacionando o log atual para <code>*.prev.log</code>
Redefinição de tráfego por período	<code>@hourly</code> , <code>@daily</code> , <code>@weekly</code> , <code>@monthly</code>	Redefine os contadores de tráfego dos inbounds (e seus clientes) que têm o período de redefinição automática configurado
Relatório do Telegram	definido nas configurações do bot (padrão <code>@daily</code>)	Envio do relatório para os administradores; quando a opção está habilitada – com o backup do banco de dados anexado (seção 16.1)
Redefinição do armazenamento hash do Telegram	a cada 2 m	Apenas quando o bot está habilitado
Monitoramento de carga de CPU para o Telegram	a cada 10 s	Apenas se o limite de CPU > 0 estiver definido

Adicionalmente:

- **Redefinição periódica de tráfego** é acionada somente para os inbounds que têm o modo de redefinição automática configurado (a cada hora/dia/semana/mês). A tarefa redefine o tráfego do próprio inbound e de todos os seus clientes.
- **Verificação de expiração e esgotamento.** A desativação de clientes por expiração e por esgotamento do limite de tráfego é feita durante a contabilização de tráfego: clientes com `expiry_time` expirado ou volume esgotado são marcados e desativados; quando necessário, o próximo prazo é calculado (para limites cíclicos e modo «contagem a partir do primeiro uso»). No «Dashboard» e nas listas, isso é refletido com os status «Expirado»/«Esgotado»/«Expirando em breve».
- **Backup automático no Telegram** – este é um efeito colateral da tarefa de relatório; não há um agendamento cron separado apenas para backup. Portanto, a frequência do backup automático é igual à frequência do relatório do bot.

16.7. Menu de console e CLI (`x-ui`)

No servidor, o painel é gerenciado pelo comando `x-ui` . Sem argumentos, abre o menu interativo «3X-UI Panel Management Script»; com um argumento, executa um subcomando específico. Itens do menu relacionados à operação:

Nº no menu	Item	Ação
1	Install	Instalação do painel (baixa e executa <code>install.sh</code>)
2	Update	Atualização de todos os componentes do x-ui para a última versão sem perda de dados; seguida de reinicialização automática
3	Update Menu	Atualização apenas do script de menu <code>x-ui</code>

Nº no menu	Item	Ação
4	Legacy Version	Instalação de uma versão específica (antiga) do painel pelo número informado (por exemplo, 2.4.0)
5	Uninstall	Remoção completa do painel e do Xray (ver 16.8)
6	Reset Username & Password	Redefinição do login/senha do administrador
7	Reset Web Base Path	Redefinição do caminho base do painel web
8	Reset Settings	Redefinição das configurações para os valores padrão
9	Change Port	Alteração da porta do painel
10	View Current Settings	Visualização das configurações atuais
11-13	Start / Stop / Restart	Iniciar, parar, reiniciar o serviço do painel
14	Restart Xray	Reiniciar apenas o Xray
15	Check Status	Status atual do serviço
16	Logs Management	Visualização e limpeza de logs (ver abaixo)
17-18	Enable / Disable Autostart	Habilitar/desabilitar inicialização automática do serviço ao iniciar o SO
25	Update Geo Files	Atualização dos arquivos geográficos (GeoIP/GeoSite)
27	PostgreSQL Management	Gerenciamento do PostgreSQL

Gerenciamento de logs na CLI (item 16)

No submenu «Logs Management»:

- **Debug Log** – visualização contínua do journal do serviço: `journalctl -u x-ui -e --no-pager -f -p debug` (no Alpine – `grep` em `/var/log/messages`).
- **Clear All logs** – limpeza do journal do sistema: `journalctl --rotate + journalctl --vacuum-time=1s` , após o que o serviço é reiniciado. (Não disponível no Alpine.)

Subcomandos diretos do `x-ui`

Todos os subcomandos disponíveis:

Comando	Descrição
<code>x-ui</code>	Abrir o menu administrativo
<code>x-ui start</code>	Iniciar o painel
<code>x-ui stop</code>	Parar o painel
<code>x-ui restart</code>	Reiniciar o painel

Comando	Descrição
<code>x-ui restart-xray</code>	Reiniciar o Xray
<code>x-ui status</code>	Status atual
<code>x-ui settings</code>	Exibir as configurações atuais
<code>x-ui enable</code>	Habilitar inicialização automática ao iniciar o SO
<code>x-ui disable</code>	Desabilitar inicialização automática
<code>x-ui log</code>	Visualização de logs
<code>x-ui banlog</code>	Visualização dos logs de bloqueios do Fail2ban
<code>x-ui setup-fail2ban</code>	Instalar e configurar o fail2ban para limite de IPs (ver 16.5)
<code>x-ui update</code>	Atualizar o painel
<code>x-ui update-all-geofiles</code>	Atualizar todos os arquivos geográficos (com reinicialização posterior)
<code>x-ui migrateDB [file]</code>	Conversão do banco <code>.db</code> para <code>.dump</code> (SQLite)
<code>x-ui legacy</code>	Instalar uma versão antiga
<code>x-ui install</code>	Instalar o painel
<code>x-ui uninstall</code>	Remover o painel

O comando `x-ui update` baixa e executa o `update.sh` oficial (o mesmo que a atualização web da seção 16.5), solicitando confirmação: «This function will update all x-ui components to the latest version, and the data will not be lost.» Ao concluir, o painel é reiniciado automaticamente.

Flags `-webCert` / `-webCertKey` no subcomando `setting`. Os caminhos para o certificado e a chave privada do painel web podem ser definidos diretamente no subcomando `x-ui setting -webCert <caminho> -webCertKey <caminho>` — informar qualquer um desses flags salva o caminho correspondente (assim como o subcomando `cert` separado), e o painel muda imediatamente para HTTPS.

Obtenção de token de API via CLI

O comando de obtenção de token de API via CLI (item de menu/comando `x-ui`) não exibe um token previamente gerado. Os tokens de API são armazenados apenas na forma de hashes, portanto um token existente não pode ser recuperado em texto claro. Se os tokens já estiverem configurados, o comando informa a quantidade deles, sugere gerenciá-los no painel (**Settings** → **API Tokens**, ver seção sobre tokens de API) e gera imediatamente um **novo token de fallback** com o nome no formato `cli-fallback-<timestamp>`, exibindo-o para que a CLI continue útil sem precisar acessar a interface.

16.8. Remoção do painel

A remoção é feita pela CLI – item de menu **5 (Uninstall)** ou comando `x-ui uninstall`. Antes da remoção, é solicitada uma confirmação (padrão «não»): «Are you sure you want to uninstall the panel? xray will also be uninstalled!».

Ao confirmar, o script:

1. Para o serviço e desabilita sua inicialização automática (`systemctl stop/disable x-ui`, ou no Alpine – `rc-service / rc-update`), remove o arquivo de unit do serviço e recarrega a configuração do systemd.
2. Remove os diretórios de dados e de aplicação (`/etc/x-ui/`, diretório de instalação) e o arquivo env do serviço (`/etc/default/x-ui`, `/etc/conf.d/x-ui` ou `/etc/sysconfig/x-ui` – dependendo da distribuição).
3. Remove o próprio script `x-ui` e exibe a mensagem «Uninstalled Successfully», bem como o comando para reinstalação.

A remoção é irreversível: junto com o painel, o Xray e todos os dados são removidos (incluindo o banco de dados). Se os dados puderem ser necessários, exporte o banco antecipadamente (seção 16.1).

16.9. Comando `x-ui migrateDB`

A partir da versão 3.3.0, o script de controle `x-ui.sh` recebeu o subcomando `migrateDB` – um wrapper em torno do binário integrado `x-ui` (`x-ui migrate-db`) para converter o banco de dados do painel SQLite entre dois formatos: binário `.db` e dump de texto portátil `.dump` (texto SQL comum).

O que o comando faz

O comando funciona em dois sentidos, e a direção é determinada **automaticamente** pelo arquivo de entrada:

Direção	Nome	O que acontece
<code>.db → .dump</code>	dump (exportação)	o banco SQLite binário é exportado para um arquivo SQL de texto
<code>.dump → .db</code>	restore (restauração)	a partir do arquivo SQL de texto, um banco SQLite binário é reconstruído

Internamente, o script chama o binário do painel:

- exportação: `x-ui migrate-db --src <entrada> --dump <saída>`
- restauração: `x-ui migrate-db --restore <entrada> --out <saída>`

Sintaxe de chamada

```
x-ui migrateDB [file.db|file.dump] [output]
```

- **[file.db|file.dump]** – arquivo de entrada (primeiro argumento). Se não especificado, usa-se o banco padrão instalado do painel: `/etc/x-ui/x-ui.db`.

- **[output]** – caminho para o arquivo de saída (segundo argumento). Opcional: se ausente, o nome é escolhido automaticamente no mesmo diretório do arquivo de entrada (ver abaixo).

Exemplos:

```
x-ui migrateDB                               # exportar /etc/x-ui/x-ui.db -> /etc/x-ui/x-
ui.dump
x-ui migrateDB /etc/x-ui/x-ui.db backup.dump
x-ui migrateDB backup.dump restored.db      # reconstruir .db a partir do dump
```

Como a direção é determinada

O script verifica a extensão do arquivo de entrada:

- *.db, *.sqlite, *.sqlite3 → modo **dump** (exportação para texto);
- *.dump, *.sql → modo **restore** (reconstrução do banco).

Se a extensão não for reconhecida, o script lê os primeiros 16 bytes do arquivo: a assinatura SQLite format 3 indica um banco binário (modo dump); caso contrário, o arquivo é considerado um dump (modo restore).

Nome do arquivo de saída, se o segundo argumento não for informado:

- na exportação – mesmo nome do arquivo de entrada, com extensão `.dump`;
- na restauração – mesmo nome com extensão `.db`.

Verificações de proteção e comportamento

- **Presença do binário.** Se o binário `x-ui` não for encontrado ou não for executável – é exibido o erro «x-ui binary not found ... Is the panel installed?».
- **Suporte da função na build.** O script verifica se o binário suporta `migrate-db --dump/--restore` (via `x-ui migrate-db -h`). Se não – sugere primeiro atualizar o painel com `x-ui update`.
- **Existência do arquivo de entrada.** Se o arquivo de entrada não existir, é exibida uma mensagem de erro e a linha com a sintaxe de chamada.
- **Sobrescrita da saída.** Se o arquivo de saída já existir, é solicitada confirmação (padrão «não»); sem confirmação, a operação é cancelada. Na restauração, o arquivo de saída antigo é removido previamente.
- **Proteção do banco em uso.** Na restauração para o banco padrão `/etc/x-ui/x-ui.db` enquanto o painel estiver em execução, a operação é recusada com a exigência de primeiro parar o painel (`x-ui stop`) ou escolher outro caminho de saída. Isso evita a sobrescrita do banco de trabalho de um serviço em execução.
- Em caso de falha na reconstrução do banco, o arquivo de saída incompleto é removido.

Para que serve

- **Backup.** O `.dump` em texto é legível por humanos, conveniente para armazenamento em sistemas de controle de versão e para visualização diferencial do conteúdo do banco.
- **Migração.** O dump é portátil entre máquinas e resistente a diferenças entre versões do formato do arquivo SQLite – em um novo servidor, um `.db` funcional é reconstruído a partir dele.

- **Diagnóstico.** A partir do `.dump` é possível visualizar a estrutura e os dados do painel sem ter ferramentas SQLite à mão.

Modo interativo

Além da chamada direta, a conversão está disponível no menu interativo. No submenu PostgreSQL (`x-ui` → seção de gerenciamento do PostgreSQL) há o item **9. Convert SQLite `.db` <--> `.dump`** : ele solicita o caminho para o arquivo de entrada (padrão `/etc/x-ui/x-ui.db`) e para o arquivo de saída (pode ser deixado vazio para nomeação automática), e a direção, como no modo CLI, é determinada automaticamente.

Documento elaborado com base no código-fonte do 3X-UI. Se algum item da interface na sua versão for diferente – prevalece o comportamento do painel e as dicas na própria UI.